



Universidad Peruana de Ciencias Aplicadas

Carrera de Ingeniería de Software

**1ACC0238**

**Aplicaciones para Dispositivos Móviles**

NRC

**13975**

**Informe del Trabajo Final**

Docente

**Quevedo Velasco, David Gerardo**

Equipo

**ADM**

Proyecto

**AniTec**

**Integrantes:**

| <b>Código</b> | <b>Apellidos y Nombres</b>            |
|---------------|---------------------------------------|
| U202217853    | Beingolea Montalvo, Sebastian Martin  |
| U202315165    | Melgarejo Quiroz, Josep Eliu          |
| U20231C019    | Ortega Muñoz, Saul                    |
| U202215979    | Sanchez Silva, Luciana Celeste        |
| U20221C554    | Villanueva Rodriguez, Giuseppe Adrian |

**Período 202620**

**Septiembre 2026**

# Índice general

---

- [Registro de Versiones del Informe](#)
- [Project Report Collaboration Insights](#)
- [Student Outcome](#)

## Capítulo I: Presentación

- [1.1. Startup Profile](#)
  - [1.1.1. Descripción de la Startup](#)
  - [1.1.2. Perfiles de integrantes del equipo](#)
- [1.2. Solution Profile](#)
  - [1.2.1. Antecedentes y problemática](#)
  - [1.2.2. Lean UX Process](#)
    - [1.2.2.1. Lean UX Problem Statements](#)
    - [1.2.2.2. Lean UX Assumptions](#)
    - [1.2.2.3. Lean UX Hypothesis Statements](#)
    - [1.2.2.4. Lean UX Canvas](#)
- [1.3. Segmentos objetivo](#)

## Capítulo II: Requirements Development and Software Solution Design

- [2.1. Competidores](#)
  - [2.1.1. Análisis competitivo](#)
  - [2.1.2. Estrategias y tácticas frente a competidores](#)
- [2.2. Entrevistas](#)
  - [2.2.1. Diseño de entrevistas](#)
  - [2.2.2. Registro de entrevistas](#)
  - [2.2.3. Análisis de entrevistas](#)
- [2.3. Needfinding](#)
  - [2.3.1. User Personas](#)
  - [2.3.2. User Task Matrix](#)
  - [2.3.3. User Journey Mapping](#)
  - [2.3.4. Empathy Mapping](#)
  - [2.3.5. Big Picture EventStorming](#)
  - [2.3.6. Ubiquitous Language](#)
- [2.4. Requirements specification](#)
  - [2.4.1. User Stories](#)
  - [2.4.2. Impact Mapping](#)
  - [2.4.3. Product Backlog](#)

- 2.5. Strategic-Level Domain-Driven Design
  - 2.5.1. EventStorming
    - 2.5.1.1. Candidate Context Discovery
    - 2.5.1.2. Domain Message Flows Modeling
    - 2.5.1.3. Bounded Context Canvases
  - 2.5.2. Context Mapping
  - 2.5.3. Software Architecture
    - 2.5.3.1. Software Architecture Context Level Diagrams
    - 2.5.3.2. Software Architecture Container Level Diagrams
    - 2.5.3.3. Software Architecture Deployment Diagrams
- 2.6. Tactical-Level Domain-Driven Design
  - 2.6.1. Bounded Context: Identity and Access Management
    - 2.6.1.1. Domain Layer
    - 2.6.1.2. Interface Layer
    - 2.6.1.3. Application Layer
    - 2.6.1.4. Infrastructure Layer
    - 2.6.1.5. Bounded Context Software Architecture Component Level Diagrams
    - 2.6.1.6. Bounded Context Software Architecture Code Level Diagrams
      - 2.6.1.6.1. Bounded Context Domain Layer Class Diagrams
      - 2.6.1.6.2. Bounded Context Database Design Diagram
  - 2.6.2. Bounded Context: Profile Management
    - 2.6.2.1. Domain Layer
    - 2.6.2.2. Interface Layer
    - 2.6.2.3. Application Layer
    - 2.6.2.4. Infrastructure Layer
    - 2.6.2.5. Bounded Context Software Architecture Component Level Diagrams
    - 2.6.2.6. Bounded Context Software Architecture Code Level Diagrams
      - 2.6.2.6.1. Bounded Context Domain Layer Class Diagrams
      - 2.6.2.6.2. Bounded Context Database Design Diagram
  - 2.6.3. Bounded Context: Livestock Management
    - 2.6.3.1. Domain Layer
    - 2.6.3.2. Interface Layer
    - 2.6.3.3. Application Layer
    - 2.6.3.4. Infrastructure Layer
    - 2.6.3.5. Bounded Context Software Architecture Component Level Diagrams
    - 2.6.3.6. Bounded Context Software Architecture Code Level Diagrams
      - 2.6.3.6.1. Bounded Context Domain Layer Class Diagrams
      - 2.6.3.6.2. Bounded Context Database Design Diagram
  - 2.6.4. Bounded Context: Sanitary Management
    - 2.6.4.1. Domain Layer
    - 2.6.4.2. Interface Layer
    - 2.6.4.3. Application Layer

- 2.6.4.4. Infrastructure Layer
- 2.6.4.5. Bounded Context Software Architecture Component Level Diagrams
- 2.6.4.6. Bounded Context Software Architecture Code Level Diagrams
  - 2.6.4.6.1. Bounded Context Domain Layer Class Diagrams
  - 2.6.4.6.2. Bounded Context Database Design Diagram
- 2.6.5. Bounded Context: Veterinary Collaboration
  - 2.6.5.1. Domain Layer
  - 2.6.5.2. Interface Layer
  - 2.6.5.3. Application Layer
  - 2.6.5.4. Infrastructure Layer
  - 2.6.5.5. Bounded Context Software Architecture Component Level Diagrams
  - 2.6.5.6. Bounded Context Software Architecture Code Level Diagrams
    - 2.6.5.6.1. Bounded Context Domain Layer Class Diagrams
    - 2.6.5.6.2. Bounded Context Database Design Diagram
- 2.6.6. Bounded Context: Activity Management
  - 2.6.6.1. Domain Layer
  - 2.6.6.2. Interface Layer
  - 2.6.6.3. Application Layer
  - 2.6.6.4. Infrastructure Layer
  - 2.6.6.5. Bounded Context Software Architecture Component Level Diagrams
  - 2.6.6.6. Bounded Context Software Architecture Code Level Diagrams
    - 2.6.6.6.1. Bounded Context Domain Layer Class Diagrams
    - 2.6.6.6.2. Bounded Context Database Design Diagram
- 2.6.7. Bounded Context: Financial Management
  - 2.6.7.1. Domain Layer
  - 2.6.7.2. Interface Layer
  - 2.6.7.3. Application Layer
  - 2.6.7.4. Infrastructure Layer
  - 2.6.7.5. Bounded Context Software Architecture Component Level Diagrams
  - 2.6.7.6. Bounded Context Software Architecture Code Level Diagrams
    - 2.6.7.6.1. Bounded Context Domain Layer Class Diagrams
    - 2.6.7.6.2. Bounded Context Database Design Diagram
- 2.6.8. Bounded Context: Subscription Management
  - 2.6.8.1. Domain Layer
  - 2.6.8.2. Interface Layer
  - 2.6.8.3. Application Layer
  - 2.6.8.4. Infrastructure Layer
  - 2.6.8.5. Bounded Context Software Architecture Component Level Diagrams
  - 2.6.8.6. Bounded Context Software Architecture Code Level Diagrams
    - 2.6.8.6.1. Bounded Context Domain Layer Class Diagrams
    - 2.6.8.6.2. Bounded Context Database Design Diagram
- 2.6.9. Bounded Context: Analytics and Reporting

- 2.6.9.1. Domain Layer
- 2.6.9.2. Interface Layer
- 2.6.9.3. Application Layer
- 2.6.9.4. Infrastructure Layer
- 2.6.9.5. Bounded Context Software Architecture Component Level Diagrams
- 2.6.9.6. Bounded Context Software Architecture Code Level Diagrams
  - 2.6.9.6.1. Bounded Context Domain Layer Class Diagrams
  - 2.6.9.6.2. Bounded Context Database Design Diagram

## Capítulo III: Solution UI/UX Design

- 3.1. Product design
  - 3.1.1. Style Guidelines
    - 3.1.1.1. General Style Guidelines
  - 3.1.2. Information Architecture
    - 3.1.2.1. Organization Systems
    - 3.1.2.2. Labelling Systems
    - 3.1.2.3. SEO Tags and Meta Tags
    - 3.1.2.4. Searching Systems
    - 3.1.2.5. Navigation Systems
  - 3.1.3. Landing Page UI Design
    - 3.1.3.1. Landing Page Wireframe
    - 3.1.3.2. Landing Page Mock-up
  - 3.1.4. Mobile Applications UX/UI Design
    - 3.1.4.1. Mobile Applications Wireframes
    - 3.1.4.2. Mobile Applications Wireflow Diagrams
    - 3.1.4.3. Mobile Applications Mock-ups
    - 3.1.4.4. Mobile Applications User Flow Diagrams
    - 3.1.4.5. Mobile Applications Prototyping

## Capítulo IV: Product Implementation & Validation

- 4. Product Implementation & Validation
  - 4.1. Software Configuration Management
    - 4.1.1. Software Development Environment Configuration
    - 4.1.2. Source Code Management
    - 4.1.3. Source Code Style Guide & Conventions
    - 4.1.4. Software Deployment Configuration
  - 4.2. Landing Page & Mobile Application Implementation
    - 4.2.1. Sprint n
      - 4.2.1.1. Sprint Planning n
      - 4.2.1.2. Aspect Leaders and Collaborators

- 4.2.1.3. Sprint Backlog n
- 4.2.1.4. Development Evidence for Sprint Review
- 4.2.1.5. Testing Suite Evidence for Sprint Review
- 4.2.1.6. Execution Evidence for Sprint Review
- 4.2.1.7. Services Documentation Evidence for Sprint Review
- 4.2.1.8. Software Deployment Evidence for Sprint Review
- 4.2.1.9. Team Collaboration Insights during Sprint
- 4.3. Validation Interviews
  - 4.3.1. Diseño de Entrevistas
  - 4.3.2. Registro de Entrevistas
  - 4.3.3. Evaluaciones según heurísticas
- Conclusiones
- Bibliografía
- Anexos

# Registro de versiones del informe

Las entradas de la versión 1 corresponden al avance AV1 del curso de Aplicaciones para Dispositivos Móviles. La fecha indica la actualización de este registro.

| Versión | Fecha      | Autor                               | Descripción de modificación                               |
|---------|------------|-------------------------------------|-----------------------------------------------------------|
| 1.0     | 16/09/2026 | Sebastian Martin Beingolea Montalvo | Redacción del capítulo I: Introducción.                   |
| 1.1     | 16/09/2026 | Saul Ortega Muñoz                   | Redacción de las secciones 2.1 a 2.4 del capítulo II.     |
| 1.2     | 16/09/2026 | Josep Eliu Melgarejo Quiroz         | Redacción de las secciones 2.6.1 a 2.6.9 del capítulo II. |
| 1.3     | 16/09/2026 | Giuseppe Villanueva Rodriguez       | Realizó una entrevista.                                   |
| 1.4     | 16/09/2026 | Luciana Celeste Sanchez Silva       | Redacción de las secciones 2.5 a 2.6 del capítulo II.     |

# Project Report Collaboration Insights

## Repositorios del proyecto

- Informe
- Landing page
- Frontend web heredado
- Backend

## AV1. Distribución del trabajo

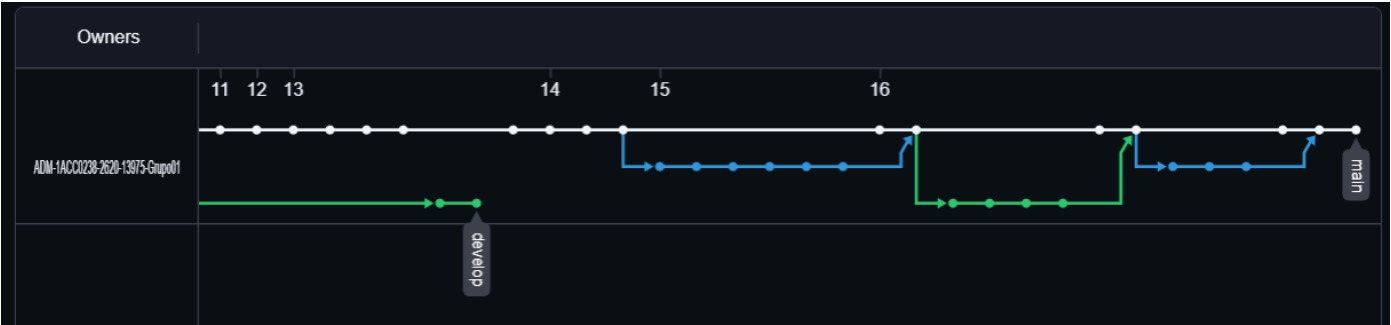
Durante AV1, el equipo adaptó el informe y el diseño de AniTec al alcance de una aplicación móvil. Las contribuciones documentadas para los cinco integrantes actuales son las siguientes:

| Integrante                          | Contribución en AV1                                  |
|-------------------------------------|------------------------------------------------------|
| Sebastian Martin Beingolea Montalvo | Redactó el capítulo I: Introducción.                 |
| Saul Ortega Muñoz                   | Redactó las secciones 2.1 a 2.4 del capítulo II.     |
| Josep Eliu Melgarejo Quiroz         | Redactó las secciones 2.6.1 a 2.6.9 del capítulo II. |
| Giuseppe Villanueva Rodriguez       | Realizó una entrevista.                              |
| Luciana Celeste Sanchez Silva       | Redactó las secciones 2.5 a 2.6 del capítulo II.     |

## GitHub Collaboration Insights

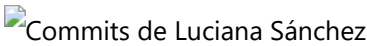
El repositorio del informe conserva el historial de los cambios documentales del equipo. Las siguientes capturas muestran el grafo general y los commits asociados a cada integrante durante el avance.

Grafo general de commits



Commits por integrante

Luciana Celeste Sanchez Silva



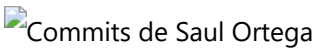
Josep Eliu Melgarejo Quiroz



Sebastian Martin Beingolea Montalvo



Saul Ortega Muñoz



Giuseppe Villanueva Rodriguez



# Student Outcome

## ABET - EAC - Student Outcome 7

**Criterio:** La capacidad de adquirir y aplicar nuevos conocimientos según sea necesario, utilizando estrategias de aprendizaje apropiadas.

En el siguiente cuadro se describen las acciones realizadas y los enunciados de conclusiones por parte del grupo durante el AV1, que permiten sustentar el logro del ABET - EAC - Student Outcome 7.

| Criterio específico                 | Acciones realizadas | Conclusiones |
|-------------------------------------|---------------------|--------------|
| Actualiza conceptos y conocimientos | AV1:                | AV1:         |

| Criterio específico                                                                                 | Acciones realizadas                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               | Conclusiones                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|-----------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| necesarios para su desarrollo profesional y en especial para su proyecto en soluciones de software. | <p><b>Luciana Celeste Sanchez Silva:</b> Revisó los conceptos de Lean UX, segmentación de usuarios y diseño centrado en las necesidades del ganadero y del veterinario. Aplicó estos conocimientos al actualizar el Startup Profile, el Solution Profile, los Problem Statements, Assumptions e Hypothesis Statements para orientar AniTec hacia una aplicación móvil.</p> <p><b>Josep Eliu Melgarejo Quiroz:</b> Analizó el statement del curso y estudió la estructura requerida para la especificación de requisitos móviles. Aplicó lo aprendido al reorganizar el índice del informe, adaptar las User Stories y Technical Stories, y distribuir el Product Backlog en tres sprints alineados con los hitos del curso.</p> <p><b>Sebastian Martin Beingolea Montalvo:</b> Profundizó en los fundamentos de Domain-Driven Design, EventStorming y lenguaje ubicuo. Aplicó estos conceptos en la identificación y delimitación de los bounded contexts, así como en la revisión de los flujos de mensajes del dominio ganadero.</p> <p><b>Saul Ortega Muñoz:</b> Investigó la notación C4, Structurizr DSL y la representación de arquitecturas para aplicaciones móviles. Aplicó estos conocimientos al apoyar la definición de los diagramas de contexto, contenedores, despliegue y componentes de AniTec.</p> <p><b>Giuseppe Villanueva Rodriguez:</b></p> | <p>El equipo actualizó conocimientos de Lean UX, Requirements Engineering, Domain-Driven Design, arquitectura C4 y desarrollo para dispositivos móviles. Estos conocimientos se aplicaron directamente en la corrección de los capítulos I y II, la definición del alcance de Android y Flutter, la elaboración de requisitos trazables y el diseño estratégico y táctico de los nueve bounded contexts. Como resultado, AniTec cuenta con una base documental coherente con el statement del curso y preparada para orientar su posterior implementación móvil.</p> |

| Criterio específico                                                                                                                           | Acciones realizadas                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                | Conclusiones                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|-----------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                                                                                               | <p>Revisó conceptos relacionados con desarrollo Android, aplicaciones multiplataforma, persistencia local, sincronización, uso de cámara y notificaciones. Aplicó estos conocimientos al verificar la coherencia entre los requisitos obligatorios del curso, las funcionalidades móviles y las historias incluidas en el Product Backlog.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| <p>Reconoce la necesidad del aprendizaje permanente para el desempeño profesional y el desarrollo de proyectos en soluciones de software.</p> | <p><b>AV1:</b></p> <p><b>Luciana Celeste Sanchez Silva:</b></p> <p>Identificó que los supuestos sobre facilidad de uso, adopción y colaboración entre ganaderos y veterinarios deben contrastarse continuamente mediante entrevistas y validaciones. Por ello, consideró la investigación con usuarios como una fuente permanente de aprendizaje para mejorar la propuesta móvil.</p> <p><b>Josep Eliu Melgarejo Quiroz:</b></p> <p>Reconoció la necesidad de revisar continuamente el statement, el Product Backlog y los criterios de evaluación para detectar brechas entre el trabajo heredado y las exigencias del nuevo curso. Utilizó la comparación sistemática y la mejora iterativa del informe como estrategia de aprendizaje.</p> <p><b>Sebastian Martin Beingolea Montalvo:</b> Comprendió que la delimitación del dominio puede evolucionar conforme aparecen nuevos eventos, reglas y dependencias. Adoptó la revisión de recursos anteriores, la discusión del</p> | <p><b>AV1:</b></p> <p>El equipo reconoció que el desarrollo de AniTec requiere aprendizaje continuo porque combina un producto web heredado con nuevas plataformas, herramientas y restricciones móviles. La revisión de documentación, el análisis de trabajos anteriores, la investigación individual, la discusión en equipo y la validación progresiva permitieron corregir supuestos y fundamentar decisiones. Estas prácticas fortalecen la capacidad del grupo para incorporar nuevos conocimientos durante los siguientes hitos y responder a cambios técnicos o de usuario sin perder coherencia en la solución.</p> |

| Criterio específico | Acciones realizadas                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             | Conclusiones |
|---------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------|
|                     | <p>lenguaje ubicuo y la contrastación entre diagramas como estrategias para refinar el diseño.</p> <p><b>Saul Ortega Muñoz:</b> Reconoció que las herramientas y convenciones de arquitectura requieren práctica y actualización constante. Consultó ejemplos, documentación y resultados exportados para comprobar que los diagramas representaran correctamente las relaciones entre usuarios, aplicaciones y servicios.</p> <p><b>Giuseppe Villanueva Rodriguez:</b> Identificó que las capacidades móviles plantean riesgos que deben investigarse antes de implementarse, especialmente en conectividad, permisos, almacenamiento local y uso de recursos del dispositivo. Consideró las Spike Stories, los prototipos y las pruebas en dispositivos físicos como estrategias necesarias para adquirir evidencia y ajustar decisiones.</p> |              |

# Capítulo I: Presentación

---

## 1.1. Startup Profile

---

En esta sección se presenta la startup responsable de AniTec y a los integrantes del equipo que desarrollarán la solución durante el curso de Aplicaciones para Dispositivos Móviles.

### 1.1.1. Descripción de la Startup

Titan es una startup tecnológica que desarrolla soluciones digitales para mejorar la gestión de pequeños y medianos productores ganaderos. Su producto, AniTec, nació como una aplicación web para centralizar la información sanitaria, productiva y económica del ganado. En este curso, la propuesta evoluciona hacia una solución móvil que permita realizar las tareas principales desde el teléfono, dispositivo que acompaña al usuario durante el trabajo de campo.

AniTec contará con una aplicación nativa para Android desarrollada con Kotlin y con una experiencia multiplataforma. La tecnología de esta última se seleccionará mediante una evaluación técnica entre las alternativas permitidas por el curso. Ambas aplicaciones consumirán la API REST propia de AniTec y ofrecerán una experiencia coherente, accesible e internacionalizada.

La solución móvil se concentrará en las siguientes capacidades:

- Registrar, consultar y actualizar animales y sus datos principales.
- Consultar el historial sanitario y registrar atenciones, tratamientos y otros eventos.
- Guardar borradores o información esencial localmente para afrontar conexiones inestables.
- Programar y recibir recordatorios de vacunaciones, tratamientos y actividades.
- Identificar animales con apoyo de la cámara, manteniendo una búsqueda manual como alternativa.
- Consultar indicadores y resúmenes visuales que faciliten decisiones de campo.
- Permitir que los veterinarios consulten los animales autorizados por sus clientes y registren el seguimiento correspondiente.
- Gestionar planes mediante un servicio externo de pagos, sujeto a validación técnica y funcional.

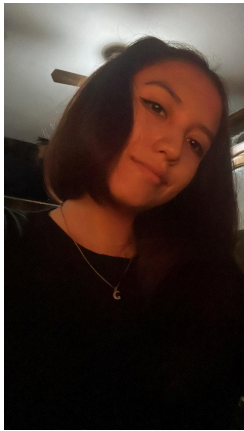
El ecosistema se complementará con la landing page existente, que comunicará la propuesta de valor y dirigirá a la descarga de las aplicaciones. El desarrollo también incluirá persistencia local, acceso a capacidades del dispositivo, manejo de errores de red, accesibilidad e internacionalización. Además, se incorporará una capacidad autónoma basada en una biblioteca o servicio no utilizado previamente en clase, cuya viabilidad se comprobará mediante una Spike Story antes de integrarla al producto.

**Misión:** Facilitar la gestión y trazabilidad del ganado de pequeños y medianos productores mediante una experiencia móvil simple, accesible y útil en el trabajo de campo, que conecte al ganadero con la información de sus animales y con el seguimiento veterinario.

**Visión:** Convertir a AniTec en una solución móvil confiable para la gestión ganadera en Latinoamérica, reconocida por adaptarse a las condiciones reales del entorno rural y por ayudar a tomar decisiones sanitarias y productivas basadas en información ordenada.

## 1.1.2. Perfiles de integrantes del equipo

Luciana Celeste Sanchez Silva



**U202215979**

Mi nombre es Luciana Celeste Sanchez Silva, tengo 20 años y vivo en Lima. En la actualidad, me encuentro estudiando el 6to ciclo de la carrera de ingeniería de software en la UPC debido a que desde una edad temprana tuve una fascinación relacionada con el uso de la tecnología y la programación. En mi tiempo libre trato de crecer y expandir mi conocimiento en todas las áreas posibles. De igual forma, me gusta nadar, escuchar música y tocar la guitarra. Me comprometo a colaborar en todo momento con la elaboración de esta startup, y llegar a un trabajo sobresaliente. Mis habilidades son: responsabilidad, resolución de problemas, y disciplina.

---

Josep Eliu Melgarejo Quiroz



**u202315165**

Mi nombre es Josep Eliu Melgarejo Quiroz, tengo 21 años y mi lugar de nacimiento es Huaral pero vivo actualmente en Lima - San miguel, me encuentro cursando el 5to ciclo de la carrera de ingeniería de software en la UPC debido a que siempre me fascino el tema tecnologico, y como era un apasionado por lo juegos que luego me conllevaron a conocer el mundo de la programacion decidi estudiar mi carrera. Me comprometo a siempre apoyar y motivar a mis compañeros en hacer el mejor trabajo posible y dar el 100% de capacidad en este trabajo

---



Sebastian Martin Beingolea Montalvo

**U202217853**

Mi nombre es Sebastian Martin Beingolea Montalvo, tengo 21 años y vivo en Lima. Actualmente estudio la carrera de Ingeniería de Software en la UPC, motivado por el interés en crear soluciones tecnológicas que aporten valor real a las personas y a los negocios. Me gusta el desarrollo de software, el trabajo en equipo y aprender nuevas herramientas que puedan aplicar en proyectos como AniTec. En mi tiempo libre disfruto explorar tecnologías, escuchar música y seguir mejorando mis habilidades de programación. Me comprometo a colaborar de forma constante con mis compañeros, aportar ideas y contribuir a

un entregable de calidad. Mis habilidades son: responsabilidad, trabajo en equipo, capacidad de aprendizaje y orientación a resultados.

---

Saul Ortega Muñoz



**u20231c019**

Soy estudiante de Ingeniería de Software con formación en programación y desarrollo web. Tengo conocimientos en algunos lenguajes de programación. Me caracterizo por ser responsable, con capacidad de aprendizaje rápido y trabajo en equipo, aportando ideas prácticas para resolver problemas y mejorar proyectos tecnológicos.

---



Giuseppe Villanueva Rodriguez

**u20221c554**

Soy estudiante de Ingeniería de Software con habilidades en programación. Poseo conocimientos en algunos lenguajes de programación. Me caracterizo por ser empático y siempre atento cuando se trata de trabajo en equipo.

---

## 1.2. Solution Profile

---

### 1.2.1. Antecedentes y problemática

#### Qué (What)

*¿Cuál es la situación problemática?*

Los pequeños y medianos ganaderos suelen distribuir la información de sus animales entre cuadernos, hojas sueltas, archivos y mensajes. Esto dificulta mantener historiales completos, encontrar datos durante una atención, recordar actividades sanitarias y compartir información confiable con el veterinario. La aplicación web tomada como base centraliza parte de esos datos, pero no cubre por sí sola la necesidad de registrar y consultar información mientras el usuario se desplaza por el campo.

#### Cuándo (When)

*¿Cuándo ocurre el problema?*

El problema aparece durante todo el ciclo de manejo del animal: registro o nacimiento, vacunaciones, tratamientos, controles, reproducción, cambios productivos y venta. Se vuelve especialmente crítico cuando una actividad debe registrarse en el momento o cuando se necesita consultar el historial para decidir una atención.

#### Dónde (Where)

### *¿Dónde se manifiesta y dónde se origina?*

Se manifiesta en unidades ganaderas rurales y semiurbanas, durante recorridos de campo y atenciones veterinarias. En estos entornos puede haber acceso limitado a una computadora y conectividad móvil intermitente, por lo que depender exclusivamente de una aplicación web o de una conexión constante interrumpe el flujo de trabajo.

### **Quién (Who)**

#### *¿Quiénes participan y quiénes usarán la solución?*

Los usuarios principales serán pequeños y medianos ganaderos responsables del manejo cotidiano del hato y veterinarios de campo que necesitan consultar antecedentes y registrar atenciones. Asociaciones, cooperativas y entidades del sector son actores relacionados, pero no constituyen los segmentos de usuario priorizados para la primera versión.

### **Por qué (Why)**

#### *¿Cuál es la causa principal del problema?*

Las herramientas disponibles no siempre se ajustan a la movilidad del trabajo ganadero, al nivel de experiencia digital de los usuarios ni a las restricciones de conectividad. Como consecuencia, el registro se posterga, la información queda incompleta o dispersa y la coordinación entre ganadero y veterinario depende de comunicaciones informales.

### **Cómo (How)**

#### *¿Cómo se implementará la solución?*

AniTec extenderá el producto web existente con una aplicación nativa Android desarrollada con Kotlin y una aplicación multiplataforma cuya tecnología se elegirá mediante evaluación técnica. Las aplicaciones consumirán la API REST propia y permitirán registrar animales y eventos, consultar historiales, recibir notificaciones, utilizar la cámara, revisar indicadores y conservar temporalmente datos o borradores en el dispositivo.

#### *¿Cómo se logrará una experiencia útil en campo?*

Se diseñarán flujos breves, formularios con validación, estados claros de sincronización y alternativas manuales cuando una función del dispositivo no esté disponible. La solución considerará accesibilidad, internacionalización, seguridad de acceso y pruebas en dispositivos físicos bajo distintas condiciones de conectividad.

### **Cuánto (How much)**

#### *¿Cuál es la magnitud del contexto?*

El IV Censo Nacional Agropecuario registró 2 199 243 productores agropecuarios como personas naturales y señaló que el 79,6 % tenía unidades de menos de cinco hectáreas (INEI, 2014). Además, durante el primer trimestre de 2025 solo el 20,5 % de los hogares rurales disponía de Internet en el hogar (INEI, 2025a). Estas cifras no cuantifican por sí solas el mercado de AniTec, pero muestran la escala de la pequeña producción agropecuaria y la necesidad de diseñar para un entorno de conectividad restringida.

Descripción consolidada de los antecedentes y la problemática

AniTec parte de una aplicación web creada en un curso anterior. El reto actual consiste en adaptar y ampliar esa solución para el uso móvil, sin limitarse a reproducir sus pantallas. El teléfono debe convertirse en la herramienta de trabajo inmediata del ganadero y del veterinario: permitir el registro en el lugar donde ocurre el evento, presentar la información relevante con rapidez y tolerar interrupciones de red.

La propuesta comprenderá una landing page, una API REST propia, una aplicación Android nativa y una experiencia multiplataforma. El alcance inicial priorizará la gestión de animales e historiales sanitarios, el trabajo local y la posterior sincronización, las notificaciones, el uso de cámara, los indicadores de seguimiento y la colaboración autorizada con veterinarios. Las integraciones complementarias, como pagos o una capacidad autónoma basada en una tecnología nueva para el equipo, se incorporarán después de validar su viabilidad y su aporte al usuario.

El objetivo del proyecto es reducir la pérdida y fragmentación de información, facilitar el cumplimiento de actividades sanitarias y mejorar el acceso al historial del animal durante el trabajo de campo. La solución deberá mantener coherencia entre plataformas, proteger los datos, comunicar los estados de conexión y funcionar de manera comprensible para usuarios con distintos niveles de experiencia digital.

## 1.2.2. Lean UX Process.

### 1.2.2.1. Lean UX Problem Statements.

#### **Problem Statement:**

El estado actual de la gestión ganadera para pequeños y medianos productores se ha centrado principalmente en controles manuales, registros en cuadernos y herramientas digitales improvisadas para administrar la información sanitaria, reproductiva y económica del hato.

Lo que los productos y servicios existentes no abordan es la necesidad de contar con una aplicación para Android sencilla, accesible y adaptada al trabajo de campo, que permita centralizar la información del ganado y mantener el registro de las actividades principales incluso cuando la conexión a Internet sea inestable.

Nuestro producto, AniTec, abordará esta brecha mediante una aplicación móvil para Android que permitirá registrar, organizar y consultar la información del ganado desde el teléfono. La solución incluirá recordatorios sanitarios, historiales por animal, reportes y almacenamiento local de información esencial para reducir errores, evitar la pérdida de datos y facilitar la toma de decisiones.

Nuestro enfoque inicial será pequeños y medianos ganaderos que actualmente dependen de registros manuales o sistemas poco organizados para gestionar su producción, junto con veterinarios que necesitan consultar información sanitaria y dar seguimiento a los animales de sus clientes desde un dispositivo Android.

Sabremos que hemos tenido éxito cuando observemos una reducción en el uso de registros manuales, un aumento en la frecuencia y precisión de los registros realizados desde la aplicación, una mejora en el cumplimiento de vacunaciones y tratamientos, y una consulta más rápida de los historiales sanitarios por parte de ganaderos y veterinarios.

### 1.2.2.2. Lean UX Assumptions.

## Business Assumptions:

1. **Creemos que nuestros usuarios necesitan** un método confiable y eficiente para registrar y supervisar desde el teléfono la salud, productividad y trazabilidad de su ganado.
2. **Creemos que esta necesidad puede satisfacerse** mediante una aplicación para Android que permita registrar información clave, conservar datos esenciales en el dispositivo, generar alertas y consultar reportes útiles para la toma de decisiones.
3. **Creemos que nuestros primeros usuarios serán** pequeños y medianos ganaderos que utilizan teléfonos Android, así como veterinarios y técnicos agropecuarios que asesoran directamente en el campo.
4. **Creemos que lo más importante para los clientes es** contar con un control ordenado y disponible durante sus actividades, evitando pérdidas de información y mejorando la trazabilidad del ganado.
5. **Creemos que los usuarios también recibirán** alertas sanitarias mediante notificaciones, reportes económicos, acceso al historial de cada animal y contenido educativo dentro de la aplicación.
6. **Creemos que conseguiremos clientes mediante** alianzas con asociaciones ganaderas, programas de desarrollo rural, recomendaciones de veterinarios y campañas digitales dirigidas a regiones con actividad ganadera.
7. **Creemos que los ingresos se generarán mediante** un modelo de suscripción con planes ajustados al tamaño del hato y licencias institucionales para asociaciones y entidades del sector agropecuario.
8. **Creemos que nuestra competencia incluye** aplicaciones genéricas de gestión ganadera, hojas de cálculo y métodos tradicionales de registro manual.
9. **Creemos que nuestra ventaja competitiva radica en** ofrecer una aplicación Android adaptada al contexto rural, fácil de usar y preparada para conservar el progreso cuando la conectividad sea limitada.
10. **Creemos que un riesgo importante es** que algunos ganaderos no adopten fácilmente la aplicación por falta de experiencia digital, conectividad inestable o limitaciones de sus dispositivos.
11. **Creemos que lo mitigaremos mediante** una interfaz intuitiva, tutoriales paso a paso, formularios breves, almacenamiento local y mensajes claros sobre el estado de sincronización.

## User Assumptions:

- **Creemos que** los principales usuarios son pequeños y medianos ganaderos, veterinarios y técnicos agropecuarios que utilizan dispositivos Android durante sus actividades. En etapas posteriores, la aplicación también podría apoyar a asociaciones, cooperativas y entidades vinculadas con la sanidad y trazabilidad del sector.
- **Creemos que** AniTec ayuda a organizar la información del hato desde el teléfono, evitando la pérdida de datos importantes y la falta de seguimiento de vacunas, partos, tratamientos y movimientos. Para los veterinarios, facilita la consulta de clientes, pacientes e historiales sanitarios durante una atención.
- **Creemos que** los usuarios valoran el registro individual de cada animal, las notificaciones sobre actividades pendientes, los reportes simples, el historial del hato y la posibilidad de conservar registros cuando la conexión sea inestable. También creemos que la facilidad de uso es esencial para su adopción.
- **Creemos que** AniTec se integra en la rutina del ganadero porque puede utilizarse en el campo cada vez que se registra un animal, tratamiento, parto, vacunación o cambio productivo. Los veterinarios pueden utilizarla antes, durante y después de una visita para consultar antecedentes y registrar la atención.

- **Creemos que** AniTec debe ofrecer una experiencia Android intuitiva, legible y estable, pensada para usuarios con distintos niveles de experiencia tecnológica. También debe proteger los datos, comunicar los errores de conexión y permitir recuperar el trabajo pendiente.

#### Feature Assumptions:

##### Feature Assumption 01

**Creemos que** una funcionalidad de registro móvil centralizado permitirá a los pequeños y medianos ganaderos gestionar la información sanitaria, reproductiva y económica de sus animales desde un dispositivo Android.

**Sabremos que esta funcionalidad es valiosa cuando** la mayoría de los usuarios registre y actualice periódicamente la información de sus animales mediante la aplicación.

##### Feature Assumption 02

**Creemos que** un sistema de notificaciones para vacunaciones, tratamientos y eventos reproductivos ayudará a los ganaderos a recordar actividades importantes y reducir los descuidos en el manejo del hato.

**Sabremos que esta funcionalidad es valiosa cuando** los usuarios atiendan o reprogramen los recordatorios recibidos en sus dispositivos Android.

##### Feature Assumption 03

**Creemos que** un módulo de reportes visuales e historial de cada animal facilitará la interpretación de la información y permitirá tomar mejores decisiones productivas, reproductivas y económicas.

**Sabremos que esta funcionalidad es valiosa cuando** los usuarios consulten los reportes y antecedentes antes de realizar acciones relacionadas con la gestión del ganado.

##### Feature Assumption 04

**Creemos que** el almacenamiento local de información esencial y borradores permitirá que los usuarios continúen registrando actividades cuando la conexión a Internet sea inestable.

**Sabremos que esta funcionalidad es valiosa cuando** los usuarios puedan recuperar y sincronizar sus registros sin repetir la información ingresada.

##### Feature Assumption 05

**Creemos que** un módulo para veterinarios con acceso autorizado a clientes, historiales y eventos sanitarios facilitará el seguimiento de los animales y mejorará la colaboración con los ganaderos.

**Sabremos que esta funcionalidad es valiosa cuando** los veterinarios utilicen la aplicación Android para registrar y consultar la información sanitaria de los animales atendidos.

#### 1.2.2.3. Lean UX Hypothesis Statements.

- **Hypothesis Statement 01:**

**Creemos que lograremos** una mayor adopción y un uso continuo de AniTec

**Si** los pequeños y medianos ganaderos

**Obtienen** una forma sencilla de registrar y consultar la información sanitaria, reproductiva y económica de sus animales desde el teléfono

**Con** una aplicación para Android que centralice y organice la información del hato.

- **Hypothesis Statement 02:**

**Creemos que lograremos** mejorar la gestión sanitaria del hato y reducir actividades olvidadas

**Si** los pequeños y medianos ganaderos

**Obtienen** recordatorios oportunos sobre vacunaciones, tratamientos y eventos reproductivos

**Con** un sistema de alertas y notificaciones integrado en la aplicación Android.

- **Hypothesis Statement 03:**

**Creemos que lograremos** una mejor toma de decisiones sobre el manejo del ganado

**Si** los pequeños y medianos ganaderos

**Obtienen** acceso desde el teléfono a reportes visuales y al historial de cada animal

**Con** un módulo móvil de reportes e historiales organizado y fácil de interpretar.

- **Hypothesis Statement 04:**

**Creemos que lograremos** reducir la pérdida de información causada por problemas de conectividad

**Si** los pequeños y medianos ganaderos y veterinarios

**Obtienen** la posibilidad de conservar el progreso cuando no dispongan de una conexión estable

**Con** almacenamiento local y sincronización posterior dentro de la aplicación Android.

- **Hypothesis Statement 05:**

**Creemos que lograremos** una mejor colaboración entre ganaderos y veterinarios y un seguimiento sanitario más eficiente

**Si** los veterinarios

**Obtienen** acceso autorizado a los clientes asignados, al historial de los animales y al registro de eventos sanitarios

**Con** un módulo para veterinarios integrado en la aplicación Android de AniTec.

#### 1.2.2.4. Lean UX Canvas.

El Lean UX Canvas es una herramienta utilizada en el marco del diseño centrado en el usuario (UX) y la metodología Lean, cuyo objetivo es apoyar la creación y mejora de productos de manera ágil y eficiente. Su propósito principal es proporcionar una estructura organizada que fomente la colaboración entre equipos multidisciplinarios. A continuación, se presenta el Lean UX Canvas elaborado por el equipo utilizando la plataforma digital Mural.

Lean UX Canvas

Title: AniTec

Date: 21/04/2025

Iteration:

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <div><div>Business Problem</div><div>What business have you identified that needs help?</div><div><div>Pequeños y medianos ganaderos no gestionan de manera eficiente la información de su ganado.</div><div>La mayoría usa métodos manuales o improvisados (cuadernos, hojas de cálculo), lo que genera errores y pérdida de datos.</div><div>Falta de trazabilidad y control impide decisiones informadas y limita el acceso a mejores mercados.</div></div></div>                                                                                                                                                                                    | <div><div>Solution ideas</div><div>List product, feature, or enhancement ideas that help your target audience achieve the benefits they're seeking.</div><div><div>Plataforma web y móvil para registrar datos de cada animal (salud, reproducción, productividad, ingresos y egresos).</div><div>Alertas automáticas sobre vacunaciones, tratamientos y eventos reproductivos.</div><div>Reportes visuales y descargables que faciliten decisiones estratégicas.</div></div></div> | <div><div>Business Outcomes</div><div>(Changes in customer behavior)</div><div>What changes in customer behavior will indicate you have solved a real problem in a way that adds value to your customers?</div><div><div>Mejora en la trazabilidad y organización del hato.</div><div>Reducción de pérdidas por descuidos sanitarios o errores de registro.</div><div>Aumento en la rentabilidad y eficiencia de pequeños y medianos ganaderos.</div></div></div>                    |
| <div><div>Users &amp; Customers</div><div>What types of users and customers should you focus on first?</div><div><div>Pequeños y medianos ganaderos.</div><div>Técnicos agropecuarios que brindan asistencia en campo.</div><div>Asociaciones, cooperativas y entidades públicas en etapas posteriores.</div></div></div>                                                                                                                                                                                                                                                                                                                               |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | <div><div>User Benefits</div><div>What are the goals your users are trying to achieve? What is motivating them to seek out your solution? (e.g., do better at my job OR get a promotion)</div><div><div>Control eficiente y organizado de su ganado.</div><div>Alertas y recordatorios que evitan errores y pérdidas.</div><div>Acceso a historial completo de cada animal y reportes visuales.</div><div>Capacitación continua mediante la Academia Ganadera.</div></div></div>     |
| <div><div>Solution ideas</div><div>Combine the assumptions from 2, 3, 4 &amp; 5 into the following template hypothesis statement: "We believe that [business outcome] will be achieved if [user] attains [benefit] with [feature]."</div><div>Each hypothesis should focus on one feature.</div><div><div>Creemos que la adopción de GanTrace permitirá a los ganaderos registrar digitalmente la información de su ganado.</div><div>Creemos que las alertas automáticas reducirán pérdidas y descuidos sanitarios.</div><div>Creemos que los reportes visuales facilitarán decisiones sobre reproducción, venta y manejo económico.</div></div></div> | <div><div>What's the most important thing we need to learn first?</div><div>List product, feature, or enhancement ideas that help your target audience achieve the benefits they're seeking.</div><div><div>Que los ganaderos puedan usar la plataforma sin problemas desde celular o computadora.</div><div>Que las alertas sean efectivas y los usuarios las sigan.</div><div>Que los reportes realmente ayuden a tomar decisiones más rápidas y precisas.</div></div></div>      | <div><div>What's the least amount of work we need to do to learn the next most important thing?</div><div>Brainstorm the types of experiments you can run to learn whether your riskiest assumption is true or false.</div><div><div>Crear un prototipo básico funcional con registro de animales y alertas.</div><div>Realizar pruebas piloto con 5-10 ganaderos</div><div>Recoger feedback sobre facilidad de uso, efectividad de alertas y utilidad de reportes</div></div></div> |

Download this canvas at: [www.jeffgothelf.com/blog/leanuxcanvas](http://www.jeffgothelf.com/blog/leanuxcanvas)

Adapted from Jeff Patton's Opportunity Canvas. Download at: <http://jeffpattonassociates.com/opportunity-canvas/>

Enlace para acceder al Lean UX Canvas en Mural: <https://tinyurl.com/LeanUxCanvasMural>

### 1.3. Segmentos objetivo

AniTec prioriza dos segmentos que participan directamente en el registro y uso de la información ganadera: pequeños y medianos ganaderos y veterinarios de campo. Esta delimitación permite diseñar los flujos móviles alrededor de tareas concretas y evita considerar como usuarios principales a organizaciones que solo cumplen un rol de apoyo o influencia.

El IV Censo Nacional Agropecuario registró 2 199 243 productores agropecuarios como personas naturales y encontró que el 79,6 % tenía unidades de menos de cinco hectáreas (INEI, 2014). Aunque estas cifras incluyen distintas actividades agropecuarias y no representan por sí solas el mercado de AniTec, evidencian la relevancia de la pequeña producción. El diseño también debe considerar la brecha de conectividad: en el primer trimestre de 2025, el 20,5 % de los hogares rurales disponía de Internet en el hogar (INEI, 2025a). A la vez, el teléfono móvil es el principal medio de acceso: en el cuarto trimestre de 2025, el 89,2 % de las personas usuarias de Internet se conectó mediante un celular y, en el área rural, la proporción fue de 88,4 % (INEI, 2026). Estas condiciones respaldan una experiencia centrada en el teléfono, con persistencia local y tolerancia a conexiones intermitentes.

## Pequeños y medianos ganaderos

Son productores responsables del manejo cotidiano de un hato pequeño o mediano en zonas rurales o semiurbanas. Registran nacimientos, vacunaciones, tratamientos, reproducción, producción y movimientos de animales. Cuando esta información se conserva en cuadernos, mensajes o archivos separados, necesitan invertir más tiempo para encontrarla y corren el riesgo de perder continuidad en el seguimiento.

Las entrevistas exploratorias disponibles comprenden tres ganaderos de entre 54 y 65 años que utilizan teléfonos Android y aplicaciones de comunicación como WhatsApp. Esta muestra ayuda a formular supuestos iniciales, pero no permite generalizar sus resultados al conjunto del segmento; deberá ampliarse y contrastarse durante la validación.

| Característica            | Descripción                                                                                                                           |
|---------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| Contexto de uso           | Campo, establo o vivienda, con acceso variable a Internet y uso frecuente del teléfono.                                               |
| Necesidades principales   | Registrar animales y eventos en el momento, consultar historiales, recordar actividades y revisar el estado general del hato.         |
| Dificultades actuales     | Información dispersa, pérdida u omisión de datos, demora al buscar antecedentes y conectividad inestable.                             |
| Valor esperado            | Menos tiempo de registro y búsqueda, mayor continuidad sanitaria y decisiones sustentadas en información organizada.                  |
| Consideraciones de diseño | Flujos cortos, lenguaje simple, controles legibles, ayuda contextual, almacenamiento local y estados de sincronización comprensibles. |

## Veterinarios de campo

Son profesionales que atienden animales de distintos productores y necesitan revisar antecedentes, registrar diagnósticos, tratamientos y recomendaciones, y dar seguimiento a los casos. Requieren acceso únicamente a los clientes y animales que les hayan concedido autorización, además de una forma clara de comunicar lo registrado al ganadero.

Las entrevistas exploratorias disponibles comprenden dos veterinarios de entre 24 y 27 años. Sus respuestas orientan los primeros supuestos sobre movilidad y consulta de historiales, pero la muestra aún no satisface el mínimo de tres a cinco entrevistas exigido para cada segmento. Por ello, será necesario entrevistar al menos a un veterinario adicional y actualizar el análisis antes de cerrar la validación del problema.

| Característica          | Descripción                                                                                                                  |
|-------------------------|------------------------------------------------------------------------------------------------------------------------------|
| Contexto de uso         | Atención en campo, desplazamiento entre clientes y consulta desde el teléfono antes, durante o después de una visita.        |
| Necesidades principales | Acceder con autorización al historial del animal, registrar atenciones y revisar pendientes de seguimiento.                  |
| Dificultades actuales   | Antecedentes incompletos, información repartida entre conversaciones y documentos, y demora para coordinar con el productor. |

| Característica            | Descripción                                                                                                                                          |
|---------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| Valor esperado            | Consulta rápida de antecedentes, registros sanitarios consistentes y mejor continuidad entre visitas.                                                |
| Consideraciones de diseño | Permisos explícitos, identificación clara del cliente y del animal, formularios breves, trazabilidad de cambios y funcionamiento ante fallas de red. |

Las asociaciones ganaderas, cooperativas, técnicos agropecuarios y entidades públicas son stakeholders externos que pueden facilitar adopción, capacitación o articulación. El equipo de desarrollo es un stakeholder interno. Estos actores se documentarán en el análisis correspondiente, pero no sustituyen a los dos segmentos objetivo priorizados.

# Capítulo II: Requirements Development and Software Solution Design

## 2.1. Competidores

El análisis competitivo permite conocer cómo otras soluciones digitales atienden la gestión ganadera y qué aspectos debe validar AniTec para diferenciarse. Se seleccionaron tres competidores directos con productos móviles vigentes: Livestock Manager, AgriWebb e iLivestock. La comparación se concentra en las capacidades relevantes para la aplicación Android de AniTec y utiliza información publicada en los sitios oficiales de cada producto, consultada el 13 de septiembre de 2026.

### 2.1.1. Análisis competitivo

#### Pregunta que orienta el análisis

¿Cómo puede AniTec diferenciar su aplicación Android frente a otras soluciones móviles de gestión ganadera en facilidad de uso, continuidad ante conexiones inestables, seguimiento veterinario y adaptación a pequeños y medianos ganaderos peruanos?

#### Competitive Analysis Landscape

| Aspecto       | AniTec                                                                                                                                                                                                                                                                                                    | Livestock Manager                                                                                                                                                                                                                         | AgriWebb                                                                                                                                                                                                                            | iLivestock                                                                                                                                                                                                     |
|---------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|               |                                                                                                                                                                                                                        |                                                                                                                                                        |                                                                                                                                                 |                                                                                                                           |
| Perfil        |                                                                                                                                                                                                                                                                                                           |                                                                                                                                                                                                                                           |                                                                                                                                                                                                                                     |                                                                                                                                                                                                                |
| Clasificación | Startup analizada.                                                                                                                                                                                                                                                                                        | Competidor directo.                                                                                                                                                                                                                       | Competidor directo.                                                                                                                                                                                                                 | Competidor directo.                                                                                                                                                                                            |
| Overview      | Solución de gestión ganadera orientada a pequeños y medianos ganaderos y veterinarios de campo. Parte de una landing page, una API REST y una aplicación web desarrolladas previamente. En el curso se desarrollará una aplicación Android nativa y se definirá la experiencia multiplataforma requerida. | Producto de Pila para gestionar animales y lotes de distintas especies desde el teléfono y la web. Integra registros de salud, alimentación, reproducción y finanzas. Su sitio declara más de 50 000 descargas, 24 especies y 15 idiomas. | Plataforma de gestión ganadera que trabaja con animales individuales o grupos. Permite registrar actividades, movimientos, tratamientos, pesos, alimentación, inventarios y tareas desde aplicaciones móviles y una plataforma web. | Plataforma para productores de ganado bovino y ovino. Reúne registros de salud, reproducción, movimientos, pesos, tratamientos y rendimiento, y se integra con equipos de identificación electrónica y pesaje. |

| Aspecto                                     | <b>AniTec</b><br>                                                                                                                            | <b>Livestock Manager</b><br>                       | <b>AgriWebb</b><br>                                                      | <b>iLivestock</b><br>                                                   |
|---------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Ventaja competitiva</b>                  | Propuesta de experiencia simple en español, adaptada al contexto de pequeños productores peruanos y con un flujo de colaboración autorizado entre ganadero y veterinario. Esta ventaja todavía debe comprobarse con usuarios. | Barrera de entrada baja mediante un plan gratuito, cobertura de múltiples especies y disponibilidad tanto móvil como web.           | Amplitud funcional para operaciones ganaderas, mapas interactivos, trabajo sin conexión e integración con lectores EID, clima y dispositivos IoT.          | Integración de software y hardware para identificación EID y pesaje, funcionamiento sin conexión y herramientas especializadas para ganado bovino y ovino. |
| <b>Valor que ofrece a los clientes</b>      | Centralizar animales e historiales, registrar actividades desde el campo, conservar datos esenciales ante interrupciones de red, recibir recordatorios y facilitar el seguimiento sanitario compartido.                       | Reunir registros productivos y económicos de distintas especies en una herramienta económica y accesible desde varios dispositivos. | Registrar las operaciones del campo en el momento, mantener inventarios y movimientos actualizados y sincronizar los datos cuando se recupera la conexión. | Reducir el trabajo manual de identificación y pesaje, mantener la trazabilidad y convertir los registros del ganado en reportes de desempeño.              |
| <b>Perfil de Marketing</b>                  |                                                                                                                                                                                                                               |                                                                                                                                     |                                                                                                                                                            |                                                                                                                                                            |
| <b>Mercado objetivo</b>                     | Pequeños y medianos ganaderos peruanos y veterinarios de campo que emplean teléfonos Android y gestionan información dispersa.                                                                                                | Desde propietarios de granjas pequeñas y aficionados hasta productores profesionales y organizaciones con múltiples granjas.        | Productores y equipos ganaderos que administran bovinos u ovinos de forma individual o por grupos y requieren coordinación operativa.                      | Productores de bovinos y ovinos, desde hatos pequeños hasta operaciones comerciales que utilizan registros, identificación EID o sistemas de pesaje.       |
| <b>Estrategias de marketing observables</b> | Propuesta preliminar basada en la landing page, demostraciones con usuarios, recomendaciones de veterinarios y alianzas con                                                                                                   | Modelo freemium sin tarjeta para iniciar, distribución en tiendas de aplicaciones, prueba inmediata del                             | Prueba gratuita, demostración interactiva, contenido educativo y contacto con ventas para operaciones que                                                  | Prueba gratuita de 14 días, presencia en tiendas de aplicaciones, distribuidores regionales y venta                                                        |

| Aspecto                        | AniTec                                                                                                                                                                                                                                                                             | Livestock Manager                                                                                                                                                                                                  | AgriWebb                                                                                                                                                                                      | iLivestock                                                                                                                                                                     |
|--------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                |                                                                                                                                                                                                   |                                                                                                                                   |                                                                                                             |                                                                                             |
|                                | asociaciones ganaderas. Todavía no existe evidencia de adquisición o retención.                                                                                                                                                                                                    | producto y uso de testimonios y cifras de descargas como prueba social.                                                                                                                                            | requieren configuración o integraciones.                                                                                                                                                      | combinada de software, lectores, balanzas y etiquetas.                                                                                                                         |
|                                | <b>Perfil de Producto</b>                                                                                                                                                                                                                                                          |                                                                                                                                                                                                                    |                                                                                                                                                                                               |                                                                                                                                                                                |
| <b>Productos y servicios</b>   | Landing page y API REST existentes; aplicación Android nativa en desarrollo. El alcance móvil prioriza animales, historiales sanitarios, notificaciones, almacenamiento local, reportes y acceso veterinario autorizado. La solución multiplataforma se definirá durante el curso. | Registro de animales y lotes, salud, medicamentos, alimentación, reproducción, finanzas y reportes. Los planes superiores incorporan colaboración, facturación, certificados, integraciones y acceso mediante API. | Gestión individual o por grupos, mapas, movimientos, tratamientos, pesos, alimentación, reproducción, inventarios, tareas, reportes e integraciones con hardware y servicios externos.        | Registros de bovinos y ovinos, grupos, tratamientos, medicamentos, movimientos, reproducción, pesos, rendimiento, reportes e integración con lectores EID y equipos de pesaje. |
| <b>Precios y costos</b>        | Precio aún no definido. El modelo de suscripción y la disposición de pago son hipótesis que deben validarse. Las cifras heredadas de la landing page y del backend no se consideran precios comerciales aprobados.                                                                 | Starter: gratuito. Growth: desde USD 3,99 y Scale: desde USD 9,99, mostrados como importes mensuales con facturación anual. Enterprise: cotización personalizada.                                                  | Precio calculado según la cantidad de animales y la región. El sitio solicita ingresar el número de bovinos u ovinos o contactar con ventas; también ofrece complementos con costo adicional. | Software: GBP 27,50 mensuales o GBP 300 anuales. Ofrece una prueba gratuita de 14 días; el hardware y las etiquetas se comercializan por separado.                             |
| <b>Canales de distribución</b> | Landing page web y, cuando se implemente y publique, Google Play para la aplicación Android. El canal de la aplicación multiplataforma permanece por definir.                                                                                                                      | Aplicaciones para Android e iOS y acceso web o de escritorio.                                                                                                                                                      | Aplicaciones para Android e iOS y aplicación web.                                                                                                                                             | Aplicaciones para Android e iOS y acceso desde smartphone, tableta, PC o Mac.                                                                                                  |

Los precios corresponden a la información visible en los sitios oficiales en la fecha de consulta y pueden cambiar según país, cantidad de animales, modalidad de facturación o configuración contratada. En el caso de AniTec, cualquier precio deberá presentarse como una propuesta hasta que sea validado con los segmentos objetivo y exista una decisión comercial consistente.

## Análisis FODA

El siguiente análisis interpreta la oferta observable de cada producto en relación con el objetivo de AniTec. Las fortalezas y debilidades describen condiciones internas de cada solución; las oportunidades y amenazas consideran su posición frente al mercado y los demás competidores.

| Dimensión            | AniTec                                                                                                                                                                                                 | Livestock Manager                                                                                                                                                                   | AgriWebb                                                                                                                                                                              | iLivestock                                                                                                                                                                       |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Fortalezas</b>    | Conocimiento inicial de los segmentos locales; propuesta en español; reutilización de una API y producto web existentes; flujo diferenciado entre ganadero y veterinario.                              | Plan gratuito; precio inicial bajo; soporte para numerosas especies e idiomas; disponibilidad móvil y web; base pública de descargas.                                               | Trabajo online y offline; gestión ganadera amplia; sincronización; mapas; roles de equipo e integraciones con EID e IoT.                                                              | Especialización en bovinos y ovinos; trabajo offline; integración propia con lectores y pesaje; trazabilidad y reportes de rendimiento.                                          |
| <b>Debilidades</b>   | Aplicación móvil aún no implementada; propuesta de precio sin validar; ausencia de usuarios activos demostrables; reconocimiento de marca limitado y capacidades offline aún por comprobar.            | Su cobertura de muchas especies puede ofrecer menos profundidad para un flujo veterinario especializado; la propuesta pública no destaca adaptación específica al contexto peruano. | La amplitud de funciones e integraciones puede elevar el esfuerzo de configuración y aprendizaje; el precio final no es visible sin introducir datos del hato o contactar con ventas. | Parte importante de su diferenciación depende del ecosistema de hardware y EID; el precio se publica en libras esterlinas y varias integraciones regulatorias dependen del país. |
| <b>Oportunidades</b> | Atender a pequeños productores peruanos con lenguaje, soporte y flujos validados localmente; simplificar la colaboración con veterinarios; diseñar para teléfonos Android y conectividad intermitente. | Convertir usuarios del plan gratuito a funciones avanzadas y ampliar sus capacidades de colaboración y automatización.                                                              | Extender sus integraciones, análisis de rendimiento y servicios para operaciones ganaderas de mayor complejidad.                                                                      | Expandir su red de distribuidores y alianzas en Latinoamérica, y aumentar la adopción de identificación y pesaje digital.                                                        |
| <b>Amenazas</b>      | Competidores consolidados ya                                                                                                                                                                           | Soluciones especializadas por                                                                                                                                                       | Aplicaciones más simples y económicas                                                                                                                                                 | Alternativas que funcionan sin                                                                                                                                                   |

| Dimensión | AniTec                                                                                                                                                                  | Livestock Manager                                                                                                                     | AgriWebb                                                                                      | iLivestock                                                                                                  |
|-----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
|           | ofrecen trabajo offline, aplicaciones Android y precios de entrada bajos; resistencia a la adopción, problemas de conectividad y desconfianza sobre el manejo de datos. | especie o región pueden ofrecer flujos más profundos; plataformas integrales pueden competir mediante mapas, hardware y cumplimiento. | pueden resultar suficientes para productores pequeños que no necesitan todas sus capacidades. | hardware adicional o con planes gratuitos pueden reducir la adopción entre productores sensibles al precio. |

El Landscape muestra que el funcionamiento sin conexión ya está presente en AgriWebb e iLivestock, mientras que Livestock Manager compite con un plan gratuito y precios de entrada bajos. Por ello, AniTec no debe presentar el almacenamiento local ni la accesibilidad económica como ventajas únicas sin validación. Su diferenciación deberá comprobarse principalmente en la adaptación al contexto peruano, la simplicidad de los flujos Android y la colaboración autorizada entre ganadero y veterinario.

Fuentes del análisis competitivo

- [Livestock Manager: producto, funciones y precios.](#)
- [AgriWebb: aplicación móvil y funcionamiento offline.](#)
- [AgriWebb: planes e integraciones.](#)
- [iLivestock: software, funciones y precio.](#)
- [iLivestock: presencia y alianzas en Latinoamérica.](#)

2.1.2. Estrategias y tácticas frente a competidores

Las siguientes estrategias son preliminares y se derivan de los hallazgos del Landscape y del análisis FODA. Sus indicadores permitirán comprobar si las tácticas producen el resultado esperado.

| Hallazgo competitivo                                                                 | Estrategia de AniTec                                                                          | Táctica preliminar                                                                                                                                        | Indicador de validación                                                                                                 |
|--------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| AgriWebb e iLivestock ya permiten trabajar sin conexión.                             | Alcanzar una continuidad de uso comparable y ofrecer estados de sincronización comprensibles. | Permitir guardar datos esenciales y borradores localmente, mostrar si existen cambios pendientes y reintentar la sincronización al recuperar la conexión. | Porcentaje de registros recuperados y sincronizados sin duplicación; cantidad de tareas abandonadas por errores de red. |
| Livestock Manager reduce la barrera de entrada con un plan gratuito y precios bajos. | Definir una propuesta de planes ajustada al valor percibido por pequeños productores.         | Probar con los segmentos una versión gratuita o de evaluación, límites por cantidad de animales y funciones pagadas antes de fijar precios comerciales.   | Comprensión de los planes, disposición de pago y porcentaje de participantes que elegiría cada alternativa.             |

| Hallazgo competitivo                                                                                                                                  | Estrategia de AniTec                                                                                         | Táctica preliminar                                                                                                                                                                 | Indicador de validación                                                                                      |
|-------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|
| Los competidores ofrecen productos amplios para mercados internacionales.                                                                             | Adaptar AniTec al contexto de los ganaderos peruanos.                                                        | Validar con usuarios el lenguaje, los formularios, las categorías sanitarias, el proceso de incorporación y las condiciones reales de uso en teléfonos Android.                    | Porcentaje de tareas completadas sin ayuda, tiempo por tarea y problemas de comprensión encontrados.         |
| Las ofertas revisadas gestionan equipos o usuarios, pero no presentan como eje principal una relación ganadero-veterinario adaptada al mercado local. | Diferenciar AniTec mediante seguimiento sanitario autorizado.                                                | Implementar permisos explícitos para asignar veterinarios, consultar historiales y registrar atenciones, mostrando al ganadero quién puede acceder a cada animal.                  | Porcentaje de asignaciones completadas, atenciones registradas y percepción de confianza sobre los permisos. |
| AniTec aún no tiene la trayectoria ni la base pública de usuarios de sus competidores.                                                                | Construir confianza mediante acompañamiento y evidencia de uso.                                              | Realizar demostraciones con ganaderos y veterinarios, ofrecer tutorial inicial y soporte por canales conocidos, y publicar únicamente resultados obtenidos en validaciones reales. | Activación del primer animal, retención semanal, solicitudes de soporte y recomendaciones obtenidas.         |
| AgriWebb e iLivestock cuentan con integraciones avanzadas de hardware y EID.                                                                          | Priorizar primero los problemas centrales del segmento y evaluar integraciones mediante aprendizaje técnico. | Desarrollar una Spike Story para la capacidad autónoma elegida y probar su utilidad antes de convertirla en una función permanente del producto.                                   | Viabilidad técnica documentada, tasa de éxito en pruebas y beneficio percibido por los usuarios.             |

Las campañas de referidos, alianzas con asociaciones, participación en ferias y acercamiento a veterinarios podrán utilizarse como tácticas de adquisición cuando exista una versión Android demostrable. Cada actividad deberá registrar participantes, usuarios activados y continuidad de uso; la asistencia a una actividad, por sí sola, no demostrará adopción del producto.

## 2.2. Entrevistas.

Las entrevistas son una herramienta esencial para comprender a fondo a nuestro público objetivo. Para que sean efectivas, deben seguir una estructura clara y directa, utilizando preguntas específicas que permitan recolectar información de valor y datos precisos de los participantes.

### 2.2.1. Diseño de entrevistas.

Objetivo: Identificar frustraciones, necesidades, dispositivos disponibles, grado de digitalización y percepción sobre el registro de información ganadera.

### 2.2.1. Diseño de entrevistas.

Segmentos entrevistados:

- Ganaderos
- Veterinarios

Formato: Entrevistas semiestructuradas, de 25-30 minutos, registradas en video con consentimiento.

Preguntas dirigidas al personal de **Ganaderos**.

Preguntas principales:

- ¿Podría indicarnos su nombre completo y su edad?
- ¿Cuánto tiempo lleva dedicado a la ganadería? ¿Qué tipo de ganado maneja actualmente?
- ¿Cuál es el tamaño aproximado de su ganado? ¿Y cuántas personas trabajan en su unidad ganadera?
- ¿Qué herramientas utiliza actualmente para llevar el control de sus animales y sus actividades?
- ¿Lleva algún registro sobre la salud, alimentación o reproducción de su ganado? ¿Cómo lo hace?
- ¿Cuáles son las principales dificultades que enfrenta en la gestión diaria del ganado?
- ¿Cómo monitorea actualmente la productividad y salud de su ganado?
- ¿Qué tan importante considera llevar un control digital del historial veterinario y productivo de cada animal?
- ¿Ha enfrentado problemas por no tener registros claros (por ejemplo, en ventas, enfermedades o reproducción)?
- ¿Confía en herramientas digitales o ha probado alguna aplicación para el manejo ganadero?
- ¿Cuánto tiempo promedio dedica al registro manual de datos (si lo realiza)?
- ¿Qué tipo de información considera más importante tener a la mano sobre su ganado?
- ¿Estaría dispuesto a usar una aplicación móvil/web para llevar el control del ganado si fuera sencilla y funcional?
- ¿Qué funcionalidades le gustaría que tenga esta herramienta (alertas, historial médico, reproductivo, reportes, etc.)?
- ¿Qué beneficios espera al adoptar una herramienta digital para su ganadería?

Preguntas dirigidas a los **Veterinarios**

Preguntas principales:

- ¿Podría proporcionarnos su nombre completo y su edad?
- ¿Cuánto tiempo lleva ejerciendo como veterinario y en qué región trabaja principalmente?
- ¿Está especializado en atención ganadera? ¿Qué tipo de ganado atiende con más frecuencia?
- ¿Cómo realiza el seguimiento del historial médico de los animales que atiende?
- ¿Utiliza actualmente alguna herramienta digital para llevar registros veterinarios?
- ¿Qué información considera fundamental registrar tras una consulta o intervención (vacunas, tratamientos, diagnóstico)?
- ¿Cómo se comunica con los ganaderos respecto al seguimiento o tratamientos posteriores?
- ¿Con qué frecuencia atiende emergencias ganaderas? ¿Cómo coordina este tipo de intervenciones?
- ¿Ha tenido casos donde la falta de información del animal haya afectado la efectividad del tratamiento?
- ¿Qué retos encuentra en su trabajo relacionado con el registro o gestión de información?
- ¿Le resultaría útil tener acceso al historial médico del animal antes de una consulta?
- ¿Qué tan dispuesto estaría a utilizar una aplicación móvil/web para registrar y acceder al historial de sus pacientes?
- ¿Qué funcionalidades considera clave en una herramienta digital veterinaria (calendario, historial, recordatorios, fichas clínicas)?
- ¿Cómo podría mejorar su trabajo con una solución que conecte a veterinarios con ganaderos en tiempo real?
- ¿Qué tan importante considera el análisis de datos (estadísticas de salud, tratamientos más comunes, etc.) en su labor?

#### Preguntas complementarias (para ambos segmentos):

- ¿Qué expectativas tendría sobre una plataforma digital que centralice la información ganadera y veterinaria?
- ¿Qué dispositivos usa con más frecuencia para sus actividades laborales (celular, laptop, tablet)? ¿Está familiarizado con el uso de apps?
- ¿Qué es lo que más valora en una herramienta digital: rapidez, facilidad de uso, seguridad de datos u otro aspecto?

#### Preguntas principales (comunes):

1. ¿Cómo lleva actualmente el registro de su ganado (peso, salud, vacunas)?
2. ¿Qué desafíos ha enfrentado por llevar registros manuales?
3. ¿Qué tan cómodo se siente utilizando un celular o computadora?

4. ¿Le sería útil recibir alertas de vacunación o reproducción?
5. ¿Ha perdido información relevante alguna vez?
6. ¿Qué contenido educativo le interesaría tener en una app?
7. ¿Qué canales digitales usa actualmente (WhatsApp, redes sociales, etc.)?

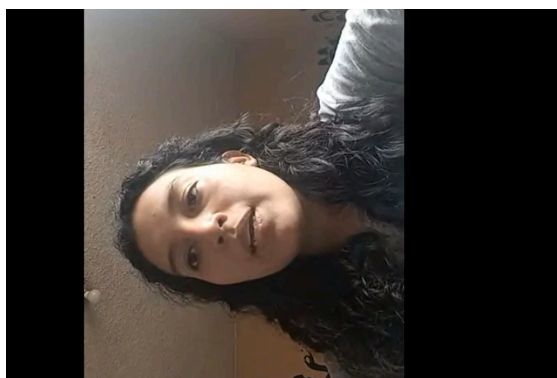
Variables demográficas a recolectar: Edad, género, distrito de residencia, educación, tipo de hacienda, frecuencia de registros, ocupación alterna, herramientas digitales que maneja, tipo de celular, acceso a internet, objetivos personales, frustraciones, marcas preferidas, influencia de técnicos o asociaciones.

## 2.2.2. Registro de entrevistas.

Entrevistas al segmento de ganaderos

### Entrevista 1: Caterina Rodriguez

| Campo                        | Información                                                                       |
|------------------------------|-----------------------------------------------------------------------------------|
| Segmento                     | Ganadero                                                                          |
| Nombres y apellidos          | Caterina Rodriguez                                                                |
| Edad                         | 33 años                                                                           |
| Distrito                     | San miguel, Lima                                                                  |
| Ocupación                    | Ganadera con más de 9 años de experiencia                                         |
| Tipo de ganado               | Ganado porcino                                                                    |
| Tamaño aproximado del ganado | 16 cabezas de ganado                                                              |
| Inicio de la entrevista      | 00:00                                                                             |
| Duración                     | 00:07:54                                                                          |
| URL del video                | <a href="https://tinyurl.com/caterinarodri">https://tinyurl.com/caterinarodri</a> |



### Resumen de la entrevista:

Caterina Rodríguez es una ganadera de 33 años con aproximadamente ocho años de experiencia en el manejo, principalmente, de ganado vacuno. Actualmente gestiona alrededor de 45 animales junto a un equipo de tres

personas (dos familiares y un cuidador). Durante la entrevista, explicó que su gestión de datos se realiza de manera fragmentada: utiliza un cuaderno físico, un cuadro de Excel y su teléfono celular (tomando fotos y notas) para registrar información sobre salud, alimentación y reproducción. Esta forma de trabajo, a la que dedica entre 20 y 30 minutos diarios, le genera dificultades para mantener la información organizada. La entrevistada señaló que los datos suelen traspapelarse o perderse al estar en distintos formatos, o resultar incomprensibles por la mala caligrafía, lo que complica el seguimiento preciso de vacunas y tratamientos.

En cuanto a su perfil tecnológico y forma de trabajo, Caterina se muestra como una persona consciente de la necesidad de optimizar su tiempo y mejorar la toma de decisiones en su ganadería. Valora tener a la mano información vital de cada animal, como edad, peso, historial de enfermedades y tratamientos. Respecto a la tecnología, confía en las herramientas digitales, aunque hasta ahora no ha utilizado ninguna aplicación especializada en el manejo ganadero. Su dispositivo principal es el celular, siendo WhatsApp la aplicación que más utiliza para registrar información y comunicarse, además de usar redes sociales como Facebook y TikTok. El uso de la computadora es mínimo y lo reserva casi exclusivamente para revisar Excel.

La entrevista permitió identificar que Caterina está totalmente dispuesta a utilizar una aplicación móvil o web para el control de su ganado. Destacó que la herramienta ideal debe centralizar la información (usando códigos por animal), contar con un historial médico y reproductivo, y emitir alertas de vacunación y atención. Un punto crucial para ella es que la plataforma sea muy sencilla e intuitiva de usar, ya que sus familiares, que son personas mayores, también deberán interactuar con ella. Además, mostró interés en recibir contenido educativo sobre prevención de enfermedades, alimentación y cuidado de terneros. Esta información sustenta el arquetipo de una ganadera joven que reconoce las limitaciones de los métodos tradicionales y busca una digitalización práctica y accesible que unifique el trabajo de su equipo familiar.

## Entrevista 2: Rebeca Noemi Quiroz Roldan

| Campo                        | Información                                                                                           |
|------------------------------|-------------------------------------------------------------------------------------------------------|
| Segmento                     | Ganadera                                                                                              |
| Nombres y apellidos          | Rebeca Noemi Quiroz Roldan                                                                            |
| Edad                         | 54 años                                                                                               |
| Distrito                     | Lima                                                                                                  |
| Ocupación                    | Productora ganadera                                                                                   |
| Tipo de ganado               | Ganado vacuno                                                                                         |
| Tamaño aproximado del ganado | Hato pequeño familiar                                                                                 |
| Inicio de la entrevista      | 00:00                                                                                                 |
| Duración                     | 00:06:53                                                                                              |
| URL del video                | <a href="https://tinyurl.com/entrevista2RebecaQuiroz">https://tinyurl.com/entrevista2RebecaQuiroz</a> |



### Resumen de la entrevista:

Rebeca Noemi Quiroz Roldan comentó que su actividad ganadera se enfoca en el cuidado de ganado vacuno y que una de sus mayores preocupaciones aparece cuando se presentan enfermedades como mastitis o cuando los terneros se enferman. Actualmente registra la información mediante cuadernos, apuntes y hojas sueltas. Aunque mencionó que logra manejar su actividad con estos medios, también reconoció que una aplicación podría ayudarle a tener mayor orden y a recordar eventos importantes, como vacunaciones, dosis de medicamentos y controles sanitarios.

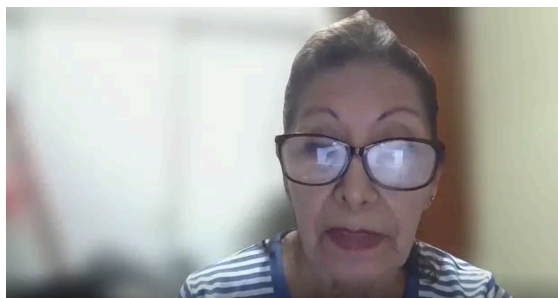
En sus respuestas se observa una personalidad responsable y preventiva. Rebeca valora la seguridad y la confianza antes de adoptar una herramienta digital, por lo que una solución como AniTec debe transmitir protección de datos, facilidad de uso y utilidad concreta. Su principal canal de interacción es el celular, especialmente WhatsApp, ya que lo usa para comunicarse con familiares, trabajadores y profesionales que la asesoran. Además, suele apoyarse en un ingeniero o especialista para atender problemas sanitarios del ganado, lo que muestra que sus decisiones están influenciadas por personal técnico de confianza.

Respecto a tecnología, utiliza smartphone y navegación básica desde el celular. No manifestó rechazo hacia las aplicaciones, pero sí dejó claro que no desea una herramienta complicada. Esta entrevista ayuda a justificar funciones como recordatorios, alertas sanitarias, registro simple de enfermedades y acceso rápido a datos importantes. Sus respuestas aportan evidencia para construir un arquetipo de ganadera que tiene disposición a digitalizarse siempre que la plataforma sea segura, clara y útil para resolver problemas reales del manejo diario.

### Entrevista 3: Ursula Trigozo Salcedo

| Campo               | Información            |
|---------------------|------------------------|
| Segmento            | Ganadero               |
| Nombres y apellidos | Ursula trigozo Salcedo |
| Edad                | 64 años                |
| Distrito            | Lurin, Lima            |
| Ocupación           | Ganadero artesanal     |
| Tipo de ganado      | Ganado vacuno          |
| Tamaño aproximado   | Hato pequeño familiar  |

| Campo                   | Información                                                                                                                                                                                                                                                                                                                                                                       |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| del ganado              |                                                                                                                                                                                                                                                                                                                                                                                   |
| Inicio de la entrevista | 00:00                                                                                                                                                                                                                                                                                                                                                                             |
| Duración                | 00:10:31                                                                                                                                                                                                                                                                                                                                                                          |
| URL del video           | <a href="https://upcedupe-my.sharepoint.com/:g/personal/u20231c019_upc_edu_pe/IQDw1Fks32sxQroa9TMJfnav=eyJyZWZlcnJhbEluZm8iOmsicmVmZXJyYWxBcHAIoiJTdHJIYW1XZWJBcHAIJCjYZWZlcnJhbFZpZXciOiJ">https://upcedupe-my.sharepoint.com/:g/personal/u20231c019_upc_edu_pe/IQDw1Fks32sxQroa9TMJfnav=eyJyZWZlcnJhbEluZm8iOmsicmVmZXJyYWxBcHAIoiJTdHJIYW1XZWJBcHAIJCjYZWZlcnJhbFZpZXciOiJ</a> |



### Resumen de la entrevista:

Úrsula Trigozo Salcedo, de 64 años, cuenta con 25 años de experiencia en la ganadería y actualmente maneja aproximadamente 35 cabezas de ganado vacuno junto con tres familiares. Su gestión se basa principalmente en cuadernos, papel y celular, donde registra vacunas, tratamientos, reproducción, partos y observaciones.

Su principal dificultad es mantener la información ordenada y disponible, ya que en ocasiones pierde tiempo buscando registros o estos se deterioran. Considera importante contar con un historial digital por animal y recibir alertas de vacunas, controles veterinarios y reproducción.

Úrsula está familiarizada con aplicaciones como WhatsApp, Facebook y YouTube, por lo que estaría dispuesta a utilizar AniTec si es simple, rápida, segura y funciona desde el celular. También valora el contenido educativo sobre alimentación, enfermedades y manejo del ganado.

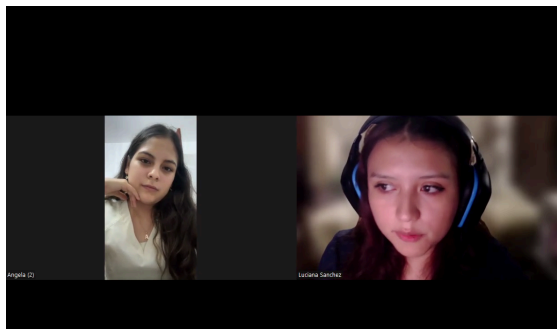
En este perfil, AniTec debe enfocarse en centralizar la información, ahorrar tiempo, evitar la pérdida de datos y facilitar el seguimiento individual de cada animal. Sus respuestas representan el arquetipo de una ganadera experimentada y abierta a la tecnología, siempre que sea sencilla y útil para su trabajo diario.

### Entrevistas al segmento de veterinarios

#### Entrevista 4: Angela Mendoza

| Campo               | Información        |
|---------------------|--------------------|
| Segmento            | Veterinaria        |
| Nombres y apellidos | Angela Mendoza     |
| Edad                | 24 años            |
| Distrito            | Camaná, Arequipa   |
| Ocupación           | Médica veterinaria |

| Campo                   | Información                                                                                             |
|-------------------------|---------------------------------------------------------------------------------------------------------|
| Zona de trabajo         | Sierra sur del Perú                                                                                     |
| Tipo de atención        | Atención sanitaria de ganado                                                                            |
| Inicio de la entrevista | 00:00                                                                                                   |
| Duración                | 00:06:55                                                                                                |
| URL del video           | <a href="https://tinyurl.com/entrevista4AngelaMendoza">https://tinyurl.com/entrevista4AngelaMendoza</a> |



### Resumen de la entrevista:

Angela Mendoza explicó que en su trabajo como veterinaria observa que muchos productores todavía gestionan la información sanitaria con cuadernos físicos, archivos simples de Excel, fotografías y conversaciones de WhatsApp. Esta dispersión de datos afecta el seguimiento de vacunas, tratamientos, diagnósticos y antecedentes clínicos. También mencionó que, en visitas de emergencia o campañas sanitarias, la falta de información ordenada puede retrasar la atención y complicar la toma de decisiones.

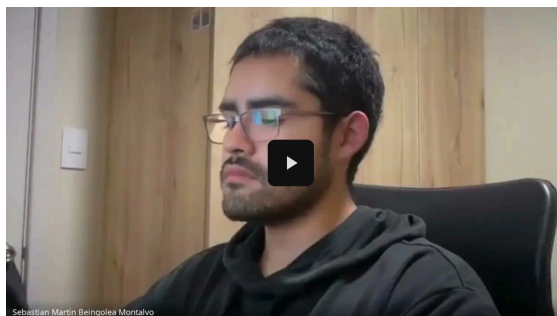
Su perfil evidencia una personalidad organizada, técnica y orientada al servicio. Angela valora la rapidez, la trazabilidad y la claridad de la información porque su trabajo depende de revisar antecedentes antes de indicar tratamientos. Sus canales principales de interacción son WhatsApp, llamadas telefónicas y archivos digitales básicos que comparte con productores. Usa smartphone y laptop para su labor diaria, y se siente cómoda navegando en Google Chrome o usando herramientas web cuando necesita consultar información técnica. Sus influencias profesionales provienen de su formación veterinaria, colegas del sector, campañas sanitarias y necesidades reales observadas en campo.

La entrevistada considera que una herramienta como AniTec sería útil si permite centralizar el historial clínico de cada animal, registrar tratamientos, programar seguimientos y mejorar la comunicación con los ganaderos. Sin embargo, remarcó que la plataforma debe ser rápida y fácil de usar, porque el trabajo veterinario de campo no permite perder tiempo en procesos largos. Esta entrevista aporta evidencia para el arquetipo de veterinaria joven que usa tecnología, pero necesita una solución enfocada en eficiencia clínica y coordinación con productores.

### Entrevista 5: Jeferson Castro Pariona

| Campo               | Información             |
|---------------------|-------------------------|
| Segmento            | Veterinario             |
| Nombres y apellidos | Jeferson Castro Pariona |
| Edad                | 27 años                 |

| Campo                   | Información                                                             |
|-------------------------|-------------------------------------------------------------------------|
| Distrito                | Ventanilla, Lima (Residencia)                                           |
| Ocupación               | Médico veterinario                                                      |
| Zona de trabajo         | Selva central del Perú                                                  |
| Tipo de atención        | Atención clínica y sanitaria de ganado en campo                         |
| Inicio de la entrevista | 00:00                                                                   |
| Duración                | 00:08:09                                                                |
| URL del video           | <a href="https://tinyurl.com/b3txtyh3">https://tinyurl.com/b3txtyh3</a> |



### Resumen de la entrevista:

Jeferson Castro Pariona comentó que el reto más grande y frecuente en la atención veterinaria ganadera es la falta de trazabilidad y la pérdida de continuidad en el historial médico de los animales. Explicó que la información suele estar dispersa en cuadernos, notas físicas, hojas de cálculo o chats de WhatsApp, y que cuando se enfrentan emergencias o se incorporan nuevos animales, debe tomar decisiones clínicas con datos limitados. Esto afecta directamente la planificación sanitaria, la efectividad de los tratamientos y el seguimiento de las intervenciones.

El entrevistado muestra un perfil analítico, práctico y orientado a la tecnología como herramienta de trabajo diario. Utiliza su smartphone para la comunicación y el registro rápido en campo, y su laptop para consultas técnicas más detalladas. Considera fundamental contar con datos concretos antes de intervenir y valora la rapidez, la facilidad de uso y la intuición en cualquier herramienta digital, especialmente considerando los retos de conectividad en el entorno rural.

Según sus respuestas, una plataforma digital (como AniTec) le aportaría un valor significativo si logra centralizar la información sin ser un sistema pesado. Sus funcionalidades clave ideales incluyen: acceso inmediato al historial sanitario del animal, un módulo rápido para registrar diagnósticos y tratamientos, alertas de vacunación o reproducción para programar rutas de campo, y un canal claro de comunicación con el ganadero. También valoró la inclusión de contenido educativo rápido, como fichas técnicas o información sobre brotes en la región. En conclusión, está totalmente dispuesto a digitalizar su labor para reducir los errores derivados de la información dispersa, siempre y cuando la herramienta se integre de forma ágil a su flujo real de trabajo en el campo.

## 2.2.3. Análisis de entrevistas.

### Análisis del segmento de Ganaderos

Para el segmento de ganaderos se analizaron 3 entrevistas: Vicente Huamán Alacutte, Rebeca Noemi Quiroz Roldan y Porfirio Salazar Rodriguez. La muestra evidencia un perfil de productores con experiencia práctica, uso frecuente de métodos tradicionales y apertura moderada hacia herramientas digitales siempre que sean simples, útiles y accesibles.

| <b>Característica identificada</b>          | <b>Resultado estadístico</b>                                                                                                   | <b>Sustento en entrevistas</b>                                                                                                                                                          | <b>Relación con el arquetipo</b>                                                                                                                       |
|---------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| Edad adulta y experiencia en ganadería      | 3 de 3 entrevistados (100%) tienen entre 54 y 65 años y experiencia directa en actividades ganaderas.                          | Vicente tiene 62 años y más de 30 años de experiencia; Rebeca tiene 54 años y gestiona ganado vacuno; Porfirio tiene 65 años y trabaja en ganadería artesanal.                          | El arquetipo debe representar a un usuario con conocimiento práctico del campo, pero no necesariamente familiarizado con sistemas digitales complejos. |
| Uso de registros manuales                   | 3 de 3 entrevistados (100%) usan cuadernos, apuntes, hojas o registros básicos para controlar información del ganado.          | Vicente usa cuaderno físico y Excel; Rebeca usa cuadernos y hojas sueltas; Porfirio trabaja con una gestión artesanal y poco digitalizada.                                              | AniTec debe priorizar registro rápido, ordenado y fácil de consultar para reemplazar gradualmente el control manual.                                   |
| Necesidad de controlar salud y tratamientos | 2 de 3 entrevistados (66.7%) mencionan directamente problemas relacionados con enfermedades, tratamientos o control sanitario. | Vicente menciona control de enfermedades y vacunación; Rebeca menciona mastitis, terneros enfermos y dosis de medicamentos.                                                             | El arquetipo necesita alertas sanitarias, historial médico y recordatorios de vacunación o tratamiento.                                                |
| Uso del celular como canal principal        | 3 de 3 entrevistados (100%) muestran familiaridad básica con el celular como medio de comunicación o consulta.                 | Vicente usa celular y WhatsApp; Rebeca usa principalmente su celular; Porfirio utiliza celular y comunicación por WhatsApp.                                                             | La solución debe funcionar bien en pantallas pequeñas y permitir una interacción directa desde dispositivos móviles.                                   |
| Interés en adoptar tecnología               | 3 de 3 entrevistados (100%) reconocen que la tecnología podría ayudar a mejorar su gestión.                                    | Vicente afirma que la tecnología puede profesionalizar el sector; Rebeca estaría dispuesta a usar una aplicación; Porfirio considera que la tecnología podría mejorar su productividad. | El arquetipo no rechaza la tecnología, pero necesita percibir un beneficio claro antes de usarla.                                                      |
| Necesidad de simplicidad                    | 3 de 3 entrevistados (100%) requieren que la herramienta sea sencilla, clara o fácil de justificar en el trabajo diario.       | Vicente pide una app sencilla para el hombre de campo; Rebeca no desea una herramienta complicada; Porfirio                                                                             | La interfaz debe usar lenguaje simple, formularios directos y pocos pasos para completar tareas.                                                       |

| Característica identificada          | Resultado estadístico                                                                                | Sustento en entrevistas                                                                                                                     | Relación con el arquetipo                                                                                                         |
|--------------------------------------|------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
|                                      |                                                                                                      | evalúa si el beneficio justifica la inversión.                                                                                              |                                                                                                                                   |
| Influencia de terceros en decisiones | 3 de 3 entrevistados (100%) dependen o se ven influenciados por otras personas del entorno ganadero. | Vicente considera compradores y técnicos; Rebeca se apoya en un especialista; Porfirio aprende de experiencia familiar y otros productores. | El arquetipo toma decisiones con apoyo de redes de confianza, por lo que la app debe transmitir seguridad y utilidad comprobable. |
| Sensibilidad al costo                | 1 de 3 entrevistados (33.3%) menciona directamente el costo como barrera.                            | Porfirio indica que necesitaría capital suficiente para pagar una solución digital.                                                         | El modelo de adopción debe considerar planes accesibles, prueba gratuita o beneficios económicos visibles.                        |

En conjunto, el análisis muestra que el ganadero objetivo de AniTec es un usuario con experiencia práctica y responsabilidad directa sobre el cuidado del ganado, pero con procesos todavía manuales. La característica más fuerte del segmento es la necesidad de orden y recordatorios, ya que el 100% de entrevistados depende de registros tradicionales y el 66.7% menciona problemas sanitarios específicos. También se observa una oportunidad clara para una solución móvil, debido a que el 100% usa el celular como canal principal o familiar.

Desde el punto de vista subjetivo, los ganaderos entrevistados valoran la confianza, la utilidad concreta, la seguridad de la información y la facilidad de uso. Estas características provienen directamente de los resúmenes: Vicente prioriza una herramienta simple para el campo, Rebeca valora la seguridad y Porfirio evalúa el beneficio frente al costo. Por ello, el arquetipo de ganadero debe construirse como una persona práctica, cuidadosa, sensible al esfuerzo de aprendizaje y dispuesta a digitalizarse si la solución demuestra valor real.

## Análisis del segmento de Veterinarios

Para el segmento de veterinarios se analizaron 2 entrevistas: Angela Mendoza y Jeferson Castro Pariona. La muestra evidencia un perfil profesional joven, con experiencia en atención de campo, necesidad de información clínica ordenada y uso de herramientas digitales básicas para comunicarse con productores.

| Característica identificada               | Resultado estadístico                                               | Sustento en entrevistas                                                          | Relación con el arquetipo                                                                                        |
|-------------------------------------------|---------------------------------------------------------------------|----------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| Profesionales jóvenes                     | 2 de 2 entrevistados (100%) tienen entre 24 y 27 años.              | Angela tiene 24 años; Jeferson tiene 27 años.                                    | El arquetipo puede representarse como un veterinario joven, con mayor disposición a usar herramientas digitales. |
| Trabajo en campo o zonas descentralizadas | 2 de 2 entrevistados (100%) trabajan o han trabajado en zonas fuera | Angela trabaja en la sierra sur; Jeferson tiene experiencia en la selva central. | La solución debe considerar rapidez, movilidad y uso en                                                          |

| Característica identificada                       | Resultado estadístico                                                                                  | Sustento en entrevistas                                                                                                                                                                           | Relación con el arquetipo                                                                                       |
|---------------------------------------------------|--------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
|                                                   | del entorno urbano principal.                                                                          |                                                                                                                                                                                                   | contextos con posible conectividad limitada.                                                                    |
| Problemas por registros incompletos o dispersos   | 2 de 2 entrevistados (100%) mencionan información sanitaria desordenada o incompleta.                  | Angela menciona cuadernos, Excel, fotos y WhatsApp; Jeferson menciona cuadernos, notas físicas, hojas de cálculo y chats de WhatsApp.                                                             | El arquetipo necesita acceso centralizado al historial clínico y sanitario de los animales.                     |
| Uso de WhatsApp y herramientas digitales básicas  | 2 de 2 entrevistados (100%) usan canales digitales simples para comunicarse o complementar su trabajo. | Angela usa WhatsApp, llamadas y archivos digitales; Jeferson usa smartphone en campo y laptop para consultas técnicas.                                                                            | AniTec debe integrarse al flujo real del veterinario y reducir la dependencia de información dispersa.          |
| Necesidad de historial clínico previo             | 2 de 2 entrevistados (100%) señalan que conocer antecedentes mejora la atención.                       | Angela necesita revisar antecedentes antes de tratamientos; Jeferson indica que la falta de trazabilidad obliga a decidir con datos limitados.                                                    | El arquetipo requiere fichas clínicas, historial sanitario y acceso rápido a tratamientos previos.              |
| Valoración de rapidez y facilidad de uso          | 2 de 2 entrevistados (100%) indican que la herramienta debe ser rápida, intuitiva y adaptada al campo. | Angela remarca que no puede perder tiempo en procesos largos; Jeferson pide rapidez, facilidad de uso e integración ágil al trabajo rural.                                                        | La interfaz veterinaria debe permitir registrar atenciones con pocos pasos y consultar datos al instante.       |
| Necesidad de mejorar comunicación con ganaderos   | 2 de 2 entrevistados (100%) consideran importante coordinar mejor con productores.                     | Angela menciona mejorar comunicación con ganaderos; Jeferson valora un canal claro de comunicación con el productor.                                                                              | El arquetipo necesita funciones de seguimiento, coordinación y comunicación clara con el productor.             |
| Actitud favorable hacia herramientas tecnológicas | 2 de 2 entrevistados (100%) muestran apertura a soluciones digitales para mejorar su trabajo.          | Angela considera útil centralizar historial y seguimiento; Jeferson está dispuesto a digitalizar su labor si la herramienta no es pesada y aporta alertas, registro rápido y contenido educativo. | El arquetipo es un usuario con mayor predisposición digital que el ganadero, pero exige eficiencia profesional. |

El análisis del segmento veterinario muestra que las principales necesidades están relacionadas con trazabilidad clínica, acceso rápido a información y coordinación con productores. El 100% de los entrevistados menciona problemas por información dispersa y el 100% considera importante contar con antecedentes del animal antes o durante la atención. Esto evidencia que el arquetipo veterinario debe estar construido alrededor de la toma de decisiones clínicas, el seguimiento sanitario y la necesidad de reducir incertidumbre durante el trabajo de campo.

En cuanto a características subjetivas, ambos veterinarios se muestran más familiarizados con la tecnología que el segmento ganadero. Angela representa un perfil organizado, técnico y orientado al servicio, mientras que Jeferson muestra un perfil analítico, práctico y orientado a la tecnología. Ambos valoran la rapidez, la trazabilidad y la utilidad real. Por ello, el arquetipo de veterinario debe representar a un profesional joven, móvil, acostumbrado a usar herramientas digitales básicas, pero que necesita una plataforma más integrada para evitar pérdida de información y mejorar la continuidad de tratamientos.

## 2.3. Needfinding.

---

En esta sección se presentarán los artefactos resultantes del proceso de análisis de la información recolectada de los segmentos objetivos. Aquí se incluyen secciones para User Personas, User Task Matrix, User Journey Maps, Empathy Mapping y As-is Scenario Mapping.

### 2.3.1. User Personas.

A continuación, se presentan los User Personas diseñados para representar a los segmentos objetivo identificados durante la fase de investigación. Estos arquetipos detallan variables demográficas, rasgos psicográficos, motivaciones y comportamientos, así como los pains (frustraciones) y gains (objetivos) que enfrentan en su gestión diaria. Asimismo, se analiza su nivel digital y su interacción con soluciones tecnológicas del sector agropecuario. Toda la información ha sido sintetizada a partir de los insights recolectados en las entrevistas y estructurada mediante la plataforma UXPressia para garantizar una representación fiel de las necesidades del usuario.

User Persona: Ganaderos

PERSONA: Jorge Luis Rivas

NOMBRE

Jorge Luis Rivas

TIPO

Guardian



### Biografía

Jorge es un ganadero de segunda generación con más de 20 años de experiencia. Gestiona una hacienda de tamaño mediano con unas 80 cabezas de ganado. Aunque conoce el negocio a la perfección, siente que la administración se le escapa de las manos por depender exclusivamente de cuadernos físicos que suelen dañarse o perderse. Busca modernizar sus procesos para dejar un negocio organizado, rentable y sostenible para sus hijos.

### Frustraciones

- La pérdida recurrente de registros manuales (cuadernos mojados o manchados).
- No recordar la fecha exacta del último tratamiento de un animal específico.
- La dificultad para calcular la rentabilidad real de cada ejemplar.
- Depender de su memoria para el ciclo de vacunación y reproducción.

### Metas

- Digitalizar el 100% del historial médico y reproductivo de su ganado.
- Reducir las pérdidas económicas por falta de control en vacunas y alimentación.
- Obtener reportes de productividad rápidos para tomar decisiones de venta.
- Lograr una comunicación más fluida y documentada con su veterinario de confianza.

### Motivaciones

- Legado:** Dejar un negocio organizado y tecnificado para su familia.
- Eficiencia:** Reducir el tiempo invertido en registros manuales.
- Reconocimiento:** Ser visto como un productor moderno y confiable.
- Crecimiento:** Incrementar la productividad y ganancias de la hacienda.

### Demografía

Masculino 45 años

Cajamarca, Perú

Ganadero Independiente

### Preferencias



### Cita

“El ganado es mi vida, pero a veces siento que paso más tiempo buscando un papel que cuidando a mis animales.”

### Habilidades

Tecnología

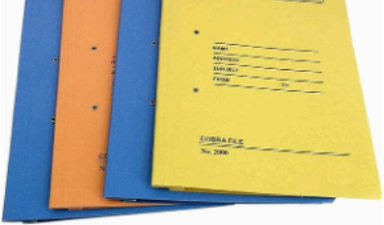


Experiencia



### Marcas e influencias





### Technology



## UXPRESSIA

This persona was built in [uxpressia.com](https://uxpressia.com)

User Persona: Veterinarios

PERSONA: Valeria Mendoza

NOMBRE

Valeria Mendoza

TIPO

Rational



### Biografía

Valeria es una veterinaria apasionada por la salud animal y la tecnología. Trabaja como consultora externa para múltiples unidades ganaderas. Su día a día consiste en realizar diagnósticos, campañas de vacunación y cirugías de campo. Se frustra constantemente cuando los ganaderos no cuentan con información previa del animal, lo que la obliga a trabajar con datos incompletos y reduce su capacidad de respuesta ante emergencias.

### Frustraciones

- La falta de trazabilidad del historial médico del ganado.
- La comunicación desordenada con los clientes mediante audios y mensajes dispersos.
- Perder tiempo llenando fichas manuales que luego se extravían.
- Diagnósticos o tratamientos menos precisos debido a información incompleta.

### Demografía

Femenino 32 años

Lima

Médico Veterinaria  
Zootecnista

### Metas

- Acceder al historial clínico completo antes de visitar la finca.
- Automatizar reportes y recetas médicas.
- Monitorear la salud del ganado mediante datos y estadísticas.
- Mantener una comunicación profesional y centralizada con los ganaderos.
- Optimizar el tiempo durante visitas y emergencias.

### Motivaciones

- **Profesionalismo:** Elevar el estándar de salud animal en la región.
- **Ciencia:** Utilizar datos y estadísticas para prevenir brotes de enfermedades.
- **Reconocimiento:** Ser valorada como una experta que utiliza herramientas de vanguardia.
- **Impacto:** Mejorar la calidad de vida y salud del ganado mediante tecnología.

### Preferencias



### Cita

“  
Un buen diagnóstico depende tanto de la experiencia como de tener la información correcta en el momento adecuado.  
”

### Habilidades

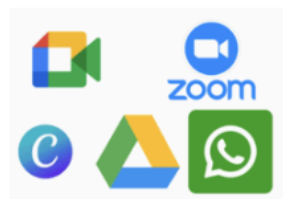
Tecnología



Experiencia



### Marcas e influencias



### Tecnología



## UXPRESSIA

This persona was built in uxpressia.com

### 2.3.2. User Task Matrix.

A través de la User Task Matrix, es posible identificar y organizar las principales actividades que los usuarios realizan actualmente dentro de su contexto de trabajo. Al categorizar estas tareas según su frecuencia e importancia, se logra comprender cuáles representan mayores dificultades y necesidades para cada perfil de usuario, permitiendo detectar oportunidades de mejora en la gestión ganadera y veterinaria.

| User Task                                                                              | Jorge Luis Rivas<br>(Frecuencia) | Jorge Luis Rivas<br>(Importancia) | Valeria Mendoza<br>(Frecuencia) | Valeria Mendoza<br>(Importancia) |
|----------------------------------------------------------------------------------------|----------------------------------|-----------------------------------|---------------------------------|----------------------------------|
| Anotar el nacimiento o compra de un nuevo animal en cuadernos físicos                  | Sometimes                        | High                              | Rarely                          | Medium                           |
| Registrar manualmente vacunas y tratamientos del ganado                                | Often                            | High                              | Always                          | High                             |
| Revisar fechas de vacunación en notas, calendarios o cuadernos                         | Often                            | High                              | Often                           | High                             |
| Recordar manualmente vacunas o controles pendientes                                    | Sometimes                        | High                              | Sometimes                       | High                             |
| Anotar peso y crecimiento del ganado durante controles                                 | Often                            | Medium                            | Often                           | Medium                           |
| Revisar manualmente información sobre productividad y rendimiento                      | Sometimes                        | Medium                            | Often                           | Medium                           |
| Compartir documentos físicos o fotografías de registros con asociaciones o compradores | Rarely                           | Medium                            | Rarely                          | Low                              |
| Buscar información o capacitaciones ganaderas en internet y redes sociales             | Sometimes                        | Low                               | Sometimes                       | Medium                           |
| Llevar el control reproductivo mediante anotaciones manuales                           | Rarely                           | Medium                            | Rarely                          | Medium                           |

| User Task                                                                | Jorge Luis Rivas<br>(Frecuencia) | Jorge Luis Rivas<br>(Importancia) | Valeria Mendoza<br>(Frecuencia) | Valeria Mendoza<br>(Importancia) |
|--------------------------------------------------------------------------|----------------------------------|-----------------------------------|---------------------------------|----------------------------------|
| Buscar antecedentes médicos y sanitarios en cuadernos o archivos físicos | Sometimes                        | High                              | Sometimes                       | High                             |

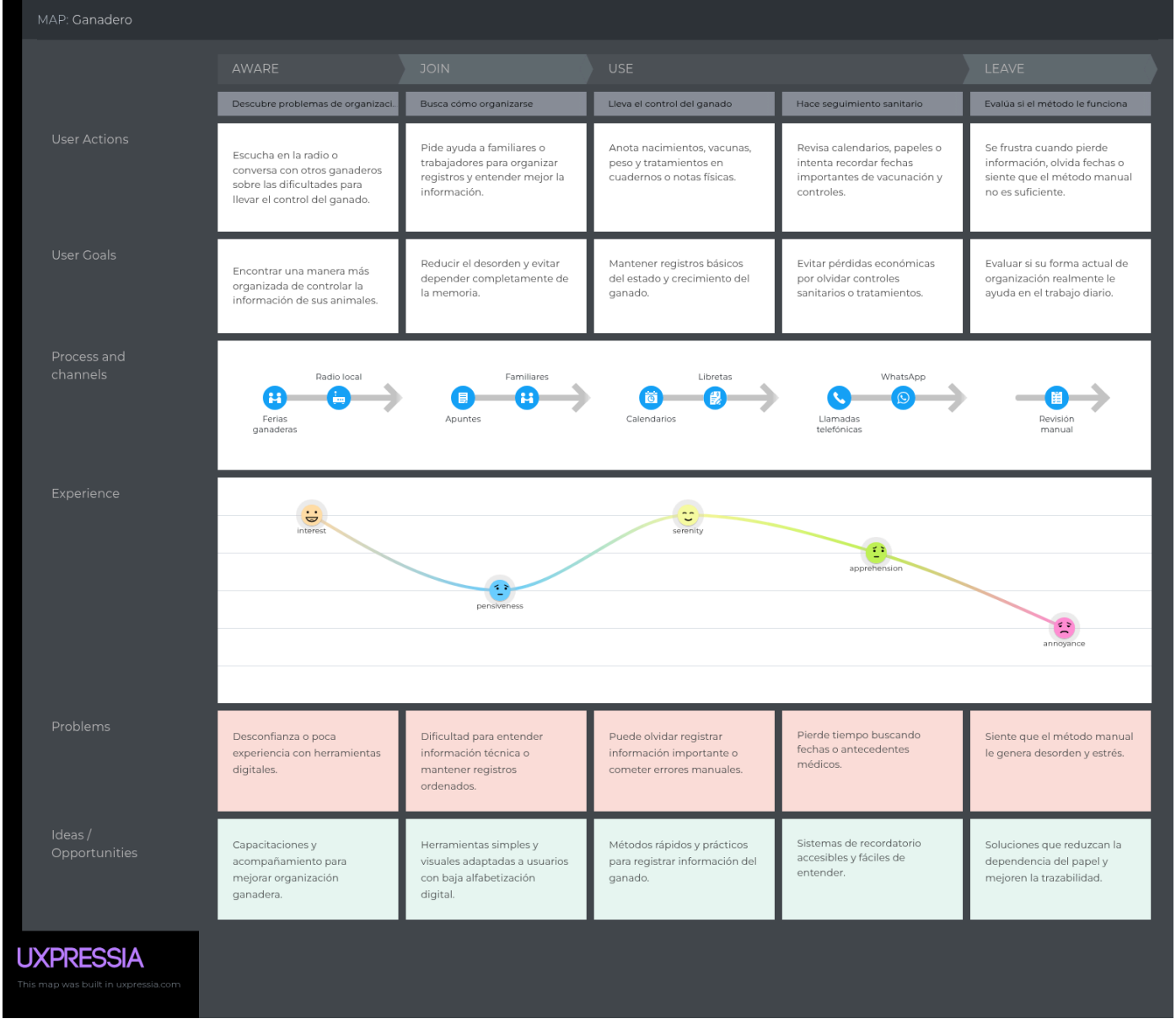
La User Task Matrix evidencia que tanto Jorge Luis como Valeria realizan constantemente actividades relacionadas con el control sanitario y el seguimiento del ganado. Mientras Jorge Luis depende principalmente de registros físicos y de su memoria para organizar la información de sus animales, Valeria necesita acceder rápidamente a datos precisos durante sus visitas de campo y procedimientos veterinarios. Asimismo, ambos perfiles presentan dificultades relacionadas con la organización, trazabilidad y acceso oportuno a la información, especialmente en procesos de vacunación, historial clínico y monitoreo del ganado. Estas tareas permiten comprender mejor el contexto actual de los usuarios e identificar necesidades reales dentro del entorno ganadero y veterinario.

### 2.3.3. User Journey Mapping.

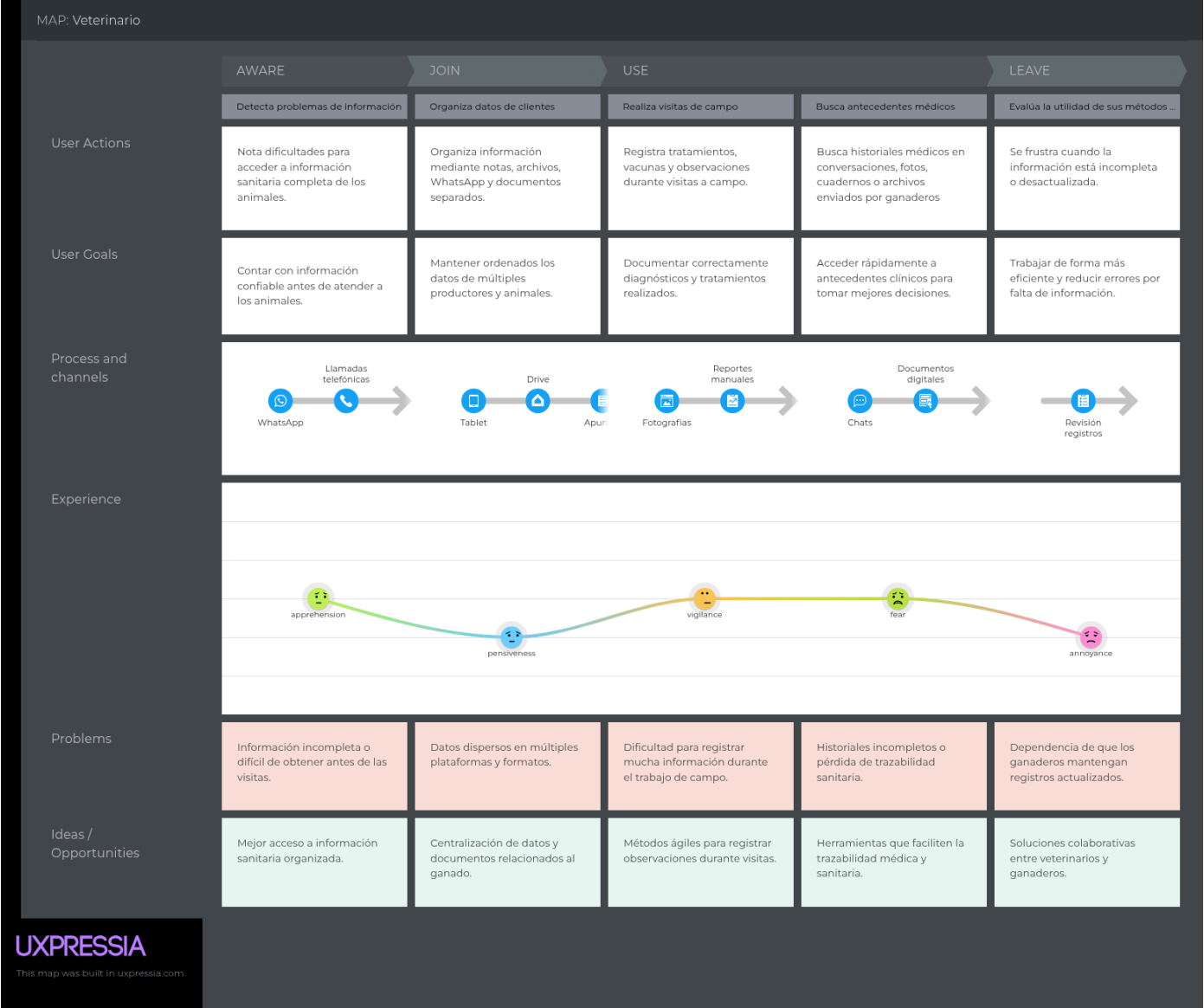
En este apartado se describe de manera detallada el recorrido y la experiencia actual de los usuarios dentro de su contexto de trabajo cotidiano, enfocándose específicamente en los dos perfiles clave identificados: productores ganaderos y médicos veterinarios. A través de los User Journey Maps As-Is, se analizan las actividades, emociones, problemas y necesidades que experimentan los usuarios durante la gestión y seguimiento del ganado, sin considerar aún la existencia de una solución tecnológica implementada.

El mapeo de este recorrido inicia desde las primeras dificultades relacionadas con la organización y acceso a la información, avanzando a través de las tareas diarias de registro, seguimiento sanitario y búsqueda de antecedentes médicos. Asimismo, se identifican los principales puntos de contacto, canales utilizados y frustraciones presentes en el proceso actual, permitiendo comprender de forma integral cómo los usuarios realizan actualmente sus actividades y cuáles son las oportunidades de mejora dentro del entorno ganadero y veterinario.

User Ganadero:



User Veterinario:



2.3.4. Empathy Mapping.

En esta sección se presenta el proceso de elaboración de los Empathy Maps correspondientes a los User Personas identificados para el proyecto: el productor ganadero y la médica veterinaria. Para el desarrollo de estos mapas de empatía, el equipo analizó la información obtenida durante la etapa de investigación y needfinding, considerando las necesidades, comportamientos, preocupaciones y motivaciones de cada perfil dentro de su contexto laboral cotidiano.

El proceso de elaboración inició colocando al User Persona en el centro del análisis, permitiendo identificar de manera estructurada qué piensa, siente, observa, escucha, dice y hace cada usuario en relación con la gestión sanitaria y administrativa del ganado. Asimismo, se identificaron los principales esfuerzos (Pains) y ganancias (Gains) presentes en sus actividades diarias, con el objetivo de comprender mejor sus frustraciones, expectativas y oportunidades de mejora dentro del entorno ganadero y veterinario.

Los Empathy Maps permitieron visualizar de forma más profunda la experiencia actual de los usuarios, evidenciando problemáticas relacionadas con la organización de información, trazabilidad sanitaria, dependencia de registros físicos y dificultades en la comunicación y acceso a datos confiables.

User Ganadero:

PERSONA: Veterinario

### 1. ¿Con quién estamos empatizando?

- **Quién:** César, 62 años, ganadero de Cajamarca.
- **Situación:** Lleva más de 25 años registrando información en cuadernos y suele olvidar fechas importantes.
- **Rol:** Dueño y gestor de su ganado; responsable de la salud, alimentación y ventas.

### 2. ¿Qué necesita hacer?

- Mantener organizados los registros de raza, edad, peso y eventos sanitarios.
- Tomar decisiones sobre vacunación, reproducción y ventas.
- Tener acceso rápido a información importante del ganado.
- **Éxito:** Reducir pérdidas y mantener un mejor control sanitario de sus animales.

### 3. ¿Qué está viendo?

- Cuadernos desordenados y registros difíciles de encontrar.
- Otros ganaderos enfrentando problemas similares con papeles y controles manuales.
- Herramientas digitales complejas o poco adaptadas al trabajo de campo.

### 4. ¿Qué está diciendo?

“

- “Yo sé todo de mi ganado, pero a veces me falla la memoria.”
- “Necesito algo sencillo y práctico para organizarme mejor.”

”



### 5. ¿Qué está haciendo?

- Anota información manualmente en cuadernos o calendarios.
- Toma fotos de registros y las comparte por WhatsApp.
- Consulta fechas importantes revisando papeles y notas antiguas.

### ESFUERZOS

- Pérdida o deterioro de cuadernos físicos.
- Dificultad para encontrar información rápidamente.
- Dependencia de la memoria para recordar controles y tratamientos.
- Señal limitada o poca experiencia con herramientas digitales.

### GANANCIAS

- Acceder fácilmente a información organizada del ganado.
- Reducir olvidos relacionados con vacunas y tratamientos.
- Tener mayor confianza y control sobre la salud de sus animales.
- Ahorrar tiempo en la búsqueda de registros.

### 7. ¿Cómo se siente y qué piensa?

“

- **Piensa:** “¿Olvidaré otra vacuna?”  
“¿Dónde anoté ese tratamiento?”
- **Siente:** preocupación por perder información importante y desconfianza hacia la tecnología.

”

### 6. ¿Qué está escuchando?

- Recomendaciones de familiares y veterinarios sobre nuevas formas de organización.
- Conversaciones sobre problemas relacionados con pérdida de registros y olvidos.
- Comentarios sobre herramientas digitales difíciles de usar.

UXPRESSIA

This persona was built in uxpressia.com

User Veterinario:

## PERSONA: Veterinario

## 1. ¿Con quién estamos empatizando?

- **Quién:** Carla, 28 años, veterinaria de campo en Apurímac.
- **Situación:** Atiende múltiples fincas semanalmente utilizando notas físicas y herramientas digitales separadas.
- **Rol:** Responsable de diagnósticos, tratamientos y asesoría sanitaria para productores ganaderos.

## 2. ¿Qué necesita hacer?

- Documentar diagnósticos, vacunas y tratamientos realizados durante visitas.
- Acceder rápidamente a antecedentes clínicos confiables.
- Organizar información de diferentes productores y animales.
- **Éxito:** Mejorar el seguimiento sanitario y reducir errores por información incompleta.

## 3. ¿Qué está viendo?

- Información dispersa en cuadernos, fotos y chats de WhatsApp.
- Productores con registros incompletos o desactualizados.
- Colegas utilizando múltiples herramientas sin integración.

## 4. ¿Qué está diciendo?

“

- “Si los productores registraran mejor la información, evitaríamos muchos problemas.”
- “Necesito toda la información organizada y accesible rápidamente.”

”



## 5. ¿Qué está haciendo?

- Registra observaciones y tratamientos durante visitas de campo.
- Toma fotografías de registros y comparte información por WhatsApp.
- Utiliza documentos digitales, notas y archivos para organizar información.

## ESFUERZOS

- Información sanitaria incompleta o desactualizada.
- Datos dispersos en diferentes plataformas y formatos.
- Dificultad para registrar información durante visitas de campo.
- Dependencia de que los productores mantengan registros organizados.

## GANANCIAS

- Acceder rápidamente a historiales clínicos organizados.
- Reducir tiempo buscando información de animales.
- Mejorar el seguimiento sanitario y la toma de decisiones.
- Trabajar de manera más eficiente y profesional.

## 7. ¿Cómo se siente y qué piensa?

“

- **Piensa:** “¿Cómo puedo tomar mejores decisiones si la información está incompleta?”
- **Siente:** frustración por la falta de organización y preocupación por cometer errores clínicos.

”

## 6. ¿Qué está escuchando?

- Quejas de productores sobre pérdida de información o dificultad para organizar registros.
- Comentarios de colegas sobre problemas de trazabilidad sanitaria.
- Recomendaciones de herramientas digitales poco adaptadas al trabajo rural.

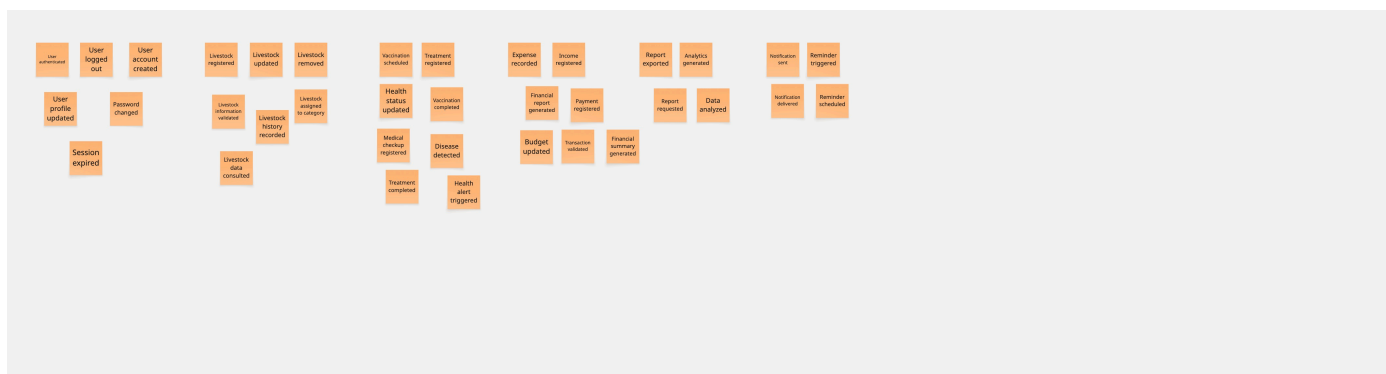
UXPRESSIA

This persona was built in uxpressia.com

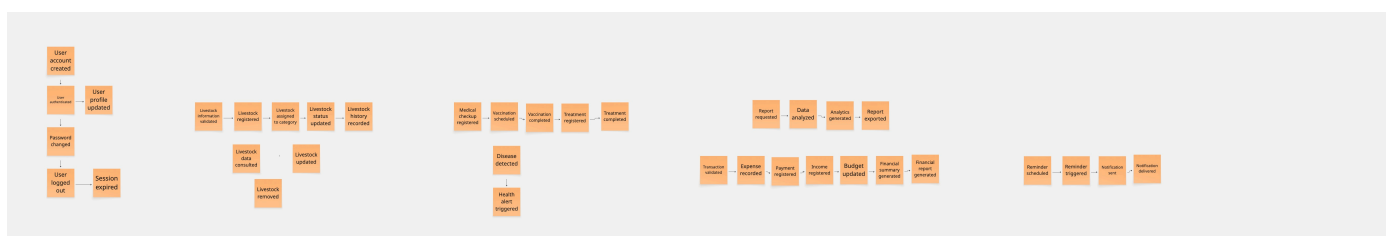
## 2.3.5. Big Picture EventStorming.

El presente Big Picture Event Storming se ha desarrollado de manera colaborativa utilizando la plataforma Miro, siguiendo la metodología de Philippe Bourgau para explorar el dominio del negocio de forma holística y establecer un entendimiento compartido. A través de un proceso iterativo en este entorno digital, que incluyó la generación de eventos de dominio, el ordenamiento cronológico y la identificación de puntos críticos dentro de los procesos, se ha logrado mapear la complejidad del sector ganadero en una narrativa visual coherente. Este artefacto no solo permitió identificar riesgos y oportunidades de mejora en la gestión de AniTec, sino que también sentó las bases para la comprensión del dominio y el diseño posterior de la arquitectura del sistema alineada con la realidad operativa de los ganaderos y veterinarios.

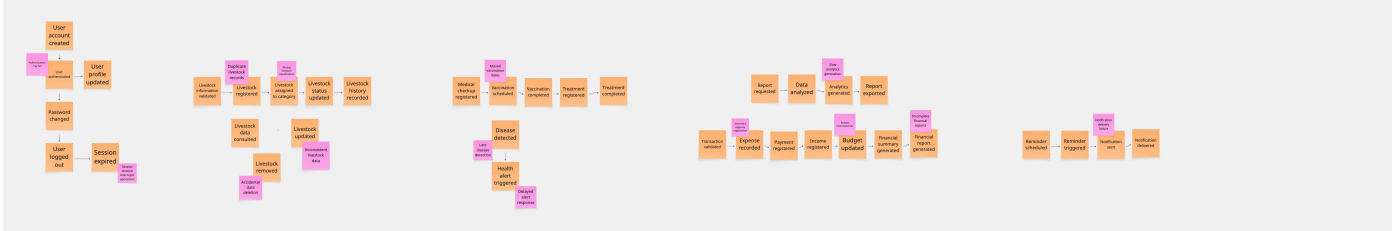
**Paso 1:** Unstructured Exploration (Exploración no estructurada) consiste en una lluvia de ideas colaborativa donde los participantes identifican y registran domain events, que son sucesos relevantes ocurridos dentro del negocio. Estos eventos deben redactarse obligatoriamente en tiempo pasado (por ejemplo, "Livestock registered") y se colocan en notas adhesivas de color naranja sobre la superficie de modelado. En esta etapa inicial, se prioriza el descubrimiento de conceptos y procesos importantes del dominio sobre el orden o la jerarquía, permitiendo identificar eventos relacionados con la autenticación de usuarios, gestión de ganado, procesos sanitarios, operaciones financieras, generación de reportes y envío de notificaciones.



**Paso 2:** Timelines, los participantes revisan los eventos de dominio generados y los organizan cronológicamente para reflejar la secuencia real de los procesos empresariales. La construcción inicia organizando el flujo principal del negocio y posteriormente incorporando relaciones, ramificaciones y dependencias entre eventos. Este paso permitió construir una narrativa visual coherente del comportamiento del sistema AniTec, facilitando la identificación de secuencias operativas, conexiones entre procesos y posibles inconsistencias dentro del dominio.



**Paso 3:** Pain Points, los participantes utilizan la línea de tiempo recién organizada para identificar los puntos críticos, riesgos o ineficiencias presentes dentro de los procesos modelados. Estos problemas se representan mediante notas adhesivas rosadas colocadas cerca de los eventos relacionados, permitiendo visualizar dificultades operativas y oportunidades de mejora dentro del sistema. Entre los principales pain points identificados destacan posibles fallos de autenticación, registros duplicados de ganado, detección tardía de enfermedades, inconsistencias presupuestarias y fallos en la entrega de notificaciones. La identificación explícita de estas debilidades permitió comprender mejor los desafíos del dominio y considerar posibles automatizaciones o mejoras futuras en el diseño del sistema.



Enlace para acceder al miro: <https://tinyurl.com/EventSorming>

### 2.3.6. Ubiquitous Language.

Siguiendo los conceptos de **Ubiquitous Language** definidos por **Eric Evans (2003)** en su obra *Domain-Driven Design: Tackling Complexity in the Heart of Software*, se presenta el siguiente glosario. Este conjunto de términos constituye el lenguaje común del proyecto, eliminando ambigüedades entre el equipo de ingeniería y los expertos del dominio ganadero.

| Term (English)     | Término (Español)        | Definition (Definición)                                                                                                                                  |
|--------------------|--------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Livestock Owner    | Ganadero                 | Usuario responsable de la administración operativa y financiera de la hacienda, encargado de registrar eventos diarios y tomar decisiones de producción. |
| Veterinarian       | Veterinario              | Profesional especializado encargado de la salud animal, responsable de emitir diagnósticos, prescribir tratamientos y validar historiales clínicos.      |
| Herd               | Hato                     | Conjunto de animales agrupados dentro de una unidad productiva o finca para su control sanitario, productivo y administrativo.                           |
| Farm               | Finca                    | Unidad productiva donde el ganadero organiza animales, actividades, registros sanitarios y movimientos financieros.                                      |
| Animal             | Animal                   | Ejemplar individual registrado por el ganadero para mantener su trazabilidad sanitaria, productiva y económica.                                          |
| Health Event       | Evento Sanitario         | Registro de una vacuna, enfermedad, diagnóstico, tratamiento, incidencia o seguimiento relacionado con la salud de un animal.                            |
| Veterinary Client  | Cliente Veterinario      | Ganadero asignado a un veterinario para que este pueda revisar sus animales, consultar historiales y registrar atenciones sanitarias.                    |
| Veterinary History | Historial Veterinario    | Expediente clínico consolidado que permite al especialista revisar antecedentes médicos antes de realizar una intervención.                              |
| Treatment Protocol | Protocolo de Tratamiento | Conjunto de instrucciones médicas y fármacos asignados a un animal para tratar una afección diagnosticada por el veterinario.                            |
| Traceability       | Trazabilidad             | Capacidad de reconstruir el historial completo de un animal, incluyendo origen, salud, peso, actividades y cambios relevantes.                           |

| Term (English)            | Término (Español)          | Definition (Definición)                                                                                                                          |
|---------------------------|----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>IoT Device</b>         | Dispositivo IoT            | Equipo asociado a una finca o animal para apoyar el monitoreo de datos como peso, temperatura, humedad, ubicación o actividad.                   |
| <b>Metric</b>             | Métrica                    | Lectura generada por un dispositivo o registro del sistema que permite analizar el estado de un animal, finca o proceso ganadero.                |
| <b>Farm Activity</b>      | Actividad Ganadera         | Tarea productiva, sanitaria, financiera o veterinaria programada para organizar el trabajo dentro de la operación ganadera.                      |
| <b>Financial Movement</b> | Movimiento Financiero      | Ingreso o egreso registrado por el ganadero para controlar la rentabilidad de su operación.                                                      |
| <b>Subscription Plan</b>  | Plan de Suscripción        | Opción de uso de la plataforma que define condiciones, beneficios y precio para el usuario.                                                      |
| <b>Payment</b>            | Pago                       | Registro económico asociado a la activación o renovación de un plan de suscripción dentro de AniTec.                                             |
| <b>Farm Management</b>    | Gestión de Hacienda        | Administración de los recursos, personal, animales, actividades y datos que ocurren dentro de la unidad productiva ganadera.                     |
| <b>Mobile Application</b> | Aplicación Móvil           | Software instalado en un smartphone o tablet que permite al ganadero o veterinario gestionar información del ganado desde el campo.              |
| <b>Native App</b>         | Aplicación Nativa          | Aplicación desarrollada específicamente para un sistema operativo móvil, como Android con Kotlin, aprovechando capacidades del dispositivo.      |
| <b>Cross-Platform App</b> | Aplicación Multiplataforma | Aplicación móvil que se ejecuta en más de un sistema operativo a partir de una base de código compartida, manteniendo una experiencia coherente. |
| <b>Push Notification</b>  | Notificación Push          | Mensaje enviado al dispositivo del usuario para alertar sobre vacunaciones, tratamientos, recordatorios o eventos sanitarios pendientes.         |
| <b>Offline Mode</b>       | Modo Sin Conexión          | Capacidad de la aplicación móvil para registrar o consultar información esencial aunque no exista conexión a internet estable.                   |
| <b>Local Persistence</b>  | Persistencia Local         | Almacenamiento de datos en el dispositivo móvil para conservar borradores, historiales básicos o registros pendientes de sincronización.         |
| <b>Synchronization</b>    | Sincronización             | Proceso mediante el cual los datos guardados en el dispositivo se actualizan con el servidor cuando se recupera la conectividad.                 |
| <b>Camera Capture</b>     | Captura con Cámara         | Uso de la cámara del dispositivo móvil para identificar animales, registrar evidencias o apoyar la búsqueda visual dentro de la aplicación.      |

| Term (English)               | Término (Español)         | Definition (Definición)                                                                                                                             |
|------------------------------|---------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Device Capability</b>     | Capacidad del Dispositivo | Función del hardware o sistema móvil utilizada por la app, como cámara, almacenamiento local, notificaciones o sensores.                            |
| <b>Mobile Session</b>        | Sesión Móvil              | Estado de autenticación del usuario dentro de la aplicación móvil que permite acceder a rutas protegidas y consumir la API de AniTec.               |
| <b>Mobile Dashboard</b>      | Panel Móvil               | Vista resumida en la aplicación móvil que muestra indicadores, alertas y accesos rápidos para facilitar decisiones en campo.                        |
| <b>Responsive Navigation</b> | Navegación Móvil          | Estructura de pantallas y menús adaptada a pantallas pequeñas para que el usuario complete tareas con pocos pasos desde el teléfono.                |
| <b>API Consumption</b>       | Consumo de API            | Comunicación de la aplicación móvil con el backend mediante servicios REST para enviar o recibir datos del dominio ganadero.                        |
| <b>Draft Record</b>          | Borrador                  | Registro temporal guardado en el dispositivo cuando el usuario aún no puede completar o sincronizar una operación por falta de red o tiempo.        |
| <b>Accessibility</b>         | Accesibilidad             | Conjunto de prácticas de diseño móvil que facilitan el uso de la aplicación a personas con distintos niveles de experiencia digital o limitaciones. |
| <b>Internationalization</b>  | Internacionalización      | Capacidad de la aplicación móvil para adaptar idioma, formatos y textos a diferentes contextos regionales sin cambiar la lógica del producto.       |

## 2.4. Requirements specification

Esta sección especifica los requisitos de la solución móvil de AniTec a partir de las entrevistas, los User Personas, el Needfinding y los supuestos Lean UX. El alcance comprende la landing page estática, la API REST propia, una aplicación Android nativa desarrollada con Kotlin y Jetpack Compose, y una aplicación multiplataforma desarrollada con Flutter y Dart. Los requisitos también incluyen almacenamiento local, uso de la cámara, notificaciones, integración con Stripe como servicio externo y un feature de aprendizaje autónomo basado en Google ML Kit Barcode Scanning.

La priorización utiliza MoSCoW: **Must Have** identifica capacidades necesarias para cumplir el alcance o completar los flujos core; **Should Have** corresponde a capacidades importantes que admiten una implementación posterior dentro del proyecto; **Could Have** representa capacidades complementarias cuya inclusión depende de la capacidad disponible. Todo elemento incluido en el Product Backlog deberá implementarse antes de la entrega final o retirarse formalmente del alcance.

### To-Be Scenario Mapping

El To-Be Scenario Mapping representa cómo realizarán sus tareas Jorge Luis Rivas, del segmento ganadero, y Valeria Mendoza, del segmento veterinario, cuando utilicen AniTec. Los escenarios incorporan el uso en campo, la conectividad intermitente, la identificación mediante cámara, los recordatorios y el acceso veterinario autorizado.

### To-Be Scenario Mapping: pequeño o mediano ganadero

**User Persona:** Jorge Luis Rivas **Escenario:** Registro y seguimiento móvil de un animal durante el trabajo de campo.

| FASES    | Revisar actividades y seleccionar la finca                                        | Identificar al animal                                     | Registrar información en campo                                                                      | Programar el seguimiento                                              | Sincronizar y compartir el historial                                                              |
|----------|-----------------------------------------------------------------------------------|-----------------------------------------------------------|-----------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| DOING    | Consulta sus actividades pendientes y los animales de la finca desde el teléfono. | Escanea el código QR del animal o lo busca manualmente.   | Registra una incidencia, control o actualización; si no hay conexión, guarda el trabajo localmente. | Define la próxima actividad y activa el recordatorio correspondiente. | Revisa los cambios pendientes y permite que se sincronicen cuando vuelve la conexión.             |
| THINKING | "Puedo saber qué debo atender antes de comenzar el recorrido".                    | "Necesito encontrar la ficha correcta sin perder tiempo". | "Quiero dejar el registro ahora para no olvidarlo después".                                         | "El teléfono debe ayudarme a recordar la siguiente acción".           | "Quiero confirmar que la información quedó guardada y disponible para el veterinario autorizado". |
| FEELING  | Orientado al tener una lista clara de pendientes.                                 | Confiado cuando identifica al animal correcto.            | Tranquilo al conservar el trabajo pese a la falta de conexión.                                      | Aliviado al delegar el recordatorio en la aplicación.                 | Seguro cuando distingue los datos sincronizados de los pendientes.                                |

### To-Be Scenario Mapping: veterinario de campo

**User Persona:** Valeria Mendoza **Escenario:** Consulta de antecedentes, atención y seguimiento de un paciente autorizado.

| FASES | Seleccionar un cliente autorizado                              | Identificar al paciente                                                     | Consultar antecedentes                                               | Registrar la atención                                                                      | Programar y sincronizar el seguimiento                                   |
|-------|----------------------------------------------------------------|-----------------------------------------------------------------------------|----------------------------------------------------------------------|--------------------------------------------------------------------------------------------|--------------------------------------------------------------------------|
| DOING | Consulta los ganaderos que mantienen una autorización vigente. | Escanea el código QR o busca al animal dentro de los pacientes del cliente. | Revisa el historial sanitario disponible y su última sincronización. | Registra diagnóstico, tratamiento y recomendaciones, incluso si la conexión se interrumpe. | Programa el próximo control y verifica la sincronización de la atención. |

|          |                                                              |                                                         |                                                                 |                                                           |                                                                         |
|----------|--------------------------------------------------------------|---------------------------------------------------------|-----------------------------------------------------------------|-----------------------------------------------------------|-------------------------------------------------------------------------|
| THINKING | "Debo consultar únicamente los clientes que me autorizaron". | "Necesito confirmar que atenderé al paciente correcto". | "Los antecedentes me ayudarán a sustentar la decisión clínica". | "La atención debe quedar registrada con mi autorización". | "El ganadero debe poder consultar las indicaciones y la próxima fecha". |
| FEELING  | Seguro al reconocer el alcance de su autorización.           | Confiado al verificar la identidad del animal.          | Preparado al disponer de antecedentes ordenados.                | Responsable al dejar un registro trazable.                | Satisfecho al mantener continuidad entre visitas.                       |

- Placeholder de captura:** Insertar aquí la captura de los To-Be Scenario Mapping elaborados en la herramienta seleccionada.
- Placeholder de enlace público:** Agregar aquí la URL pública de los To-Be Scenario Mapping.

### 2.4.1. User Stories

Las Epics y las historias siguientes describen resultados esperados para la landing page, las aplicaciones móviles y los servicios. Las User Stories evitan decisiones de interfaz en sus criterios de aceptación; las Technical Stories describen capacidades sin interacción directa y utilizan el rol Developer. Cada historia incluye escenarios comprobables en presente y tercera persona mediante Given-When-Then.

#### Epics

| Epic ID | Title                               | Description                                                                                                           |
|---------|-------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| EP-001  | Landing Page                        | Comunica la propuesta de valor, los segmentos atendidos y los canales de acceso a las aplicaciones móviles de AniTec. |
| EP-002  | Identity and Access Management      | Gestiona el registro, la autenticación, la sesión y la autorización de ganaderos y veterinarios.                      |
| EP-003  | Farm and Livestock Management       | Permite organizar fincas y registrar, consultar, actualizar y archivar animales desde las aplicaciones móviles.       |
| EP-004  | Sanitary Management                 | Centraliza incidencias, diagnósticos, tratamientos, controles e historiales sanitarios de los animales.               |
| EP-005  | Veterinary Collaboration            | Gestiona la relación autorizada entre ganaderos y veterinarios para el seguimiento de clientes y pacientes.           |
| EP-006  | Activities and Notifications        | Organiza actividades ganaderas y sanitarias y genera recordatorios en los dispositivos móviles.                       |
| EP-007  | Offline Storage and Synchronization | Mantiene información esencial y trabajo pendiente en el dispositivo y lo sincroniza cuando vuelve la conexión.        |

| Epic ID | Title                                              | Description                                                                                                               |
|---------|----------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| EP-008  | Animal Identification and Autonomous Feature       | Identifica animales mediante códigos QR y la cámara, incorporando una tecnología investigada de forma autónoma.           |
| EP-009  | Analytics and Reports                              | Presenta indicadores sanitarios y operativos relevantes para ganaderos y veterinarios.                                    |
| EP-010  | Financial and Subscription Management              | Permite consultar finanzas, planes, pagos y el estado de la suscripción mediante un servicio externo.                     |
| EP-011  | Accessibility, Internationalization and Resilience | Asegura una experiencia comprensible, accesible, internacionalizada y tolerante a errores.                                |
| EP-012  | Mobile Platforms and Services                      | Agrupar la arquitectura y las integraciones técnicas de Android nativo, Flutter, API REST, almacenamiento y distribución. |

## User Stories

| Story ID    | User                                       | Priority  | Epic   |
|-------------|--------------------------------------------|-----------|--------|
| US-001      | Visitante                                  | Must Have | EP-001 |
| Title       | Comprender la propuesta de valor de AniTec |           |        |
| Description |                                            |           |        |

Como visitante, quiero conocer el problema que resuelve AniTec y sus beneficios para determinar si la solución se relaciona con mis necesidades.

### Acceptance Criteria

#### Scenario 1: Propuesta disponible

**Given** el contenido público de AniTec está publicado

**When** el visitante accede a la dirección de la landing page

**Then** el sistema presenta la propuesta de valor y los beneficios principales para la gestión ganadera

#### Scenario 2: Contenido verificable

**Given** la startup todavía no dispone de resultados comerciales validados

**When** el visitante consulta cifras o testimonios

**Then** el sistema diferencia las metas y testimonios reales de cualquier información todavía no validada

| Story ID    | User                                      | Priority  | Epic   |
|-------------|-------------------------------------------|-----------|--------|
| US-002      | Visitante                                 | Must Have | EP-001 |
| Title       | Conocer las soluciones para cada segmento |           |        |
| Description |                                           |           |        |

Como visitante, quiero conocer cómo AniTec ayuda a ganaderos y veterinarios para identificar la aplicación móvil adecuada para mi perfil.

### Acceptance Criteria

#### Scenario 1: Información para ganaderos

**Given** el visitante pertenece al segmento ganadero

**When** el visitante consulta la información dirigida a su segmento

**Then** el sistema explica las capacidades móviles destinadas a la gestión del hato

#### Scenario 2: Información para veterinarios

**Given** el visitante pertenece al segmento veterinario

**When** el visitante consulta la información dirigida a su segmento

**Then** el sistema explica las capacidades de seguimiento de clientes y pacientes

| Story ID           | User                                                      | Priority    | Epic   |
|--------------------|-----------------------------------------------------------|-------------|--------|
| US-003             | Visitante                                                 | Should Have | EP-001 |
| <b>Title</b>       | Acceder a una landing page adaptable e internacionalizada |             |        |
| <b>Description</b> |                                                           |             |        |

Como visitante, quiero consultar la landing page desde distintos dispositivos y en un idioma disponible para comprender la información y acceder a los canales de contacto o descarga.

### Acceptance Criteria

#### Scenario 1: Adaptación al dispositivo

**Given** el visitante utiliza un teléfono, una tableta o una computadora

**When** el visitante accede a la landing page

**Then** el contenido conserva su legibilidad y permite completar las acciones principales

#### Scenario 2: Idioma disponible

**Given** la landing page ofrece más de un idioma

**When** el visitante selecciona un idioma

**Then** el sistema muestra el contenido traducido y conserva la preferencia durante la navegación

| Story ID           | User                              | Priority  | Epic   |
|--------------------|-----------------------------------|-----------|--------|
| US-004             | Ganadero o veterinario            | Must Have | EP-002 |
| <b>Title</b>       | Registrar una cuenta según el rol |           |        |
| <b>Description</b> |                                   |           |        |

Como ganadero o veterinario, quiero crear una cuenta con mi rol para acceder a las capacidades que corresponden a mi actividad.

### Acceptance Criteria

#### Scenario 1: Registro válido

**Given** la persona proporciona datos válidos y selecciona un rol permitido

**When** la persona solicita crear su cuenta

**Then** el sistema registra la cuenta y asocia el rol seleccionado

Scenario 2: Datos duplicados o inválidos

**Given** ya existe una cuenta con el mismo correo o los datos incumplen una regla

**When** la persona solicita crear su cuenta

**Then** el sistema rechaza el registro e informa la causa sin exponer información sensible

| Story ID                                                                                                                    | User               | Priority  | Epic   |
|-----------------------------------------------------------------------------------------------------------------------------|--------------------|-----------|--------|
| US-005                                                                                                                      | Usuario registrado | Must Have | EP-002 |
| Title                                                                                                                       | Iniciar sesión     |           |        |
| Description                                                                                                                 |                    |           |        |
| Como usuario registrado, quiero autenticarme con mis credenciales para consultar de manera segura mi información en AniTec. |                    |           |        |
| Acceptance Criteria                                                                                                         |                    |           |        |

Scenario 1: Credenciales válidas

**Given** existe una cuenta activa con las credenciales proporcionadas

**When** el usuario solicita iniciar sesión

**Then** el sistema autentica al usuario y habilita las capacidades de su rol

Scenario 2: Credenciales inválidas

**Given** las credenciales no corresponden a una cuenta activa

**When** el usuario solicita iniciar sesión

**Then** el sistema rechaza el acceso mediante un mensaje que no revela qué dato es incorrecto

| Story ID | User                                                                                                                                      | Priority  | Epic   |
|----------|-------------------------------------------------------------------------------------------------------------------------------------------|-----------|--------|
| US-006   | Usuario autenticado                                                                                                                       | Must Have | EP-002 |
| Title    | Mantener y finalizar la sesión móvil                                                                                                      |           |        |
|          | Description                                                                                                                               |           |        |
|          | Como usuario autenticado, quiero conservar mi sesión de manera segura y poder finalizarla para evitar accesos no autorizados a mis datos. |           |        |
|          | Acceptance Criteria                                                                                                                       |           |        |

Scenario 1: Sesión vigente

**Given** el usuario tiene una sesión válida almacenada de forma segura

**When** el usuario vuelve a abrir la aplicación

**Then** el sistema recupera la sesión y valida su vigencia antes de entregar información protegida

Scenario 2: Cierre de sesión

**Given** el usuario mantiene una sesión activa

**When** el usuario solicita finalizarla

**Then** el sistema elimina las credenciales locales y bloquea el acceso a la información protegida

| Story ID | User                | Priority  | Epic   |
|----------|---------------------|-----------|--------|
| US-007   | Usuario autenticado | Must Have | EP-002 |

| Title                                                                                                                                                        | Acceder únicamente a información autorizada |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------|
| Description                                                                                                                                                  |                                             |
| Como usuario autenticado, quiero acceder solamente a los datos permitidos para mi rol y relaciones vigentes para proteger la información ganadera y clínica. |                                             |
| Acceptance Criteria                                                                                                                                          |                                             |

**Scenario 1: Acceso permitido**

- Given** el usuario posee el rol y la relación requeridos por una operación
- When** el usuario solicita consultar o modificar información
- Then** el sistema procesa la operación dentro del alcance autorizado

**Scenario 2: Acceso denegado**

- Given** el usuario no posee el rol o la relación requeridos
- When** el usuario solicita consultar o modificar información protegida
- Then** el sistema rechaza la operación y registra el resultado de autorización

| Story ID    | User                             | Priority  | Epic   |
|-------------|----------------------------------|-----------|--------|
| US-008      | Ganadero                         | Must Have | EP-003 |
| Title       | Consultar las fincas registradas |           |        |
| Description |                                  |           |        |

Como ganadero, quiero consultar mis fincas para organizar los animales según su unidad productiva.

| Acceptance Criteria                                                            |  |  |  |
|--------------------------------------------------------------------------------|--|--|--|
| <b>Scenario 1: Fincas existentes</b>                                           |  |  |  |
| <b>Given</b> el ganadero tiene una o más fincas registradas                    |  |  |  |
| <b>When</b> el ganadero consulta sus fincas                                    |  |  |  |
| <b>Then</b> el sistema devuelve únicamente las fincas que le pertenecen        |  |  |  |
| <b>Scenario 2: Sin fincas</b>                                                  |  |  |  |
| <b>Given</b> el ganadero todavía no registra fincas                            |  |  |  |
| <b>When</b> el ganadero consulta sus fincas                                    |  |  |  |
| <b>Then</b> el sistema informa que no existen unidades productivas registradas |  |  |  |

| Story ID    | User                             | Priority  | Epic   |
|-------------|----------------------------------|-----------|--------|
| US-009      | Ganadero                         | Must Have | EP-003 |
| Title       | Registrar y actualizar una finca |           |        |
| Description |                                  |           |        |

Como ganadero, quiero registrar y mantener los datos de una finca para asociar correctamente mis animales y actividades.

| Acceptance Criteria |  |  |  |
|---------------------|--|--|--|
|---------------------|--|--|--|

Scenario 1: Registro válido

**Given** el ganadero proporciona los datos obligatorios de una nueva finca

**When** el ganadero solicita registrarla

**Then** el sistema crea la finca y la deja disponible para asociar animales

Scenario 2: Actualización válida

**Given** existe una finca perteneciente al ganadero

**When** el ganadero modifica información permitida

**Then** el sistema conserva los cambios y la relación con sus animales

| Story ID                                                                                                                 | User                        | Priority  | Epic   |
|--------------------------------------------------------------------------------------------------------------------------|-----------------------------|-----------|--------|
| US-010                                                                                                                   | Ganadero                    | Must Have | EP-003 |
| Title                                                                                                                    | Consultar y buscar animales |           |        |
| Description                                                                                                              |                             |           |        |
| Como ganadero, quiero consultar y buscar los animales de mis fincas para localizar rápidamente el registro que necesito. |                             |           |        |
| Acceptance Criteria                                                                                                      |                             |           |        |

Scenario 1: Consulta autorizada

**Given** el ganadero tiene animales registrados

**When** el ganadero consulta sus animales

**Then** el sistema devuelve únicamente animales asociados a sus fincas

Scenario 2: Búsqueda

**Given** existen animales que coinciden con un código, nombre, especie o raza

**When** el ganadero realiza una búsqueda

**Then** el sistema devuelve los animales coincidentes

| Story ID                                                                                                            | User                | Priority  | Epic   |
|---------------------------------------------------------------------------------------------------------------------|---------------------|-----------|--------|
| US-011                                                                                                              | Ganadero            | Must Have | EP-003 |
| Title                                                                                                               | Registrar un animal |           |        |
| Description                                                                                                         |                     |           |        |
| Como ganadero, quiero registrar un animal en una de mis fincas para iniciar su trazabilidad sanitaria y productiva. |                     |           |        |
| Acceptance Criteria                                                                                                 |                     |           |        |

Scenario 1: Registro válido

**Given** el ganadero tiene una finca y proporciona los datos obligatorios del animal

**When** el ganadero solicita registrar el animal

**Then** el sistema crea un identificador único y asocia el animal con la finca

Scenario 2: Código duplicado

**Given** ya existe un animal del ganadero con el mismo código de identificación

**When** el ganadero solicita registrar otro animal con ese código

**Then** el sistema rechaza la operación e informa la duplicidad

| Story ID                                                                                                           | User                            | Priority  | Epic   |
|--------------------------------------------------------------------------------------------------------------------|---------------------------------|-----------|--------|
| US-012                                                                                                             | Ganadero                        | Must Have | EP-003 |
| Title                                                                                                              | Actualizar o archivar un animal |           |        |
| Description                                                                                                        |                                 |           |        |
| Como ganadero, quiero actualizar o archivar un animal para mantener vigente el inventario sin perder su historial. |                                 |           |        |
| Acceptance Criteria                                                                                                |                                 |           |        |

#### Scenario 1: Actualización válida

**Given** el animal pertenece al ganadero

**When** el ganadero modifica datos permitidos

**Then** el sistema conserva los cambios y mantiene el historial asociado

#### Scenario 2: Archivado

**Given** el animal ya no forma parte del hato activo

**When** el ganadero solicita archivarlo

**Then** el sistema lo excluye del inventario activo y conserva su historial para consulta

| Story ID                                                                                                                                   | User                              | Priority  | Epic   |
|--------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------|-----------|--------|
| US-013                                                                                                                                     | Usuario autorizado                | Must Have | EP-003 |
| Title                                                                                                                                      | Consultar el detalle de un animal |           |        |
| Description                                                                                                                                |                                   |           |        |
| Como usuario autorizado, quiero consultar la ficha de un animal para conocer sus datos e historial relevante antes de realizar una acción. |                                   |           |        |
| Acceptance Criteria                                                                                                                        |                                   |           |        |

#### Scenario 1: Detalle autorizado

**Given** el usuario está autorizado para consultar el animal

**When** el usuario solicita su detalle

**Then** el sistema devuelve los datos generales y las referencias a su historial

#### Scenario 2: Animal no autorizado

**Given** el usuario no tiene relación autorizada con el animal

**When** el usuario solicita su detalle

**Then** el sistema rechaza la consulta sin revelar información del animal

| Story ID    | User                         | Priority  | Epic   |
|-------------|------------------------------|-----------|--------|
| US-014      | Usuario autorizado           | Must Have | EP-004 |
| Title       | Consultar eventos sanitarios |           |        |
| Description |                              |           |        |

Como usuario autorizado, quiero consultar los eventos sanitarios de los animales a mi alcance para conocer su situación de salud.

Acceptance Criteria

Scenario 1: Eventos existentes

- Given** existen eventos sanitarios de animales autorizados
- When** el usuario consulta los eventos
- Then** el sistema devuelve la fecha, el tipo, el animal y el responsable de cada registro

Scenario 2: Sin eventos

- Given** no existen eventos sanitarios dentro del alcance autorizado
- When** el usuario realiza la consulta
- Then** el sistema informa que no existen registros disponibles

| Story ID | User                               | Priority  | Epic   |
|----------|------------------------------------|-----------|--------|
| US-015   | Ganadero                           | Must Have | EP-004 |
| Title    | Registrar una incidencia sanitaria |           |        |
|          | Description                        |           |        |

Como ganadero, quiero registrar una incidencia observada en uno de mis animales para dejar evidencia y solicitar seguimiento oportuno.

Acceptance Criteria

Scenario 1: Incidencia válida

- Given** el animal pertenece al ganadero y los datos obligatorios están completos
- When** el ganadero registra la incidencia
- Then** el sistema incorpora el evento al historial del animal

Scenario 2: Animal ajeno

- Given** el animal no pertenece al ganadero
- When** el ganadero intenta registrar una incidencia
- Then** el sistema rechaza la operación

| Story ID    | User                                | Priority  | Epic   |
|-------------|-------------------------------------|-----------|--------|
| US-016      | Veterinario                         | Must Have | EP-004 |
| Title       | Registrar diagnóstico y tratamiento |           |        |
| Description |                                     |           |        |

Como veterinario, quiero registrar el diagnóstico, tratamiento y recomendaciones de un paciente autorizado para documentar la atención realizada.

Acceptance Criteria

Scenario 1: Atención autorizada

- Given** el paciente pertenece a un cliente que autorizó al veterinario
- When** el veterinario registra una atención con datos válidos

**Then** el sistema incorpora el registro al historial e identifica al profesional responsable

### Scenario 2: Atención no autorizada

**Given** el veterinario no tiene acceso vigente al paciente

**When** el veterinario intenta registrar una atención

**Then** el sistema rechaza la operación

| Story ID                                                                                                                                           | User                                          | Priority  | Epic   |
|----------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------|-----------|--------|
| US-017                                                                                                                                             | Usuario autorizado                            | Must Have | EP-004 |
| Title                                                                                                                                              | Consultar el historial sanitario de un animal |           |        |
| Description                                                                                                                                        |                                               |           |        |
| Como usuario autorizado, quiero consultar cronológicamente el historial sanitario de un animal para tomar decisiones con base en sus antecedentes. |                                               |           |        |
| Acceptance Criteria                                                                                                                                |                                               |           |        |

### Scenario 1: Historial disponible

**Given** el animal tiene eventos sanitarios y el usuario posee acceso

**When** el usuario consulta el historial

**Then** el sistema devuelve los eventos ordenados y con su responsable

### Scenario 2: Conectividad interrumpida

**Given** existe una copia local vigente del historial autorizado

**When** el usuario consulta el historial sin conexión

**Then** el sistema entrega la información disponible e indica cuándo fue actualizada

| Story ID                                                                                                                                                 | User                                            | Priority    | Epic   |
|----------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------|-------------|--------|
| US-018                                                                                                                                                   | Autor de un registro sanitario                  | Should Have | EP-004 |
| Title                                                                                                                                                    | Corregir un registro sanitario con trazabilidad |             |        |
| Description                                                                                                                                              |                                                 |             |        |
| Como autor de un registro sanitario, quiero corregir información incorrecta dejando constancia del cambio para preservar la confiabilidad del historial. |                                                 |             |        |
| Acceptance Criteria                                                                                                                                      |                                                 |             |        |

### Scenario 1: Corrección permitida

**Given** el usuario es autor del registro y proporciona el motivo

**When** el usuario solicita corregir información permitida

**Then** el sistema guarda la nueva versión e identifica la fecha, el autor y el motivo

### Scenario 2: Eliminación no permitida

**Given** el registro clínico ya forma parte del historial

**When** el usuario intenta eliminarlo definitivamente

**Then** el sistema conserva el registro y ofrece el mecanismo de corrección o anulación trazable

| Story ID                                                                                                           | User                                     | Priority    | Epic   |
|--------------------------------------------------------------------------------------------------------------------|------------------------------------------|-------------|--------|
| US-019                                                                                                             | Veterinario                              | Should Have | EP-004 |
| Title                                                                                                              | Programar un control sanitario posterior |             |        |
| Description                                                                                                        |                                          |             |        |
| Como veterinario, quiero programar el próximo control de un paciente para dar continuidad al tratamiento indicado. |                                          |             |        |
| Acceptance Criteria                                                                                                |                                          |             |        |

- Scenario 1: Control programado**
- Given** el veterinario registra una atención autorizada
- When** el veterinario establece una fecha futura de control
- Then** el sistema crea la actividad asociada al paciente y al cliente
- Scenario 2: Fecha inválida**
- Given** la fecha propuesta no es posterior a la atención
- When** el veterinario solicita programar el control
- Then** el sistema rechaza la programación e informa la regla incumplida

| Story ID                                                                                                                                        | User                                             | Priority  | Epic   |
|-------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------|-----------|--------|
| US-020                                                                                                                                          | Ganadero                                         | Must Have | EP-005 |
| Title                                                                                                                                           | Recibir una solicitud de seguimiento veterinario |           |        |
| Description                                                                                                                                     |                                                  |           |        |
| Como ganadero, quiero recibir solicitudes de veterinarios para decidir quién puede consultar y registrar información sanitaria de mis animales. |                                                  |           |        |
| Acceptance Criteria                                                                                                                             |                                                  |           |        |

- Scenario 1: Solicitud válida**
- Given** un veterinario registrado solicita relacionarse con el ganadero
- When** el sistema procesa la solicitud
- Then** el ganadero recibe la solicitud con la identidad del profesional y su estado pendiente
- Scenario 2: Solicitud duplicada**
- Given** ya existe una solicitud pendiente o una relación vigente
- When** el mismo veterinario envía otra solicitud
- Then** el sistema evita crear una relación duplicada

| Story ID | User                                  | Priority  | Epic   |
|----------|---------------------------------------|-----------|--------|
| US-021   | Ganadero                              | Must Have | EP-005 |
| Title    | Aceptar o rechazar acceso veterinario |           |        |
|          | Description                           |           |        |

Como ganadero, quiero aceptar o rechazar una solicitud veterinaria para controlar el acceso a la información de mis animales.

#### Acceptance Criteria

##### Scenario 1: Solicitud aceptada

**Given** existe una solicitud pendiente

**When** el ganadero la acepta

**Then** el sistema activa la relación y autoriza al veterinario dentro del alcance definido

##### Scenario 2: Solicitud rechazada

**Given** existe una solicitud pendiente

**When** el ganadero la rechaza

**Then** el sistema cierra la solicitud sin conceder acceso

| Story ID    | User                                | Priority  | Epic   |
|-------------|-------------------------------------|-----------|--------|
| US-022      | Ganadero                            | Must Have | EP-005 |
| Title       | Revocar el acceso de un veterinario |           |        |
| Description |                                     |           |        |

Como ganadero, quiero revocar una autorización veterinaria para impedir futuras consultas o registros sobre mis animales.

#### Acceptance Criteria

##### Scenario 1: Revocación válida

**Given** existe una relación veterinaria vigente

**When** el ganadero revoca la autorización

**Then** el sistema impide nuevas operaciones del veterinario y conserva la autoría de registros anteriores

##### Scenario 2: Relación inexistente

**Given** no existe una relación vigente con el veterinario

**When** el ganadero solicita revocarla

**Then** el sistema informa que no existe acceso activo

| Story ID    | User                                       | Priority  | Epic   |
|-------------|--------------------------------------------|-----------|--------|
| US-023      | Veterinario                                | Must Have | EP-005 |
| Title       | Consultar clientes y pacientes autorizados |           |        |
| Description |                                            |           |        |

Como veterinario, quiero consultar mis clientes y sus pacientes autorizados para organizar las atenciones de campo.

#### Acceptance Criteria

##### Scenario 1: Clientes autorizados

**Given** el veterinario tiene relaciones vigentes

**When** el veterinario consulta sus clientes

**Then** el sistema devuelve únicamente los ganaderos y pacientes autorizados

**Scenario 2: Sin clientes**

**Given** el veterinario no tiene relaciones vigentes

**When** el veterinario consulta sus clientes

**Then** el sistema informa que no existen clientes autorizados

| Story ID                                                                                                                   | User                                             | Priority  | Epic   |
|----------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------|-----------|--------|
| US-024                                                                                                                     | Veterinario                                      | Must Have | EP-005 |
| <b>Title</b>                                                                                                               | Consultar antecedentes de un paciente autorizado |           |        |
| <b>Description</b>                                                                                                         |                                                  |           |        |
| Como veterinario, quiero consultar los antecedentes de un paciente autorizado para sustentar el diagnóstico y seguimiento. |                                                  |           |        |
| <b>Acceptance Criteria</b>                                                                                                 |                                                  |           |        |

**Scenario 1: Acceso vigente**

**Given** el ganadero mantiene autorización vigente

**When** el veterinario consulta los antecedentes del paciente

**Then** el sistema devuelve la información sanitaria permitida

**Scenario 2: Acceso revocado**

**Given** el ganadero revocó la autorización

**When** el veterinario vuelve a solicitar los antecedentes

**Then** el sistema rechaza la consulta

| Story ID                                                                                                               | User                              | Priority  | Epic   |
|------------------------------------------------------------------------------------------------------------------------|-----------------------------------|-----------|--------|
| US-025                                                                                                                 | Usuario autenticado               | Must Have | EP-006 |
| Title                                                                                                                  | Consultar actividades programadas |           |        |
| Description                                                                                                            |                                   |           |        |
| Como usuario autenticado, quiero consultar mis actividades ganaderas o sanitarias para organizar el trabajo pendiente. |                                   |           |        |
| Acceptance Criteria                                                                                                    |                                   |           |        |

**Scenario 1: Actividades existentes**

**Given** el usuario tiene actividades dentro de un periodo

**When** el usuario consulta su programación

**Then** el sistema devuelve las actividades ordenadas por fecha y prioridad

**Scenario 2: Actividades por rol**

**Given** existen actividades de distintos propietarios o profesionales

**When** el usuario realiza la consulta

**Then** el sistema devuelve solamente las actividades dentro de su alcance

| Story ID | User | Priority | Epic |
|----------|------|----------|------|
|----------|------|----------|------|

|                                                                                                                                         |                                        |             |        |
|-----------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------|-------------|--------|
| US-026                                                                                                                                  | Usuario autenticado                    | Must Have   | EP-006 |
| Title                                                                                                                                   | Gestionar una actividad o recordatorio |             |        |
| Description                                                                                                                             |                                        |             |        |
| Como usuario autenticado, quiero crear, actualizar o cancelar una actividad para mantener vigente mi planificación.                     |                                        |             |        |
| Acceptance Criteria                                                                                                                     |                                        |             |        |
| Scenario 1: Actividad válida                                                                                                            |                                        |             |        |
| Given el usuario proporciona una fecha y datos válidos                                                                                  |                                        |             |        |
| When el usuario guarda una actividad                                                                                                    |                                        |             |        |
| Then el sistema registra la actividad y programa el recordatorio cuando corresponde                                                     |                                        |             |        |
| Scenario 2: Cancelación                                                                                                                 |                                        |             |        |
| Given existe una actividad pendiente perteneciente al usuario                                                                           |                                        |             |        |
| When el usuario solicita cancelarla                                                                                                     |                                        |             |        |
| Then el sistema cambia su estado y evita recordatorios posteriores                                                                      |                                        |             |        |
| Story ID                                                                                                                                | User                                   | Priority    | Epic   |
| US-027                                                                                                                                  | Usuario autenticado                    | Must Have   | EP-006 |
| Title                                                                                                                                   | Recibir una notificación de actividad  |             |        |
| Description                                                                                                                             |                                        |             |        |
| Como usuario autenticado, quiero recibir notificaciones de actividades próximas para reducir olvidos de controles y tareas importantes. |                                        |             |        |
| Acceptance Criteria                                                                                                                     |                                        |             |        |
| Scenario 1: Permiso concedido                                                                                                           |                                        |             |        |
| Given existe una actividad pendiente y el dispositivo permite notificaciones                                                            |                                        |             |        |
| When llega el momento configurado                                                                                                       |                                        |             |        |
| Then el sistema entrega una notificación relacionada con la actividad                                                                   |                                        |             |        |
| Scenario 2: Permiso denegado                                                                                                            |                                        |             |        |
| Given el dispositivo no permite notificaciones                                                                                          |                                        |             |        |
| When se aproxima una actividad                                                                                                          |                                        |             |        |
| Then el sistema conserva la actividad para consulta y comunica que las notificaciones están deshabilitadas                              |                                        |             |        |
| Story ID                                                                                                                                | User                                   | Priority    | Epic   |
| US-028                                                                                                                                  | Usuario autenticado                    | Should Have | EP-006 |
| Title                                                                                                                                   | Atender o reprogramar una actividad    |             |        |
| Description                                                                                                                             |                                        |             |        |
| Como usuario autenticado, quiero marcar una actividad como atendida o reprogramarla para mantener actualizado su seguimiento.           |                                        |             |        |
| Acceptance Criteria                                                                                                                     |                                        |             |        |

Scenario 1: Actividad atendida

**Given** existe una actividad pendiente

**When** el usuario registra que fue atendida

**Then** el sistema actualiza su estado y conserva la fecha de atención

Scenario 2: Actividad reprogramada

**Given** existe una actividad pendiente

**When** el usuario establece una nueva fecha válida

**Then** el sistema actualiza la programación y reemplaza el recordatorio anterior

| Story ID                                                                                                                                              | User                                        | Priority  | Epic   |
|-------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------|-----------|--------|
| US-029                                                                                                                                                | Usuario autenticado                         | Must Have | EP-007 |
| Title                                                                                                                                                 | Consultar información esencial sin conexión |           |        |
| Description                                                                                                                                           |                                             |           |        |
| Como usuario autenticado, quiero consultar información esencial previamente sincronizada cuando no tengo conexión para continuar mi trabajo en campo. |                                             |           |        |
| Acceptance Criteria                                                                                                                                   |                                             |           |        |

Scenario 1: Datos locales disponibles

**Given** el dispositivo conserva datos autorizados y no existe conexión

**When** el usuario consulta fincas, animales, actividades o historiales disponibles

**Then** el sistema entrega la copia local e informa su última sincronización

Scenario 2: Datos locales inexistentes

**Given** no existe conexión ni una copia local de la información solicitada

**When** el usuario realiza la consulta

**Then** el sistema informa que los datos requieren una sincronización inicial

| Story ID                                                                                                                                        | User                                   | Priority  | Epic   |
|-------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------|-----------|--------|
| US-030                                                                                                                                          | Usuario autenticado                    | Must Have | EP-007 |
| Title                                                                                                                                           | Guardar trabajo pendiente sin conexión |           |        |
| Description                                                                                                                                     |                                        |           |        |
| Como usuario autenticado, quiero guardar localmente un registro pendiente cuando no tengo conexión para evitar perder la información ingresada. |                                        |           |        |
| Acceptance Criteria                                                                                                                             |                                        |           |        |

Scenario 1: Guardado local

**Given** la conexión no está disponible y los datos cumplen las reglas locales

**When** el usuario guarda un animal, incidencia, atención o actividad

**Then** el sistema conserva la operación como pendiente de sincronización

Scenario 2: Datos inválidos

**Given** los datos incumplen una regla que puede validarse localmente  
**When** el usuario intenta guardar la operación  
**Then** el sistema no la incorpora a la cola y explica la corrección necesaria

| Story ID    | User                               | Priority  | Epic   |
|-------------|------------------------------------|-----------|--------|
| US-031      | Usuario autenticado                | Must Have | EP-007 |
| Title       | Sincronizar operaciones pendientes |           |        |
| Description |                                    |           |        |

Como usuario autenticado, quiero sincronizar el trabajo pendiente al recuperar la conexión para mantener consistentes el dispositivo y el servidor.

#### Acceptance Criteria

##### Scenario 1: Sincronización exitosa

**Given** existen operaciones locales pendientes y la conexión vuelve a estar disponible  
**When** el sistema inicia la sincronización  
**Then** el servidor procesa cada operación una sola vez y el dispositivo actualiza su estado

##### Scenario 2: Sincronización parcial

**Given** una operación es rechazada y otras son válidas  
**When** el sistema procesa la cola  
**Then** el sistema conserva como pendiente solo la operación rechazada e informa su causa

| Story ID           | User                                            | Priority    | Epic   |
|--------------------|-------------------------------------------------|-------------|--------|
| US-032             | Usuario autenticado                             | Should Have | EP-007 |
| <b>Title</b>       | Resolver errores o conflictos de sincronización |             |        |
| <b>Description</b> |                                                 |             |        |

Como usuario autenticado, quiero conocer y resolver los conflictos de sincronización para evitar sobrescribir información válida.

#### Acceptance Criteria

##### Scenario 1: Conflicto detectado

**Given** el registro local y el remoto fueron modificados después de la última sincronización  
**When** el sistema intenta sincronizarlos  
**Then** el sistema conserva ambas referencias y solicita aplicar una regla de resolución válida

##### Scenario 2: Reintento seguro

**Given** una operación falló por una causa temporal  
**When** el usuario o el sistema reintenta la sincronización  
**Then** el servidor evita duplicar la operación ya procesada

| Story ID | User               | Priority  | Epic   |
|----------|--------------------|-----------|--------|
| US-033   | Usuario autorizado | Must Have | EP-008 |

| Title               | Identificar un animal mediante código QR                                                                          |
|---------------------|-------------------------------------------------------------------------------------------------------------------|
| Description         | Como usuario autorizado, quiero escanear el código QR de un animal con la cámara para abrir rápidamente su ficha. |
| Acceptance Criteria |                                                                                                                   |

Scenario 1: Código reconocido

- Given** el dispositivo concede acceso a la cámara y el código pertenece a un animal autorizado
- When** el usuario realiza el escaneo
- Then** el sistema identifica al animal y recupera su información disponible

Scenario 2: Código no autorizado

- Given** el código corresponde a un animal fuera del alcance del usuario
- When** el usuario realiza el escaneo
- Then** el sistema rechaza la consulta sin revelar datos del animal

| Story ID | User               | Priority  | Epic   |
|----------|--------------------|-----------|--------|
| US-034   | Usuario autorizado | Must Have | EP-008 |

| Title               | Identificar un animal sin utilizar la cámara                                                                                               |
|---------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| Description         | Como usuario autorizado, quiero buscar manualmente un animal cuando la cámara o el código QR no estén disponibles para continuar la tarea. |
| Acceptance Criteria |                                                                                                                                            |

Scenario 1: Permiso de cámara denegado

- Given** el dispositivo no concede acceso a la cámara
- When** el usuario intenta iniciar la identificación
- Then** el sistema mantiene disponible la búsqueda por código o nombre

Scenario 2: Código ilegible

- Given** la cámara no reconoce un código válido
- When** el usuario concluye el intento de escaneo
- Then** el sistema informa el resultado y permite identificar el animal manualmente

| Story ID | User     | Priority    | Epic   |
|----------|----------|-------------|--------|
| US-035   | Ganadero | Should Have | EP-009 |

| Title               | Consultar indicadores del hato                                                                                                     |
|---------------------|------------------------------------------------------------------------------------------------------------------------------------|
| Description         | Como ganadero, quiero consultar indicadores de mis animales, actividades y eventos sanitarios para priorizar decisiones de manejo. |
| Acceptance Criteria |                                                                                                                                    |

Scenario 1: Indicadores con datos

**Given** el ganadero tiene información registrada

**When** el ganadero consulta sus indicadores

**Then** el sistema calcula resultados únicamente con datos de sus fincas y comunica el periodo considerado

Scenario 2: Indicadores sin datos suficientes

**Given** no existen datos suficientes para calcular un indicador

**When** el ganadero realiza la consulta

**Then** el sistema informa la falta de información sin presentar resultados engañosos

| Story ID                                                                                                                               | User                                         | Priority    | Epic   |
|----------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------|-------------|--------|
| US-036                                                                                                                                 | Veterinario                                  | Should Have | EP-009 |
| Title                                                                                                                                  | Consultar indicadores sanitarios de clientes |             |        |
| Description                                                                                                                            |                                              |             |        |
| Como veterinario, quiero consultar indicadores sanitarios de mis clientes autorizados para priorizar pacientes y controles pendientes. |                                              |             |        |
| Acceptance Criteria                                                                                                                    |                                              |             |        |

Scenario 1: Indicadores autorizados

**Given** el veterinario tiene clientes vigentes con información sanitaria

**When** el veterinario consulta los indicadores

**Then** el sistema calcula resultados únicamente con los datos autorizados

Scenario 2: Autorización revocada

**Given** un cliente deja de autorizar al veterinario

**When** el veterinario actualiza la consulta

**Then** el sistema excluye los datos de ese cliente

| Story ID                                                                                                                       | User                                          | Priority   | Epic   |
|--------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------|------------|--------|
| US-037                                                                                                                         | Ganadero                                      | Could Have | EP-010 |
| Title                                                                                                                          | Registrar y consultar movimientos financieros |            |        |
| Description                                                                                                                    |                                               |            |        |
| Como ganadero, quiero registrar ingresos y egresos y consultar mi balance para conocer el resultado económico de mi operación. |                                               |            |        |
| Acceptance Criteria                                                                                                            |                                               |            |        |

Scenario 1: Movimiento válido

**Given** el ganadero proporciona tipo, monto, categoría y fecha válidos

**When** el ganadero registra el movimiento

**Then** el sistema incorpora el movimiento y actualiza el balance correspondiente

Scenario 2: Consulta por periodo

**Given** existen movimientos del ganadero en distintos periodos

**When** el ganadero consulta un periodo

**Then** el sistema calcula ingresos, egresos y balance únicamente con sus movimientos

| Story ID    | User                            | Priority  | Epic   |
|-------------|---------------------------------|-----------|--------|
| US-038      | Usuario autenticado             | Must Have | EP-010 |
| Title       | Consultar planes de suscripción |           |        |
| Description |                                 |           |        |

Como usuario autenticado, quiero comparar los planes vigentes para seleccionar una alternativa acorde con mi operación.

#### Acceptance Criteria

##### Scenario 1: Planes disponibles

**Given** existen planes activos definidos por AniTec

**When** el usuario consulta las alternativas

**Then** el sistema devuelve precio, moneda, periodicidad, límites y capacidades de cada plan

##### Scenario 2: Precio por validar

**Given** la startup todavía no aprueba un precio comercial

**When** el usuario consulta un entorno de validación

**Then** el sistema identifica claramente la información como propuesta o prueba

| Story ID           | User                                          | Priority  | Epic   |
|--------------------|-----------------------------------------------|-----------|--------|
| US-039             | Usuario autenticado                           | Must Have | EP-010 |
| <b>Title</b>       | Iniciar un pago mediante un proveedor externo |           |        |
| <b>Description</b> |                                               |           |        |

Como usuario autenticado, quiero iniciar el pago de un plan mediante un proveedor externo para contratar una suscripción sin entregar datos bancarios directamente a AniTec.

#### Acceptance Criteria

##### Scenario 1: Sesión de pago creada

**Given** el usuario selecciona un plan activo

**When** el usuario solicita continuar con el pago

**Then** el sistema crea una sesión asociada al usuario y transfiere el proceso al proveedor externo

##### Scenario 2: Solicitud repetida

**Given** ya existe una operación en proceso para la misma referencia

**When** el usuario repite la solicitud

**Then** el sistema evita registrar cobros duplicados

| Story ID | User                | Priority  | Epic   |
|----------|---------------------|-----------|--------|
| US-040   | Usuario autenticado | Must Have | EP-010 |

| Title                                                                                                                                                     | Consultar el resultado del pago y la suscripción |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------|
| Description                                                                                                                                               |                                                  |
| Como usuario autenticado, quiero consultar el resultado de mis pagos y el estado de mi suscripción para conocer las capacidades disponibles en mi cuenta. |                                                  |
| Acceptance Criteria                                                                                                                                       |                                                  |

**Scenario 1: Pago confirmado**  
**Given** el proveedor externo confirma una operación válida  
**When** el sistema procesa la confirmación  
**Then** el sistema activa o actualiza la suscripción una sola vez y conserva la referencia del pago

**Scenario 2: Pago cancelado o fallido**  
**Given** el proveedor externo no confirma la operación  
**When** el usuario vuelve a AniTec  
**Then** el sistema conserva la suscripción anterior e informa el estado sin registrar un pago exitoso

| Story ID | User    | Priority    | Epic   |
|----------|---------|-------------|--------|
| US-041   | Usuario | Should Have | EP-011 |

| Title                                                                                                                            | Cambiar el idioma de la aplicación |
|----------------------------------------------------------------------------------------------------------------------------------|------------------------------------|
| Description                                                                                                                      |                                    |
| Como usuario, quiero elegir un idioma disponible para comprender la información y realizar mis tareas con términos consistentes. |                                    |
| Acceptance Criteria                                                                                                              |                                    |

**Scenario 1: Cambio de idioma**  
**Given** la aplicación dispone de las traducciones requeridas  
**When** el usuario selecciona otro idioma  
**Then** el sistema actualiza los textos y formatos localizables

**Scenario 2: Preferencia conservada**  
**Given** el usuario eligió un idioma  
**When** el usuario vuelve a abrir la aplicación  
**Then** el sistema aplica la preferencia almacenada

| Story ID | User    | Priority  | Epic   |
|----------|---------|-----------|--------|
| US-042   | Usuario | Must Have | EP-011 |

| Title                                                                                                                                              | Utilizar la aplicación con necesidades de accesibilidad |
|----------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------|
| Description                                                                                                                                        |                                                         |
| Como usuario con necesidades de accesibilidad, quiero percibir y comprender el contenido para completar las tareas principales de manera autónoma. |                                                         |
| Acceptance Criteria                                                                                                                                |                                                         |

---

**Scenario 1: Escalado de texto**

**Given** el dispositivo utiliza un tamaño de fuente mayor

**When** el usuario accede a una tarea principal

**Then** el contenido mantiene su legibilidad y no oculta información necesaria

**Scenario 2: Tecnología de asistencia**

**Given** el usuario utiliza un lector de pantalla

**When** el usuario recorre la información y las acciones principales

**Then** el sistema comunica nombres, estados y propósitos comprensibles

| Story ID                                                                                                                                 | User                                         | Priority  | Epic   |
|------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------|-----------|--------|
| US-043                                                                                                                                   | Usuario                                      | Must Have | EP-011 |
| Title                                                                                                                                    | Comprender errores y estados de conectividad |           |        |
| Description                                                                                                                              |                                              |           |        |
| Como usuario, quiero recibir información clara ante errores o cambios de conectividad para decidir cómo continuar sin perder mi trabajo. |                                              |           |        |
| Acceptance Criteria                                                                                                                      |                                              |           |        |

---

**Scenario 1: Error recuperable**

**Given** una operación falla por una causa temporal

**When** el sistema recibe el error

**Then** el sistema conserva el trabajo posible y comunica una acción de recuperación

**Scenario 2: Cambio de conectividad**

**Given** el dispositivo pierde o recupera la conexión

**When** el sistema detecta el cambio

**Then** el sistema actualiza el estado de trabajo local y sincronización sin afirmar que una operación terminó antes de confirmarla

**Technical Stories**

| Story ID                                                                                                                                                                   | User                                    | Priority  | Epic   |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------|-----------|--------|
| TS-001                                                                                                                                                                     | Developer                               | Must Have | EP-012 |
| Title                                                                                                                                                                      | Configurar la aplicación Android nativa |           |        |
| Description                                                                                                                                                                |                                         |           |        |
| Como Developer, quiero configurar el proyecto Android con Kotlin y Jetpack Compose para implementar una aplicación nativa mantenible y ejecutable en dispositivos físicos. |                                         |           |        |
| Acceptance Criteria                                                                                                                                                        |                                         |           |        |

---

**Scenario 1: Proyecto reproducible**

**Given** el repositorio contiene la configuración documentada

**When** el Developer compila el proyecto en un ambiente limpio

**Then** el sistema genera una aplicación instalable sin depender de archivos locales no versionados

**Scenario 2: Dispositivo físico**

**Given** existe una compilación de desarrollo válida

**When** el Developer instala y ejecuta la aplicación en un dispositivo Android compatible

**Then** las funciones base inician sin errores bloqueantes

| Story ID | User                                                                                                                                                     | Priority  | Epic   |
|----------|----------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|--------|
| TS-002   | Developer                                                                                                                                                | Must Have | EP-012 |
| Title    | Configurar la aplicación multiplataforma con Flutter                                                                                                     |           |        |
|          | Description                                                                                                                                              |           |        |
|          | Como Developer, quiero configurar un proyecto Flutter con Dart para entregar una experiencia multiplataforma coherente con la aplicación Android nativa. |           |        |
|          | Acceptance Criteria                                                                                                                                      |           |        |

**Scenario 1: Proyecto reproducible**

**Given** el repositorio contiene las versiones y dependencias documentadas

**When** el Developer ejecuta la compilación desde un ambiente limpio

**Then** el sistema genera una aplicación para las plataformas objetivo definidas

**Scenario 2: Consistencia funcional**

**Given** existe un flujo core implementado en Android nativo

**When** el Developer implementa el mismo flujo en Flutter

**Then** ambas aplicaciones aplican las mismas reglas de negocio y contrato de servicio

| Story ID                                                                                                                                                  | User                                                       | Priority  | Epic   |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------|-----------|--------|
| TS-003                                                                                                                                                    | Developer                                                  | Must Have | EP-012 |
| Title                                                                                                                                                     | Definir la arquitectura móvil por capas y bounded contexts |           |        |
| Description                                                                                                                                               |                                                            |           |        |
| Como Developer, quiero definir una arquitectura móvil alineada con Domain-Driven Design para separar presentación, aplicación, dominio e infraestructura. |                                                            |           |        |
| Acceptance Criteria                                                                                                                                       |                                                            |           |        |

**Scenario 1: Límites definidos**

**Given** se conocen los bounded contexts y dependencias permitidas

**When** el equipo documenta la arquitectura móvil

**Then** cada módulo identifica sus responsabilidades y evita dependencias contrarias a la dirección acordada

**Scenario 2: Regla de dominio aislada**

**Given** existe una regla que no depende del sistema operativo

**When** el Developer implementa y prueba la regla

**Then** la regla se ejecuta sin requerir componentes de interfaz o infraestructura

| Story ID | User      | Priority  | Epic   |
|----------|-----------|-----------|--------|
| TS-004   | Developer | Must Have | EP-012 |

**Title** Integrar las aplicaciones con la API REST interna

#### Description

Como Developer, quiero implementar clientes HTTP autenticados en Android y Flutter para consumir de forma consistente la API REST de AniTec.

#### Acceptance Criteria

##### Scenario 1: Respuesta exitosa

**Given** la API devuelve una respuesta válida

**When** la aplicación solicita un recurso

**Then** el cliente transforma la respuesta al modelo correspondiente sin exponer el formato de transporte al dominio

##### Scenario 2: Respuesta de error

**Given** la API devuelve un código de error o no responde

**When** la aplicación procesa la solicitud

**Then** el cliente clasifica el resultado para que la aplicación pueda informar o reintentar de forma segura

| Story ID | User      | Priority  | Epic   |
|----------|-----------|-----------|--------|
| TS-005   | Developer | Must Have | EP-007 |

**Title** Implementar persistencia local segura en Android

#### Description

Como Developer, quiero implementar persistencia local con Room para almacenar información esencial y operaciones pendientes en la aplicación Android.

#### Acceptance Criteria

##### Scenario 1: Persistencia

**Given** la aplicación recibe información autorizada

**When** el repositorio local la almacena

**Then** los datos permanecen disponibles después de reiniciar la aplicación

##### Scenario 2: Protección y limpieza

**Given** el usuario finaliza la sesión o cambia de cuenta

**When** la aplicación procesa el cambio

**Then** los datos protegidos que no deben compartirse se eliminan o aíslan según la política definida

| Story ID | User      | Priority  | Epic   |
|----------|-----------|-----------|--------|
| TS-006   | Developer | Must Have | EP-007 |

**Title** Implementar persistencia local en Flutter

#### Description

Como Developer, quiero implementar una base local compatible con Flutter para ofrecer el mismo comportamiento offline en la aplicación multiplataforma.

#### Acceptance Criteria

##### Scenario 1: Esquema equivalente

**Given** Android y Flutter comparten las reglas de datos offline

**When** el Developer configura el almacenamiento local de Flutter

**Then** los registros esenciales y operaciones pendientes conservan los campos necesarios para sincronizarse

##### Scenario 2: Migración local

**Given** existe una versión anterior del esquema en el dispositivo

**When** la aplicación se actualiza

**Then** la migración conserva los datos compatibles sin bloquear el inicio

| Story ID    | User                                   | Priority  | Epic   |
|-------------|----------------------------------------|-----------|--------|
| TS-007      | Developer                              | Must Have | EP-007 |
| Title       | Implementar sincronización idempotente |           |        |
| Description |                                        |           |        |

Como Developer, quiero implementar una cola de sincronización idempotente para enviar operaciones pendientes sin duplicarlas ni perderlas.

#### Acceptance Criteria

##### Scenario 1: Operación identificada

**Given** una operación local pendiente posee un identificador único

**When** el cliente la envía más de una vez por un fallo temporal

**Then** el servidor produce un único efecto y devuelve el resultado correspondiente

##### Scenario 2: Conflicto

**Given** el servidor detecta una versión incompatible

**When** el cliente intenta sincronizar

**Then** el sistema conserva el conflicto y aplica la estrategia documentada antes de reemplazar información

| Story ID    | User                                    | Priority  | Epic   |
|-------------|-----------------------------------------|-----------|--------|
| TS-008      | Developer                               | Must Have | EP-002 |
| Title       | Proteger credenciales y datos de sesión |           |        |
| Description |                                         |           |        |

Como Developer, quiero almacenar tokens mediante mecanismos seguros de cada plataforma para reducir la exposición de credenciales.

#### Acceptance Criteria

##### Scenario 1: Almacenamiento seguro

**Given** el servidor entrega un token válido

**When** la aplicación conserva la sesión

**Then** el token se almacena mediante el mecanismo seguro definido para la plataforma

### Scenario 2: Token vencido

**Given** el servidor rechaza un token por expiración

**When** el cliente recibe la respuesta

**Then** la aplicación invalida la sesión o ejecuta la renovación autorizada sin reutilizar credenciales inválidas

| Story ID                                                                                                                                            | User                               | Priority  | Epic   |
|-----------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------|-----------|--------|
| TS-009                                                                                                                                              | Developer                          | Must Have | EP-006 |
| Title                                                                                                                                               | Implementar notificaciones móviles |           |        |
| Description                                                                                                                                         |                                    |           |        |
| Como Developer, quiero integrar notificaciones locales o push para entregar recordatorios sanitarios respetando los permisos del sistema operativo. |                                    |           |        |
| Acceptance Criteria                                                                                                                                 |                                    |           |        |

### Scenario 1: Programación

**Given** existe una actividad válida y el permiso correspondiente

**When** la aplicación programa el recordatorio

**Then** el sistema operativo recibe una notificación con una referencia válida a la actividad

### Scenario 2: Permiso ausente

**Given** el permiso no fue concedido

**When** la aplicación procesa una actividad con recordatorio

**Then** la actividad permanece registrada y la aplicación informa que no puede entregar la notificación

| Story ID                                                                                                                                                               | User                                              | Priority  | Epic   |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------|-----------|--------|
| TS-010                                                                                                                                                                 | Developer                                         | Must Have | EP-008 |
| Title                                                                                                                                                                  | Integrar identificación QR mediante Google ML Kit |           |        |
| Description                                                                                                                                                            |                                                   |           |        |
| Como Developer, quiero integrar Google ML Kit Barcode Scanning y una alternativa compatible en Flutter para identificar animales mediante códigos QR usando la cámara. |                                                   |           |        |
| Acceptance Criteria                                                                                                                                                    |                                                   |           |        |

### Scenario 1: Código compatible

**Given** la cámara recibe un código QR válido

**When** la biblioteca procesa la imagen

**Then** la aplicación obtiene el identificador sin almacenar imágenes innecesarias

### Scenario 2: Lectura fallida

**Given** la biblioteca no reconoce un código válido

**When** finaliza el intento configurado

**Then** la aplicación devuelve un resultado controlado y conserva la alternativa manual

| Story ID | User      | Priority  | Epic   |
|----------|-----------|-----------|--------|
| TS-011   | Developer | Must Have | EP-010 |

**Title**

Integrar el checkout externo de Stripe

**Description**

Como Developer, quiero integrar el checkout de Stripe mediante el backend y enlaces seguros para procesar pagos de prueba desde las aplicaciones móviles.

**Acceptance Criteria****Scenario 1: Creación de checkout**

**Given** el cliente solicita pagar un plan activo

**When** el backend valida la solicitud

**Then** el servicio crea una sesión de Stripe y devuelve una URL permitida

**Scenario 2: Confirmación idempotente**

**Given** Stripe informa el resultado mediante el mecanismo configurado

**When** el backend procesa el evento

**Then** el pago y la suscripción se actualizan una sola vez

| Story ID | User      | Priority  | Epic   |
|----------|-----------|-----------|--------|
| TS-012   | Developer | Must Have | EP-011 |

**Title**

Aplicar internacionalización y accesibilidad móvil

**Description**

Como Developer, quiero centralizar recursos traducibles y aplicar prácticas de accesibilidad para mantener una experiencia coherente en Android y Flutter.

**Acceptance Criteria****Scenario 1: Recursos traducibles**

**Given** existen idiomas soportados

**When** el Developer agrega o modifica contenido

**Then** los textos visibles provienen de recursos localizables y no quedan cadenas funcionales sin traducir

**Scenario 2: Verificación accesible**

**Given** existen flujos core implementados

**When** el equipo ejecuta verificaciones con escalado de texto y lector de pantalla

**Then** los problemas encontrados se documentan y los bloqueantes se corrigen

| Story ID | User      | Priority  | Epic   |
|----------|-----------|-----------|--------|
| TS-013   | Developer | Must Have | EP-012 |

**Title**

Adaptar y documentar los servicios backend para móviles

**Description**

Como Developer, quiero adaptar los endpoints existentes y documentarlos con OpenAPI para soportar los flujos móviles y las reglas de autorización.

#### Acceptance Criteria

##### Scenario 1: Contrato documentado

**Given** un endpoint forma parte del alcance móvil

**When** el Developer publica la documentación OpenAPI

**Then** el contrato describe autenticación, parámetros, respuestas exitosas y errores

##### Scenario 2: Autorización

**Given** una solicitud autenticada intenta acceder a un recurso ajeno

**When** la API evalúa el rol y la relación

**Then** la API rechaza la operación mediante el código HTTP correspondiente

| Story ID           | User                                               | Priority    | Epic   |
|--------------------|----------------------------------------------------|-------------|--------|
| TS-014             | Developer                                          | Should Have | EP-012 |
| <b>Title</b>       | Automatizar pruebas de los flujos móviles críticos |             |        |
| <b>Description</b> |                                                    |             |        |

Como Developer, quiero automatizar pruebas unitarias y de integración para detectar regresiones en dominio, persistencia, sincronización y consumo de servicios.

#### Acceptance Criteria

##### Scenario 1: Pruebas de dominio

**Given** existen reglas críticas de autorización, estado o sincronización

**When** el equipo ejecuta la suite

**Then** las pruebas verifican resultados exitosos y alternativos de manera determinista

##### Scenario 2: Integración controlada

**Given** existen repositorios locales y clientes de servicio

**When** el equipo ejecuta pruebas con dobles o ambientes controlados

**Then** la suite comprueba persistencia, mapeo y tratamiento de errores

| Story ID           | User                                                       | Priority  | Epic   |
|--------------------|------------------------------------------------------------|-----------|--------|
| TS-015             | Developer                                                  | Must Have | EP-012 |
| <b>Title</b>       | Configurar compilación y distribución de versiones móviles |           |        |
| <b>Description</b> |                                                            |           |        |

Como Developer, quiero automatizar compilaciones firmadas y distribuir versiones mediante Firebase App Distribution para probarlas en dispositivos físicos.

#### Acceptance Criteria

##### Scenario 1: Compilación versionada

**Given** el repositorio alcanza un hito de entrega

**When** el pipeline genera la versión

**Then** el artefacto incluye un número de versión trazable y una configuración válida

## Scenario 2: Distribución

**Given** existe un artefacto aprobado para pruebas

**When** el equipo publica la versión en Firebase App Distribution

**Then** los evaluadores autorizados pueden instalarla y se registra la versión distribuida

## Spike Stories

La siguiente Spike Story sustenta el feature de aprendizaje autónomo. Su resultado deberá documentar la investigación y determinar la viabilidad antes de cerrar la implementación definitiva.

| Story ID                                                                                                                                                                                                                              | User                                                    | Priority  | Epic   |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------|-----------|--------|
| SP-001                                                                                                                                                                                                                                | Developer                                               | Must Have | EP-008 |
| Title                                                                                                                                                                                                                                 | Investigar identificación de animales con Google ML Kit |           |        |
| Description                                                                                                                                                                                                                           |                                                         |           |        |
| Como Developer, quiero investigar y prototipar Google ML Kit Barcode Scanning en Android y una integración compatible en Flutter para determinar su viabilidad como feature de aprendizaje autónomo e identificación mediante cámara. |                                                         |           |        |
| Acceptance Criteria                                                                                                                                                                                                                   |                                                         |           |        |

### Scenario 1: Investigación documentada

**Given** el equipo definió el caso de identificación mediante QR

**When** el Developer compara compatibilidad, permisos, operación offline, licencias, privacidad y esfuerzo en ambas plataformas

**Then** el equipo documenta fuentes, alternativas, riesgos y criterios de selección

### Scenario 2: Prototipo y conclusión

**Given** existen códigos de prueba y al menos un dispositivo físico

**When** el equipo prueba el prototipo bajo distintas condiciones de iluminación y códigos inválidos

**Then** el equipo registra resultados, limita el tratamiento de imágenes y concluye si la tecnología es viable o qué alternativa debe utilizarse

## Matriz de coherencia con el alcance móvil y los requisitos del curso

La siguiente matriz hace trazable cada condición tecnológica obligatoria con una decisión verificable y con los elementos del backlog que permiten implementarla. Ambas aplicaciones consumen el mismo dominio y la misma API REST, pero conservan implementaciones y pruebas propias para demostrar el desarrollo Android nativo y el desarrollo multiplataforma.

| Aspecto requerido               | Decisión para AniTec                                                                                                           | Evidencia esperada                                                                                                  | Historias relacionadas                            |
|---------------------------------|--------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|---------------------------------------------------|
| Aplicación Android nativa       | Kotlin y Jetpack Compose, con arquitectura por capas y funcionalidades core de ganaderos y veterinarios.                       | Proyecto Android compilable, ejecución y pruebas en un dispositivo Android físico.                                  | TS-001, TS-003, TS-004 y US-004 a US-043.         |
| Aplicación multiplataforma      | Flutter y Dart, con el mismo alcance funcional y contratos de servicio que la aplicación Android.                              | Proyecto Flutter compilable y comparación de los flujos core en las plataformas objetivo definidas por el equipo.   | TS-002, TS-003, TS-004, TS-006 y US-004 a US-043. |
| Almacenamiento local            | Room en Android y una base de datos local compatible en Flutter para caché, operaciones pendientes y estado de sincronización. | Consulta y registro de información esencial sin conexión, seguidos por una sincronización idempotente.              | US-029 a US-032 y TS-005 a TS-007.                |
| Recurso interno del dispositivo | Cámara del teléfono para leer el código QR que identifica al animal, con alternativa de búsqueda manual.                       | Prueba con permiso concedido, permiso denegado, código válido, código inválido y condiciones reales de iluminación. | US-033, US-034, TS-010 y SP-001.                  |
| Servicio REST interno           | API REST propia de AniTec, documentada con OpenAPI y protegida mediante autenticación y autorización por rol y relación.       | Backend público al 70 % en Sprint 1 y al 100 % desde Sprint 2, con documentación accesible.                         | US-004 a US-040, TS-004, TS-008 y TS-013.         |
| Servicio externo                | Stripe Checkout para iniciar pagos y confirmar el estado de la suscripción sin almacenar datos de tarjeta en AniTec.           | Flujos de pago aprobado, rechazado o cancelado y validación del resultado desde el backend.                         | US-038 a US-040 y TS-011.                         |
| Feature de aprendizaje autónomo | Investigación, prototipo e integración de Google ML Kit Barcode Scanning para reconocer códigos QR en el dispositivo.          | Informe del Spike, fuentes consultadas, alternativas, riesgos, prueba física y conclusión de viabilidad.            | US-033, TS-010 y SP-001.                          |
| Notificaciones móviles          | Recordatorios locales o remotos asociados con actividades ganaderas y sanitarias.                                              | Recepción, atención y reprogramación de recordatorios con permisos habilitados o denegados.                         | US-025 a US-028 y TS-009.                         |

| Aspecto requerido              | Decisión para AniTec                                                                            | Evidencia esperada                                                                         | Historias relacionadas |
|--------------------------------|-------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|------------------------|
| Despliegue y validación física | Versiones firmadas y distribuidas mediante Firebase App Distribution o un servicio equivalente. | Instalación trazable en dispositivos físicos y registro de versión para la Release Review. | TS-014 y TS-015.       |

### 2.4.2. Impact Mapping

El Impact Mapping conecta objetivos SMART del piloto con los User Personas, los cambios de comportamiento esperados, los entregables y las historias que los habilitan. Los objetivos se medirán con analítica de uso, registros de sincronización y pruebas moderadas antes de concluir el Sprint 3.

| Business Goal                                                                                                                                                                                          | Actor / User Persona        | Impact                                                                                               | Deliverable                                                          | User Stories                                                                                                                              |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------|------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| <b>BG-01:</b> Lograr que al menos 8 de 10 ganaderos participantes registren tres animales y dos eventos desde una aplicación móvil durante un piloto de cuatro semanas, antes del cierre del Sprint 3. | Jorge Luis Rivas — Ganadero | Registra la información en el lugar donde ocurre y deja de postergarla por falta de una computadora. | Gestión móvil de fincas y animales.                                  | Como ganadero, deseo registrar y consultar mis animales desde el teléfono para mantener su trazabilidad. US-008 a US-013.                 |
|                                                                                                                                                                                                        | Jorge Luis Rivas — Ganadero | Continúa trabajando durante interrupciones de red y confirma después la sincronización.              | Persistencia local, cola de operaciones y estados de sincronización. | Como ganadero, deseo guardar y sincronizar registros sin repetirlos para trabajar con conectividad intermitente. US-029 a US-032.         |
| <b>BG-02:</b> Conseguir que al menos el 70 % de los recordatorios sanitarios creados durante el piloto se marquen como atendidos o reprogramados dentro de su plazo, antes del cierre del Sprint 3.    | Jorge Luis Rivas — Ganadero | Registra incidencias y consulta actividades pendientes con mayor frecuencia.                         | Historial sanitario, calendario y notificaciones móviles.            | Como ganadero, deseo registrar incidencias y recibir recordatorios para cumplir actividades sanitarias. US-014, US-015 y US-025 a US-028. |
|                                                                                                                                                                                                        | Valeria Mendoza —           | Programa el control posterior como parte                                                             | Registro clínico trazable y                                          | Como veterinaria, deseo registrar                                                                                                         |

| Business Goal                                                                                                                                                                                                              | Actor /<br>User<br>Persona            | Impact                                                                                           | Deliverable                                               | User Stories                                                                                                                          |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
|                                                                                                                                                                                                                            | Veterinaria                           | de la atención.                                                                                  | actividad de seguimiento.                                 | tratamientos y próximos controles para mantener continuidad clínica. US-016 a US-019.                                                 |
| <b>BG-03:</b> Lograr que al menos 4 de 5 veterinarios participantes consulten antecedentes y registren una atención autorizada en menos de cinco minutos y sin ayuda, durante la validación previa al cierre del Sprint 3. | Valeria Mendoza —<br>Veterinaria      | Consulta únicamente clientes y pacientes autorizados antes de atenderlos.                        | Gestión de relaciones, clientes, pacientes e historiales. | Como veterinaria, deseo consultar clientes y pacientes autorizados para preparar y documentar la atención. US-023 y US-024.           |
|                                                                                                                                                                                                                            | Jorge Luis Rivas —<br>Ganadero        | Controla quién puede consultar y registrar información sanitaria.                                | Solicitudes, aceptación y revocación de acceso.           | Como ganadero, deseo conceder o revocar acceso veterinario para proteger los datos de mis animales. US-020 a US-022.                  |
| <b>BG-04:</b> Conseguir que al menos el 80 % de los participantes identifique correctamente un animal mediante QR o la alternativa manual en menos de 30 segundos, durante las pruebas del Sprint 2.                       | Jorge Luis Rivas y<br>Valeria Mendoza | Localizan la ficha correcta con menos tiempo y mantienen una alternativa cuando la cámara falla. | Escaneo QR con Google ML Kit y búsqueda manual.           | Como usuario autorizado, deseo identificar un animal mediante cámara o búsqueda para acceder rápidamente a su ficha. US-033 y US-034. |

**Placeholder de captura:** Insertar aquí la captura actualizada del Impact Mapping en la herramienta indicada.

**Placeholder de enlace público:** Agregar aquí la URL pública del Impact Mapping.

### 2.4.3. Product Backlog

El Product Backlog contiene todas las User Stories, Technical Stories y Spike Stories definidas en esta sección. El orden comienza con la landing page, como exige el statement, y continúa con incrementos verticales de valor móvil. Las historias técnicas se ubican cerca del resultado que habilitan y todas las estimaciones utilizan la escala 1, 2, 3, 5 u 8.

La distribución considera los hitos del curso:

- **Sprint 1 — TB1, semana 7:** landing page desplegada, backend al 70 %, bases de Android y Flutter y pantallas core de acceso y gestión de animales.
- **Sprint 2 — AV2, semana 12:** backend al 100 % y principales funciones core de sanidad, colaboración veterinaria, notificaciones, trabajo offline e identificación mediante cámara.
- **Sprint 3 — TB2, semana 15:** cierre de trazabilidad sanitaria y sincronización, reportes, pagos, accesibilidad, pruebas, distribución y aplicación completa según el backlog.

| #<br>Orden | User Story<br>Id | Título                                                     | Story Points (1 / 2 / 3 / 5 / 8) | Sprint   |
|------------|------------------|------------------------------------------------------------|----------------------------------|----------|
| 1          | US-001           | Comprender la propuesta de valor de AniTec                 | 3                                | Sprint 1 |
| 2          | US-002           | Conocer las soluciones para cada segmento                  | 3                                | Sprint 1 |
| 3          | US-003           | Acceder a una landing page adaptable e internacionalizada  | 5                                | Sprint 1 |
| 4          | TS-001           | Configurar la aplicación Android nativa                    | 5                                | Sprint 1 |
| 5          | TS-002           | Configurar la aplicación multiplataforma con Flutter       | 5                                | Sprint 1 |
| 6          | TS-003           | Definir la arquitectura móvil por capas y bounded contexts | 5                                | Sprint 1 |
| 7          | TS-013           | Adaptar y documentar los servicios backend para móviles    | 8                                | Sprint 1 |
| 8          | US-004           | Registrar una cuenta según el rol                          | 5                                | Sprint 1 |
| 9          | US-005           | Iniciar sesión                                             | 3                                | Sprint 1 |
| 10         | US-006           | Mantener y finalizar la sesión móvil                       | 3                                | Sprint 1 |
| 11         | US-007           | Acceder únicamente a información autorizada                | 5                                | Sprint 1 |
| 12         | TS-008           | Proteger credenciales y datos de sesión                    | 5                                | Sprint 1 |
| 13         | US-008           | Consultar las fincas registradas                           | 3                                | Sprint 1 |
| 14         | US-009           | Registrar y actualizar una finca                           | 5                                | Sprint 1 |

| #<br>Orden | User Story<br>Id | Título                                                  | Story Points (1 / 2 / 3 / 5 / 8) | Sprint   |
|------------|------------------|---------------------------------------------------------|----------------------------------|----------|
| 15         | US-010           | Consultar y buscar animales                             | 5                                | Sprint 1 |
| 16         | US-011           | Registrar un animal                                     | 5                                | Sprint 1 |
| 17         | US-012           | Actualizar o archivar un animal                         | 5                                | Sprint 1 |
| 18         | US-013           | Consultar el detalle de un animal                       | 3                                | Sprint 1 |
| 19         | TS-004           | Integrar las aplicaciones con la API REST interna       | 5                                | Sprint 1 |
| 20         | TS-005           | Implementar persistencia local segura en Android        | 5                                | Sprint 1 |
| 21         | TS-006           | Implementar persistencia local en Flutter               | 5                                | Sprint 2 |
| 22         | SP-001           | Investigar identificación de animales con Google ML Kit | 5                                | Sprint 1 |
| 23         | US-014           | Consultar eventos sanitarios                            | 3                                | Sprint 2 |
| 24         | US-015           | Registrar una incidencia sanitaria                      | 5                                | Sprint 2 |
| 25         | US-016           | Registrar diagnóstico y tratamiento                     | 5                                | Sprint 2 |
| 26         | US-017           | Consultar el historial sanitario de un animal           | 5                                | Sprint 2 |
| 27         | US-018           | Corregir un registro sanitario con trazabilidad         | 5                                | Sprint 3 |
| 28         | US-019           | Programar un control sanitario posterior                | 3                                | Sprint 3 |
| 29         | US-020           | Recibir una solicitud de seguimiento veterinario        | 3                                | Sprint 2 |
| 30         | US-021           | Aceptar o rechazar acceso veterinario                   | 3                                | Sprint 2 |
| 31         | US-022           | Revocar el acceso de un veterinario                     | 3                                | Sprint 2 |

| #<br>Orden | User Story<br>Id | Título                                            | Story Points (1 / 2 / 3 / 5 / 8) | Sprint   |
|------------|------------------|---------------------------------------------------|----------------------------------|----------|
| 32         | US-023           | Consultar clientes y pacientes autorizados        | 5                                | Sprint 2 |
| 33         | US-024           | Consultar antecedentes de un paciente autorizado  | 3                                | Sprint 2 |
| 34         | US-025           | Consultar actividades programadas                 | 3                                | Sprint 2 |
| 35         | US-026           | Gestionar una actividad o recordatorio            | 5                                | Sprint 2 |
| 36         | US-027           | Recibir una notificación de actividad             | 5                                | Sprint 2 |
| 37         | US-028           | Atender o reprogramar una actividad               | 3                                | Sprint 3 |
| 38         | TS-009           | Implementar notificaciones móviles                | 5                                | Sprint 2 |
| 39         | US-029           | Consultar información esencial sin conexión       | 5                                | Sprint 2 |
| 40         | US-030           | Guardar trabajo pendiente sin conexión            | 5                                | Sprint 2 |
| 41         | US-031           | Sincronizar operaciones pendientes                | 8                                | Sprint 2 |
| 42         | US-032           | Resolver errores o conflictos de sincronización   | 8                                | Sprint 3 |
| 43         | TS-007           | Implementar sincronización idempotente            | 8                                | Sprint 2 |
| 44         | US-033           | Identificar un animal mediante código QR          | 5                                | Sprint 2 |
| 45         | US-034           | Identificar un animal sin utilizar la cámara      | 3                                | Sprint 2 |
| 46         | TS-010           | Integrar identificación QR mediante Google ML Kit | 5                                | Sprint 2 |
| 47         | US-035           | Consultar indicadores del hato                    | 5                                | Sprint 3 |
| 48         | US-036           | Consultar indicadores sanitarios de clientes      | 5                                | Sprint 3 |

| #<br>Orden | User Story<br>Id | Título                                                     | Story Points (1 / 2 / 3 / 5 / 8) | Sprint   |
|------------|------------------|------------------------------------------------------------|----------------------------------|----------|
| 49         | US-037           | Registrar y consultar movimientos financieros              | 5                                | Sprint 3 |
| 50         | US-038           | Consultar planes de suscripción                            | 3                                | Sprint 3 |
| 51         | US-039           | Iniciar un pago mediante un proveedor externo              | 5                                | Sprint 3 |
| 52         | US-040           | Consultar el resultado del pago y la suscripción           | 5                                | Sprint 3 |
| 53         | TS-011           | Integrar el checkout externo de Stripe                     | 8                                | Sprint 3 |
| 54         | US-041           | Cambiar el idioma de la aplicación                         | 3                                | Sprint 3 |
| 55         | US-042           | Utilizar la aplicación con necesidades de accesibilidad    | 5                                | Sprint 3 |
| 56         | US-043           | Comprender errores y estados de conectividad               | 3                                | Sprint 3 |
| 57         | TS-012           | Aplicar internacionalización y accesibilidad móvil         | 5                                | Sprint 3 |
| 58         | TS-014           | Automatizar pruebas de los flujos móviles críticos         | 8                                | Sprint 3 |
| 59         | TS-015           | Configurar compilación y distribución de versiones móviles | 5                                | Sprint 3 |

**Placeholder de captura:** Insertar aquí la captura del Product Backlog actualizado en Jira, YouTrack o Trello.

**Placeholder de enlace público:** Agregar aquí la URL pública del Product Backlog.

## 2.5. Strategic-Level Domain-Driven Design

El diseño estratégico de AniTec utiliza Domain-Driven Design para separar el dominio ganadero en modelos con responsabilidades, reglas y lenguaje propios. El proceso parte del Big Picture EventStorming de la sección 2.3.5, lo contrasta con los User Personas, entrevistas y requisitos de la sección 2.4, y concluye con bounded contexts y relaciones que pueden mantenerse de forma coherente en el backend y en las aplicaciones móviles.

La revisión recupera la evidencia del trabajo anterior —especialmente los pasos 4 al 10 del EventStorming, los canvases de Livestock, Sanitary, Financiamiento y Analytics, y la estructura modular del backend— y la adapta al alcance actual: una aplicación Android nativa, una aplicación multiplataforma, operación offline, identificación QR, colaboración veterinaria, notificaciones y pagos mediante Stripe.

## 2.5.1. EventStorming

El equipo utilizó Miro para construir una narrativa compartida del dominio de AniTec. La sesión identificó actores, comandos, eventos de dominio, políticas, read models, sistemas externos, agregados, puntos problemáticos y eventos pivote. El análisis se organizó en bloques de trabajo acotados para evitar que la exploración se convirtiera en un inventario de funcionalidades sin secuencia de negocio.

| Dato de la sesión | Registro                                                                                                         |
|-------------------|------------------------------------------------------------------------------------------------------------------|
| Herramienta       | Miro                                                                                                             |
| Participantes     | Equipo de AniTec                                                                                                 |
| Fecha real        | <b>Placeholder:</b> completar con la fecha de la sesión registrada por el equipo.                                |
| Duración real     | <b>Placeholder:</b> completar con la duración de la sesión; el time-box recomendado es de una a dos horas.       |
| Entradas          | Entrevistas, User Personas, Needfinding, Ubiquitous Language y Product Backlog móvil.                            |
| Resultados        | Flujo actualizado del dominio, eventos pivote, nueve bounded contexts candidatos, Domain Stories y dependencias. |

### Convención visual

| Elemento        | Color utilizado       | Propósito                                                        |
|-----------------|-----------------------|------------------------------------------------------------------|
| Domain Event    | Naranja               | Hecho relevante del negocio redactado en pasado.                 |
| Command         | Azul                  | Intención que puede producir un evento.                          |
| Actor           | Amarillo              | Persona o rol que ejecuta una acción.                            |
| Policy          | Morado                | Regla que reacciona ante un evento y puede generar otro comando. |
| Read Model      | Verde                 | Información que un actor consulta antes de tomar una decisión.   |
| External System | Rosado                | Sistema ajeno a AniTec que participa en el flujo.                |
| Hotspot         | Rojo o rosado intenso | Duda, riesgo, conflicto o decisión todavía abierta.              |
| Aggregate       | Amarillo amplio       | Frontera de consistencia que recibe comandos y produce eventos.  |

### Proceso de exploración y refinamiento

**Paso 1 — Unstructured Exploration.** Los participantes registraron hechos relevantes sin imponer inicialmente una estructura. Se identificaron eventos relacionados con autenticación, fincas, animales, sanidad, finanzas, analítica, actividades y suscripciones.

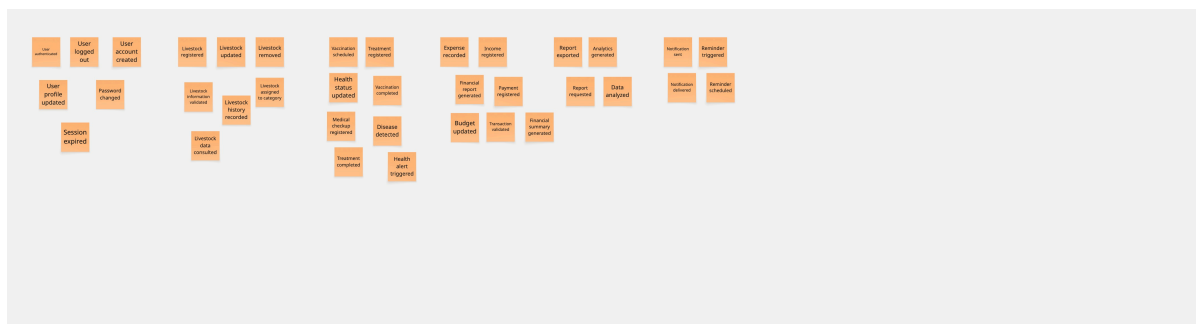


Figura 2.5.1. Exploración no estructurada del dominio. Fuente: elaboración propia.

**Paso 2 — Timeline.** Los eventos fueron ordenados para formar secuencias de negocio. El orden permitió distinguir el acceso al sistema, la gestión del hato, la atención sanitaria, el trabajo de campo y la obtención de indicadores.

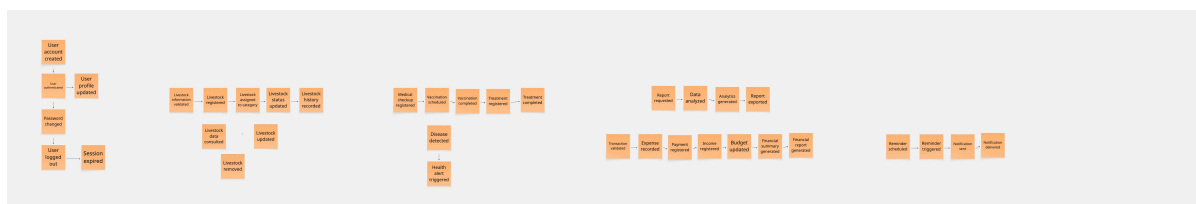


Figura 2.5.2. Línea de tiempo inicial. Fuente: elaboración propia.

**Paso 3 — Pain Points.** Se marcaron problemas como credenciales inválidas, animales duplicados, detección tardía de enfermedades, registros financieros incompletos y fallos de notificación. Para el alcance móvil se añadieron conectividad intermitente, conflictos de sincronización, cámara no disponible, QR ilegible, autorización veterinaria vencida y pago no confirmado.

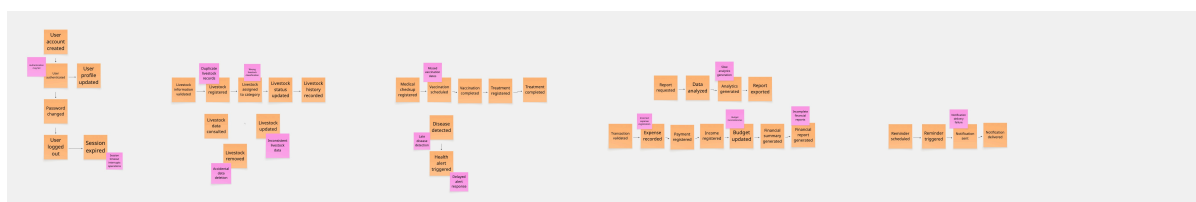


Figura 2.5.3. Pain points identificados. Fuente: elaboración propia.

**Paso 4 — Pivotal Events.** Se buscaron hechos que cambian el estado o la responsabilidad del proceso. Los principales eventos pivote son UserRegistered, AnimalRegistered, AnimalIdentified, VeterinaryAccessGranted, HealthEventRecorded, DataSynchronized, PaymentConfirmed y SubscriptionActivated. Estos eventos ayudaron a proponer límites entre modelos.

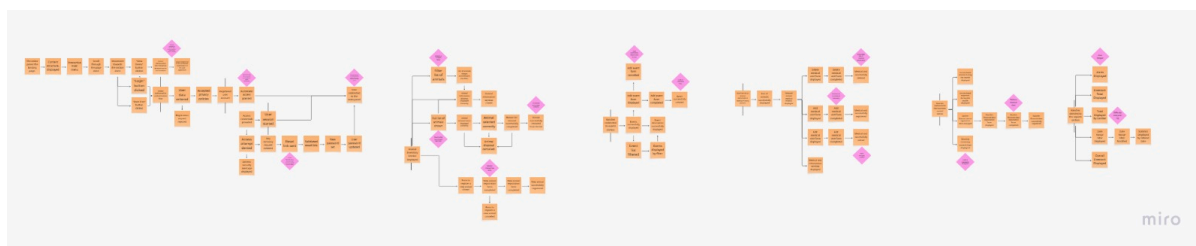


Figura 2.5.4. Identificación de eventos pivote. Fuente: elaboración propia.

**Paso 5 — Commands and Actors.** Cada evento se relacionó con el comando que lo origina y con el actor correspondiente. Por ejemplo, el ganadero ejecuta RegisterAnimal, mientras que el veterinario ejecuta RecordVeterinaryCare únicamente cuando existe una autorización vigente.

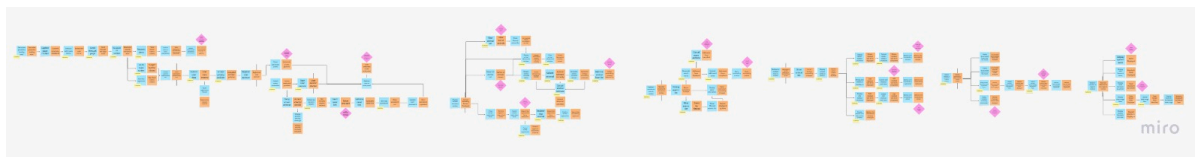


Figura 2.5.5. Incorporación de comandos y actores. Fuente: elaboración propia.

**Paso 6 — Políticas.** Se representaron reglas reactivas. Cuando una atención requiere seguimiento, se solicita programar una actividad; cuando la conexión vuelve, se solicita sincronizar operaciones pendientes; cuando Stripe confirma un pago válido, se solicita activar la suscripción.

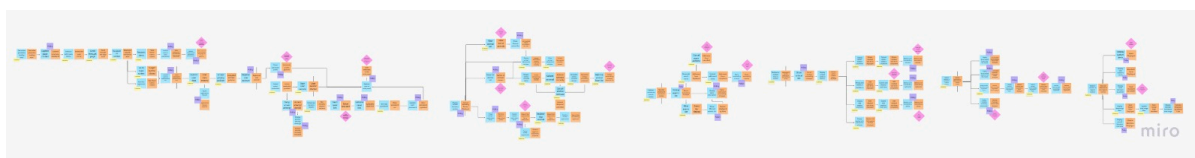


Figura 2.5.6. Políticas y automatizaciones. Fuente: elaboración propia.

**Paso 7 — Read Models.** Se incorporó la información necesaria para decidir: lista de fincas, ficha del animal, historial sanitario, clientes autorizados, actividades pendientes, resumen financiero, indicadores y estado de suscripción. En la aplicación móvil, cada read model debe indicar si procede del servidor o de una copia local y cuándo fue actualizado.

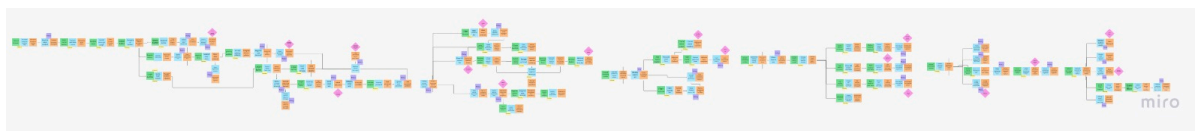


Figura 2.5.7. Read models del dominio. Fuente: elaboración propia.

**Paso 8 — External Systems.** Stripe participa en el checkout y la confirmación de pagos. El proveedor de notificaciones entrega recordatorios cuando se utiliza notificación remota. Google ML Kit se trata como una dependencia técnica ejecutada en el dispositivo para decodificar el QR y no como parte del modelo de negocio.

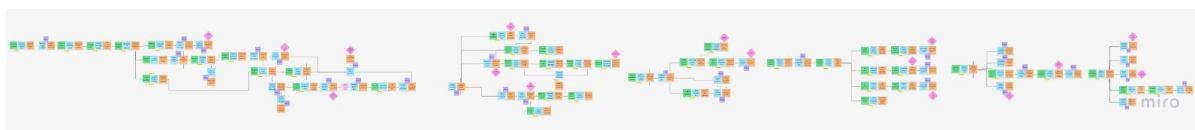


Figura 2.5.8. Sistemas externos. Fuente: elaboración propia.

**Paso 9 — Aggregates.** Los comandos y eventos se agruparon alrededor de agregados con consistencia propia: User, Profile, Farm, Animal, HealthHistory, VeterinaryAuthorization, FarmActivity, FinancialRecord, Subscription y las proyecciones de analítica.



Figura 2.5.9. Agregados candidatos. Fuente: elaboración propia.

La siguiente síntesis actualiza el resultado anterior con las capacidades móviles del Product Backlog. Su propósito es mantener visibles los cambios de estado del negocio sin convertir Room, Flutter, Android, sincronización o ML Kit en conceptos del dominio.

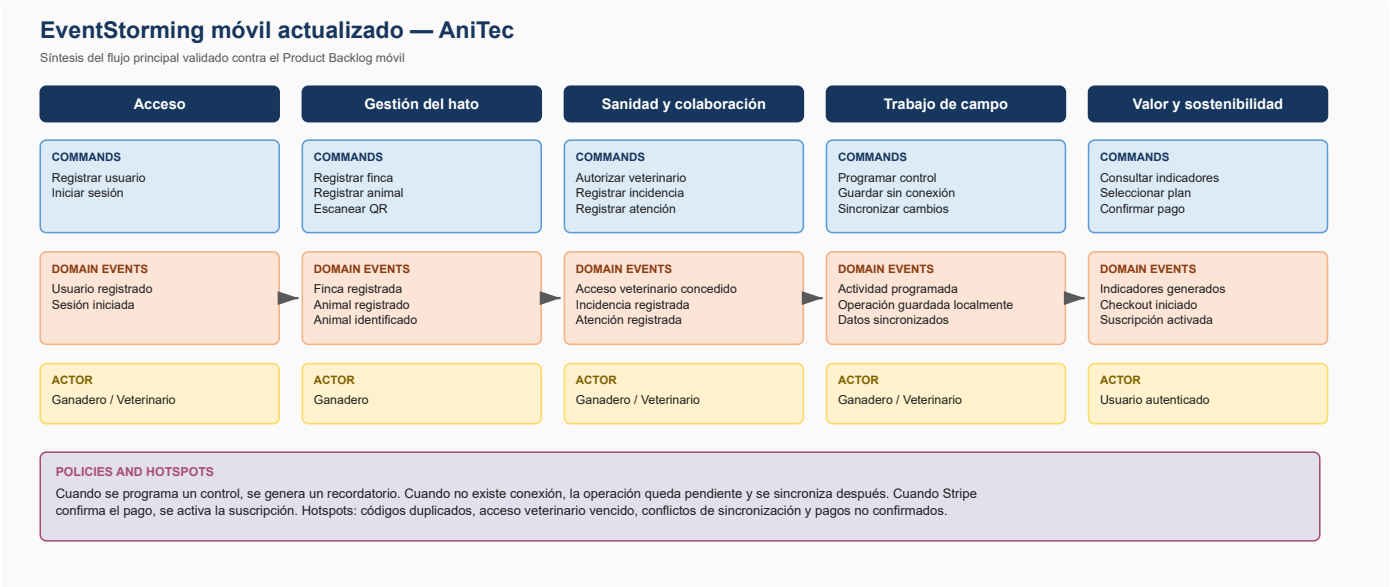


Figura 2.5.10. Síntesis actualizada del EventStorming móvil. Fuente: elaboración propia.

**Enlace del tablero de trabajo:** <https://tinyurl.com/EventSorming>

El equipo debe actualizar en ese tablero las notas correspondientes a QR, autorización veterinaria, offline, sincronización, Stripe y aplicaciones móviles para que la evidencia pública coincida con esta versión del informe.

Decisiones derivadas del EventStorming

- La gestión del hato, la sanidad y la colaboración veterinaria concentran el valor diferencial y se clasifican como Core Domain.
- La autorización veterinaria posee reglas y ciclo de vida propios, por lo que no se modela como un simple atributo del perfil.
- La suscripción y los movimientos financieros representan conceptos distintos: una suscripción controla el acceso a un plan; un movimiento financiero describe la operación económica del ganadero.
- La analítica consume información de otros contextos mediante proyecciones y no modifica sus agregados.
- La sincronización offline pertenece a la infraestructura móvil. El contexto propietario mantiene las reglas para aceptar, rechazar o detectar conflictos.
- La lectura QR es una forma de identificar un animal; el dominio continúa trabajando con AnimalId y AnimalCode aunque cambie la tecnología de escaneo.

2.5.1.1. Candidate Context Discovery

El Candidate Context Discovery aplicó dos técnicas complementarias. **Start-with-Value** permitió separar las capacidades que generan el valor principal para ganaderos y veterinarios. **Look-for-Pivotal-Events** permitió ubicar

cambios de estado y de responsabilidad en la línea de tiempo. Después se contrastaron los candidatos con el Ubiquitous Language, el Product Backlog y los módulos existentes del backend.

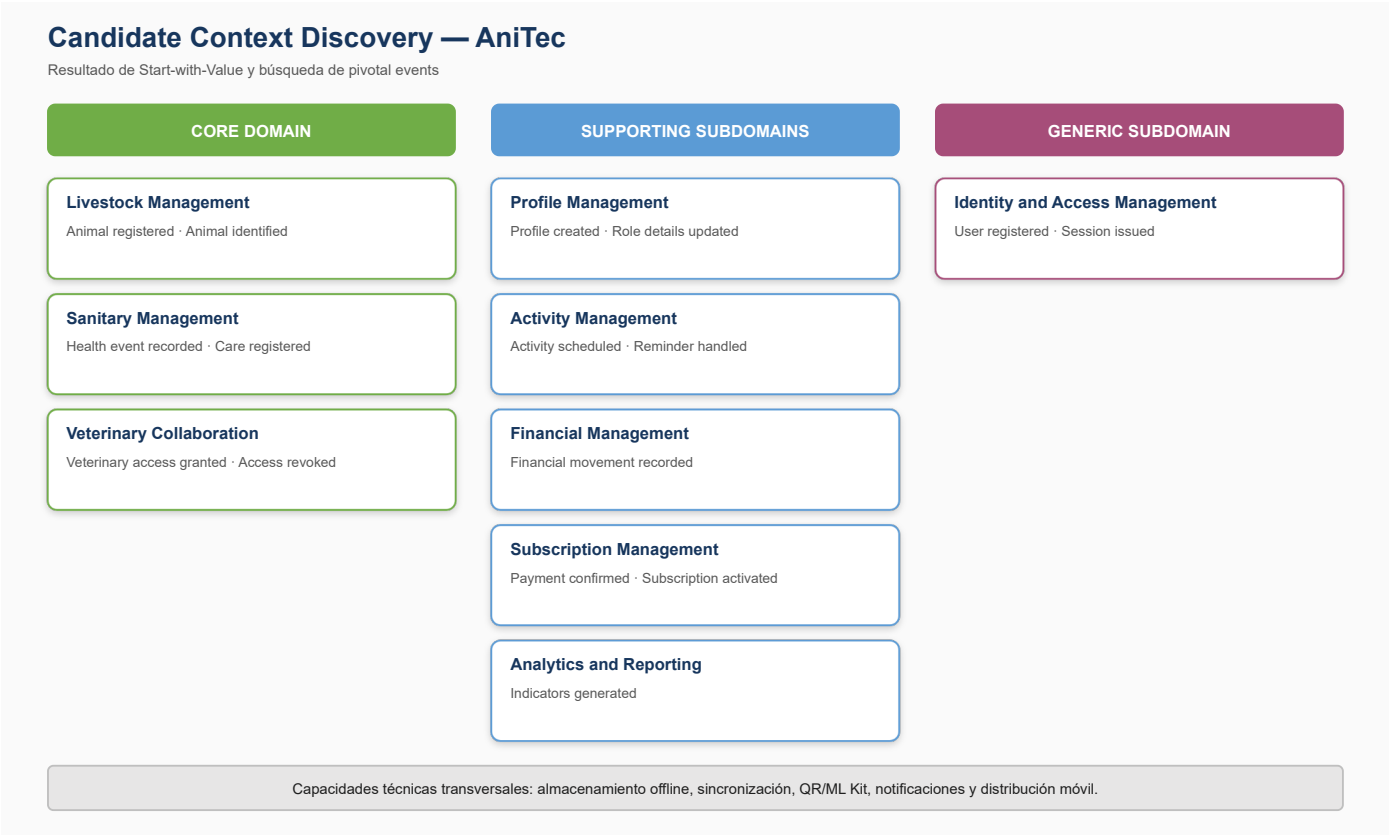


Figura 2.5.11. Clasificación de los bounded contexts candidatos. Fuente: elaboración propia.

| Bounded Context                | Clasificación | Responsabilidad y eventos pivote                                                                               | Correspondencia técnica actual                           |
|--------------------------------|---------------|----------------------------------------------------------------------------------------------------------------|----------------------------------------------------------|
| Identity and Access Management | Generic       | Identidad, credenciales, sesiones y roles.<br>UserRegistered, SessionIssued.                                   | lam                                                      |
| Profile Management             | Supporting    | Datos personales y profesionales. ProfileCreated, ProfileUpdated.                                              | Profiles                                                 |
| Livestock Management           | Core          | Fincas, hatos, animales e identificación.<br>AnimalRegistered, AnimalIdidentified.                             | Livestock                                                |
| Sanitary Management            | Core          | Incidencias, diagnósticos, tratamientos e historial.<br>HealthEventRecorded, VeterinaryCareRecorded.           | Sanitary                                                 |
| Veterinary Collaboration       | Core          | Solicitudes y autorizaciones entre ganaderos y veterinarios. VeterinaryAccessGranted, VeterinaryAccessRevoked. | Clients, que deberá adoptar el lenguaje de colaboración. |
| Activity Management            | Supporting    | Actividades, controles y recordatorios.<br>ActivityScheduled, ReminderDue.                                     | Activities                                               |
| Financial Management           | Supporting    | Ingresos, egresos y resumen financiero.<br>FinancialMovementRecorded.                                          | Financial                                                |

| Bounded Context         | Clasificación | Responsabilidad y eventos pivote                                                          | Correspondencia técnica actual |
|-------------------------|---------------|-------------------------------------------------------------------------------------------|--------------------------------|
| Subscription Management | Supporting    | Planes, checkout, pago confirmado y suscripción. PaymentConfirmed, SubscriptionActivated. | Subscriptions                  |
| Analytics and Reporting | Supporting    | Indicadores y read models para cada rol. IndicatorsGenerated.                             | Analytics                      |

### Decisiones sobre los límites

| Elemento analizado                  | Decisión                                   | Justificación                                                                                                        |
|-------------------------------------|--------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| Offline Storage and Synchronization | No constituye bounded context.             | Es una capacidad técnica de Android y Flutter. Cada operación sigue perteneciendo al contexto que define sus reglas. |
| Animal Identification con QR        | Capability de Livestock Management.        | ML Kit traduce una imagen a un código; Livestock resuelve el animal y verifica autorización.                         |
| Notifications                       | Capability técnica de Activity Management. | Activities decide cuándo existe un recordatorio; el proveedor solo lo entrega.                                       |
| Landing Page                        | Producto informativo, no bounded context.  | No contiene reglas relevantes del dominio ganadero.                                                                  |
| Shared                              | Infraestructura compartida, no subdominio. | Repositorio base, Unit of Work y manejo de errores son mecanismos técnicos.                                          |
| Devices y Metrics                   | Fuera del alcance actual.                  | Pertenecen a la propuesta IoT anterior y no están respaldados por el Product Backlog móvil vigente.                  |

El resultado corrige la diferencia existente entre la lista anterior de cinco contextos, los cuatro canvases históricos y los contextos adicionales utilizados en los Domain Stories. Los nueve contextos seleccionados serán la referencia única para Context Mapping, Software Architecture y Tactical-Level DDD.

#### 2.5.1.2. Domain Message Flows Modeling

Los Domain Message Flows aplican Domain Storytelling para mostrar cómo los actores, aplicaciones móviles y bounded contexts colaboran en los escenarios principales. Cada historia identifica quién inicia el flujo, qué comando se transmite, quién aplica las reglas y qué evento informa el resultado. Los términos Android, Flutter, base local y ML Kit aparecen únicamente cuando explican el transporte o la interacción con el dispositivo.

| Domain Story | Objetivo                                                        | Requisitos relacionados                            |
|--------------|-----------------------------------------------------------------|----------------------------------------------------|
| DS-01        | Registrar un animal con conectividad disponible o interrumpida. | US-008 a US-013, US-029 a US-031, TS-005 a TS-007. |

| Domain Story | Objetivo                                                       | Requisitos relacionados            |
|--------------|----------------------------------------------------------------|------------------------------------|
| DS-02        | Identificar un animal mediante QR y conservar búsqueda manual. | US-033, US-034, TS-010 y SP-001.   |
| DS-03        | Registrar una incidencia y programar seguimiento.              | US-014 a US-019 y US-025 a US-028. |
| DS-04        | Autorizar al veterinario y registrar una atención trazable.    | US-016 a US-024.                   |
| DS-05        | Sincronizar operaciones idempotentes y comunicar conflictos.   | US-029 a US-032 y TS-007.          |
| DS-06        | Procesar una suscripción mediante Stripe.                      | US-038 a US-040 y TS-011.          |

DS-01. Registro móvil de un animal

El ganadero registra un animal en la finca seleccionada. La aplicación guarda un comando identificado de forma única. Con conexión, el comando se envía inmediatamente; sin conexión, permanece pendiente. Livestock Management valida la propiedad de la finca y la unicidad del código antes de producir AnimalRegistered.

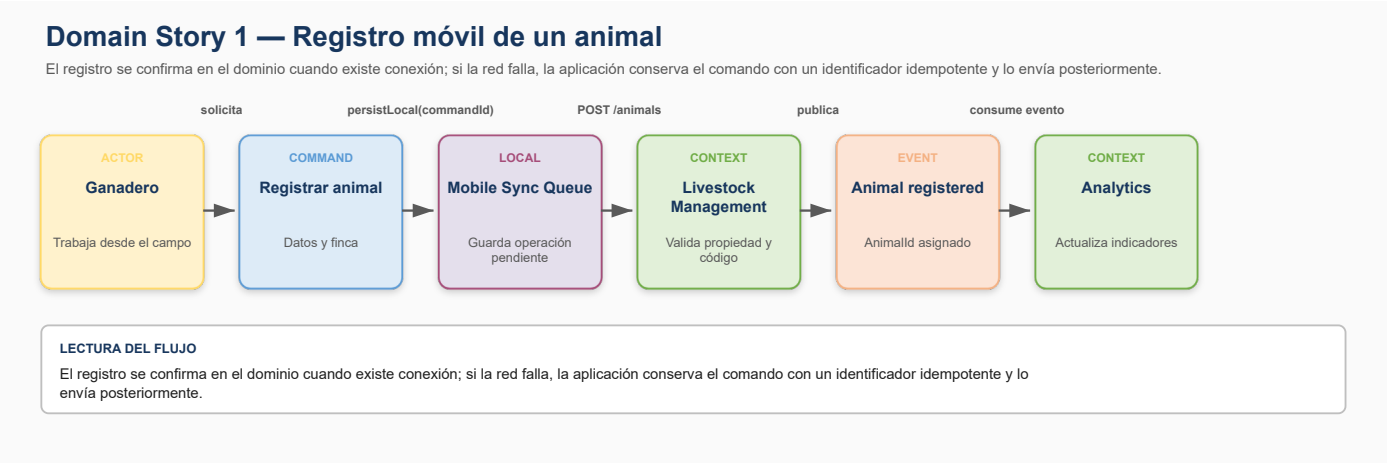


Figura 2.5.12. Domain Story del registro móvil de un animal. Fuente: elaboración propia.

DS-02. Identificación mediante QR

La cámara proporciona la imagen y ML Kit decodifica el QR dentro del dispositivo. El código resultante se envía a Livestock Management, que resuelve AnimalId y comprueba que el usuario pueda consultar la ficha. Si no se obtiene un código válido, la búsqueda manual mantiene el resultado de negocio disponible.

### Domain Story 2 — Identificación de un animal mediante QR

La cámara y ML Kit interpretan el QR en el dispositivo. Livestock Management resuelve el identificador y aplica autorización; la búsqueda manual mantiene el flujo disponible si el escaneo falla.



#### LECTURA DEL FLUJO

La cámara y ML Kit interpretan el QR en el dispositivo. Livestock Management resuelve el identificador y aplica autorización; la búsqueda manual mantiene el flujo disponible si el escaneo falla.

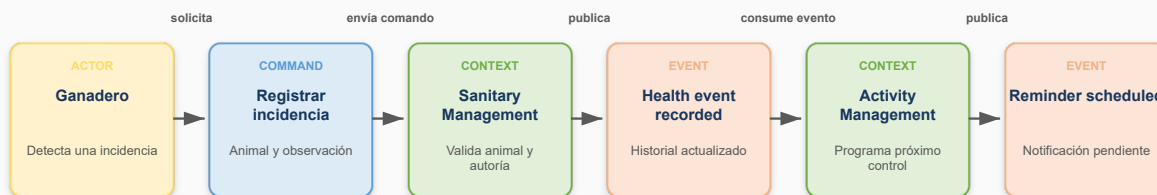
Figura 2.5.13. Domain Story de identificación mediante QR. Fuente: elaboración propia.

## DS-03. Incidencia y seguimiento sanitario

Sanitary Management incorpora la incidencia al historial del animal. Cuando el evento requiere seguimiento, publica FollowUpRequired; Activity Management lo transforma en una actividad con fecha y produce ReminderScheduled para que el mecanismo móvil correspondiente programe la notificación.

### Domain Story 3 — Incidencia y seguimiento sanitario

El ganadero registra la incidencia, Sanitary Management incorpora el evento al historial y Activity Management programa el seguimiento y la notificación correspondiente.



#### LECTURA DEL FLUJO

El ganadero registra la incidencia, Sanitary Management incorpora el evento al historial y Activity Management programa el seguimiento y la notificación correspondiente.

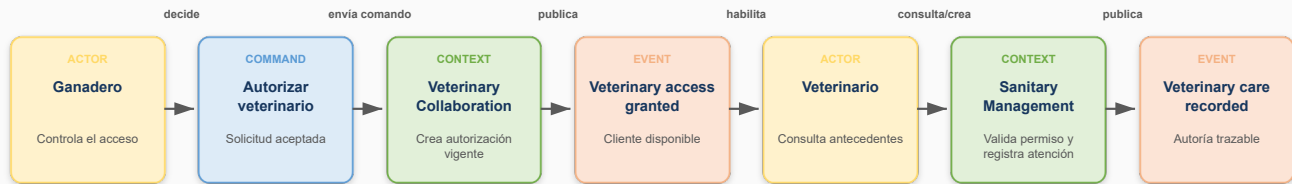
Figura 2.5.14. Domain Story de incidencia y seguimiento sanitario. Fuente: elaboración propia.

## DS-04. Autorización y atención veterinaria

Veterinary Collaboration administra la relación entre el ganadero y el veterinario. Sanitary Management consulta ese contrato antes de exponer antecedentes o registrar una atención. El evento final conserva la identidad profesional para asegurar trazabilidad.

### Domain Story 4 — Autorización y atención veterinaria

Veterinary Collaboration decide el alcance del acceso. Sanitary Management solo entrega antecedentes y acepta la atención cuando la autorización permanece vigente.



#### LECTURA DEL FLUJO

Veterinary Collaboration decide el alcance del acceso. Sanitary Management solo entrega antecedentes y acepta la atención cuando la autorización permanece vigente.

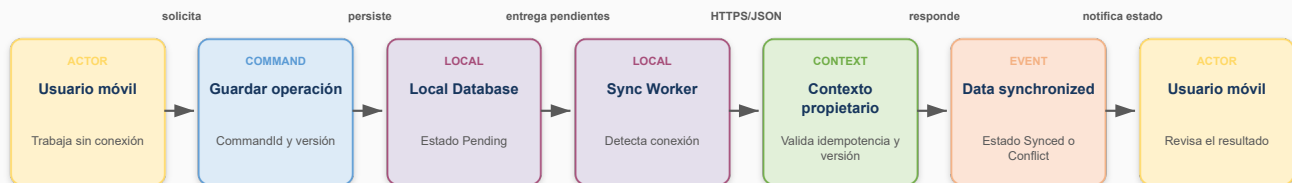
Figura 2.5.15. Domain Story de autorización y atención veterinaria. Fuente: elaboración propia.

## DS-05. Sincronización y resolución de conflictos

La base local y el Sync Worker pertenecen a la infraestructura móvil. El comando conserva CommandId, versión y fecha. El contexto propietario comprueba idempotencia y concurrencia y responde con Synced, Rejected o Conflict; la aplicación muestra el estado sin duplicar la operación.

### Domain Story 5 — Sincronización y resolución de conflictos

La sincronización es una responsabilidad de infraestructura móvil. Cada comando conserva identidad, versión y fecha; el contexto propietario decide si aplica, rechaza o devuelve un conflicto para resolución.



#### LECTURA DEL FLUJO

La sincronización es una responsabilidad de infraestructura móvil. Cada comando conserva identidad, versión y fecha; el contexto propietario decide si aplica, rechaza o devuelve un conflicto para resolución.

Figura 2.5.16. Domain Story de sincronización y resolución de conflictos. Fuente: elaboración propia.

## DS-06. Suscripción mediante Stripe

Subscription Management crea el checkout para el plan seleccionado. La Anti-Corruption Layer transforma el modelo interno al contrato de Stripe y valida la respuesta. La suscripción solo cambia a activa después de que el backend comprueba el pago; una redirección del cliente por sí sola no confirma la operación.

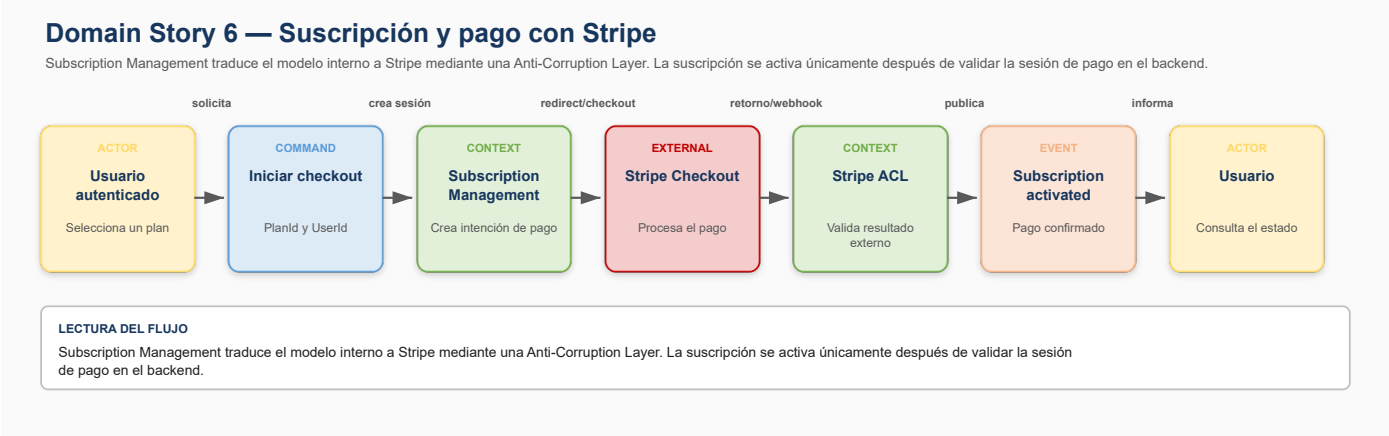


Figura 2.5.17. Domain Story de suscripción mediante Stripe. Fuente: elaboración propia.

Tipos de colaboración identificados

- Las consultas directas desde Android y Flutter utilizan JSON sobre HTTPS mediante la API REST.
- Los eventos como AnimalRegistered, HealthEventRecorded o FollowUpRequired representan mensajes del dominio y permiten desacoplar consumidores.
- Los comandos offline se procesan con idempotencia para que una retransmisión no genere duplicados.
- Analytics utiliza read models y no consulta ni modifica directamente los agregados de otros contextos.
- Stripe y el proveedor de notificaciones se aíslan mediante Anti-Corruption Layers.

2.5.1.3. Bounded Context Canvases

Los canvases de Livestock Management, Animal Health, Financial Control y Analytics del trabajo anterior se utilizaron como punto de partida. La revisión actual normaliza sus nombres, actualiza reglas según el Product Backlog y agrega los contextos ausentes. Cada canvas se elaboró siguiendo este ciclo:

1. **Context Overview Definition:** propósito y clasificación estratégica.
2. **Business Rules Distillation and Ubiquitous Language Capture:** reglas y términos exclusivos.
3. **Capability Analysis:** comandos, consultas y eventos principales.
4. **Capability Layering:** distinción entre dominio, soporte e infraestructura.
5. **Dependencies Capture:** comunicaciones entrantes, salientes y contextos relacionados.
6. **Design Critique:** supuestos, métricas, preguntas abiertas y revisión de límites.

| Bounded Context          | Resultado de la revisión                                          |
|--------------------------|-------------------------------------------------------------------|
| IAM                      | Se separaron credenciales y sesiones de los datos del perfil.     |
| Profiles                 | Se delimitó la identidad descriptiva de ganaderos y veterinarios. |
| Livestock                | Se incorporaron finca, identificación QR y alternativa manual.    |
| Sanitary                 | Se añadieron autorización, autoría y correcciones trazables.      |
| Veterinary Collaboration | Se formalizó como modelo con ciclo de vida propio.                |
| Activities               | Se relacionaron controles, estados y recordatorios.               |
| Financial                | Se separó la operación financiera de los pagos de suscripción.    |

| Bounded Context | Resultado de la revisión                                    |
|-----------------|-------------------------------------------------------------|
| Subscriptions   | Se reemplazó el pago simulado por Stripe y una ACL.         |
| Analytics       | Se limitó a indicadores respaldados por historias vigentes. |

## Capability Analysis and Layering

La revisión separa capacidades del dominio de mecanismos de aplicación e infraestructura. Esta separación evita crear bounded contexts para cada librería, pantalla o recurso del dispositivo.

| Bounded Context          | Capacidades del dominio                                       | Capacidades de aplicación o infraestructura                |
|--------------------------|---------------------------------------------------------------|------------------------------------------------------------|
| IAM                      | Registro, autenticación, sesión y autorización.               | JWT, BCrypt y almacenamiento seguro del token.             |
| Profiles                 | Creación y actualización de datos personales y profesionales. | Validación, localización y exposición REST del perfil.     |
| Livestock                | Fincas, animales, códigos e identificación.                   | Cámara, ML Kit, caché local y adaptadores REST.            |
| Sanitary                 | Historial, incidencia, atención, tratamiento y corrección.    | Persistencia local, sincronización y control de versiones. |
| Veterinary Collaboration | Solicitud, concesión, alcance y revocación del acceso.        | Consulta REST y aplicación de políticas de autorización.   |
| Activities               | Programación, atención y reprogramación.                      | Scheduler y adaptador del proveedor de notificaciones.     |
| Financial                | Ingresos, egresos, categorías y balance.                      | Caché local y exposición REST.                             |
| Subscriptions            | Planes, estado de suscripción y confirmación de pago.         | Stripe Checkout y Anti-Corruption Layer.                   |
| Analytics                | Definición de indicadores por rol y periodo.                  | Proyecciones, agregación y caché de read models.           |

## Identity and Access Management

Gestiona cuentas, credenciales, sesiones y autorización. Publica un identificador estable y el rol; no administra los datos personales del perfil.

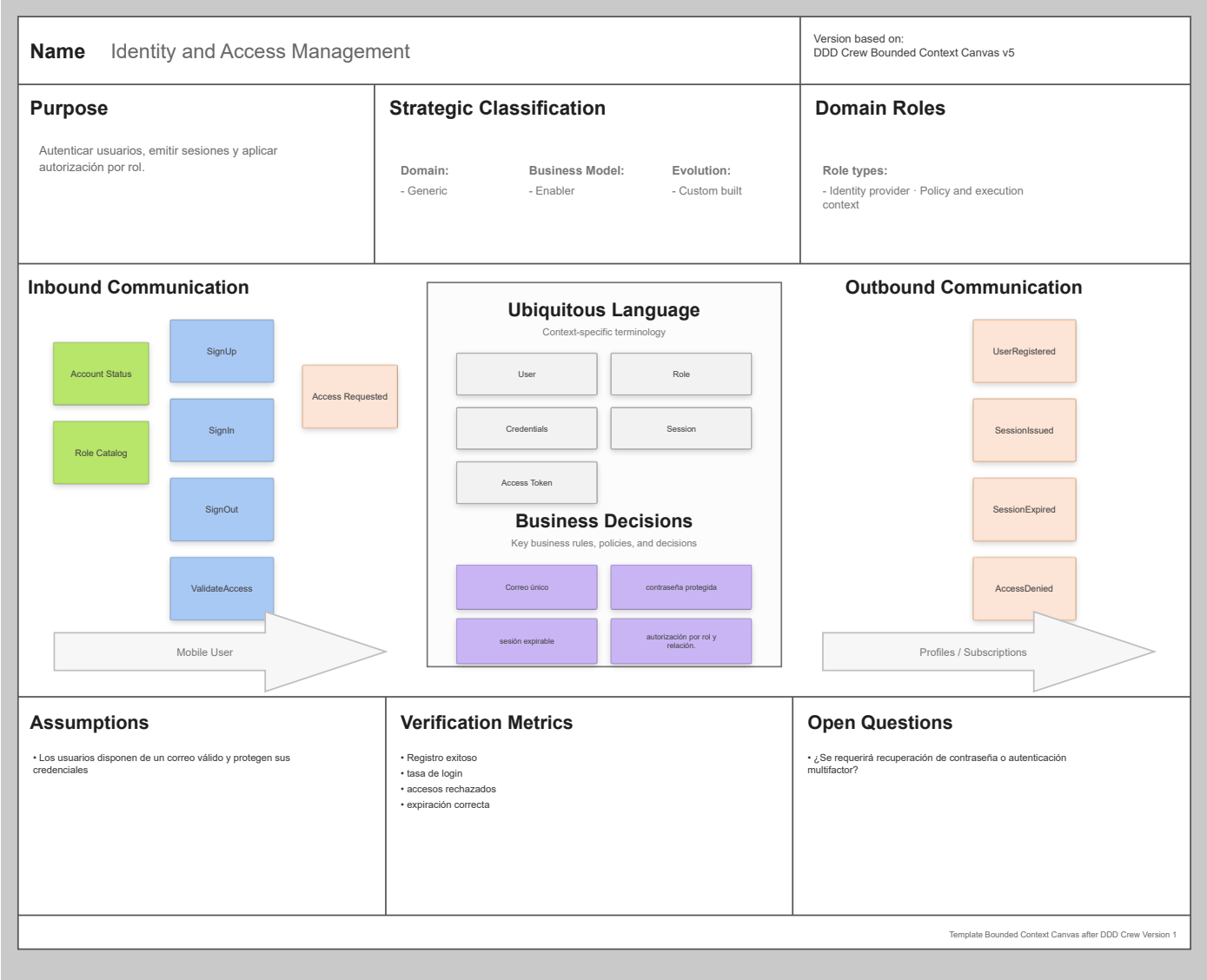


Figura 2.5.18. Bounded Context Canvas de IAM. Fuente: elaboración propia.

Profile Management

Mantiene la información personal y profesional asociada con la identidad. Su separación evita que los cambios del perfil alteren el modelo de autenticación.

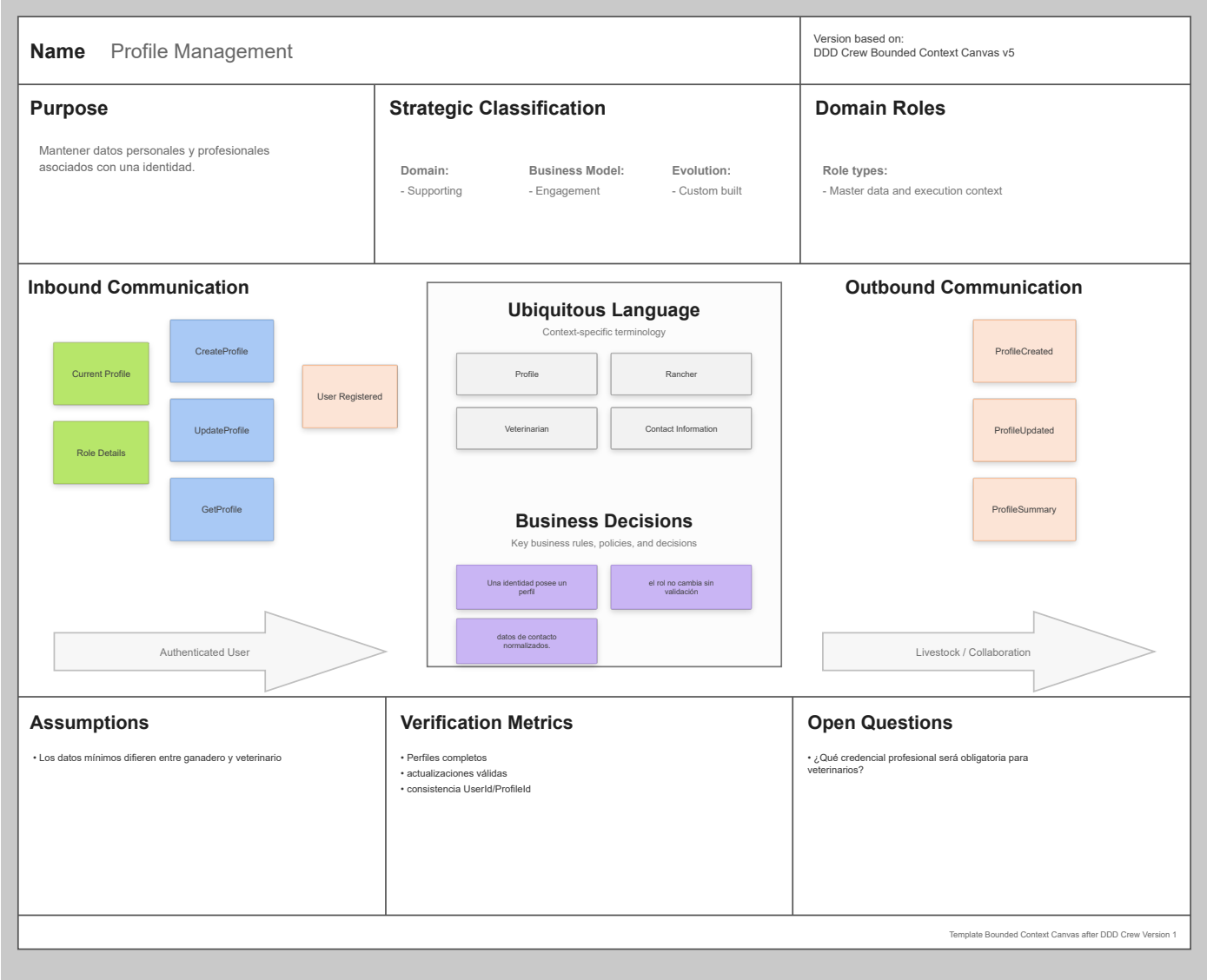


Figura 2.5.19. Bounded Context Canvas de Profile Management. Fuente: elaboración propia.

Livestock Management

Concentra el registro de fincas, hatos y animales. El QR es un medio de identificación y no sustituye el identificador estable del animal.

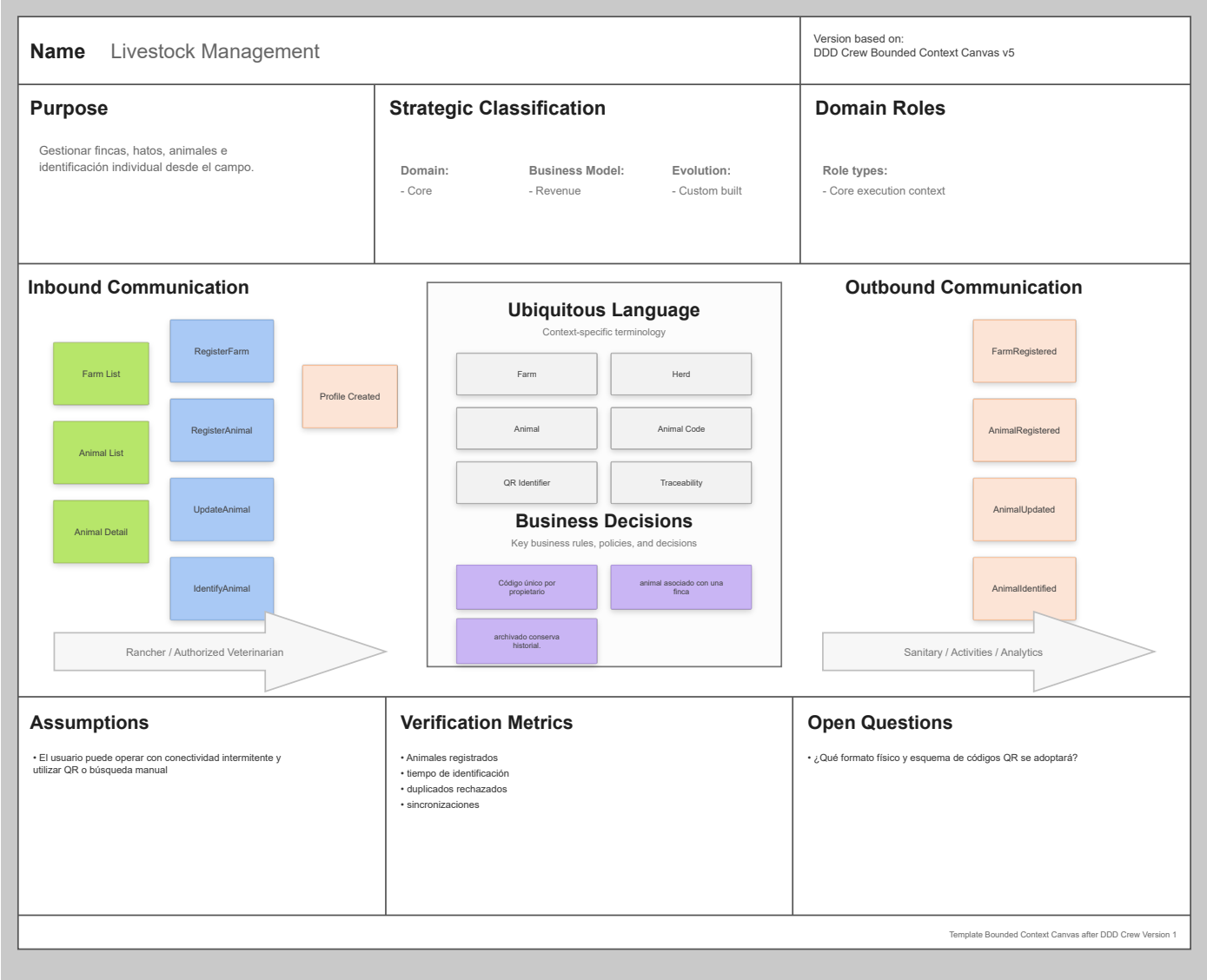


Figura 2.5.20. Bounded Context Canvas de Livestock Management. Fuente: elaboración propia.

Sanitary Management

Mantiene el historial sanitario, las atenciones y la trazabilidad clínica. Depende de Livestock para reconocer el animal y de Veterinary Collaboration para validar al profesional.

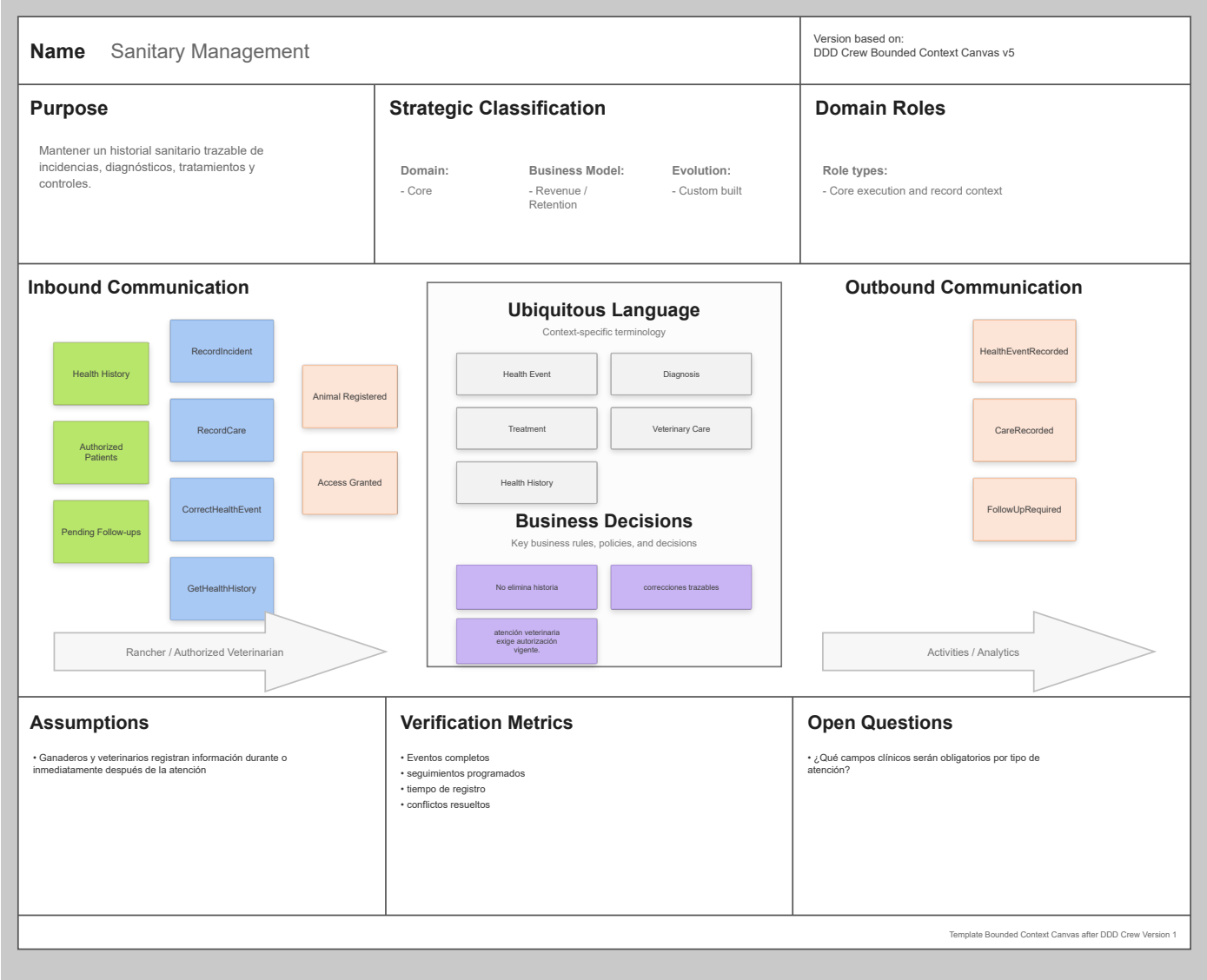


Figura 2.5.21. Bounded Context Canvas de Sanitary Management. Fuente: elaboración propia.

Veterinary Collaboration

Define solicitudes, autorizaciones, revocaciones, clientes y pacientes. La autorización es temporal y delimita qué información puede consultar o modificar el veterinario.

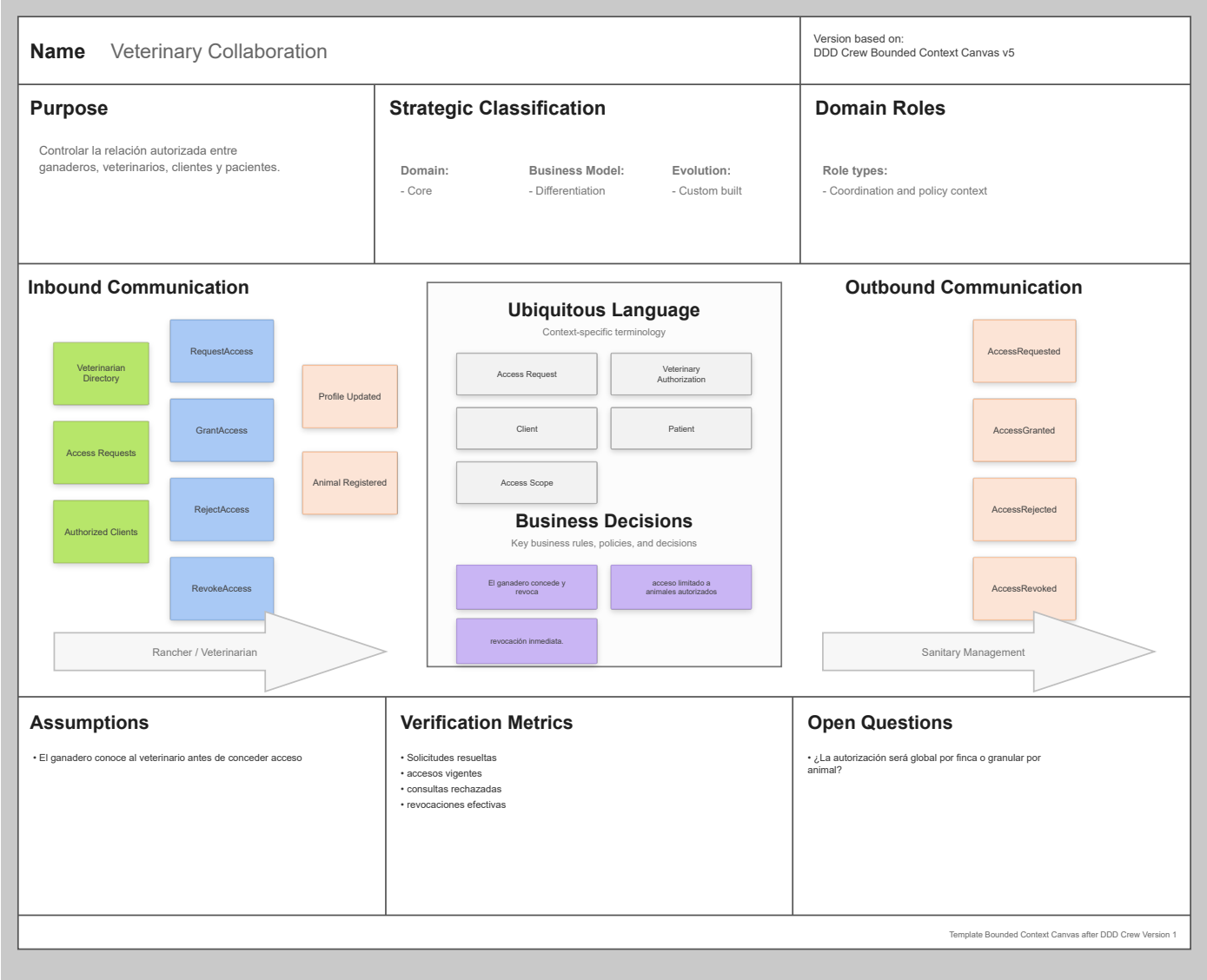


Figura 2.5.22. Bounded Context Canvas de Veterinary Collaboration. Fuente: elaboración propia.

Activity Management

Administra actividades, fechas, responsables, estados y reprogramaciones. El mecanismo de notificación ejecuta la entrega, mientras Activity Management conserva la decisión de negocio sobre cuándo corresponde recordar.

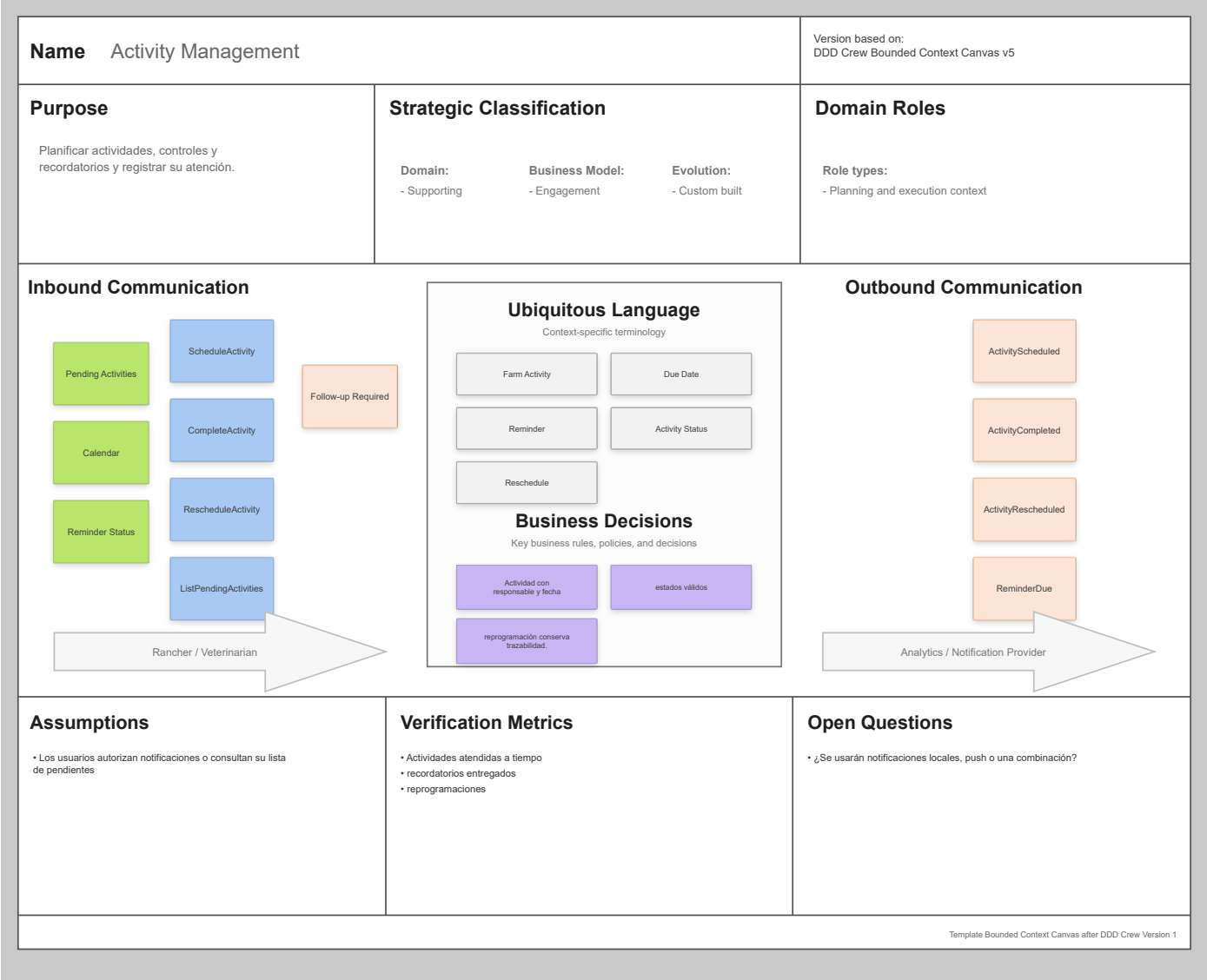


Figura 2.5.23. Bounded Context Canvas de Activity Management. Fuente: elaboración propia.

Financial Management

Registra ingresos y egresos operativos del ganadero. Su alcance inicial proporciona visibilidad económica básica y no reemplaza un sistema contable o tributario.

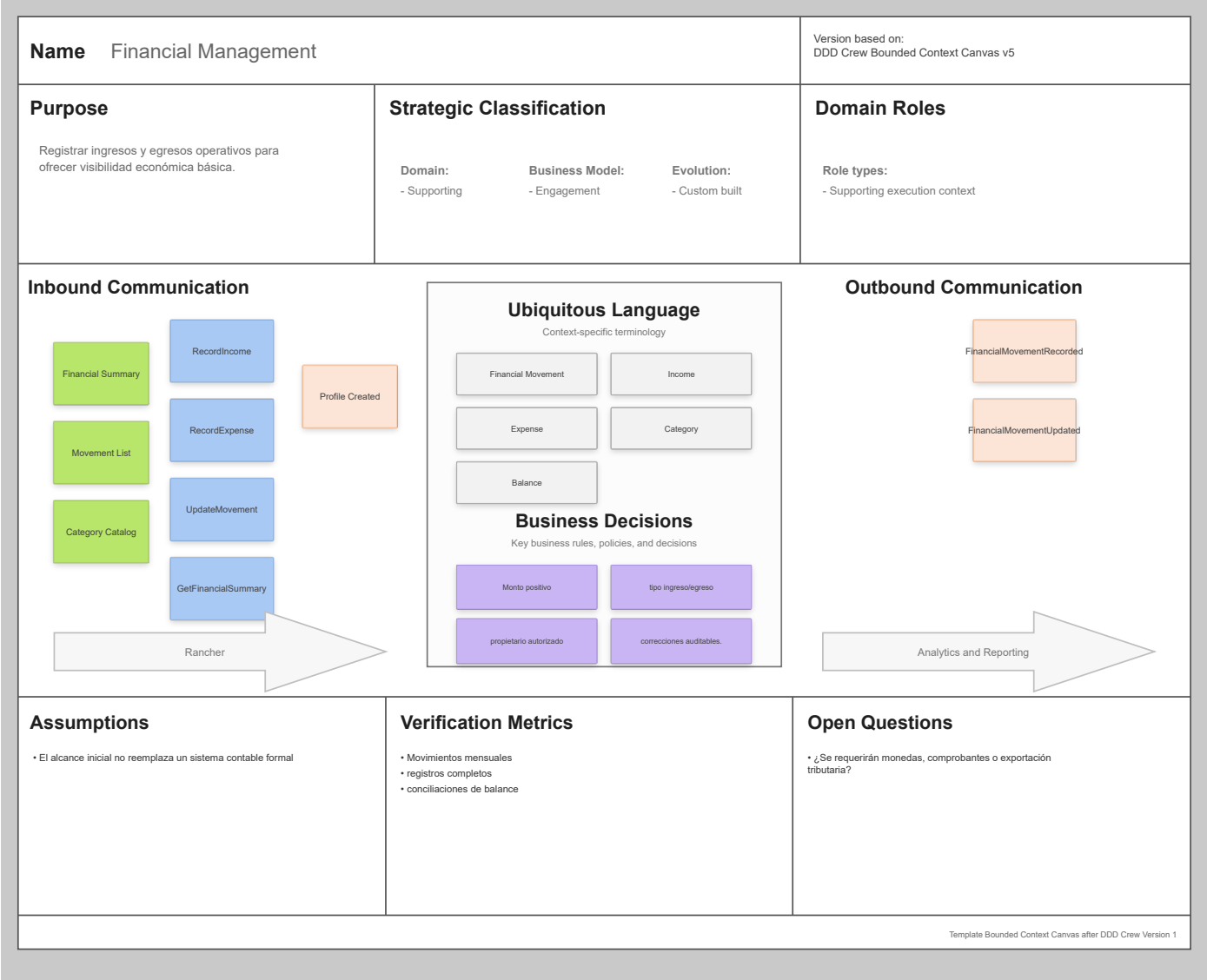


Figura 2.5.24. Bounded Context Canvas de Financial Management. Fuente: elaboración propia.

Subscription Management

Administra planes, checkout, pagos confirmados y estado de suscripción. Stripe permanece fuera del modelo y se traduce mediante una Anti-Corruption Layer.

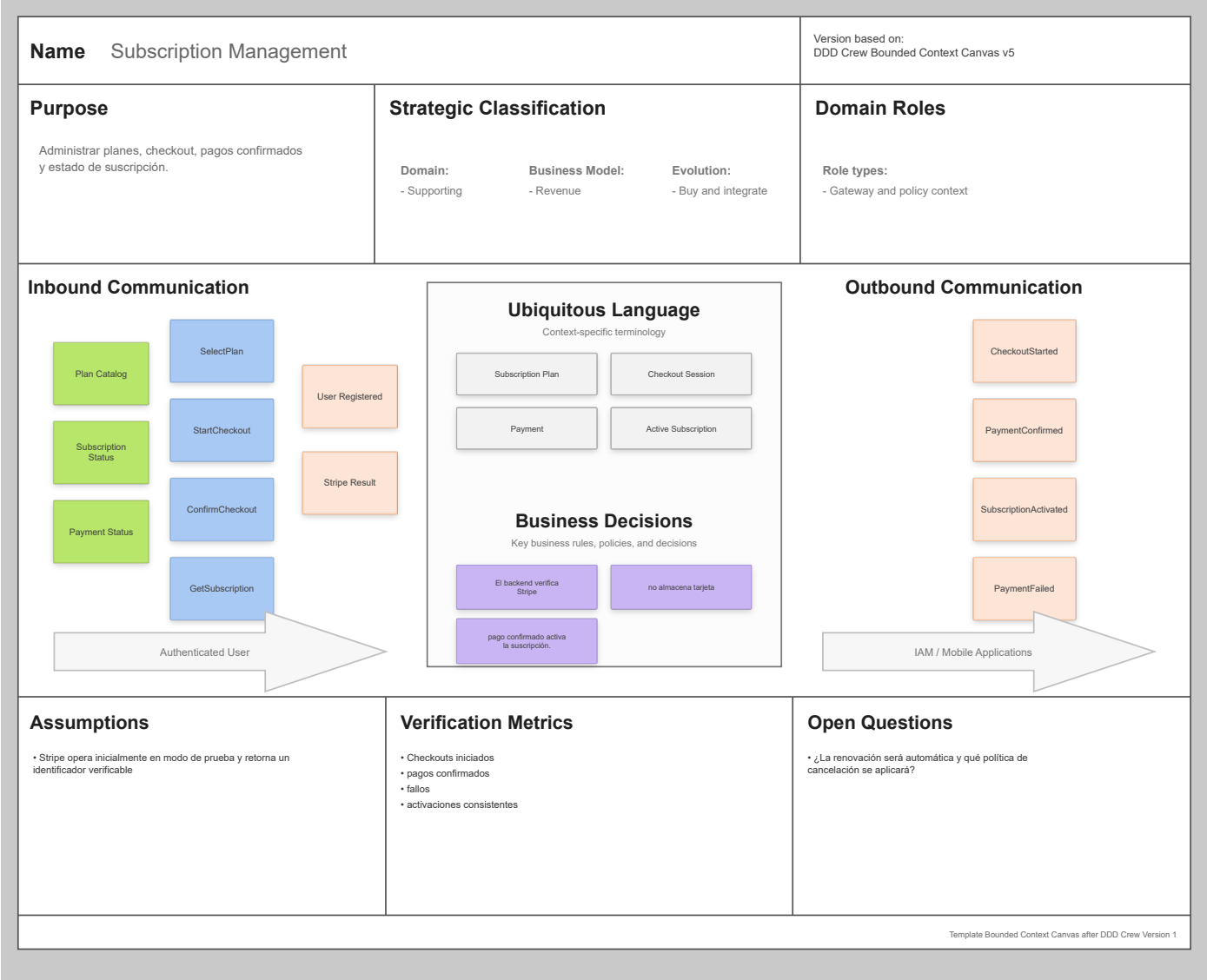


Figura 2.5.25. Bounded Context Canvas de Subscription Management. Fuente: elaboración propia.

**Analytics and Reporting**

Construye read models para indicadores de ganaderos y veterinarios. Consume información publicada por los contextos fuente y no adquiere propiedad sobre sus reglas o registros.

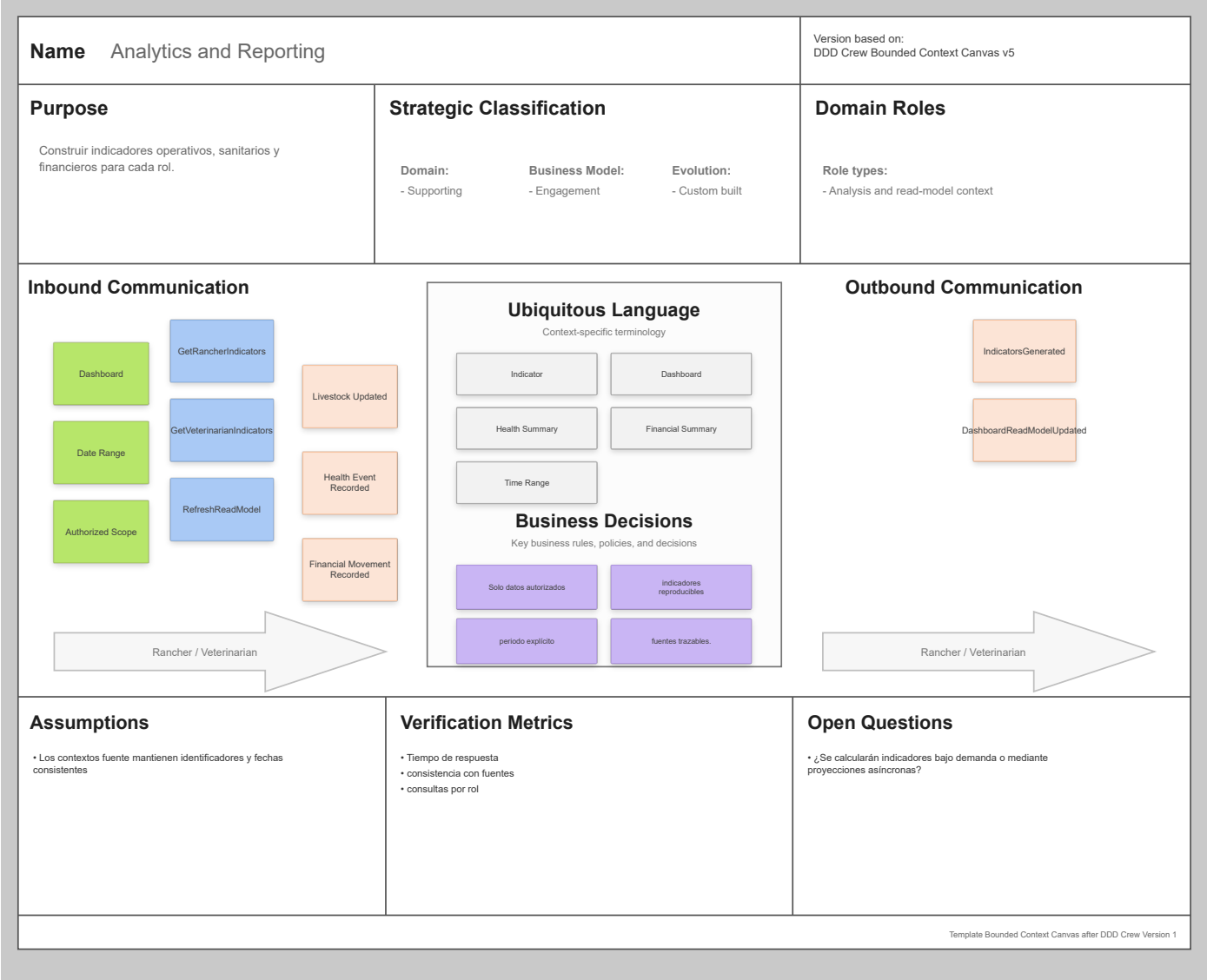


Figura 2.5.26. Bounded Context Canvas de Analytics and Reporting. Fuente: elaboración propia.

Las preguntas abiertas de cada canvas forman parte del Design Critique y deberán resolverse durante los sprints sin modificar silenciosamente el lenguaje o los límites. Cualquier decisión que cambie responsabilidades deberá reflejarse primero en el Context Map y después en el diseño táctico.

2.5.2. Context Mapping

El Context Mapping compara las relaciones estructurales entre los bounded contexts y explicita qué modelo actúa como proveedor, qué modelo consume sus contratos y qué patrón reduce el acoplamiento. Antes de seleccionar el mapa final se evaluaron tres alternativas.

| Alternativa                   | Descripción                                                                                         | Evaluación                                                                                                                   |
|-------------------------------|-----------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| A. Contextos amplios          | Unir perfiles, colaboración, sanidad y actividades dentro de un único contexto de gestión ganadera. | Rechazada porque mezcla ciclos de vida, vocabularios y reglas de autorización; también dificulta la evolución independiente. |
| B. Replicar todos los módulos | Considerar como contextos todos los módulos del backend, incluidos Devices,                         | Rechazada porque confunde infraestructura con subdominios y conserva capacidades IoT fuera                                   |

| Alternativa                            | Descripción                                                                                                             | Evaluación                                                                                                                     |
|----------------------------------------|-------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| anteriores                             | Metrics y Shared.                                                                                                       | del backlog actual.                                                                                                            |
| C. Nueve contextos alineados con valor | Separar IAM, Profiles, Livestock, Sanitary, Veterinary Collaboration, Activities, Financial, Subscriptions y Analytics. | Seleccionada porque mantiene reglas cohesivas, coincide con el alcance móvil y aprovecha la modularidad existente del backend. |

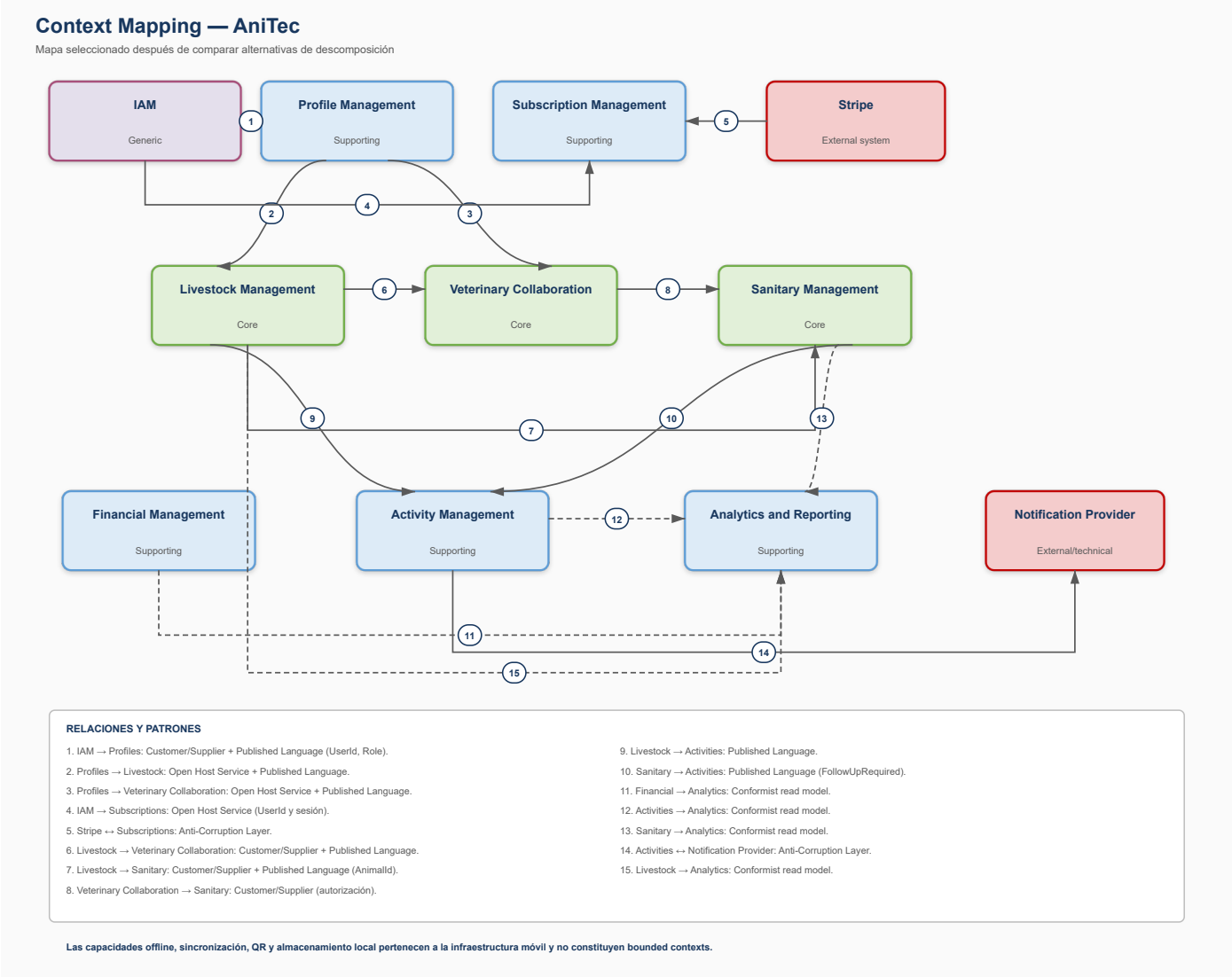


Figura 2.5.27. Context Map seleccionado para AniTec. Fuente: elaboración propia.

Relaciones entre bounded contexts

| Upstream / proveedor | Downstream / consumidor | Patrón                                 | Contrato principal              |
|----------------------|-------------------------|----------------------------------------|---------------------------------|
| IAM                  | Profiles                | Customer/Supplier y Published Language | UserId, Role, UserRegistered.   |
| IAM                  | Subscription Management | Open Host Service                      | Identidad y sesión autenticada. |
| Profile Management   | Livestock Management    | Open Host Service y Published Language | Resumen del propietario.        |

| Upstream /<br>proveedor                        | Downstream /<br>consumidor  | Patrón                                    | Contrato principal                                                    |
|------------------------------------------------|-----------------------------|-------------------------------------------|-----------------------------------------------------------------------|
| Profile Management                             | Veterinary<br>Collaboration | Open Host Service y<br>Published Language | Identidad y rol de las partes.                                        |
| Livestock Management                           | Veterinary<br>Collaboration | Customer/Supplier y<br>Published Language | Propietario, finca y animales.                                        |
| Livestock Management                           | Sanitary<br>Management      | Customer/Supplier y<br>Published Language | AnimalId, estado y propiedad.                                         |
| Veterinary<br>Collaboration                    | Sanitary<br>Management      | Customer/Supplier                         | Estado y alcance de la autorización.                                  |
| Livestock Management                           | Activity<br>Management      | Published Language                        | Referencia de animal y finca.                                         |
| Sanitary Management                            | Activity<br>Management      | Published Language                        | FollowUpRequired.                                                     |
| Livestock, Sanitary,<br>Activities y Financial | Analytics and<br>Reporting  | Conformist read models                    | Eventos y resúmenes publicados para<br>proyecciones.                  |
| Stripe                                         | Subscription<br>Management  | Anti-Corruption Layer                     | Checkout Session y resultado de pago<br>traducidos al modelo interno. |
| Notification Provider                          | Activity<br>Management      | Anti-Corruption Layer                     | Solicitud de entrega y resultado de<br>notificación.                  |

## Design Critique del Context Map

- **Coherencia con el negocio:** cada contexto contiene conceptos que cambian por las mismas reglas y razones.
- **Protección del Core Domain:** Livestock, Sanitary y Veterinary Collaboration no dependen del modelo externo de Stripe ni del proveedor de notificaciones.
- **Autonomía de Analytics:** sus proyecciones pueden reconstruirse sin modificar los datos fuente.
- **Movilidad:** Android y Flutter consumen contratos publicados por la API; no introducen un segundo modelo del dominio.
- **Offline:** los clientes almacenan comandos y read models locales, mientras las reglas de aceptación siguen en el contexto propietario.
- **Evolución:** reemplazar ML Kit, Stripe o el proveedor de notificaciones no obliga a cambiar el Ubiquitous Language.
- **Trazabilidad:** los mismos nueve contextos deberán utilizarse en el capítulo 2.6 y en los futuros diagramas C4.

El mapa seleccionado será la base de 2.5.3 Software Architecture. En esa sección se representarán la landing page, Android, Flutter, almacenamiento local, API REST, MySQL, Stripe, ML Kit, notificaciones y Firebase App Distribution sin convertir los contenedores técnicos en bounded contexts adicionales.

## 2.5.3. Software Architecture

La arquitectura de AniTec se documenta con el modelo C4 para representar la solución con distintos niveles de detalle. Las tres vistas se construyen desde un único modelo en Structurizr DSL, de modo que los actores, dependencias, tecnologías y relaciones conserven el mismo significado. El archivo fuente se encuentra en [anitec-software-architecture.dsl](#).

La solución reutiliza la API REST y la persistencia desarrolladas en el proyecto anterior, pero reemplaza el frontend web como canal operativo por una aplicación Android nativa y una aplicación multiplataforma. La landing page mantiene una función informativa. Los nueve bounded contexts definidos en las secciones anteriores pertenecen al backend y no se modelan como contenedores independientes en este nivel; su estructura interna se desarrollará en el diseño táctico del punto 2.6.

### 2.5.3.1. Software Architecture Context Level Diagrams

El Context Level Diagram delimita a AniTec como un único sistema de software y muestra quién lo usa y de qué servicios externos depende. El **Rancher** administra su operación ganadera; el **Veterinarian** consulta pacientes autorizados y registra atenciones; el **Visitor** conoce la propuesta mediante la landing page; y el **Evaluator / Tester** instala las versiones móviles distribuidas para la validación del curso.

Stripe procesa el checkout de las suscripciones, Firebase Cloud Messaging entrega notificaciones push, Google ML Kit reconoce códigos QR en el dispositivo y Firebase App Distribution distribuye compilaciones firmadas. Estos servicios permanecen fuera del límite de AniTec y se integran mediante contratos que evitan trasladar sus modelos al dominio.

| Elemento externo          | Relación con AniTec                                           | Resultado esperado                                                                |
|---------------------------|---------------------------------------------------------------|-----------------------------------------------------------------------------------|
| Visitor                   | Consulta la landing page.                                     | Comprende la propuesta de valor y encuentra el canal de acceso al producto.       |
| Rancher                   | Usa las aplicaciones móviles.                                 | Gestiona fincas, animales, sanidad, actividades y finanzas.                       |
| Veterinarian              | Usa las aplicaciones móviles con autorización del ganadero.   | Consulta clientes y pacientes y mantiene la trazabilidad sanitaria permitida.     |
| Evaluator / Tester        | Recibe e instala builds mediante Firebase App Distribution.   | Valida la aplicación en un dispositivo físico y una versión identificable.        |
| Stripe                    | Recibe solicitudes de checkout y devuelve resultados de pago. | AniTec activa o conserva la suscripción según una confirmación válida.            |
| Firebase Cloud Messaging  | Recibe solicitudes de entrega de notificaciones.              | El dispositivo presenta recordatorios definidos por Activity Management.          |
| Google ML Kit             | Procesa imágenes de la cámara localmente.                     | La aplicación obtiene el código que Livestock Management resuelve como un animal. |
| Firebase App Distribution | Publica builds firmadas para usuarios autorizados.            | Las versiones de prueba y entrega se distribuyen con trazabilidad.                |

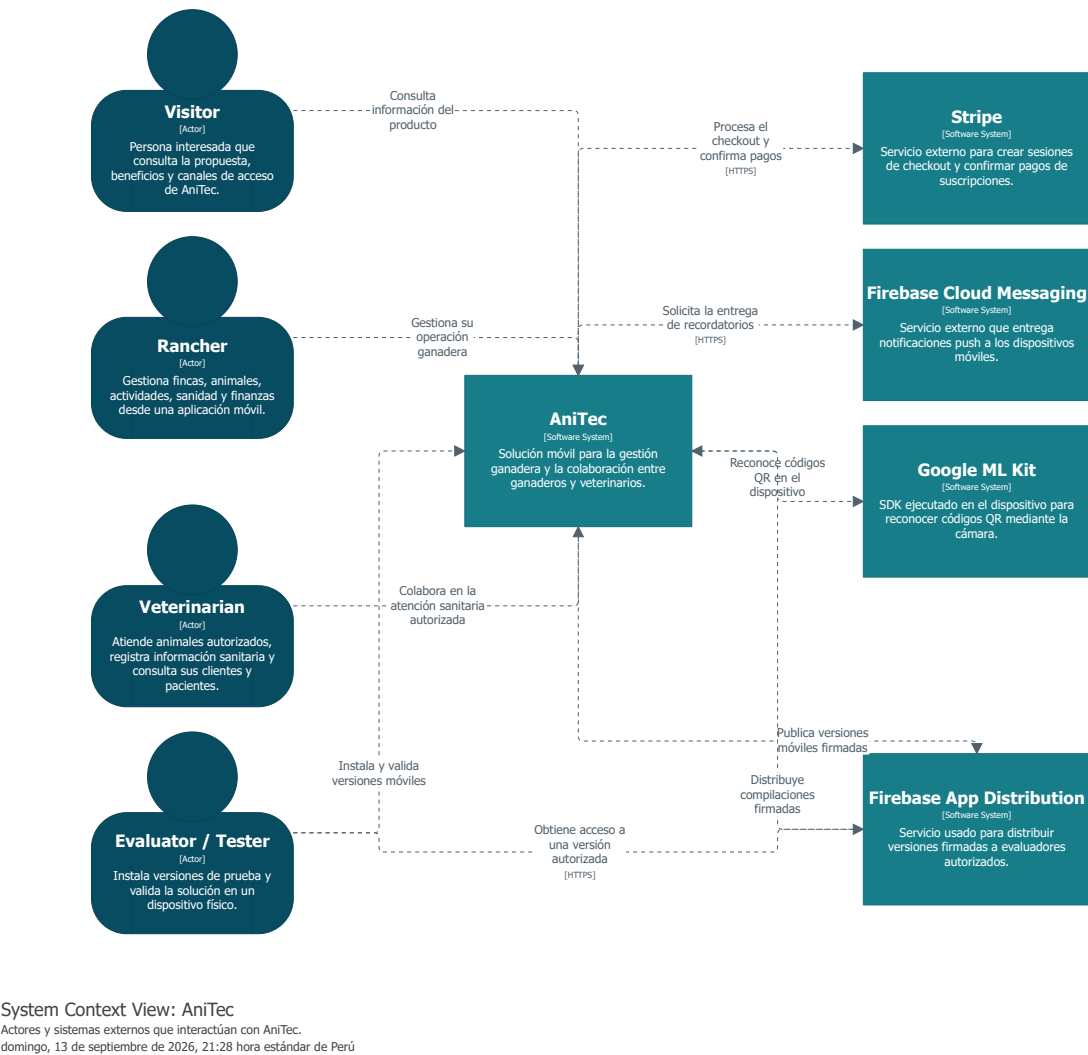


Figura 2.5.28. Software Architecture Context Level Diagram de AniTec. Fuente: elaboración propia con Structurizr DSL.

2.5.3.2. Software Architecture Container Level Diagrams

El Container Level Diagram descompone AniTec en unidades ejecutables o de almacenamiento. La landing page comunica el producto; las dos aplicaciones móviles implementan la experiencia de los roles; cada cliente mantiene almacenamiento local para caché y operaciones pendientes; la API centraliza autenticación, autorización y reglas de negocio; y MySQL conserva la fuente de verdad compartida.

Android y Flutter consumen los mismos contratos JSON sobre HTTPS y se autentican con JWT. La persistencia local mejora la continuidad ante conectividad limitada, pero las reglas de aceptación continúan bajo el bounded context propietario en la API. La cámara y ML Kit actúan como capacidades del dispositivo: decodifican un QR, mientras Livestock Management determina qué animal representa y si el usuario puede consultarlo.

| Contenedor                 | Responsabilidad                                                                              | Tecnología                |
|----------------------------|----------------------------------------------------------------------------------------------|---------------------------|
| Landing Page               | Presenta la propuesta de valor, segmentos, características y acceso a versiones autorizadas. | HTML, CSS y JavaScript.   |
| Native Android Application | Implementa los flujos móviles nativos, utiliza cámara y notificaciones y sincroniza datos.   | Kotlin y Jetpack Compose. |

| Contenedor                        | Responsabilidad                                                                                 | Tecnología                                              |
|-----------------------------------|-------------------------------------------------------------------------------------------------|---------------------------------------------------------|
| Android Local Database            | Almacena caché, read models y operaciones pendientes de sincronización.                         | Room sobre SQLite.                                      |
| Cross-Platform Mobile Application | Implementa el mismo alcance funcional mediante una base de código multiplataforma.              | Flutter y Dart.                                         |
| Flutter Local Database            | Proporciona persistencia local equivalente para la aplicación multiplataforma.                  | SQLite mediante una biblioteca compatible con Flutter.  |
| AniTec REST API                   | Expone casos de uso, aplica reglas de negocio y autorización y coordina integraciones externas. | C#, ASP.NET Core, Entity Framework Core, JWT y OpenAPI. |
| AniTec Database                   | Persiste la información transaccional y las proyecciones de los bounded contexts.               | MySQL.                                                  |

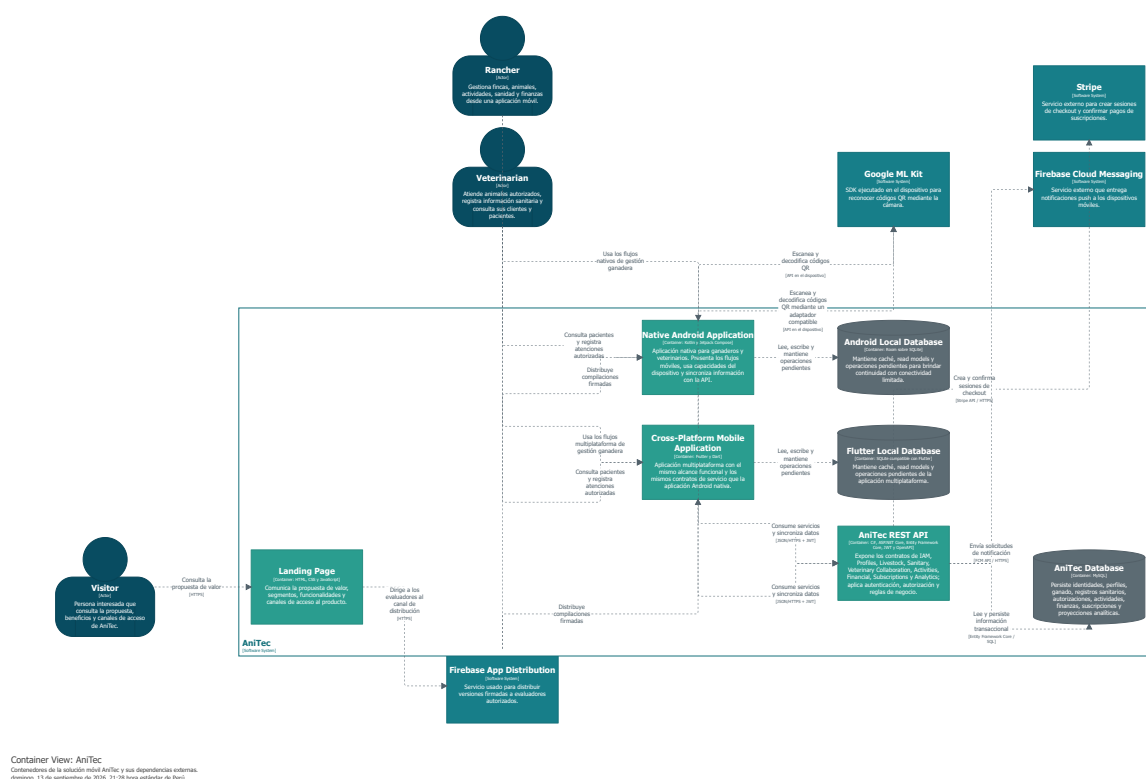


Figura 2.5.29. Software Architecture Container Level Diagram de AniTec. Fuente: elaboración propia con Structurizr DSL.

### 2.5.3.3. Software Architecture Deployment Diagrams

El Deployment Diagram representa la topología objetivo para las entregas del curso. La aplicación Android nativa se instala en un dispositivo Android físico y conserva su base Room dentro del mismo dispositivo. La aplicación Flutter se instala en el dispositivo físico correspondiente a la plataforma objetivo definida por el equipo y mantiene su propia base SQLite local. En ambos casos, la cámara y el reconocimiento QR se ejecutan dentro del dispositivo.

La landing page se publica como contenido estático en GitHub Pages. La API ASP.NET Core se ejecuta como Web Service en Render y accede a MySQL administrado en Filess.io, destinos recuperados del despliegue funcional del proyecto anterior. Stripe, Firebase Cloud Messaging y Firebase App Distribution se mantienen como servicios

externos. Esta topología deberá actualizarse si durante los sprints cambia algún proveedor, sin alterar las responsabilidades de los contenedores.

| Nodo de despliegue           | Artefacto o instancia                                                 | Comunicación principal                                                        |
|------------------------------|-----------------------------------------------------------------------|-------------------------------------------------------------------------------|
| Physical Android Device      | Native Android Application, Room/SQLite, cámara y ML Kit.             | HTTPS con la API; almacenamiento y reconocimiento local.                      |
| Cross-Platform Target Device | Flutter Application, SQLite, cámara y adaptador de reconocimiento QR. | HTTPS con la API; almacenamiento y reconocimiento local.                      |
| GitHub Pages                 | Landing Page.                                                         | HTTPS con visitantes y acceso al canal de distribución.                       |
| Render                       | AniTec REST API sobre el runtime de .NET.                             | JSON/HTTPS con clientes móviles; SQL con MySQL; HTTPS con servicios externos. |
| Filess.io                    | AniTec Database sobre MySQL.                                          | Conexión restringida desde la API.                                            |
| Firebase App Distribution    | Builds móviles firmadas y versionadas.                                | Distribución a evaluadores autorizados.                                       |
| Firebase Cloud Messaging     | Servicio de entrega de notificaciones.                                | Solicitudes desde el backend y entrega al dispositivo.                        |
| Stripe Cloud                 | Checkout y confirmación de pago.                                      | Stripe API sobre HTTPS.                                                       |

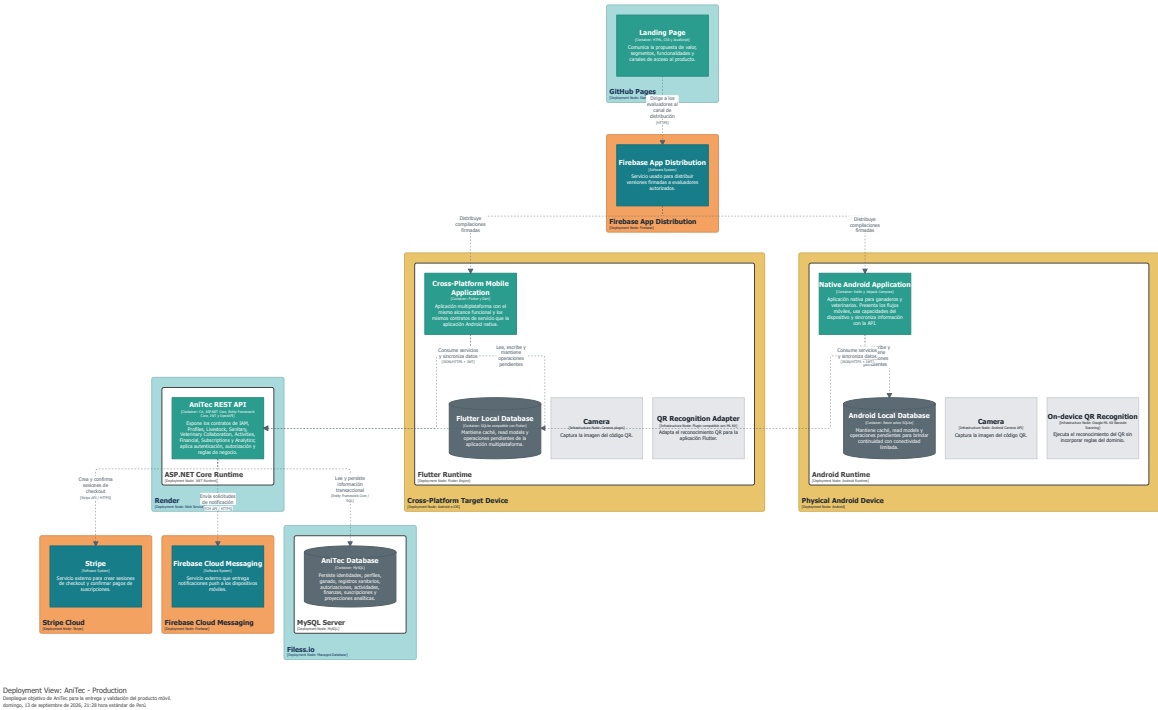


Figura 2.5.30. Software Architecture Deployment Diagram de AniTec. Fuente: elaboración propia con Structurizr DSL.

Las tres imágenes deben exportarse desde el mismo archivo DSL. Los nombres y rutas requeridos se detallan en [codefordiagrams/README.md](#), de modo que al copiar los PNG exportados los placeholders del informe se resuelvan sin modificar nuevamente el contenido.

## 2.6. Tactical-Level Domain-Driven Design

Esta sección describe cómo los nueve bounded contexts identificados en el diseño estratégico se materializan en clases, capas, componentes y estructuras de persistencia. La propuesta utiliza un enfoque híbrido trazable: conserva nombres y responsabilidades del backend heredado y presenta como diseño objetivo los elementos requeridos para Android, Flutter y la evolución del dominio.

Para todos los contextos se aplican las siguientes convenciones:

- La **Domain Layer** mantiene entidades, aggregates, value objects, servicios de dominio e interfaces de repositorio.
- La **Application Layer** orquesta comandos, consultas, eventos y casos de uso sin depender de frameworks de presentación.
- La **Interface Layer** traduce HTTP o acciones de la interfaz móvil hacia los casos de uso.
- La **Infrastructure Layer** implementa persistencia, red, seguridad, sincronización e integraciones externas.
- MySQL es la fuente autoritativa; Room y SQLite mantienen caché y un outbox idempotente.
- Las reglas de negocio permanecen en la API y el dominio; Android y Flutter reutilizan los mismos contratos y lenguaje ubicuo.

| Sección | Bounded Context                                | Módulo heredado | Propósito                                                                                                                                        |
|---------|------------------------------------------------|-----------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| 2.6.1   | <a href="#">Identity and Access Management</a> | lam             | Administrar identidades, credenciales, roles y sesiones para que cada operación de AniTec se ejecute con una identidad autenticada y autorizada. |
| 2.6.2   | <a href="#">Profile Management</a>             | Profiles        | Mantener la información personal y de contacto asociada con una identidad sin mezclarla con credenciales o reglas de autenticación.              |
| 2.6.3   | <a href="#">Livestock Management</a>           | Livestock       | Gestionar fincas, hatos, animales e identificadores QR como fuente de referencia para los demás procesos ganaderos.                              |
| 2.6.4   | <a href="#">Sanitary Management</a>            | Sanitary        | Conservar la historia sanitaria del animal y controlar el registro de diagnósticos, tratamientos, prescripciones y seguimientos autorizados.     |
| 2.6.5   | <a href="#">Veterinary Collaboration</a>       | Clients         | Administrar solicitudes y autorizaciones entre ganaderos y veterinarios, delimitando clientes, pacientes y alcance de acceso.                    |
| 2.6.6   | <a href="#">Activity Management</a>            | Activities      | Planificar actividades ganaderas y sanitarias, controlar su estado y decidir cuándo corresponde generar un recordatorio.                         |
| 2.6.7   | <a href="#">Financial Management</a>           | Financial       | Registrar ingresos y egresos operativos y producir resúmenes económicos básicos para apoyar decisiones del ganadero.                             |
| 2.6.8   | <a href="#">Subscription Management</a>        | Subscriptions   | Administrar planes, vigencia de suscripciones y pagos confirmados sin introducir conceptos propios de Stripe en el dominio.                      |

| Sección | Bounded Context                         | Módulo heredado | Propósito                                                                                                                             |
|---------|-----------------------------------------|-----------------|---------------------------------------------------------------------------------------------------------------------------------------|
| 2.6.9   | <a href="#">Analytics and Reporting</a> | Analytics       | Construir proyecciones y métricas de consulta para los dashboards sin adquirir propiedad sobre los registros de los contextos fuente. |

## 2.6.1. Bounded Context: Identity and Access Management

Administrar identidades, credenciales, roles y sesiones para que cada operación de AniTec se ejecute con una identidad autenticada y autorizada.

La base implementada se encuentra en el módulo lam de la API ASP.NET Core. El diseño móvil de Android y Flutter se presenta como **diseño objetivo** porque esos clientes todavía no existen en el workspace. Los tres productos comparten contratos REST y lenguaje ubicuo, mientras la API conserva las reglas autoritativas.

### 2.6.1.1. Domain Layer

Esta capa documenta el modelo que representa el núcleo de **Identity and Access Management**. Las reglas autoritativas se ejecutan en la API; Android y Flutter mantienen modelos equivalentes para presentación, validación inmediata y trabajo offline. La columna de estado distingue el código heredado de la arquitectura objetivo.

| Clase                | Categoría            | Producto y estado                                                                     | Propósito                                                       | Atributos                                                                                                                                                               | Métodos u operaciones                                                                                                                     | Relaciones                                                                                                                                                                                 |
|----------------------|----------------------|---------------------------------------------------------------------------------------|-----------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| User                 | Aggregate Root       | Backend; modelo equivalente en Android y Flutter<br><b>Implementado en el backend</b> | Mantener identidad, credenciales protegidas y rol del usuario.  | id: int<br>username: string<br>passwordHash: string<br>fullName: string<br>role: UserRole                                                                               | UpdateUsername(username: string): User<br>UpdatePasswordHash(hash: string): User<br>UpdateProfile(fullName: string, role: UserRole): User | User se relaciona con UserRole : has;<br>AuthenticatedSession se relaciona con User : belongs to; CredentialPolicy depende de User : validates; IUserRepository depende de User : persists |
| UserRole             | Enumeration          | Backend, Android y Flutter (modelo canónico)<br><b>Diseño objetivo</b>                | Definir los valores válidos de UserRole.                        | Rancher<br>Veterinarian<br>Administrator                                                                                                                                | —                                                                                                                                         | User se relaciona con UserRole : has                                                                                                                                                       |
| AuthenticatedSession | Entity               | Backend, Android y Flutter (modelo canónico)<br><b>Diseño objetivo</b>                | Controlar vigencia y revocación de una sesión autenticada.      | userId: int<br>tokenReference: string<br>expiresAt: DateTime                                                                                                            | IsExpired(now: DateTime): bool<br>Revoke(): void                                                                                          | AuthenticatedSession se relaciona con User : belongs to                                                                                                                                    |
| CredentialPolicy     | Domain Service       | Backend, Android y Flutter (modelo canónico)<br><b>Diseño objetivo</b>                | Validar reglas de nombre de usuario y contraseña.               | —                                                                                                                                                                       | ValidateUsername(username: string): bool<br>ValidatePassword(password: string): bool                                                      | CredentialPolicy depende de User : validates                                                                                                                                               |
| IUserRepository      | Repository Interface | Backend; modelo equivalente en Android y Flutter<br><b>Implementado en el backend</b> | Abstraer la persistencia de User.                               | —                                                                                                                                                                       | FindById(id: int): User?<br>FindByUsername(username: string): User?<br>Add(user: User): void                                              | IUserRepository depende de User : persists                                                                                                                                                 |
| IamError             | Enumeration          | Backend; modelo equivalente en Android y Flutter<br><b>Implementado</b>               | Código controlado para errores de autenticación y autorización. | None<br>UserNotFound<br>UsernameAlreadyTaken<br>InvalidCredentials<br>InvalidRole<br>OperationCancelled<br>DatabaseError<br>InternalServerError<br>ExternalServiceError | —                                                                                                                                         | Es traducido a respuestas HTTP por Interface Layer.                                                                                                                                        |

## 2.6.1.2. Interface Layer

Esta capa recibe las acciones relacionadas con **registro de cuentas, autenticación, consulta de identidad y control de sesión** y las traduce a casos de uso. Los controllers y resources corresponden a la API; las pantallas y controladores de estado representan la presentación objetivo en Android y Flutter. Ninguna de estas clases implementa reglas de negocio.

| Clase                     | Categoría          | Producto y estado                                            | Propósito                                                                                           | Atributos                                                                                                                                                                                         | Métodos u operaciones                                                                                                                                                  | Relaciones                                                                                     |
|---------------------------|--------------------|--------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| AuthenticationController  | REST Controller    | Backend<br>ASP.NET Core<br><b>Implementado en el backend</b> | Publicar por HTTP las capacidades de Identity and Access Management.                                | IUserCommandService<br>userCommandService<br>IStringLocalizer<ErrorMessages><br>errorLocalizer<br>IStringLocalizer<IamMessages><br>iamLocalizer<br>ProblemDetailsFactory<br>problemDetailsFactory | SignIn(SignInResource<br>signInResource, CancellationTokens<br>cancellationToken)<br>SignUp(SignUpResource<br>signUpResource, CancellationTokens<br>cancellationToken) | Recibe resources, invoca servicios de aplicación y devuelve resources HTTP.                    |
| UsersController           | REST Controller    | Backend<br>ASP.NET Core<br><b>Implementado en el backend</b> | Publicar por HTTP las capacidades de Identity and Access Management.                                | IUserQueryService<br>userQueryService<br>IStringLocalizer<ErrorMessages><br>errorLocalizer<br>ProblemDetailsFactory<br>problemDetailsFactory                                                      | GetUserById(int id, CancellationTokens<br>cancellationToken)<br>GetAllUsers(CancellationTokens<br>cancellationToken)                                                   | Recibe resources, invoca servicios de aplicación y devuelve resources HTTP.                    |
| SignInResource            | Resource/Assembler | Backend<br>ASP.NET Core<br><b>Implementado en el backend</b> | Definir un contrato estable de entrada o salida para la API REST.                                   | string Username<br>string Password                                                                                                                                                                | Create(...)<br>Deconstruct(...)                                                                                                                                        | Es construido o traducido por assemblers y consumido por el controller y los clientes móviles. |
| SignUpResource            | Resource/Assembler | Backend<br>ASP.NET Core<br><b>Implementado en el backend</b> | Definir un contrato estable de entrada o salida para la API REST.                                   | string Username<br>string Password<br>string FullName = ""<br>string Role = "Rancher"                                                                                                             | Create(...)<br>Deconstruct(...)                                                                                                                                        | Es construido o traducido por assemblers y consumido por el controller y los clientes móviles. |
| AuthenticatedUserResource | Resource/Assembler | Backend<br>ASP.NET Core<br><b>Implementado en el backend</b> | Definir un contrato estable de entrada o salida para la API REST.                                   | int Id<br>string Username<br>string FullName<br>string Role<br>string Token                                                                                                                       | Create(...)<br>Deconstruct(...)                                                                                                                                        | Es construido o traducido por assemblers y consumido por el controller y los clientes móviles. |
| UserResource              | Resource/Assembler | Backend<br>ASP.NET Core<br><b>Implementado en el backend</b> | Definir un contrato estable de entrada o salida para la API REST.                                   | int Id<br>string Username<br>string FullName<br>string Role                                                                                                                                       | Create(...)<br>Deconstruct(...)                                                                                                                                        | Es construido o traducido por assemblers y consumido por el controller y los clientes móviles. |
| UserScreen                | Composable         | Android / Jetpack<br>Compose<br><b>Diseño objetivo</b>       | Presentar registro de cuentas, autenticación, consulta de identidad y control de sesión en Android. | uiState<br>onAction<br>navigation                                                                                                                                                                 | Render()<br>Submit()<br>Retry()                                                                                                                                        | Observa UserViewModel y emite acciones de interfaz.                                            |
| UserViewModel             | Presentation Model | Android / Kotlin<br><b>Diseño objetivo</b>                   | Mantener el estado observable y traducir acciones de Android a casos de uso.                        | state<br>observeUseCase<br>syncUseCase                                                                                                                                                            | Load()<br>Submit(action)<br>RetrySync()                                                                                                                                | Invoca casos de uso de Application Layer y publica un UI State inmutable.                      |
| UserPage                  | Widget             | Flutter / Dart<br><b>Diseño objetivo</b>                     | Presentar registro de cuentas, autenticación, consulta de identidad y control de sesión en Flutter. | state<br>onAction<br>router                                                                                                                                                                       | build(context)<br>submit()<br>retry()                                                                                                                                  | Observa UserController y emite intenciones del usuario.                                        |
| UserController            | State Controller   | Flutter / Dart<br><b>Diseño objetivo</b>                     | Mantener el estado de presentación de Flutter y coordinar casos de uso.                             | state<br>observeUseCase<br>syncUseCase                                                                                                                                                            | load()<br>submit(action)<br>retrySync()                                                                                                                                | Invoca Application Layer y publica estados de carga, éxito y error.                            |

## 2.6.1.3. Application Layer

Esta capa coordina las capacidades de **registro de cuentas, autenticación, consulta de identidad y control de sesión**. Los commands y queries expresan intenciones; los handlers cargan aggregates, aplican reglas, persisten cambios y reaccionan a eventos. Los casos de uso móviles coordinan lectura local, actualización remota y sincronización idempotente.

| Clase                  | Categoría           | Producto y estado                                            | Propósito                                                                                                                | Atributos                                                                                                                                                             | Métodos u operaciones                                                                                                                                                                                                                  | Relaciones                                                                                      |
|------------------------|---------------------|--------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| UserCommandService     | Application Service | Backend<br>ASP.NET Core<br><b>Implementado en el backend</b> | Orquestar registro de cuentas, autenticación, consulta de identidad y control de sesión sin contener reglas del dominio. | IUserRepository userRepository<br>ITokenService tokenService<br>IHashingService hashingService<br>IUnitOfWork unitOfWork<br>IStringLocalizer<ErrorMessages> localizer | Handle(SignInCommand command, CancellationToken cancellationToken)<br>Handle(SignUpCommand command, CancellationToken cancellationToken)                                                                                               | Invoca agregados y repositories; confirma la transacción mediante Unit of Work.                 |
| UserQueryService       | Application Service | Backend<br>ASP.NET Core<br><b>Implementado en el backend</b> | Orquestar registro de cuentas, autenticación, consulta de identidad y control de sesión sin contener reglas del dominio. | IUserRepository userRepository                                                                                                                                        | Handle(GetUserByIdQuery query, CancellationToken cancellationToken)<br>Handle(GetAllUsersQuery query, CancellationToken cancellationToken)<br>Handle(GetUserByUsernameQuery query, CancellationToken cancellationToken)                | Invoca agregados y repositories; confirma la transacción mediante Unit of Work.                 |
| IamContextFacade       | Application Service | Backend<br>ASP.NET Core<br><b>Implementado en el backend</b> | Orquestar registro de cuentas, autenticación, consulta de identidad y control de sesión sin contener reglas del dominio. | IUserCommandService userCommandService<br>IUserQueryService userQueryService                                                                                          | CreateUser(string username, string password, CancellationToken cancellationToken)<br>FetchUserIdByUsername(string username, CancellationToken cancellationToken)<br>FetchUsernameById(int userId, CancellationToken cancellationToken) | Invoca agregados y repositories; confirma la transacción mediante Unit of Work.                 |
| SignInCommand          | Command/Query       | Backend<br>ASP.NET Core<br><b>Implementado en el backend</b> | Transportar una intención o consulta tipada hacia su handler.                                                            | string Username<br>string Password                                                                                                                                    | —                                                                                                                                                                                                                                      | Es recibida por un handler o servicio de aplicación y no contiene lógica de negocio.            |
| SignUpCommand          | Command/Query       | Backend<br>ASP.NET Core<br><b>Implementado en el backend</b> | Transportar una intención o consulta tipada hacia su handler.                                                            | string Username<br>string Password<br>string FullName<br>string Role                                                                                                  | —                                                                                                                                                                                                                                      | Es recibida por un handler o servicio de aplicación y no contiene lógica de negocio.            |
| GetUserByIdQuery       | Command/Query       | Backend<br>ASP.NET Core<br><b>Implementado en el backend</b> | Transportar una intención o consulta tipada hacia su handler.                                                            | int Id                                                                                                                                                                | —                                                                                                                                                                                                                                      | Es recibida por un handler o servicio de aplicación y no contiene lógica de negocio.            |
| GetUserByUsernameQuery | Command/Query       | Backend<br>ASP.NET Core<br><b>Implementado en el backend</b> | Transportar una intención o consulta tipada hacia su handler.                                                            | string Username                                                                                                                                                       | —                                                                                                                                                                                                                                      | Es recibida por un handler o servicio de aplicación y no contiene lógica de negocio.            |
| ObserveUserUseCase     | Use Case            | Android y Flutter<br><b>Diseño objetivo</b>                  | Entregar primero datos locales y actualizar la consulta cuando exista conectividad.                                      | localRepository<br>remoteRepository<br>connectivityMonitor                                                                                                            | Execute(criteria): Stream<Result>                                                                                                                                                                                                      | Es invocado por ViewModel/Controller y coordina repositorios móviles.                           |
| SyncUserUseCase        | Use Case            | Android y Flutter<br><b>Diseño objetivo</b>                  | Procesar operaciones móviles pendientes de manera idempotente.                                                           | outboxRepository<br>remoteRepository<br>conflictResolver                                                                                                              | Execute(): SyncResult                                                                                                                                                                                                                  | Lee el outbox local, consume la API y actualiza el estado de sincronización.                    |
| SignUpCommandHandler   | Command Handler     | Backend<br>ASP.NET Core<br><b>Diseño objetivo</b>            | Ejecutar una intención concreta, aplicar reglas del agregado y confirmar la transacción.                                 | repository<br>unitOfWork<br>domainPolicy                                                                                                                              | Handle(command): Result                                                                                                                                                                                                                | Consume un Command, carga el aggregate mediante su repository y puede publicar un Domain Event. |
| SignInCommandHandler   | Command Handler     | Backend<br>ASP.NET Core<br><b>Diseño objetivo</b>            | Ejecutar una intención concreta, aplicar reglas del agregado y                                                           | repository<br>unitOfWork<br>domainPolicy                                                                                                                              | Handle(command): Result                                                                                                                                                                                                                | Consume un Command, carga el aggregate mediante su repository y puede publicar un Domain Event. |

| Clase                         | Categoría     | Producto y estado      | Propósito                                                                  | Atributos                                              | Métodos u operaciones     | Relaciones                                                                                           |
|-------------------------------|---------------|------------------------|----------------------------------------------------------------------------|--------------------------------------------------------|---------------------------|------------------------------------------------------------------------------------------------------|
| UserAuthenticatedEventHandler | Event Handler |                        | confirmar la transacción.                                                  |                                                        |                           |                                                                                                      |
|                               |               | Backend ASP.NET Core   | Reaccionar al evento confirmado y actualizar proyecciones o integraciones. | projectionRepository<br>notificationPort<br>unitOfWork | Handle(domainEvent): Task | Consume un Domain Event y utiliza puertos de infraestructura sin modificar directamente el agregado. |
|                               |               | <b>Diseño objetivo</b> |                                                                            |                                                        |                           |                                                                                                      |

2.6.1.4. Infrastructure Layer

Esta capa implementa los puertos definidos hacia el interior de **Identity and Access Management** y concentra acceso a base de datos, red, almacenamiento local e integraciones externas. Las clases de infraestructura traducen errores y contratos técnicos antes de devolver resultados a Application Layer.

| Clase                  | Categoría                 | Producto y estado                                                    | Propósito                                                           | Atributos                                      | Métodos u operaciones                                                                                                                                    | Relaciones                                                                                       |
|------------------------|---------------------------|----------------------------------------------------------------------|---------------------------------------------------------------------|------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| UserRepository         | Repository Adapter        | Backend / Entity Framework Core<br><b>Implementado en el backend</b> | Implementar el puerto de persistencia definido por Domain Layer.    | AppDbContext<br>context                        | FindByUsernameAsync(string username, CancellationToken cancellationToken)<br>ExistsByUsernameAsync(string username, CancellationToken cancellationToken) | Implementa IUserRepository; utiliza AppDbContext/MySQL y reconstruye el aggregate.               |
| ModelBuilderExtensions | Persistence Configuration | Backend / Entity Framework Core<br><b>Implementado en el backend</b> | Mapear entidades y value objects del contexto al modelo relacional. | EntityTypeBuilder<br>configuration             | ApplyConfiguration(modelBuilder)                                                                                                                         | Configura tablas, claves, relaciones, restricciones y conversiones de Entity Framework Core.     |
| UserApiDataSource      | Remote Adapter            | Android / Kotlin<br><b>Diseño objetivo</b>                           | Implementar el acceso remoto del cliente móvil a la API.            | httpClient<br>tokenProvider<br>serializer      | Get(criteria)<br>Create(dto)<br>Update(dto)<br>Delete(id)                                                                                                | Consume controllers REST por HTTPS/JSON y traduce errores HTTP al modelo de aplicación.          |
| UserDao                | Room Adapter              | Android / Room<br><b>Diseño objetivo</b>                             | Implementar persistencia local y observación reactiva en Android.   | roomDatabase<br>entityMapper                   | Observe(criteria)<br>Upsert(entity)<br>Delete(id)<br>Pending()                                                                                           | Implementa el puerto local mediante Room y participa en la estrategia de caché/outbox.           |
| UserRemoteDataSource   | Remote Adapter            | Flutter / Dart<br><b>Diseño objetivo</b>                             | Implementar el acceso remoto del cliente móvil a la API.            | httpClient<br>tokenProvider<br>serializer      | Get(criteria)<br>Create(dto)<br>Update(dto)<br>Delete(id)                                                                                                | Consume controllers REST por HTTPS/JSON y traduce errores HTTP al modelo de aplicación.          |
| UserLocalDataSource    | SQLite Adapter            | Flutter / Dart<br><b>Diseño objetivo</b>                             | Implementar persistencia local equivalente en Flutter.              | sqliteDatabase<br>entityMapper                 | watch(criteria)<br>upsert(entity)<br>delete(id)<br>pending()                                                                                             | Implementa el puerto local mediante SQLite y participa en la estrategia de caché/outbox.         |
| HashingService         | Security Adapter          | Backend ASP.NET Core<br><b>Implementado</b>                          | Aislar una dependencia externa detrás de un puerto explícito.       | password,<br>passwordHash                      | HashPassword(password): string<br>VerifyPassword(password, passwordHash): bool                                                                           | Implementa el puerto de hashing y utiliza BCrypt.                                                |
| TokenService           | Token Adapter             | Backend ASP.NET Core<br><b>Implementado</b>                          | Aislar una dependencia externa detrás de un puerto explícito.       | issuer, audience,<br>signingKey,<br>expiration | GenerateToken(user): string<br>ValidateToken(token): Task<int?>                                                                                          | Implementa el puerto de tokens y utiliza JWT.                                                    |
| SecureSessionStore     | Secure Storage Adapter    | Android y Flutter<br><b>Diseño objetivo</b>                          | Aislar una dependencia externa detrás de un puerto explícito.       | tokenReference,<br>expiresAt                   | Save(session)<br>Read()<br>Clear()                                                                                                                       | Implementa el puerto local de sesión sobre Keystore/EncryptedSharedPreferences o Secure Storage. |

2.6.1.5. Bounded Context Software Architecture Component Level Diagrams

El archivo [component-level.dsl](#) contiene las vistas BC1-ApiComponents, BC1-AndroidComponents y BC1-FlutterComponents. Las tres parten del mismo modelo C4 y muestran la separación entre presentación, aplicación, dominio y adaptadores.

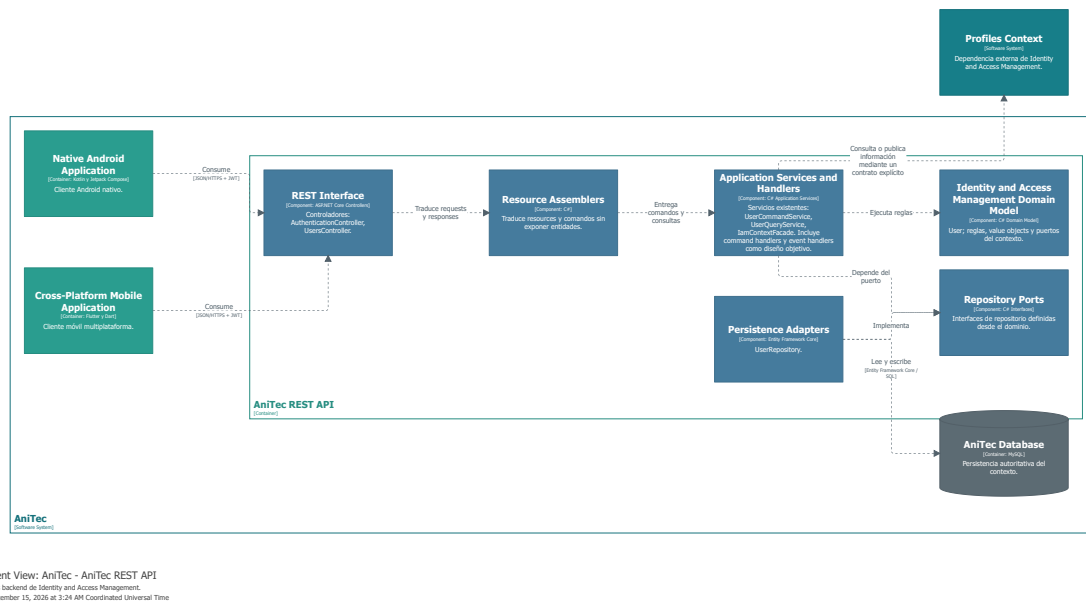


Figura 2.6.1.1. Componentes de la API para Identity and Access Management. Fuente: elaboración propia con Structurizr DSL.

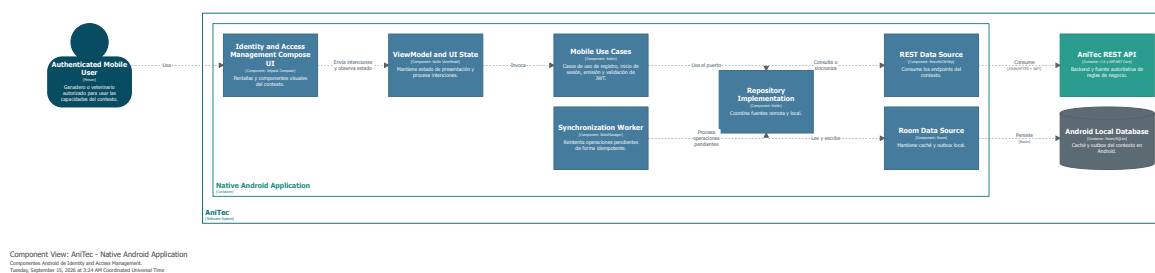


Figura 2.6.1.2. Componentes Android para Identity and Access Management. Fuente: elaboración propia con Structurizr DSL.

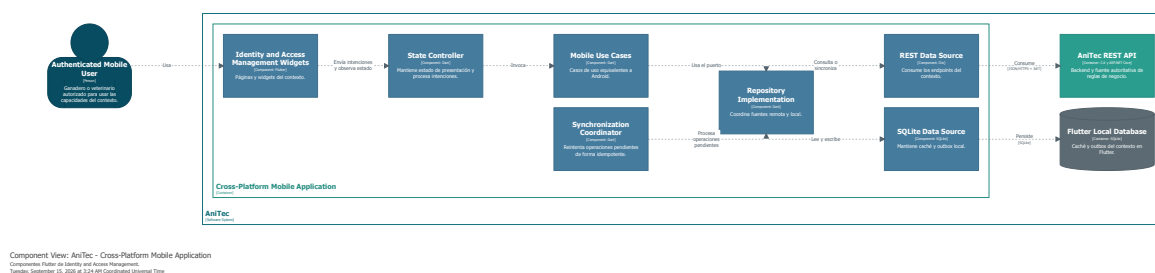


Figura 2.6.1.3. Componentes Flutter para Identity and Access Management. Fuente: elaboración propia con Structurizr DSL.

## 2.6.1.6. Bounded Context Software Architecture Code Level Diagrams

Los diagramas de código detallan el modelo del dominio y los objetos de persistencia. El UML diferencia los elementos existentes de las incorporaciones objetivo, mientras los esquemas SQL señalan mediante comentarios las columnas propuestas. Los archivos ERD quedan disponibles para completar la importación manual.

### 2.6.1.6.1. Bounded Context Domain Layer Class Diagrams

El Class Diagram incluye agregados, entidades, value objects, enumeraciones, servicios de dominio e interfaces de repositorio con atributos, operaciones, visibilidad y multiplicidades.

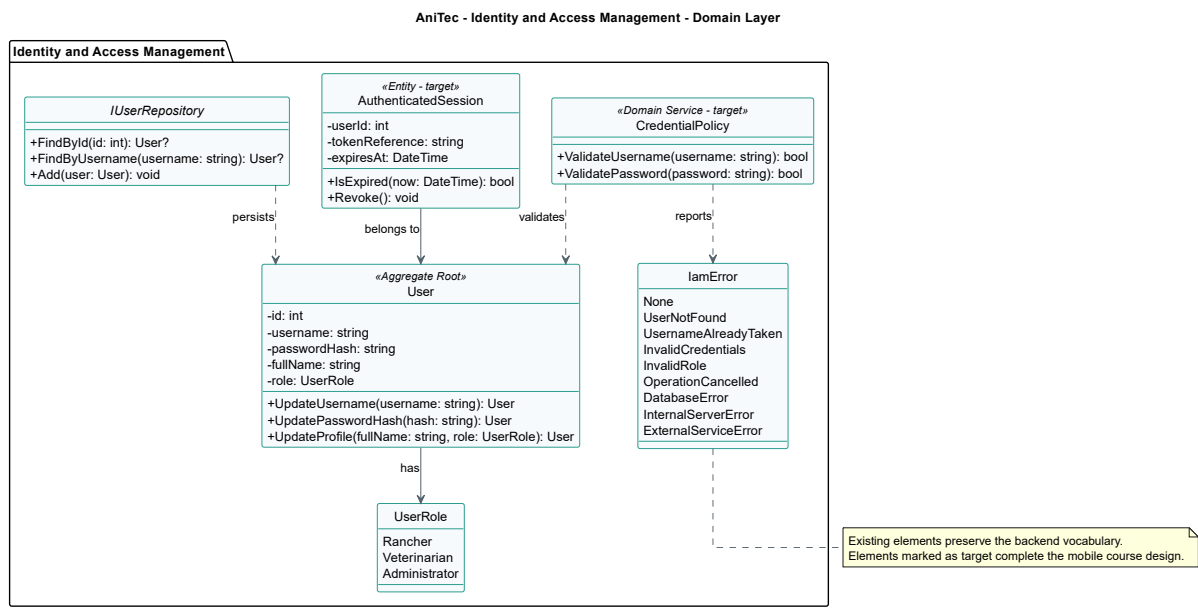


Figura 2.6.1.4. Domain Layer Class Diagram de Identity and Access Management. Fuente: elaboración propia con PlantUML.

2.6.1.6.2. Bounded Context Database Design Diagram

MySQL mantiene la persistencia autoritativa. Room y SQLite contienen únicamente caché, metadatos de sincronización y operaciones pendientes; no sustituyen las reglas ni la fuente de verdad del backend. En IAM, las credenciales y tokens permanecen fuera de las tablas locales y se almacenan mediante mecanismos seguros del sistema operativo.

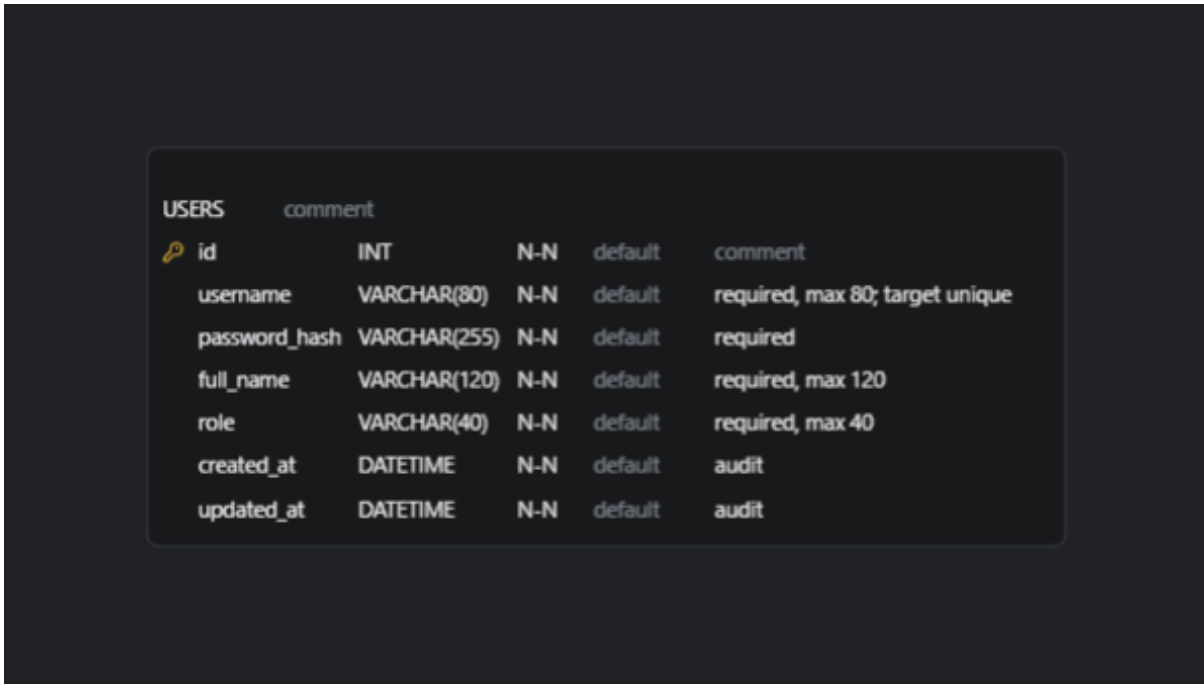


Figura 2.6.1.5. MySQL Database Design de Identity and Access Management. Fuente: elaboración propia a partir del esquema SQL.

|                         |         |      |         |         |                            |         |      |         |         |
|-------------------------|---------|------|---------|---------|----------------------------|---------|------|---------|---------|
| CACHED_IDENTITY comment |         |      |         |         | PENDING_OPERATIONS comment |         |      |         |         |
| 🔑 user_id               | INTEGER | NULL | default | comment | 🔑 operation_id             | TEXT    | NULL | default | comment |
| username                | TEXT    | N-N  | default | comment | aggregate_type             | TEXT    | N-N  | default | comment |
| full_name               | TEXT    | N-N  | default | comment | aggregate_id               | TEXT    | N-N  | default | comment |
| role                    | TEXT    | N-N  | default | comment | operation_type             | TEXT    | N-N  | default | comment |
| expires_at              | TEXT    | N-N  | default | comment | payload_json               | TEXT    | N-N  | default | comment |
| sync_state              | TEXT    | N-N  | default | comment | created_at                 | TEXT    | N-N  | default | comment |
|                         |         |      |         |         | retry_count                | INTEGER | N-N  | 0       | comment |
|                         |         |      |         |         | status                     | TEXT    | N-N  | default | comment |

Figura 2.6.1.6. Android Room Database Design de Identity and Access Management. Fuente: elaboración propia a partir del esquema SQL.

## 2.6.2. Bounded Context: Profile Management

Mantener la información personal y de contacto asociada con una identidad sin mezclarla con credenciales o reglas de autenticación.

La base implementada se encuentra en el módulo Profiles de la API ASP.NET Core. El diseño móvil de Android y Flutter se presenta como **diseño objetivo** porque esos clientes todavía no existen en el workspace. Los tres productos comparten contratos REST y lenguaje ubicuo, mientras la API conserva las reglas autoritativas.

### 2.6.2.1. Domain Layer

Esta capa documenta el modelo que representa el núcleo de **Profile Management**. Las reglas autoritativas se ejecutan en la API; Android y Flutter mantienen modelos equivalentes para presentación, validación inmediata y trabajo offline. La columna de estado distingue el código heredado de la arquitectura objetivo.

| Clase        | Categoría      | Producto y estado                                                                     | Propósito                                                         | Atributos                                                                                                                  | Métodos u operaciones                                                                                       | Relaciones                                                                                                                                |
|--------------|----------------|---------------------------------------------------------------------------------------|-------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| Profile      | Aggregate Root | Backend; modelo equivalente en Android y Flutter<br><b>Implementado en el backend</b> | Mantener la información personal y de contacto de un propietario. | id: int<br>ownerId: int<br>name: string<br>PersonName<br>email: string<br>EmailAddress<br>address: string<br>StreetAddress | FullName(): string<br>ChangeEmail(email: EmailAddress): void<br>ChangeAddress(address: StreetAddress): void | Profile compone PersonName; Profile compone EmailAddress; Profile compone StreetAddress; IProfileRepository depende de Profile : persists |
| PersonName   | Value Object   | Backend; modelo equivalente en Android y Flutter<br><b>Implementado en el backend</b> | Representar un nombre personal válido.                            | firstName: string<br>lastName: string                                                                                      | FullName(): string                                                                                          | Profile compone PersonName                                                                                                                |
| EmailAddress | Value Object   | Backend; modelo equivalente en Android y Flutter<br><b>Implementado en el backend</b> | Representar y validar un correo electrónico.                      | address: string                                                                                                            | IsValid(): bool                                                                                             | Profile compone EmailAddress                                                                                                              |

| Clase              | Categoría            | Producto y estado                                                                     | Propósito                                                  | Atributos                                                                                 | Métodos u operaciones                                                                                                                 | Relaciones                                                        |
|--------------------|----------------------|---------------------------------------------------------------------------------------|------------------------------------------------------------|-------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------|
| StreetAddress      | Value Object         | Backend; modelo equivalente en Android y Flutter<br><b>Implementado en el backend</b> | Representar una dirección postal completa.                 | street: string<br>number: string<br>city: string<br>postalCode: string<br>country: string | FullAddress(): string                                                                                                                 | Profile compone StreetAddress                                     |
| IProfileRepository | Repository Interface | Backend; modelo equivalente en Android y Flutter<br><b>Implementado en el backend</b> | Abstraer la persistencia de Profile.                       | —                                                                                         | FindById(id: int): Profile?<br>FindByOwnerId(ownerId: int): Profile?<br>Add(profile: Profile): void<br>Update(profile: Profile): void | IProfileRepository depende de Profile : persists                  |
| ProfileOwnerId     | Value Object         | Backend; modelo equivalente en Android y Flutter<br><b>Diseño objetivo</b>            | Representar y validar la identidad propietaria del perfil. | value: int                                                                                | IsValid(): bool                                                                                                                       | Referencia una identidad de IAM sin incorporar su modelo.         |
| ProfileUpdated     | Domain Event         | Backend; modelo equivalente en Android y Flutter<br><b>Diseño objetivo</b>            | Comunicar un cambio confirmado del perfil.                 | profileId: int<br>ownerId: int<br>occurredAt: DateTime                                    | —                                                                                                                                     | Es emitido por Profile y atendido por ProfileUpdatedEventHandler. |

## 2.6.2.2. Interface Layer

Esta capa recibe las acciones relacionadas con **creación, actualización y consulta de perfiles** y las traduce a casos de uso. Los controllers y resources corresponden a la API; las pantallas y controladores de estado representan la presentación objetivo en Android y Flutter. Ninguna de estas clases implementa reglas de negocio.

| Clase                 | Categoría          | Producto y estado                                         | Propósito                                                                    | Atributos                                                                                                                                                                                             | Métodos u operaciones                                                                                                                                                                                              | Relaciones                                                                                     |
|-----------------------|--------------------|-----------------------------------------------------------|------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| ProfilesController    | REST Controller    | Backend ASP.NET Core<br><b>Implementado en el backend</b> | Publicar por HTTP las capacidades de Profile Management.                     | IProfileCommandService<br>profileCommandService<br>IProfileQueryService<br>profileQueryService<br>IStringLocalizer<ErrorMessages><br>errorLocalizer<br>ProblemDetailsFactory<br>problemDetailsFactory | GetProfileById(int profileId, Cancellation token cancellationToken)<br>CreateProfile(CreateProfileResource resource, Cancellation token cancellationToken)<br>GetAllProfiles(Cancellation token cancellationToken) | Recibe resources, invoca servicios de aplicación y devuelve resources HTTP.                    |
| CreateProfileResource | Resource/Assembler | Backend ASP.NET Core<br><b>Implementado en el backend</b> | Definir un contrato estable de entrada o salida para la API REST.            | string FirstName<br>string LastName<br>string Email<br>string Street<br>string Number<br>string City<br>string PostalCode<br>string Country                                                           | Create(...)<br>Deconstruct(...)                                                                                                                                                                                    | Es construido o traducido por assemblers y consumido por el controller y los clientes móviles. |
| ProfileResource       | Resource/Assembler | Backend ASP.NET Core<br><b>Implementado en el backend</b> | Definir un contrato estable de entrada o salida para la API REST.            | int Id<br>string FullName<br>string Email<br>string StreetAddress                                                                                                                                     | Create(...)<br>Deconstruct(...)                                                                                                                                                                                    | Es construido o traducido por assemblers y consumido por el controller y los clientes móviles. |
| ProfileScreen         | Composable         | Android / Jetpack Compose<br><b>Diseño objetivo</b>       | Presentar creación, actualización y consulta de perfiles en Android.         | uiState<br>onAction<br>navigation                                                                                                                                                                     | Render()<br>Submit()<br>Retry()                                                                                                                                                                                    | Observa ProfileViewModel y emite acciones de interfaz.                                         |
| ProfileViewModel      | Presentation Model | Android / Kotlin<br><b>Diseño objetivo</b>                | Mantener el estado observable y traducir acciones de Android a casos de uso. | state<br>observeUseCase<br>syncUseCase                                                                                                                                                                | Load()<br>Submit(action)<br>RetrySync()                                                                                                                                                                            | Invoca casos de uso de Application Layer y publica un UI State inmutable.                      |
| ProfilePage           | Widget             | Flutter / Dart<br><b>Diseño objetivo</b>                  | Presentar creación, actualización y consulta de perfiles en Flutter.         | state<br>onAction<br>router                                                                                                                                                                           | build(context)<br>submit()<br>retry()                                                                                                                                                                              | Observa ProfileController y emite intenciones del usuario.                                     |
| ProfileController     | State Controller   | Flutter / Dart<br><b>Diseño objetivo</b>                  | Mantener el estado de presentación de Flutter y coordinar casos de uso.      | state<br>observeUseCase<br>syncUseCase                                                                                                                                                                | load()<br>submit(action)<br>retrySync()                                                                                                                                                                            | Invoca Application Layer y publica estados de carga, éxito y error.                            |

## 2.6.2.3. Application Layer

Esta capa coordina las capacidades de **creación, actualización y consulta de perfiles**. Los commands y queries expresan intenciones; los handlers cargan aggregates, aplican reglas, persisten cambios y reaccionan a eventos. Los casos de uso móviles coordinan lectura local, actualización remota y sincronización idempotente.

| Clase                       | Categoría           | Producto y estado                                      | Propósito                                                                                 | Atributos                                                                                                                                   | Métodos u operaciones                                                                                                                                                                                                                                         | Relaciones                                                                                      |
|-----------------------------|---------------------|--------------------------------------------------------|-------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| ProfileCommandService       | Application Service | Backend ASP.NET Core <b>Implementado en el backend</b> | Orquestar creación, actualización y consulta de perfiles sin contener reglas del dominio. | IProfileRepository<br>profileRepository<br>IUnitOfWork unitOfWork<br>IStringLocalizer<ErrorMessages> localizer                              | Handle(CreateProfileCommand command, CancellationToken cancellationToken)                                                                                                                                                                                     | Invoca agregados y repositories; confirma la transacción mediante Unit of Work.                 |
| ProfileQueryService         | Application Service | Backend ASP.NET Core <b>Implementado en el backend</b> | Orquestar creación, actualización y consulta de perfiles sin contener reglas del dominio. | IProfileRepository<br>profileRepository                                                                                                     | Handle(GetAllProfilesQuery query, CancellationToken cancellationToken)<br>Handle(GetProfileByEmailQuery query, CancellationToken cancellationToken)<br>Handle(GetProfileByIdQuery query, CancellationToken cancellationToken)                                 | Invoca agregados y repositories; confirma la transacción mediante Unit of Work.                 |
| ProfilesContextFacade       | Application Service | Backend ASP.NET Core <b>Implementado en el backend</b> | Orquestar creación, actualización y consulta de perfiles sin contener reglas del dominio. | IProfileCommandService<br>profileCommandService<br>IProfileQueryService<br>profileQueryService                                              | CreateProfile(string firstName, string lastName, string email, string street, string number, string city, string postalCode, string country, CancellationToken cancellationToken)<br>FetchProfileByIdEmail(string email, CancellationToken cancellationToken) | Invoca agregados y repositories; confirma la transacción mediante Unit of Work.                 |
| CreateProfileCommand        | Command/Query       | Backend ASP.NET Core <b>Implementado en el backend</b> | Transportar una intención o consulta tipada hacia su handler.                             | string FirstName<br>string LastName<br>string Email<br>string Street<br>string Number<br>string City<br>string PostalCode<br>string Country | —                                                                                                                                                                                                                                                             | Es recibida por un handler o servicio de aplicación y no contiene lógica de negocio.            |
| GetProfileByIdQuery         | Command/Query       | Backend ASP.NET Core <b>Implementado en el backend</b> | Transportar una intención o consulta tipada hacia su handler.                             | int ProfileId                                                                                                                               | —                                                                                                                                                                                                                                                             | Es recibida por un handler o servicio de aplicación y no contiene lógica de negocio.            |
| GetProfileByEmailQuery      | Command/Query       | Backend ASP.NET Core <b>Implementado en el backend</b> | Transportar una intención o consulta tipada hacia su handler.                             | EmailAddress Email                                                                                                                          | —                                                                                                                                                                                                                                                             | Es recibida por un handler o servicio de aplicación y no contiene lógica de negocio.            |
| ObserveProfileUseCase       | Use Case            | Android y Flutter <b>Diseño objetivo</b>               | Entregar primero datos locales y actualizar la consulta cuando exista conectividad.       | localRepository<br>remoteRepository<br>connectivityMonitor                                                                                  | Execute(criteria): Stream<Result>                                                                                                                                                                                                                             | Es invocado por ViewModel/Controller y coordina repositorios móviles.                           |
| SyncProfileUseCase          | Use Case            | Android y Flutter <b>Diseño objetivo</b>               | Procesar operaciones móviles pendientes de manera idempotente.                            | outboxRepository<br>remoteRepository<br>conflictResolver                                                                                    | Execute(): SyncResult                                                                                                                                                                                                                                         | Lee el outbox local, consume la API y actualiza el estado de sincronización.                    |
| CreateProfileCommandHandler | Command Handler     | Backend ASP.NET Core <b>Diseño objetivo</b>            | Ejecutar una intención concreta, aplicar reglas del agregado y confirmar la transacción.  | repository<br>unitOfWork<br>domainPolicy                                                                                                    | Handle(command): Result                                                                                                                                                                                                                                       | Consume un Command, carga el aggregate mediante su repository y puede publicar un Domain Event. |
| UpdateProfileCommandHandler | Command Handler     | Backend ASP.NET Core                                   | Ejecutar una intención concreta, aplicar                                                  | repository<br>unitOfWork<br>domainPolicy                                                                                                    | Handle(command): Result                                                                                                                                                                                                                                       | Consume un Command, carga el aggregate mediante su repository y                                 |

| Clase                      | Categoría     | Producto y estado                           | Propósito                                                                  | Atributos                                              | Métodos u operaciones     | Relaciones                                                                                           |
|----------------------------|---------------|---------------------------------------------|----------------------------------------------------------------------------|--------------------------------------------------------|---------------------------|------------------------------------------------------------------------------------------------------|
|                            |               | <b>Diseño objetivo</b>                      | reglas del agregado y confirmar la transacción.                            |                                                        |                           | puede publicar un Domain Event.                                                                      |
| ProfileUpdatedEventHandler | Event Handler | Backend ASP.NET Core <b>Diseño objetivo</b> | Reaccionar al evento confirmado y actualizar proyecciones o integraciones. | projectionRepository<br>notificationPort<br>unitOfWork | Handle(domainEvent): Task | Consume un Domain Event y utiliza puertos de infraestructura sin modificar directamente el agregado. |

2.6.2.4. Infrastructure Layer

Esta capa implementa los puertos definidos hacia el interior de **Profile Management** y concentra acceso a base de datos, red, almacenamiento local e integraciones externas. Las clases de infraestructura traducen errores y contratos técnicos antes de devolver resultados a Application Layer.

| Clase                   | Categoría                 | Producto y estado                                                 | Propósito                                                           | Atributos                                 | Métodos u operaciones                                                            | Relaciones                                                                                   |
|-------------------------|---------------------------|-------------------------------------------------------------------|---------------------------------------------------------------------|-------------------------------------------|----------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| ProfileRepository       | Repository Adapter        | Backend / Entity Framework Core <b>Implementado en el backend</b> | Implementar el puerto de persistencia definido por Domain Layer.    | AppDbContext<br>context                   | FindProfileByEmailAsync(EmailAddress email, CancellationToken cancellationToken) | Implementa IProfileRepository; utiliza AppDbContext/MySQL y reconstruye el aggregate.        |
| ModelBuilderExtensions  | Persistence Configuration | Backend / Entity Framework Core <b>Implementado en el backend</b> | Mapear entidades y value objects del contexto al modelo relacional. | EntityTypeBuilder<br>configuration        | ApplyConfiguration(modelBuilder)                                                 | Configura tablas, claves, relaciones, restricciones y conversiones de Entity Framework Core. |
| ProfileApiDataSource    | Remote Adapter            | Android / Kotlin <b>Diseño objetivo</b>                           | Implementar el acceso remoto del cliente móvil a la API.            | httpClient<br>tokenProvider<br>serializer | Get(criteria)<br>Create(dto)<br>Update(dto)<br>Delete(id)                        | Consume controllers REST por HTTPS/JSON y traduce errores HTTP al modelo de aplicación.      |
| ProfileDao              | Room Adapter              | Android / Room <b>Diseño objetivo</b>                             | Implementar persistencia local y observación reactiva en Android.   | roomDatabase<br>entityMapper              | Observe(criteria)<br>Upsert(entity)<br>Delete(id)<br>Pending()                   | Implementa el puerto local mediante Room y participa en la estrategia de caché/outbox.       |
| ProfileRemoteDataSource | Remote Adapter            | Flutter / Dart <b>Diseño objetivo</b>                             | Implementar el acceso remoto del cliente móvil a la API.            | httpClient<br>tokenProvider<br>serializer | Get(criteria)<br>Create(dto)<br>Update(dto)<br>Delete(id)                        | Consume controllers REST por HTTPS/JSON y traduce errores HTTP al modelo de aplicación.      |
| ProfileLocalDataSource  | SQLite Adapter            | Flutter / Dart <b>Diseño objetivo</b>                             | Implementar persistencia local equivalente en Flutter.              | sqliteDatabase<br>entityMapper            | watch(criteria)<br>upsert(entity)<br>delete(id)<br>pending()                     | Implementa el puerto local mediante SQLite y participa en la estrategia de caché/outbox.     |
| IamProfileOwnerAdapter  | Context Adapter           | Backend ASP.NET Core <b>Diseño objetivo</b>                       | Aislar una dependencia externa detrás de un puerto explícito.       | iamFacade                                 | Exists(ownerId): bool                                                            | Implementa el puerto de verificación del propietario y consume IAM Context Facade.           |
| ProfileCacheStore       | Local Cache Adapter       | Android y Flutter <b>Diseño objetivo</b>                          | Aislar una dependencia externa detrás de un puerto explícito.       | profileDao, clock                         | Read(ownerId)<br>Save(profile)<br>Invalidate(ownerId)                            | Implementa el puerto de caché mediante Room o SQLite.                                        |

2.6.2.5. Bounded Context Software Architecture Component Level Diagrams

El archivo [component-level.dsl](#) contiene las vistas BC2-APIComponents, BC2-AndroidComponents y BC2-FlutterComponents. Las tres parten del mismo modelo C4 y muestran la separación entre presentación, aplicación, dominio y adaptadores.

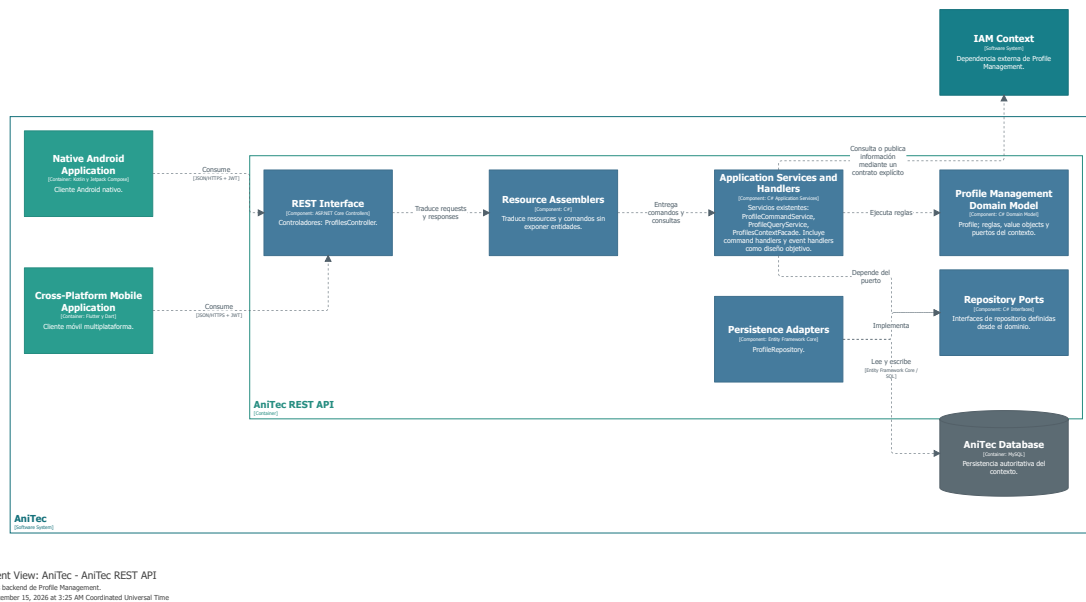


Figura 2.6.2.1. Componentes de la API para Profile Management. Fuente: elaboración propia con Structurizr DSL.

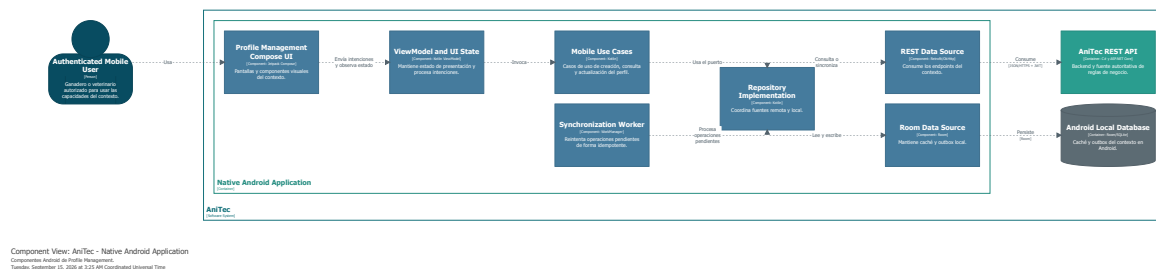


Figura 2.6.2.2. Componentes Android para Profile Management. Fuente: elaboración propia con Structurizr DSL.

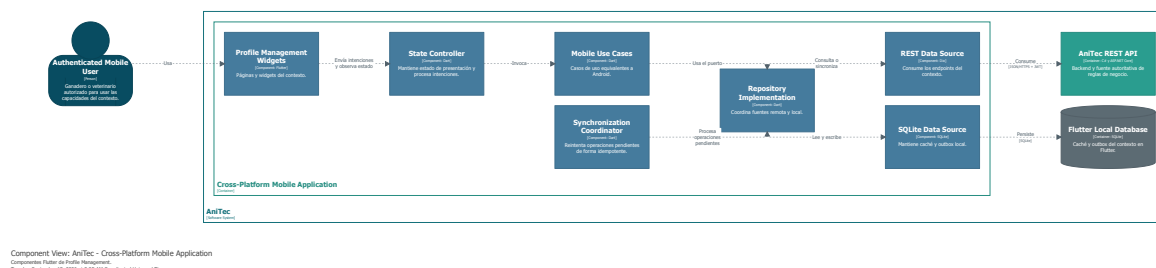


Figura 2.6.2.3. Componentes Flutter para Profile Management. Fuente: elaboración propia con Structurizr DSL.

## 2.6.2.6. Bounded Context Software Architecture Code Level Diagrams

Los diagramas de código detallan el modelo del dominio y los objetos de persistencia. El UML diferencia los elementos existentes de las incorporaciones objetivo, mientras los esquemas SQL señalan mediante comentarios las columnas propuestas. Los archivos ERD quedan disponibles para completar la importación manual.

### 2.6.2.6.1. Bounded Context Domain Layer Class Diagrams

El Class Diagram incluye agregados, entidades, value objects, enumeraciones, servicios de dominio e interfaces de repositorio con atributos, operaciones, visibilidad y multiplicidades.

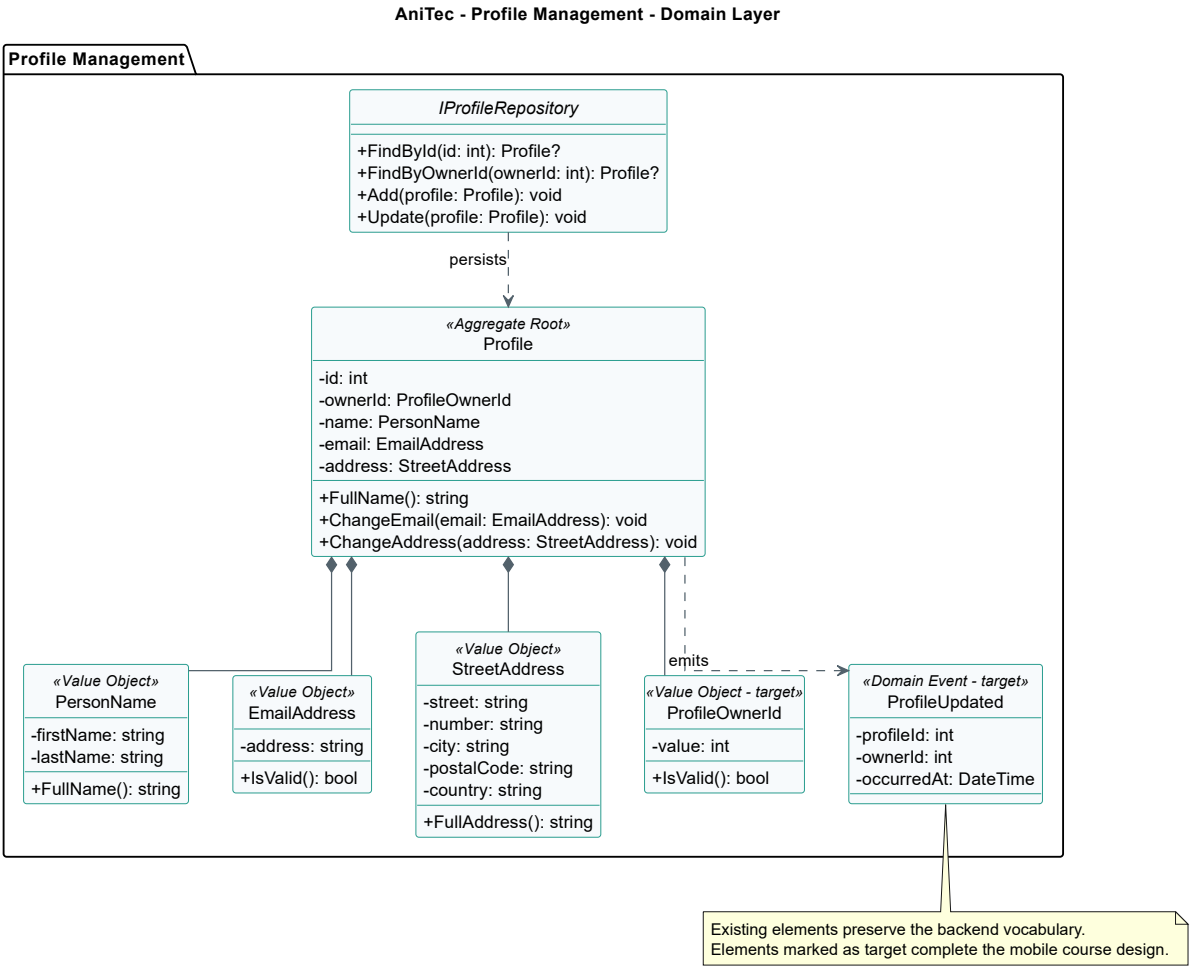


Figura 2.6.2.4. Domain Layer Class Diagram de Profile Management. Fuente: elaboración propia con PlantUML.

2.6.2.6.2. Bounded Context Database Design Diagram

MySQL mantiene la persistencia autoritativa. Room y SQLite contienen únicamente caché, metadatos de sincronización y operaciones pendientes; no sustituyen las reglas ni la fuente de verdad del backend. En IAM, las credenciales y tokens permanecen fuera de las tablas locales y se almacenan mediante mecanismos seguros del sistema operativo.

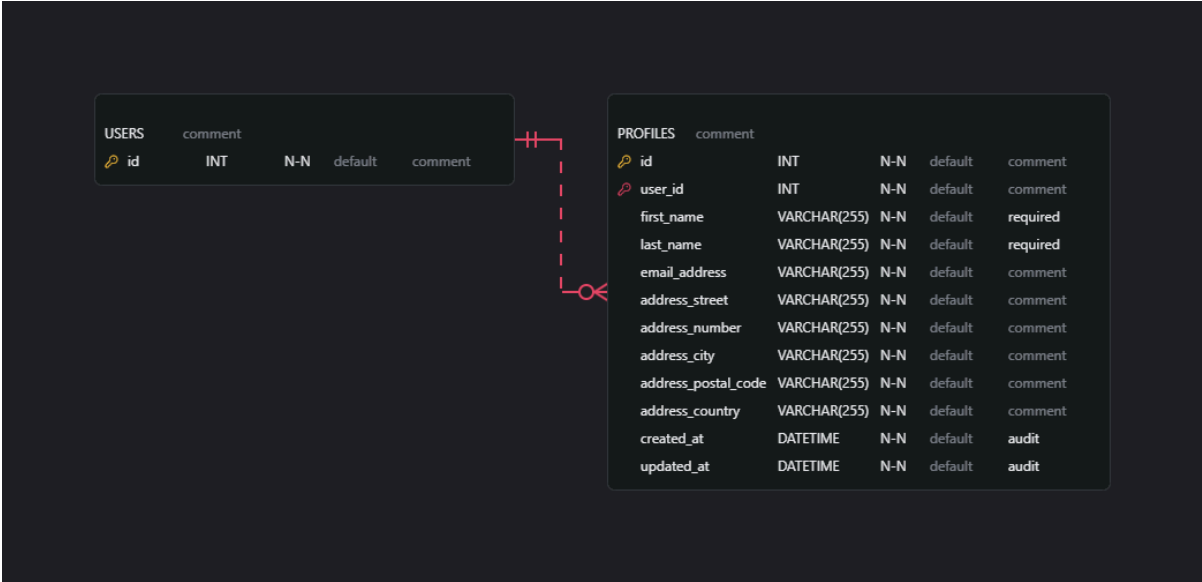


Figura 2.6.2.5. MySQL Database Design de Profile Management. Fuente: elaboración propia a partir del esquema SQL.

| CACHED_PROFILES comment |              |         |      |                 | PENDING_OPERATIONS comment |                |         |       |                 |
|-------------------------|--------------|---------|------|-----------------|----------------------------|----------------|---------|-------|-----------------|
| 🔑                       | profile_id   | INTEGER | NULL | default comment | 🔑                          | operation_id   | TEXT    | NULL  | default comment |
|                         | user_id      | INTEGER | N-N  | default comment |                            | aggregate_type | TEXT    | N-N   | default comment |
|                         | first_name   | TEXT    | N-N  | default comment |                            | aggregate_id   | TEXT    | N-N   | default comment |
|                         | last_name    | TEXT    | N-N  | default comment |                            | operation_type | TEXT    | N-N   | default comment |
|                         | email        | TEXT    | N-N  | default comment |                            | payload_json   | TEXT    | N-N   | default comment |
|                         | address_json | TEXT    | N-N  | default comment |                            | created_at     | TEXT    | N-N   | default comment |
|                         | updated_at   | TEXT    | N-N  | default comment |                            | retry_count    | INTEGER | N-N 0 | comment         |
|                         | sync_state   | TEXT    | N-N  | default comment |                            | status         | TEXT    | N-N   | default comment |

Figura 2.6.2.6. Android Room Database Design de Profile Management. Fuente: elaboración propia a partir del esquema SQL.

## 2.6.3. Bounded Context: Livestock Management

Gestionar fincas, hatos, animales e identificadores QR como fuente de referencia para los demás procesos ganaderos.

La base implementada se encuentra en el módulo Livestock de la API ASP.NET Core. El diseño móvil de Android y Flutter se presenta como **diseño objetivo** porque esos clientes todavía no existen en el workspace. Los tres productos comparten contratos REST y lenguaje ubicuo, mientras la API conserva las reglas autoritativas.

### 2.6.3.1. Domain Layer

Esta capa documenta el modelo que representa el núcleo de **Livestock Management**. Las reglas autoritativas se ejecutan en la API; Android y Flutter mantienen modelos equivalentes para presentación, validación inmediata y trabajo offline. La columna de estado distingue el código heredado de la arquitectura objetivo.

| Clase     | Categoría      | Producto y estado                                                                     | Propósito                                                 | Atributos                                                                                                                                          | Métodos u operaciones                                                           | Relaciones                                                                                                                                               |
|-----------|----------------|---------------------------------------------------------------------------------------|-----------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Farm      | Aggregate Root | Backend, Android y Flutter (modelo canónico)<br><b>Diseño objetivo</b>                | Agrupar los hatos de un propietario.                      | id: int<br>ownerId: int<br>name: string<br>location: string                                                                                        | RegisterHerd(herd: Herd): void<br>BelongsTo(ownerId: int): bool                 | Farm compone Herd                                                                                                                                        |
| Herd      | Entity         | Backend; modelo equivalente en Android y Flutter<br><b>Implementado en el backend</b> | Administrar un conjunto de animales dentro de una finca.  | id: int<br>farmId: int<br>name: string<br>mainType: string<br>veterinarianId: int?                                                                 | AssignVeterinarian(id: int): void<br>AddAnimal(animal: Animal): void            | Farm compone Herd; Herd compone Animal;<br>IHerdRepository depende de Herd                                                                               |
| Animal    | Entity         | Backend; modelo equivalente en Android y Flutter<br><b>Implementado en el backend</b> | Mantener identificación y estado productivo de un animal. | id: int<br>herdId: int<br>tag: AnimalTag<br>qrIdentifier: QrIdentifier<br>name: string<br>species: string<br>breed: string<br>status: AnimalStatus | UpdateWeight(weight: decimal): void<br>ChangeStatus(status: AnimalStatus): void | Herd compone Animal; Animal compone AnimalTag;<br>Animal compone QrIdentifier; Animal se relaciona con AnimalStatus; IAnimalRepository depende de Animal |
| AnimalTag | Value Object   | Backend, Android y Flutter (modelo                                                    | Representar el identificador visible del                  | value: string                                                                                                                                      | IsValid(): bool                                                                 | Animal compone AnimalTag                                                                                                                                 |

| Clase                 | Categoría            | Producto y estado                                                                      | Propósito                                         | Atributos                  | Métodos u operaciones                                                                      | Relaciones                                            |
|-----------------------|----------------------|----------------------------------------------------------------------------------------|---------------------------------------------------|----------------------------|--------------------------------------------------------------------------------------------|-------------------------------------------------------|
|                       |                      | canónico)<br><b>Diseño objetivo</b>                                                    | animal.                                           |                            |                                                                                            |                                                       |
| QrIdentifier          | Value Object         | Backend, Android y Flutter (modelo canónico)<br><b>Diseño objetivo</b>                 | Representar la carga QR que identifica un animal. | value: string              | AsPayload(): string                                                                        | Animal compone QrIdentifier                           |
| AnimalStatus          | Enumeration          | Backend y modelos equivalentes<br>Android/Flutter<br><b>Implementado en el backend</b> | Definir los valores válidos de AnimalStatus.      | Active<br>Sold<br>Deceased | —                                                                                          | Animal se relaciona con AnimalStatus                  |
| IAAnimalRepository    | Repository Interface | Backend; modelo equivalente en Android y Flutter<br><b>Implementado en el backend</b>  | Abstraer la persistencia de Animal.               | —                          | FindById(id: int): Animal?<br>FindByQr(code: string): Animal?<br>Add(animal: Animal): void | IAAnimalRepository depende de Animal                  |
| IHerdRepository       | Repository Interface | Backend; modelo equivalente en Android y Flutter<br><b>Implementado en el backend</b>  | Abstraer la persistencia de Herd.                 | —                          | FindById(id: int): Herd?<br>Add(herd: Herd): void                                          | IHerdRepository depende de Herd                       |
| AnimalOwnershipPolicy | Domain Service       | Backend; modelo equivalente en Android y Flutter<br><b>Diseño objetivo</b>             | Decidir si un actor puede administrar un animal.  | —                          | CanManage(actorId: int, animalId: int): bool                                               | Consulta Farm, Herd y Animal sin asumir persistencia. |

## 2.6.3.2. Interface Layer

Esta capa recibe las acciones relacionadas con **gestión de fincas, hatos, animales e identificación QR** y las traduce a casos de uso. Los controllers y resources corresponden a la API; las pantallas y controladores de estado representan la presentación objetivo en Android y Flutter. Ninguna de estas clases implementa reglas de negocio.

| Clase             | Categoría          | Producto y estado                                            | Propósito                                                         | Atributos                                                                                                         | Métodos u operaciones                                                                                                                                                                                                                                                                                                                    | Relaciones                                                                                     |
|-------------------|--------------------|--------------------------------------------------------------|-------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| HerdController    | REST Controller    | Backend<br>ASP.NET Core<br><b>Implementado en el backend</b> | Publicar por HTTP las capacidades de Livestock Management.        | IHerdCommandService<br>commandService<br>IHerdQueryService<br>queryService                                        | GetAll(Cancellation token, cancellationToken)<br>GetById(int id, Cancellation token, cancellationToken)<br>Create(CreateHerdResource resource, Cancellation token, cancellationToken)<br>Update(int id, CreateHerdResource resource, Cancellation token, cancellationToken)<br>Delete(int id, Cancellation token, cancellationToken)     | Recibe resources, invoca servicios de aplicación y devuelve resources HTTP.                    |
| AnimalsController | REST Controller    | Backend<br>ASP.NET Core<br><b>Implementado en el backend</b> | Publicar por HTTP las capacidades de Livestock Management.        | IAAnimalCommandService<br>commandService<br>IAAnimalQueryService<br>queryService                                  | GetAll(Cancellation token, cancellationToken)<br>GetById(int id, Cancellation token, cancellationToken)<br>Create(CreateAnimalResource resource, Cancellation token, cancellationToken)<br>Update(int id, CreateAnimalResource resource, Cancellation token, cancellationToken)<br>Delete(int id, Cancellation token, cancellationToken) | Recibe resources, invoca servicios de aplicación y devuelve resources HTTP.                    |
| HerdResource      | Resource/Assembler | Backend<br>ASP.NET Core<br><b>Implementado en el backend</b> | Definir un contrato estable de entrada o salida para la API REST. | int Id<br>string Name<br>string Location<br>string Owner<br>int OwnerId<br>int? VeterinarianId<br>string MainType | Create(...)<br>Deconstruct(...)                                                                                                                                                                                                                                                                                                          | Es construido o traducido por assemblers y consumido por el controller y los clientes móviles. |

| Clase                     | Categoría          | Producto y estado                                            | Propósito                                                                    | Atributos                                                                                                                                                      | Métodos u operaciones                   | Relaciones                                                                                     |
|---------------------------|--------------------|--------------------------------------------------------------|------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------|------------------------------------------------------------------------------------------------|
| AnimalResource            | Resource/Assembler | Backend<br>ASP.NET Core<br><b>Implementado en el backend</b> | Definir un contrato estable de entrada o salida para la API REST.            | int Id<br>string Tag<br>string Name<br>string Species<br>string Breed<br>string Gender<br>DateOnly? BirthDate<br>decimal Weight<br>string Status<br>int HerdId | Create(...)<br>Deconstruct(...)         | Es construido o traducido por assemblers y consumido por el controller y los clientes móviles. |
| CreateHerdResource        | Resource/Assembler | Backend<br>ASP.NET Core<br><b>Implementado en el backend</b> | Definir un contrato estable de entrada o salida para la API REST.            | string Name<br>string Location<br>string Owner<br>int OwnerId<br>int? VeterinarianId<br>string MainType                                                        | Create(...)<br>Deconstruct(...)         | Es construido o traducido por assemblers y consumido por el controller y los clientes móviles. |
| CreateAnimalResource      | Resource/Assembler | Backend<br>ASP.NET Core<br><b>Implementado en el backend</b> | Definir un contrato estable de entrada o salida para la API REST.            | string Tag<br>string Name<br>string Species<br>string Breed<br>string Gender<br>DateOnly? BirthDate<br>decimal Weight<br>string Status<br>int HerdId           | Create(...)<br>Deconstruct(...)         | Es construido o traducido por assemblers y consumido por el controller y los clientes móviles. |
| FarmHerdYAnimalScreen     | Composable         | Android / Jetpack<br>Compose<br><b>Diseño objetivo</b>       | Presentar gestión de fincas, hatos, animales e identificación QR en Android. | uiState<br>onAction<br>navigation                                                                                                                              | Render()<br>Submit()<br>Retry()         | Observa<br>FarmHerdYAnimalViewModel y emite acciones de interfaz.                              |
| FarmHerdYAnimalViewModel  | Presentation Model | Android / Kotlin<br><b>Diseño objetivo</b>                   | Mantener el estado observable y traducir acciones de Android a casos de uso. | state<br>observeUseCase<br>syncUseCase                                                                                                                         | Load()<br>Submit(action)<br>RetrySync() | Invoca casos de uso de Application Layer y publica un UI State inmutable.                      |
| FarmHerdYAnimalPage       | Widget             | Flutter / Dart<br><b>Diseño objetivo</b>                     | Presentar gestión de fincas, hatos, animales e identificación QR en Flutter. | state<br>onAction<br>router                                                                                                                                    | build(context)<br>submit()<br>retry()   | Observa<br>FarmHerdYAnimalController y emite intenciones del usuario.                          |
| FarmHerdYAnimalController | State Controller   | Flutter / Dart<br><b>Diseño objetivo</b>                     | Mantener el estado de presentación de Flutter y coordinar casos de uso.      | state<br>observeUseCase<br>syncUseCase                                                                                                                         | load()<br>submit(action)<br>retrySync() | Invoca Application Layer y publica estados de carga, éxito y error.                            |

### 2.6.3.3. Application Layer

Esta capa coordina las capacidades de **gestión de fincas, hatos, animales e identificación QR**. Los commands y queries expresan intenciones; los handlers cargan aggregates, aplican reglas, persisten cambios y reaccionan a eventos. Los casos de uso móviles coordinan lectura local, actualización remota y sincronización idempotente.

| Clase              | Categoría           | Producto y estado                                            | Propósito                                                                                         | Atributos                                                  | Métodos u operaciones                                                                                                                                                                                                      | Relaciones                                                                      |
|--------------------|---------------------|--------------------------------------------------------------|---------------------------------------------------------------------------------------------------|------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| HerdCommandService | Application Service | Backend<br>ASP.NET Core<br><b>Implementado en el backend</b> | Orquestar gestión de fincas, hatos, animales e identificación QR sin contener reglas del dominio. | IHerdRepository<br>repository<br>IUnitOfWork<br>unitOfWork | Handle(CreateHerdCommand command, CancellationToken cancellationToken)<br>Handle(UpdateHerdCommand command, CancellationToken cancellationToken)<br>Handle(DeleteHerdCommand command, CancellationToken cancellationToken) | Invoca agregados y repositories; confirma la transacción mediante Unit of Work. |
| HerdQueryService   | Application Service | Backend<br>ASP.NET Core                                      | Orquestar gestión de fincas, hatos, animales e identificación QR sin                              | IHerdRepository<br>repository                              | Handle(GetHerdByIdQuery query, CancellationToken cancellationToken)                                                                                                                                                        | Invoca agregados y repositories; confirma la                                    |

| Clase                         | Categoría           | Producto y estado                                      | Propósito                                                                                         | Atributos                                                                                                                                                      | Métodos u operaciones                                                                                                                                                                                                            | Relaciones                                                                                      |
|-------------------------------|---------------------|--------------------------------------------------------|---------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
|                               |                     | <b>Implementado en el backend</b>                      | contener reglas del dominio.                                                                      |                                                                                                                                                                | Handle(GetAllHerdsQuery query, CancellationToken cancellationToken)                                                                                                                                                              | transacción mediante Unit of Work.                                                              |
| AnimalCommandService          | Application Service | Backend ASP.NET Core <b>Implementado en el backend</b> | Orquestar gestión de fincas, hatos, animales e identificación QR sin contener reglas del dominio. | IAAnimalRepository<br>IUnitOfWork<br>unitOfWork                                                                                                                | Handle(CreateAnimalCommand command, CancellationToken cancellationToken)<br>Handle(UpdateAnimalCommand command, CancellationToken cancellationToken)<br>Handle(DeleteAnimalCommand command, CancellationToken cancellationToken) | Invoca agregados y repositories; confirma la transacción mediante Unit of Work.                 |
| AnimalQueryService            | Application Service | Backend ASP.NET Core <b>Implementado en el backend</b> | Orquestar gestión de fincas, hatos, animales e identificación QR sin contener reglas del dominio. | IAAnimalRepository repository                                                                                                                                  | Handle(GetAnimalByIdQuery query, CancellationToken cancellationToken)<br>Handle(GetAllAnimalsQuery query, CancellationToken cancellationToken)                                                                                   | Invoca agregados y repositories; confirma la transacción mediante Unit of Work.                 |
| CreateHerdCommand             | Command/Query       | Backend ASP.NET Core <b>Implementado en el backend</b> | Transportar una intención o consulta tipada hacia su handler.                                     | string Name<br>string Location<br>string Owner<br>int OwnerId<br>int? VeterinarianId<br>string MainType                                                        | —                                                                                                                                                                                                                                | Es recibida por un handler o servicio de aplicación y no contiene lógica de negocio.            |
| UpdateHerdCommand             | Command/Query       | Backend ASP.NET Core <b>Implementado en el backend</b> | Transportar una intención o consulta tipada hacia su handler.                                     | int Id<br>string Name<br>string Location<br>string Owner<br>int OwnerId<br>int? VeterinarianId<br>string MainType                                              | —                                                                                                                                                                                                                                | Es recibida por un handler o servicio de aplicación y no contiene lógica de negocio.            |
| CreateAnimalCommand           | Command/Query       | Backend ASP.NET Core <b>Implementado en el backend</b> | Transportar una intención o consulta tipada hacia su handler.                                     | string Tag<br>string Name<br>string Species<br>string Breed<br>string Gender<br>DateOnly? BirthDate<br>decimal Weight<br>string Status<br>int HerdId           | —                                                                                                                                                                                                                                | Es recibida por un handler o servicio de aplicación y no contiene lógica de negocio.            |
| UpdateAnimalCommand           | Command/Query       | Backend ASP.NET Core <b>Implementado en el backend</b> | Transportar una intención o consulta tipada hacia su handler.                                     | int Id<br>string Tag<br>string Name<br>string Species<br>string Breed<br>string Gender<br>DateOnly? BirthDate<br>decimal Weight<br>string Status<br>int HerdId | —                                                                                                                                                                                                                                | Es recibida por un handler o servicio de aplicación y no contiene lógica de negocio.            |
| GetAnimalByIdQuery            | Command/Query       | Backend ASP.NET Core <b>Implementado en el backend</b> | Transportar una intención o consulta tipada hacia su handler.                                     | int Id                                                                                                                                                         | —                                                                                                                                                                                                                                | Es recibida por un handler o servicio de aplicación y no contiene lógica de negocio.            |
| ObserveFarmHerdYAnimalUseCase | Use Case            | Android y Flutter <b>Diseño objetivo</b>               | Entregar primero datos locales y actualizar la consulta cuando exista conectividad.               | localRepository<br>remoteRepository<br>connectivityMonitor                                                                                                     | Execute(criteria); Stream<Result>                                                                                                                                                                                                | Es invocado por ViewModel/Controller y coordina repositorios móviles.                           |
| SyncFarmHerdYAnimalUseCase    | Use Case            | Android y Flutter <b>Diseño objetivo</b>               | Procesar operaciones móviles pendientes de manera idempotente.                                    | outboxRepository<br>remoteRepository<br>conflictResolver                                                                                                       | Execute(): SyncResult                                                                                                                                                                                                            | Lee el outbox local, consume la API y actualiza el estado de sincronización.                    |
| CreateHerdCommandHandler      | Command Handler     | Backend ASP.NET Core <b>Diseño objetivo</b>            | Ejecutar una intención concreta, aplicar reglas del agregado y confirmar la transacción.          | repository<br>unitOfWork<br>domainPolicy                                                                                                                       | Handle(command): Result                                                                                                                                                                                                          | Consume un Command, carga el aggregate mediante su repository y puede publicar un Domain Event. |
| CreateAnimalCommandHandler    | Command Handler     | Backend ASP.NET Core <b>Diseño objetivo</b>            | Ejecutar una intención concreta, aplicar reglas del agregado y confirmar la transacción.          | repository<br>unitOfWork<br>domainPolicy                                                                                                                       | Handle(command): Result                                                                                                                                                                                                          | Consume un Command, carga el aggregate mediante su repository y puede publicar un Domain Event. |

| Clase                        | Categoría     | Producto y estado                                 | Propósito                                                                  | Atributos                                              | Métodos u operaciones     | Relaciones                                                                                            |
|------------------------------|---------------|---------------------------------------------------|----------------------------------------------------------------------------|--------------------------------------------------------|---------------------------|-------------------------------------------------------------------------------------------------------|
| AnimalRegisteredEventHandler | Event Handler | Backend<br>ASP.NET Core<br><b>Diseño objetivo</b> | Reaccionar al evento confirmado y actualizar proyecciones o integraciones. | projectionRepository<br>notificationPort<br>unitOfWork | Handle(domainEvent): Task | Consumes un Domain Event y utiliza puertos de infraestructura sin modificar directamente el agregado. |

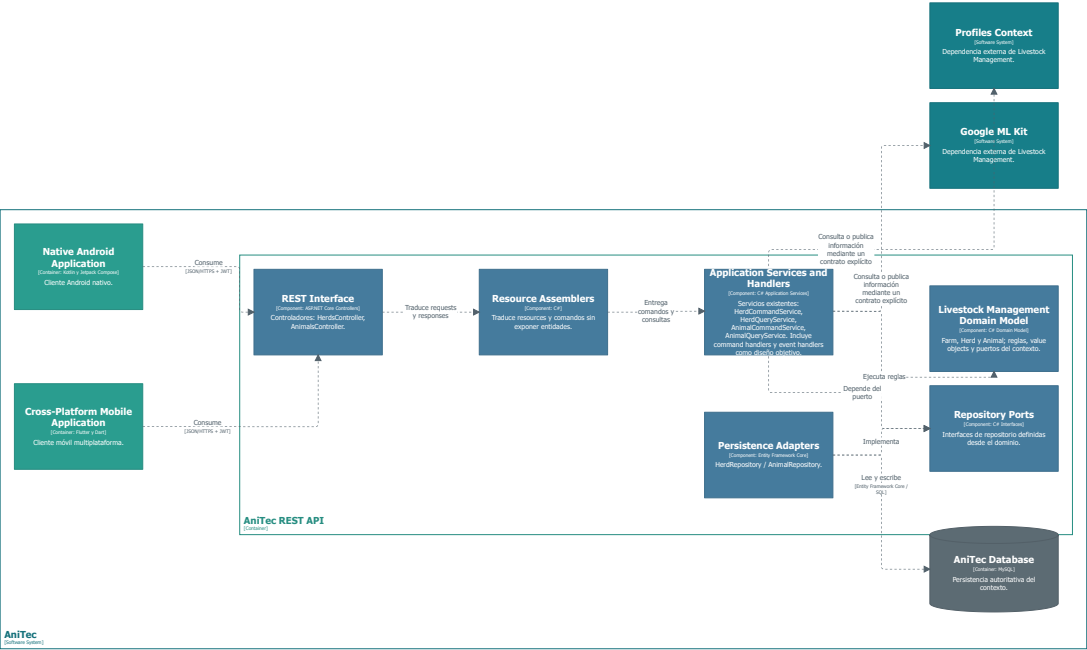
## 2.6.3.4. Infrastructure Layer

Esta capa implementa los puertos definidos hacia el interior de **Livestock Management** y concentra acceso a base de datos, red, almacenamiento local e integraciones externas. Las clases de infraestructura traducen errores y contratos técnicos antes de devolver resultados a Application Layer.

| Clase                             | Categoría                 | Producto y estado                                                    | Propósito                                                           | Atributos                                                                                  | Métodos u operaciones                                           | Relaciones                                                                                             |
|-----------------------------------|---------------------------|----------------------------------------------------------------------|---------------------------------------------------------------------|--------------------------------------------------------------------------------------------|-----------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| HerdRepository / AnimalRepository | Repository Adapter        | Backend / Entity Framework Core<br><b>Implementado en el backend</b> | Implementar el puerto de persistencia definido por Domain Layer.    | HerdRepository;<br>AppDbContext<br>context<br>AnimalRepository;<br>AppDbContext<br>context | FindById(id)<br>Add(entity)<br>Update(entity)<br>Delete(entity) | Implementa IHerdRepository / IAnimalRepository; utiliza AppDbContext/MySQL y reconstruye el aggregate. |
| ModelBuilderExtensions            | Persistence Configuration | Backend / Entity Framework Core<br><b>Implementado en el backend</b> | Mapear entidades y value objects del contexto al modelo relacional. | EntityTypeBuilder configuration                                                            | ApplyConfiguration(modelBuilder)                                | Configura tablas, claves, relaciones, restricciones y conversiones de Entity Framework Core.           |
| FarmHerdYAnimalApiDataSource      | Remote Adapter            | Android / Kotlin<br><b>Diseño objetivo</b>                           | Implementar el acceso remoto del cliente móvil a la API.            | httpClient<br>tokenProvider<br>serializer                                                  | Get(criteria)<br>Create(dto)<br>Update(dto)<br>Delete(id)       | Consumes controllers REST por HTTPS/JSON y traduce errores HTTP al modelo de aplicación.               |
| FarmHerdYAnimalDao                | Room Adapter              | Android / Room<br><b>Diseño objetivo</b>                             | Implementar persistencia local y observación reactiva en Android.   | roomDatabase<br>entityMapper                                                               | Observe(criteria)<br>Upsert(entity)<br>Delete(id)<br>Pending()  | Implementa el puerto local mediante Room y participa en la estrategia de caché/outbox.                 |
| FarmHerdYAnimalRemoteDataSource   | Remote Adapter            | Flutter / Dart<br><b>Diseño objetivo</b>                             | Implementar el acceso remoto del cliente móvil a la API.            | httpClient<br>tokenProvider<br>serializer                                                  | Get(criteria)<br>Create(dto)<br>Update(dto)<br>Delete(id)       | Consumes controllers REST por HTTPS/JSON y traduce errores HTTP al modelo de aplicación.               |
| FarmHerdYAnimalLocalDataSource    | SQLite Adapter            | Flutter / Dart<br><b>Diseño objetivo</b>                             | Implementar persistencia local equivalente en Flutter.              | sqliteDatabase<br>entityMapper                                                             | watch(criteria)<br>upsert(entity)<br>delete(id)<br>pending()    | Implementa el puerto local mediante SQLite y participa en la estrategia de caché/outbox.               |
| ProfilesOwnerAdapter              | Context Adapter           | Backend ASP.NET Core<br><b>Diseño objetivo</b>                       | Aislar una dependencia externa detrás de un puerto explícito.       | profilesFacade                                                                             | ValidateOwner(ownerId): bool                                    | Implementa el puerto de propietarios y consume Profile Management.                                     |
| MiKitQrScanner                    | Device Adapter            | Android / ML Kit<br><b>Diseño objetivo</b>                           | Aislar una dependencia externa detrás de un puerto explícito.       | scannerClient                                                                              | Scan(image): QrIdentifier                                       | Implementa el puerto de lectura QR mediante la cámara y Google ML Kit.                                 |
| LivestockOutboxStore              | Offline Adapter           | Android y Flutter<br><b>Diseño objetivo</b>                          | Aislar una dependencia externa detrás de un puerto explícito.       | database, serializer                                                                       | Enqueue(operation)<br>Pending()<br>MarkSynced(id)               | Implementa el puerto de sincronización local sobre Room o SQLite.                                      |

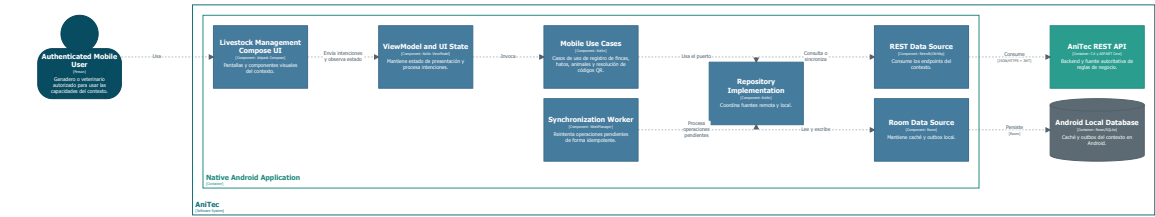
## 2.6.3.5. Bounded Context Software Architecture Component Level Diagrams

El archivo [component-level.dsl](#) contiene las vistas BC3-ApiComponents, BC3-AndroidComponents y BC3-FlutterComponents. Las tres parten del mismo modelo C4 y muestran la separación entre presentación, aplicación, dominio y adaptadores.



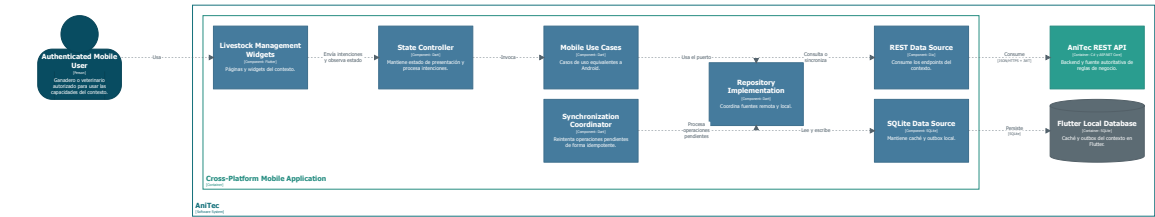
Component View: AniTec - AniTec REST API  
Component: Backend de Livestock Management.  
Tuesday, September 15, 2026 at 3:25 AM Coordinated Universal Time

Figura 2.6.3.1. Componentes de la API para Livestock Management. Fuente: elaboración propia con Structurizr DSL.



Component View: AniTec - Native Android Application  
Component: Frontend de Livestock Management.  
Tuesday, September 15, 2026 at 3:25 AM Coordinated Universal Time

Figura 2.6.3.2. Componentes Android para Livestock Management. Fuente: elaboración propia con Structurizr DSL.



Component View: AniTec - Cross-Platform Mobile Application  
Component: Frontend de Livestock Management.  
Tuesday, September 15, 2026 at 3:25 AM Coordinated Universal Time

Figura 2.6.3.3. Componentes Flutter para Livestock Management. Fuente: elaboración propia con Structurizr DSL.

2.6.3.6. Bounded Context Software Architecture Code Level Diagrams

Los diagramas de código detallan el modelo del dominio y los objetos de persistencia. El UML diferencia los elementos existentes de las incorporaciones objetivo, mientras los esquemas SQL señalan mediante comentarios las columnas propuestas. Los archivos ERD quedan disponibles para completar la importación manual.

2.6.3.6.1. Bounded Context Software Architecture Domain Layer Class Diagrams

El Class Diagram incluye agregados, entidades, value objects, enumeraciones, servicios de dominio e interfaces de repositorio con atributos, operaciones, visibilidad y multiplicidades.

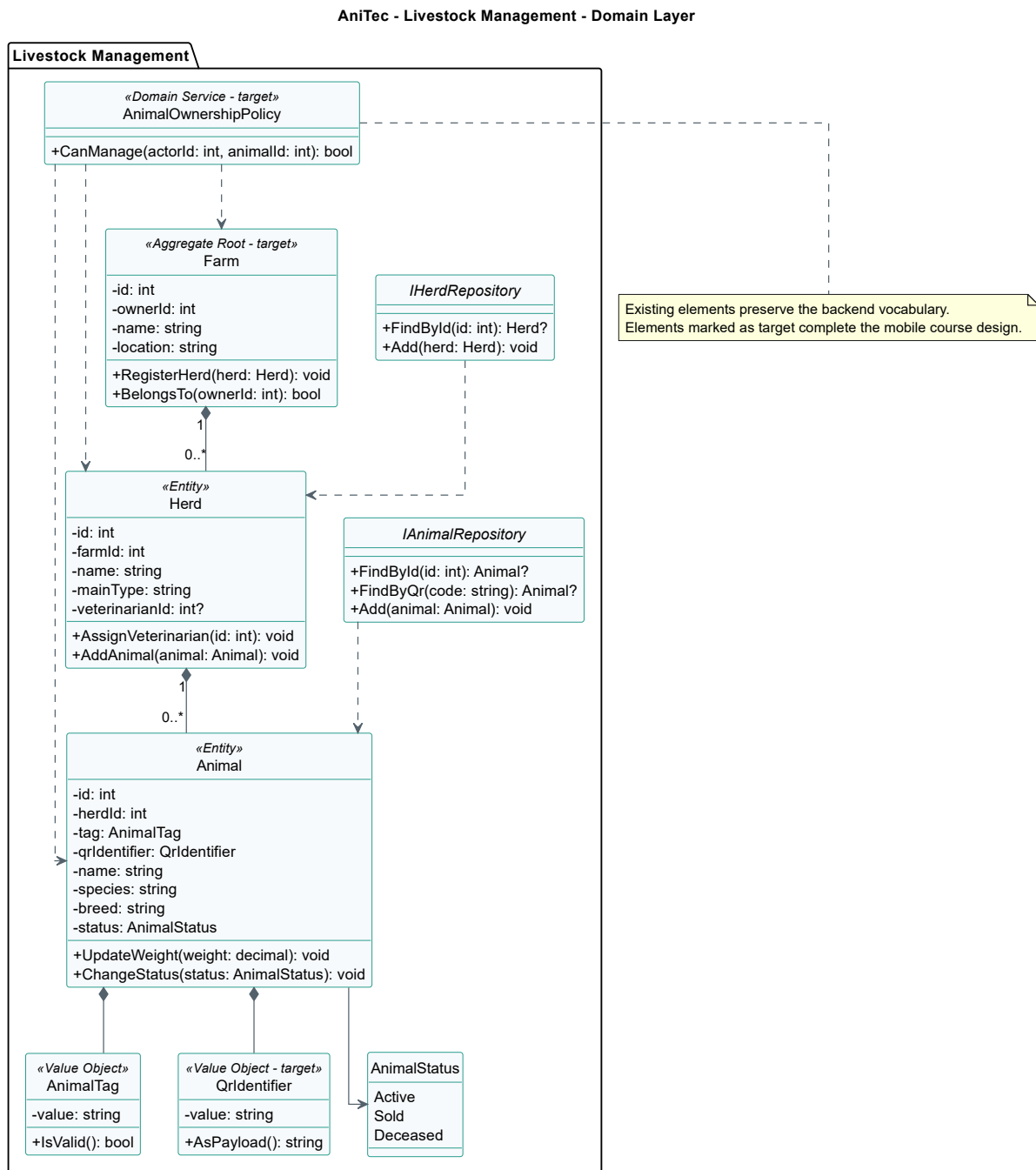


Figura 2.6.3.4. Domain Layer Class Diagram de Livestock Management. Fuente: elaboración propia con PlantUML.

### 2.6.3.6.2. Bounded Context Database Design Diagram

MySQL mantiene la persistencia autoritativa. Room y SQLite contienen únicamente caché, metadatos de sincronización y operaciones pendientes; no sustituyen las reglas ni la fuente de verdad del backend. En IAM, las credenciales y tokens permanecen fuera de las tablas locales y se almacenan mediante mecanismos seguros del sistema operativo.

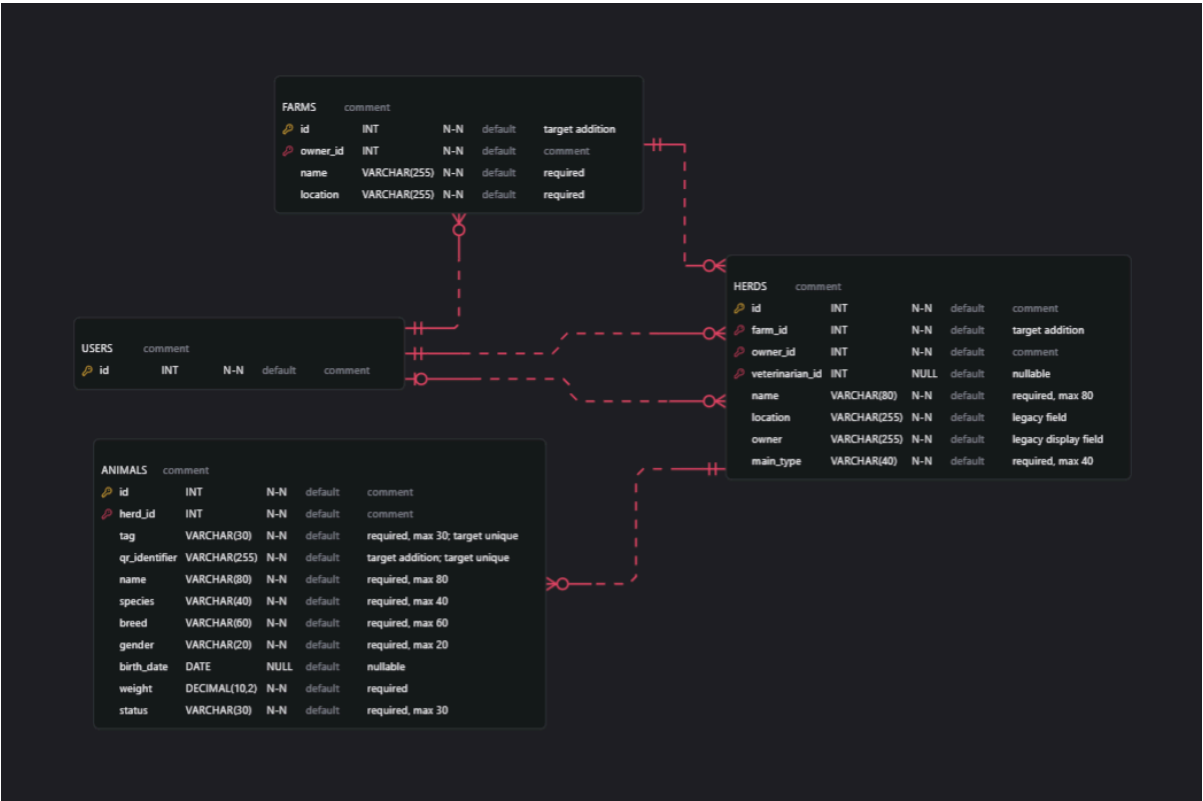


Figura 2.6.3.5. MySQL Database Design de Livestock Management. Fuente: elaboración propia a partir del esquema SQL.

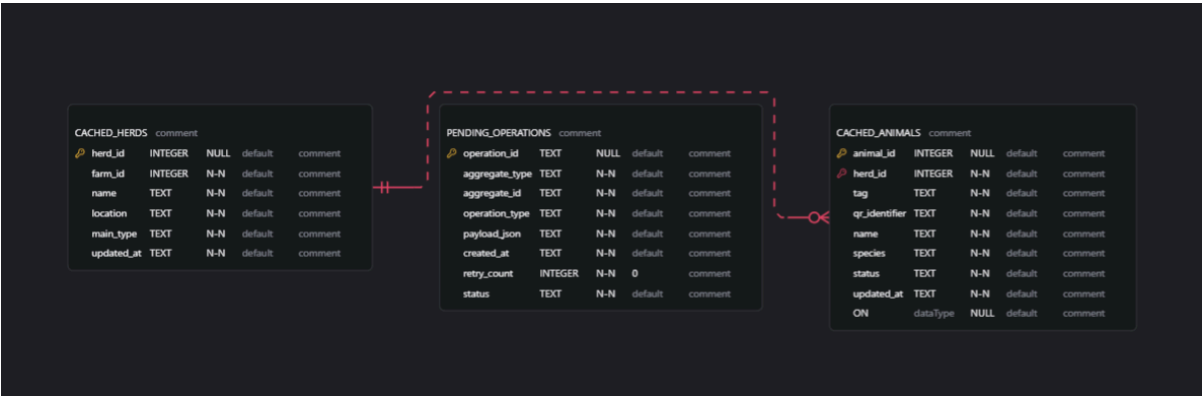


Figura 2.6.3.6. Android Room Database Design de Livestock Management. Fuente: elaboración propia a partir del esquema SQL.

## 2.6.4. Bounded Context: Sanitary Management

Conservar la historia sanitaria del animal y controlar el registro de diagnósticos, tratamientos, prescripciones y seguimientos autorizados.

La base implementada se encuentra en el módulo Sanitary de la API ASP.NET Core. El diseño móvil de Android y Flutter se presenta como **diseño objetivo** porque esos clientes todavía no existen en el workspace. Los tres productos comparten contratos REST y lenguaje ubicuo, mientras la API conserva las reglas autoritativas.

### 2.6.4.1. Domain Layer

Esta capa documenta el modelo que representa el núcleo de **Sanitary Management**. Las reglas autoritativas se ejecutan en la API; Android y Flutter mantienen modelos equivalentes para presentación, validación inmediata y trabajo offline. La columna de estado distingue el código heredado de la arquitectura objetivo.

| Clase                       | Categoría            | Producto y estado                                                                     | Propósito                                                                   | Atributos                                                                                                                                                                                                          | Métodos u operaciones                                                                                                                                  | Relaciones                                                                                                                                                                                                                                      |
|-----------------------------|----------------------|---------------------------------------------------------------------------------------|-----------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| HealthEvent                 | Aggregate Root       | Backend; modelo equivalente en Android y Flutter<br><b>Implementado en el backend</b> | Mantener un acontecimiento clínico dentro de la historia sanitaria.         | id: int<br>animalId: int<br>type: HealthEventType<br>date: Date<br>veterinarianId: int?<br>description: string<br>diagnosis: Diagnosis<br>treatment: Treatment<br>prescription: Prescription<br>nextDueDate: Date? | RescheduleFollowUp(date: Date): void<br>UpdateClinicalData(diagnosis: Diagnosis, treatment: Treatment): void                                           | HealthEvent compone Diagnosis; HealthEvent compone Treatment; HealthEvent compone Prescription; HealthEvent se relaciona con HealthEventType; SanitaryAuthorizationPolicy depende de HealthEvent; IHealthEventRepository depende de HealthEvent |
| Diagnosis                   | Value Object         | Backend, Android y Flutter (modelo canónico)<br><b>Diseño objetivo</b>                | Representar el diagnóstico clínico.                                         | description: string                                                                                                                                                                                                | IsEmpty(): bool                                                                                                                                        | HealthEvent compone Diagnosis                                                                                                                                                                                                                   |
| Treatment                   | Value Object         | Backend, Android y Flutter (modelo canónico)<br><b>Diseño objetivo</b>                | Representar instrucciones de tratamiento.                                   | instructions: string                                                                                                                                                                                               | IsValid(): bool                                                                                                                                        | HealthEvent compone Treatment                                                                                                                                                                                                                   |
| Prescription                | Value Object         | Backend, Android y Flutter (modelo canónico)<br><b>Diseño objetivo</b>                | Representar la prescripción asociada a un evento sanitario.                 | details: string                                                                                                                                                                                                    | IsRequired(): bool                                                                                                                                     | HealthEvent compone Prescription                                                                                                                                                                                                                |
| HealthEventType             | Enumeration          | Backend y modelos equivalentes Android/Flutter<br><b>Implementado en el backend</b>   | Definir los valores válidos de HealthEventType.                             | MedicalVisit<br>Vaccination<br>Treatment<br>SanitaryControl                                                                                                                                                        | —                                                                                                                                                      | HealthEvent se relaciona con HealthEventType                                                                                                                                                                                                    |
| SanitaryAuthorizationPolicy | Domain Service       | Backend, Android y Flutter (modelo canónico)<br><b>Diseño objetivo</b>                | Decidir si un actor puede registrar información sanitaria.                  | —                                                                                                                                                                                                                  | CanRegister(actorId: int, animalId: int): bool                                                                                                         | SanitaryAuthorizationPolicy depende de HealthEvent                                                                                                                                                                                              |
| IHealthEventRepository      | Repository Interface | Backend; modelo equivalente en Android y Flutter<br><b>Implementado en el backend</b> | Abstraer la persistencia de HealthEvent.                                    | —                                                                                                                                                                                                                  | FindById(id: int): HealthEvent?<br>FindByAnimal(animalId: int): List~HealthEvent~<br>Add(event: HealthEvent): void<br>Update(event: HealthEvent): void | IHealthEventRepository depende de HealthEvent                                                                                                                                                                                                   |
| MedicalVisit                | Entity               | Backend; modelo equivalente en Android y Flutter<br><b>Diseño objetivo</b>            | Representar una visita veterinaria asociada con el historial sanitario.     | id: int<br>healthEventId: int<br>veterinarianId: int<br>scheduledAt: DateTime<br>status: string                                                                                                                    | Reschedule(date): Complete()                                                                                                                           | Pertenece a HealthEvent y referencia al veterinario autorizado.                                                                                                                                                                                 |
| SanitaryAlert               | Entity               | Backend; modelo equivalente en Android y Flutter<br><b>Diseño objetivo</b>            | Representar una alerta por vacunación, tratamiento o seguimiento pendiente. | id: int<br>animalId: int<br>dueDate: Date<br>severity: string                                                                                                                                                      | IsDue(on: Date): bool<br>Dismiss()                                                                                                                     | Se origina desde HealthEvent y puede generar una actividad.                                                                                                                                                                                     |

## 2.6.4.2. Interface Layer

Esta capa recibe las acciones relacionadas con **registro y seguimiento de eventos sanitarios, diagnósticos y tratamientos** y las traduce a casos de uso. Los controllers y resources corresponden a la API; las pantallas y

controladores de estado representan la presentación objetivo en Android y Flutter. Ninguna de estas clases implementa reglas de negocio.

| Clase                     | Categoría          | Producto y estado                                            | Propósito                                                                                       | Atributos                                                                                                                                                                                                      | Métodos u operaciones                                                                                                                                                                                                                                                                                                                    | Relaciones                                                                                     |
|---------------------------|--------------------|--------------------------------------------------------------|-------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| HealthEventsController    | REST Controller    | Backend<br>ASP.NET Core<br><b>Implementado en el backend</b> | Publicar por HTTP las capacidades de Sanitary Management.                                       | IHealthEventCommandService<br>commandService<br>IHealthEventQueryService<br>queryService                                                                                                                       | GetAll(CancellationToken cancellationToken)<br>GetById(int id, CancellationToken cancellationToken)<br>Create(CreateHealthEventResource resource, CancellationToken cancellationToken)<br>Update(int id, CreateHealthEventResource resource, CancellationToken cancellationToken)<br>Delete(int id, CancellationToken cancellationToken) | Recibe resources, invoca servicios de aplicación y devuelve resources HTTP.                    |
| CreateHealthEventResource | Resource/Assembler | Backend<br>ASP.NET Core<br><b>Implementado en el backend</b> | Definir un contrato estable de entrada o salida para la API REST.                               | int AnimalId<br>string Type<br>DateOnly Date<br>string Description<br>string Veterinarian<br>string Diagnosis<br>string Treatment<br>string Prescription<br>string FollowUp<br>DateOnly? NextDueDate           | Create(...)<br>Deconstruct(...)                                                                                                                                                                                                                                                                                                          | Es construido o traducido por assemblers y consumido por el controller y los clientes móviles. |
| HealthEventResource       | Resource/Assembler | Backend<br>ASP.NET Core<br><b>Implementado en el backend</b> | Definir un contrato estable de entrada o salida para la API REST.                               | int Id<br>int AnimalId<br>string Type<br>DateOnly Date<br>string Description<br>string Veterinarian<br>string Diagnosis<br>string Treatment<br>string Prescription<br>string FollowUp<br>DateOnly? NextDueDate | Create(...)<br>Deconstruct(...)                                                                                                                                                                                                                                                                                                          | Es construido o traducido por assemblers y consumido por el controller y los clientes móviles. |
| HealthEventScreen         | Composable         | Android / Jetpack<br>Compose<br><b>Diseño objetivo</b>       | Presentar registro y seguimiento de eventos sanitarios, diagnósticos y tratamientos en Android. | uiState<br>onAction<br>navigation                                                                                                                                                                              | Render()<br>Submit()<br>Retry()                                                                                                                                                                                                                                                                                                          | Observa HealthEventViewModel y emite acciones de interfaz.                                     |
| HealthEventViewModel      | Presentation Model | Android / Kotlin<br><b>Diseño objetivo</b>                   | Mantener el estado observable y traducir acciones de Android a casos de uso.                    | state<br>observeUseCase<br>syncUseCase                                                                                                                                                                         | Load()<br>Submit(action)<br>RetrySync()                                                                                                                                                                                                                                                                                                  | Invoca casos de uso de Application Layer y publica un UI State inmutable.                      |
| HealthEventPage           | Widget             | Flutter / Dart<br><b>Diseño objetivo</b>                     | Presentar registro y seguimiento de eventos sanitarios, diagnósticos y tratamientos en Flutter. | state<br>onAction<br>router                                                                                                                                                                                    | build(context)<br>submit()<br>retry()                                                                                                                                                                                                                                                                                                    | Observa HealthEventController y emite intenciones del usuario.                                 |
| HealthEventController     | State Controller   | Flutter / Dart<br><b>Diseño objetivo</b>                     | Mantener el estado de presentación de Flutter y coordinar casos de uso.                         | state<br>observeUseCase<br>syncUseCase                                                                                                                                                                         | load()<br>submit(action)<br>retrySync()                                                                                                                                                                                                                                                                                                  | Invoca Application Layer y publica estados de carga, éxito y error.                            |

## 2.6.4.3. Application Layer

Esta capa coordina las capacidades de **registro y seguimiento de eventos sanitarios, diagnósticos y tratamientos**. Los commands y queries expresan intenciones; los handlers cargan aggregates, aplican reglas, persisten cambios y reaccionan a eventos. Los casos de uso móviles coordinan lectura local, actualización remota y sincronización idempotente.

| Clase                           | Categoría           | Producto y estado                                      | Propósito                                                                                                            | Atributos                                                                                                                                                                                                                   | Métodos u operaciones                                                                                                                                                                                                                           | Relaciones                                                                                      |
|---------------------------------|---------------------|--------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| HealthEventCommandService       | Application Service | Backend ASP.NET Core <b>Implementado en el backend</b> | Orquestar registro y seguimiento de eventos sanitarios, diagnósticos y tratamientos sin contener reglas del dominio. | IHealthEventRepository repository<br>IUnitOfWork unitOfWork                                                                                                                                                                 | Handle(CreateHealthEventCommand command, CancellationToken cancellationToken)<br>Handle(UpdateHealthEventCommand command, CancellationToken cancellationToken)<br>Handle(DeleteHealthEventCommand command, CancellationToken cancellationToken) | Invoca agregados y repositories; confirma la transacción mediante Unit of Work.                 |
| HealthEventQueryService         | Application Service | Backend ASP.NET Core <b>Implementado en el backend</b> | Orquestar registro y seguimiento de eventos sanitarios, diagnósticos y tratamientos sin contener reglas del dominio. | IHealthEventRepository repository                                                                                                                                                                                           | Handle(GetHealthEventByIdQuery query, CancellationToken cancellationToken)<br>Handle(GetAllHealthEventsQuery query, CancellationToken cancellationToken)                                                                                        | Invoca agregados y repositories; confirma la transacción mediante Unit of Work.                 |
| CreateHealthEventCommand        | Command/Query       | Backend ASP.NET Core <b>Implementado en el backend</b> | Transportar una intención o consulta tipada hacia su handler.                                                        | int AnimalId<br>string Type<br>DateOnly Date<br>string Description<br>string Veterinarian<br>string Diagnosis<br>string Treatment<br>string Prescription<br>string FollowUp<br>DateOnly? DateOnly?<br>NextDueDate           | —                                                                                                                                                                                                                                               | Es recibida por un handler o servicio de aplicación y no contiene lógica de negocio.            |
| UpdateHealthEventCommand        | Command/Query       | Backend ASP.NET Core <b>Implementado en el backend</b> | Transportar una intención o consulta tipada hacia su handler.                                                        | int Id<br>int AnimalId<br>string Type<br>DateOnly Date<br>string Description<br>string Veterinarian<br>string Diagnosis<br>string Treatment<br>string Prescription<br>string FollowUp<br>DateOnly? DateOnly?<br>NextDueDate | —                                                                                                                                                                                                                                               | Es recibida por un handler o servicio de aplicación y no contiene lógica de negocio.            |
| DeleteHealthEventCommand        | Command/Query       | Backend ASP.NET Core <b>Implementado en el backend</b> | Transportar una intención o consulta tipada hacia su handler.                                                        | int Id                                                                                                                                                                                                                      | —                                                                                                                                                                                                                                               | Es recibida por un handler o servicio de aplicación y no contiene lógica de negocio.            |
| GetHealthEventByIdQuery         | Command/Query       | Backend ASP.NET Core <b>Implementado en el backend</b> | Transportar una intención o consulta tipada hacia su handler.                                                        | int Id                                                                                                                                                                                                                      | —                                                                                                                                                                                                                                               | Es recibida por un handler o servicio de aplicación y no contiene lógica de negocio.            |
| ObserveHealthEventUseCase       | Use Case            | Android y Flutter <b>Diseño objetivo</b>               | Entregar primero datos locales y actualizar la consulta cuando exista conectividad.                                  | localRepository<br>remoteRepository<br>connectivityMonitor                                                                                                                                                                  | Execute(criteria): Stream<Result>                                                                                                                                                                                                               | Es invocado por ViewModel/Controller y coordina repositorios móviles.                           |
| SyncHealthEventUseCase          | Use Case            | Android y Flutter <b>Diseño objetivo</b>               | Procesar operaciones móviles pendientes de manera idempotente.                                                       | outboxRepository<br>remoteRepository<br>conflictResolver                                                                                                                                                                    | Execute(): SyncResult                                                                                                                                                                                                                           | Lee el outbox local, consume la API y actualiza el estado de sincronización.                    |
| CreateHealthEventCommandHandler | Command Handler     | Backend ASP.NET Core <b>Diseño objetivo</b>            | Ejecutar una intención concreta, aplicar reglas del agregado y confirmar la transacción.                             | repository<br>unitOfWork<br>domainPolicy                                                                                                                                                                                    | Handle(command): Result                                                                                                                                                                                                                         | Consume un Command, carga el aggregate mediante su repository y puede publicar un Domain Event. |
| UpdateHealthEventCommandHandler | Command Handler     | Backend ASP.NET Core <b>Diseño objetivo</b>            | Ejecutar una intención concreta, aplicar reglas del agregado y confirmar la transacción.                             | repository<br>unitOfWork<br>domainPolicy                                                                                                                                                                                    | Handle(command): Result                                                                                                                                                                                                                         | Consume un Command, carga el aggregate mediante su repository y puede publicar un Domain Event. |

| Clase                             | Categoría     | Producto y estado                              | Propósito                                                                  | Atributos                                              | Métodos u operaciones     | Relaciones                                                                                           |
|-----------------------------------|---------------|------------------------------------------------|----------------------------------------------------------------------------|--------------------------------------------------------|---------------------------|------------------------------------------------------------------------------------------------------|
| HealthEventRegisteredEventHandler | Event Handler | Backend ASP.NET Core<br><b>Diseño objetivo</b> | Reaccionar al evento confirmado y actualizar proyecciones o integraciones. | projectionRepository<br>notificationPort<br>unitOfWork | Handle(domainEvent): Task | Consume un Domain Event y utiliza puertos de infraestructura sin modificar directamente el agregado. |

## 2.6.4.4. Infrastructure Layer

Esta capa implementa los puertos definidos hacia el interior de **Sanitary Management** y concentra acceso a base de datos, red, almacenamiento local e integraciones externas. Las clases de infraestructura traducen errores y contratos técnicos antes de devolver resultados a Application Layer.

| Clase                          | Categoría                 | Producto y estado                                                    | Propósito                                                           | Atributos                                 | Métodos u operaciones                                           | Relaciones                                                                                   |
|--------------------------------|---------------------------|----------------------------------------------------------------------|---------------------------------------------------------------------|-------------------------------------------|-----------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| HealthEventRepository          | Repository Adapter        | Backend / Entity Framework Core<br><b>Implementado en el backend</b> | Implementar el puerto de persistencia definido por Domain Layer.    | AppDbContext<br>context                   | FindById(id)<br>Add(entity)<br>Update(entity)<br>Delete(entity) | Implementa IHealthEventRepository; utiliza AppDbContext/MySQL y reconstruye el aggregate.    |
| ModelBuilderExtensions         | Persistence Configuration | Backend / Entity Framework Core<br><b>Implementado en el backend</b> | Mapear entidades y value objects del contexto al modelo relacional. | EntityTypeBuilder<br>configuration        | ApplyConfiguration(modelBuilder)                                | Configura tablas, claves, relaciones, restricciones y conversiones de Entity Framework Core. |
| HealthEventApiDataSource       | Remote Adapter            | Android / Kotlin<br><b>Diseño objetivo</b>                           | Implementar el acceso remoto del cliente móvil a la API.            | httpClient<br>tokenProvider<br>serializer | Get(criteria)<br>Create(dto)<br>Update(dto)<br>Delete(id)       | Consume controllers REST por HTTPS/JSON y traduce errores HTTP al modelo de aplicación.      |
| HealthEventDao                 | Room Adapter              | Android / Room<br><b>Diseño objetivo</b>                             | Implementar persistencia local y observación reactiva en Android.   | roomDatabase<br>entityMapper              | Observe(criteria)<br>Upsert(entity)<br>Delete(id)<br>Pending()  | Implementa el puerto local mediante Room y participa en la estrategia de caché/outbox.       |
| HealthEventRemoteDataSource    | Remote Adapter            | Flutter / Dart<br><b>Diseño objetivo</b>                             | Implementar el acceso remoto del cliente móvil a la API.            | httpClient<br>tokenProvider<br>serializer | Get(criteria)<br>Create(dto)<br>Update(dto)<br>Delete(id)       | Consume controllers REST por HTTPS/JSON y traduce errores HTTP al modelo de aplicación.      |
| HealthEventLocalDataSource     | SQLite Adapter            | Flutter / Dart<br><b>Diseño objetivo</b>                             | Implementar persistencia local equivalente en Flutter.              | sqliteDatabase<br>entityMapper            | watch(criteria)<br>upsert(entity)<br>delete(id)<br>pending()    | Implementa el puerto local mediante SQLite y participa en la estrategia de caché/outbox.     |
| LivestockAnimalAdapter         | Context Adapter           | Backend ASP.NET Core<br><b>Diseño objetivo</b>                       | Aislar una dependencia externa detrás de un puerto explícito.       | livestockFacade                           | GetAnimal(animalId)<br>Exists(animalId): bool                   | Implementa el puerto de consulta de animales y consume Livestock Management.                 |
| VeterinaryAuthorizationAdapter | Context Adapter           | Backend ASP.NET Core<br><b>Diseño objetivo</b>                       | Aislar una dependencia externa detrás de un puerto explícito.       | collaborationFacade                       | CanWrite(veterinarianId, animalId): bool                        | Implementa el puerto de autorización y consume Veterinary Collaboration.                     |
| SanitaryOutboxStore            | Offline Adapter           | Android y Flutter<br><b>Diseño objetivo</b>                          | Aislar una dependencia externa detrás de un puerto explícito.       | database, serializer                      | Enqueue(event)<br>Pending()<br>MarkSynced(id)                   | Implementa el puerto de sincronización sanitaria sobre Room o SQLite.                        |

## 2.6.4.5. Bounded Context Software Architecture Component Level Diagrams

El archivo [component-level.dsl](#) contiene las vistas BC4-APIComponents, BC4-AndroidComponents y BC4-FlutterComponents. Las tres parten del mismo modelo C4 y muestran la separación entre presentación, aplicación, dominio y adaptadores.

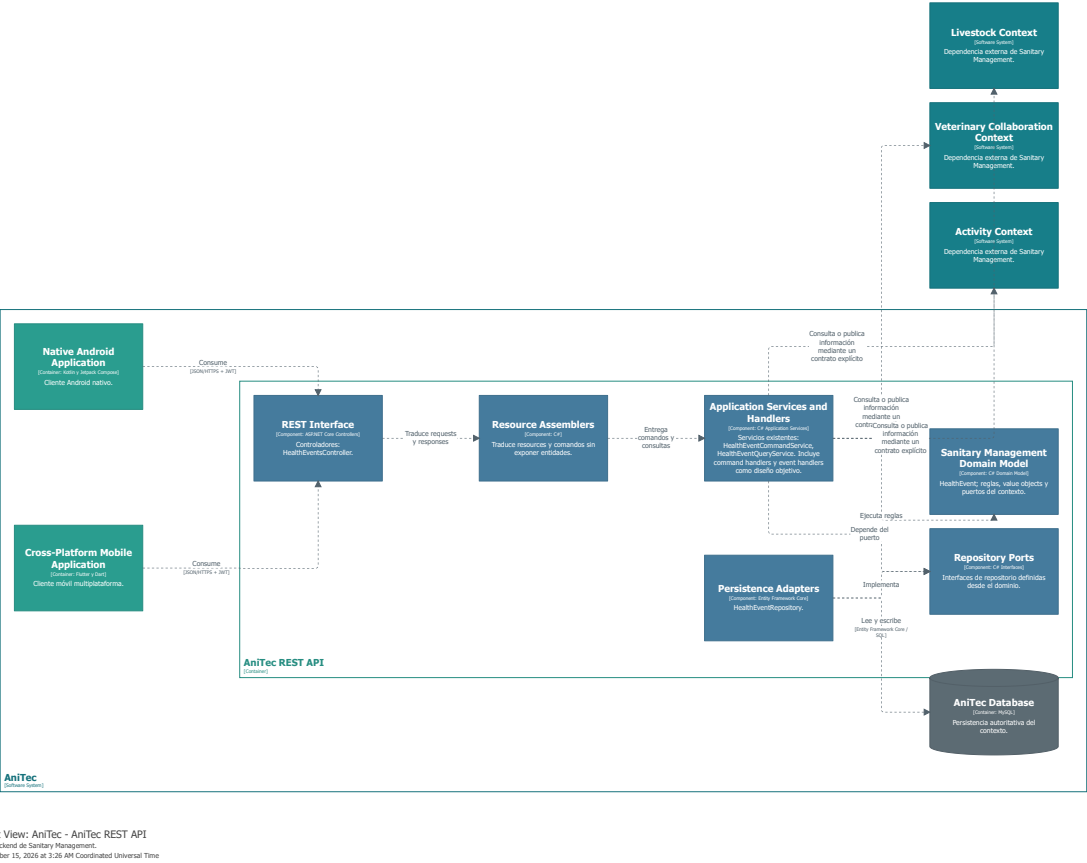


Figura 2.6.4.1. Componentes de la API para Sanitary Management. Fuente: elaboración propia con Structurizr DSL.

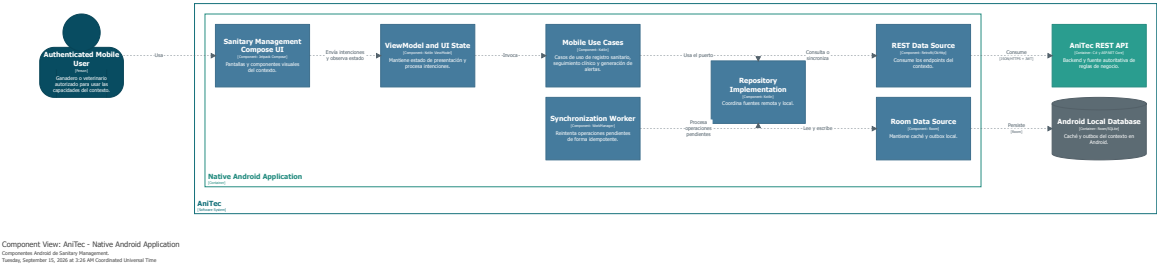


Figura 2.6.4.2. Componentes Android para Sanitary Management. Fuente: elaboración propia con Structurizr DSL.

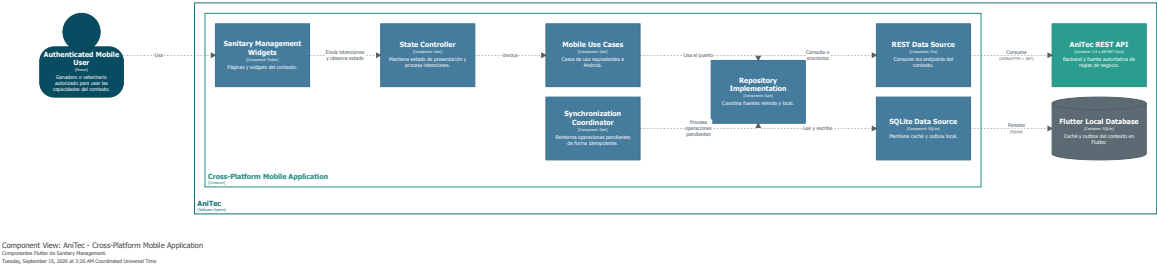


Figura 2.6.4.3. Componentes Flutter para Sanitary Management. Fuente: elaboración propia con Structurizr DSL.

2.6.4.6. Bounded Context Software Architecture Code Level Diagrams

Los diagramas de código detallan el modelo del dominio y los objetos de persistencia. El UML diferencia los elementos existentes de las incorporaciones objetivo, mientras los esquemas SQL señalan mediante comentarios las

columnas propuestas. Los archivos ERD quedan disponibles para completar la importación manual.

### 2.6.4.6.1. Bounded Context Domain Layer Class Diagrams

El Class Diagram incluye agregados, entidades, value objects, enumeraciones, servicios de dominio e interfaces de repositorio con atributos, operaciones, visibilidad y multiplicidades.

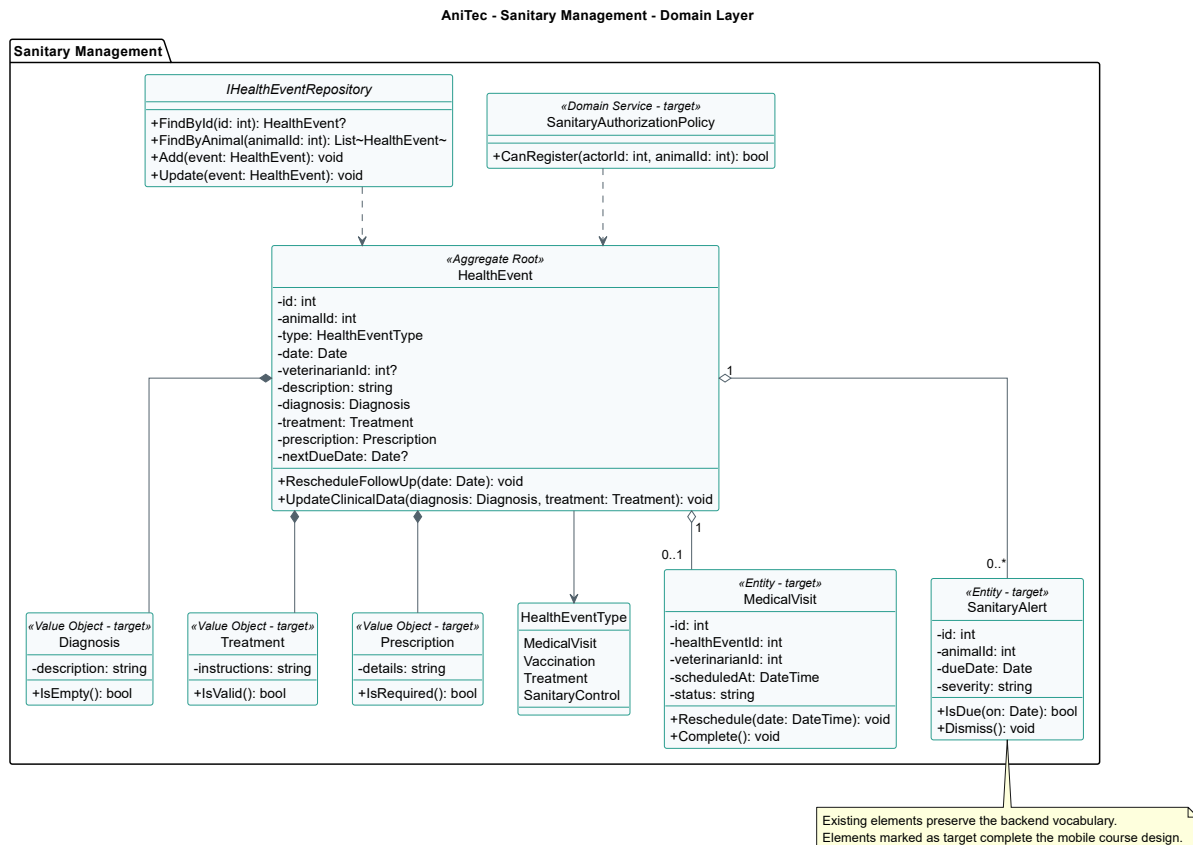


Figura 2.6.4.4. Domain Layer Class Diagram de Sanitary Management. Fuente: elaboración propia con PlantUML.

### 2.6.4.6.2. Bounded Context Database Design Diagram

MySQL mantiene la persistencia autoritativa. Room y SQLite contienen únicamente caché, metadatos de sincronización y operaciones pendientes; no sustituyen las reglas ni la fuente de verdad del backend. En IAM, las credenciales y tokens permanecen fuera de las tablas locales y se almacenan mediante mecanismos seguros del sistema operativo.

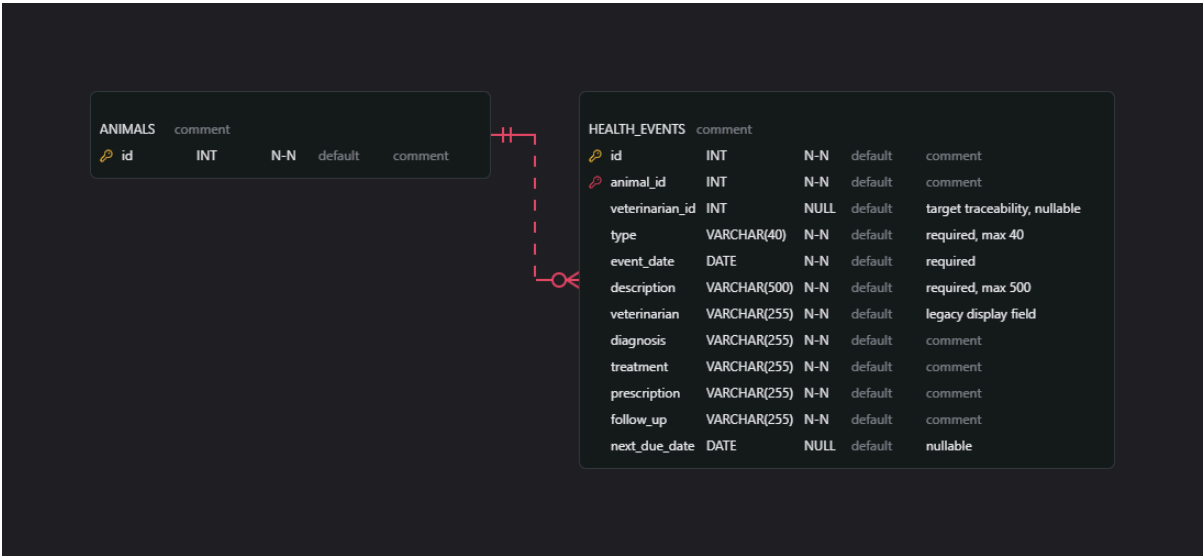


Figura 2.6.4.5. MySQL Database Design de Sanitary Management. Fuente: elaboración propia a partir del esquema SQL.

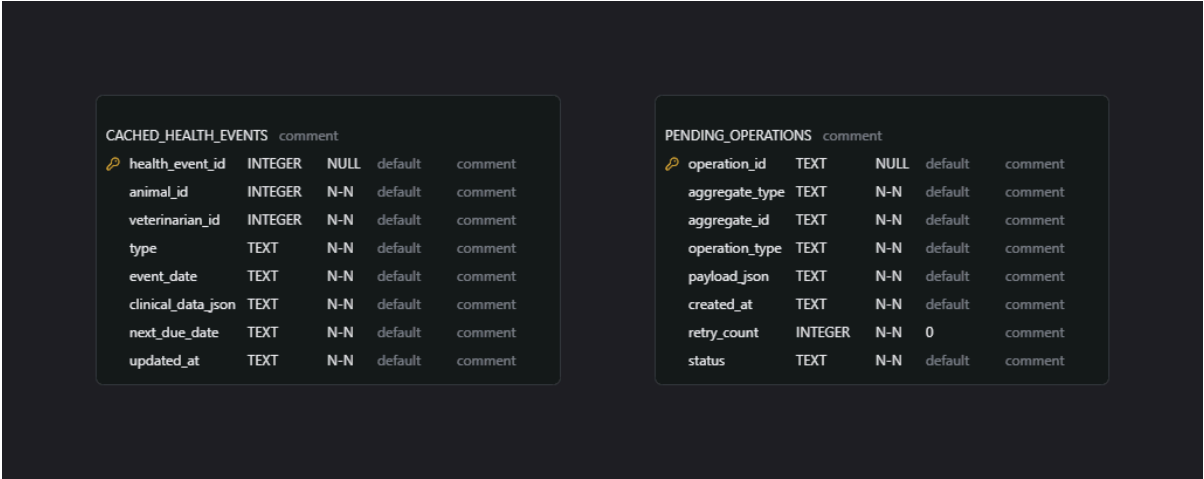


Figura 2.6.4.6. Android Room Database Design de Sanitary Management. Fuente: elaboración propia a partir del esquema SQL.

## 2.6.5. Bounded Context: Veterinary Collaboration

Administrar solicitudes y autorizaciones entre ganaderos y veterinarios, delimitando clientes, pacientes y alcance de acceso.

La base implementada se encuentra en el módulo Clients de la API ASP.NET Core. El diseño móvil de Android y Flutter se presenta como **diseño objetivo** porque esos clientes todavía no existen en el workspace. Los tres productos comparten contratos REST y lenguaje ubicuo, mientras la API conserva las reglas autoritativas.

### 2.6.5.1. Domain Layer

Esta capa documenta el modelo que representa el núcleo de **Veterinary Collaboration**. Las reglas autoritativas se ejecutan en la API; Android y Flutter mantienen modelos equivalentes para presentación, validación inmediata y trabajo offline. La columna de estado distingue el código heredado de la arquitectura objetivo.

| Clase                         | Categoría            | Producto y estado                                                                     | Propósito                                                          | Atributos                                                                                                                                                                                    | Métodos u operaciones                                                                                                                                                                                            | Relaciones                                                                                                                                                                                                             |
|-------------------------------|----------------------|---------------------------------------------------------------------------------------|--------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| VeterinarianClient            | Aggregate Root       | Backend; modelo equivalente en Android y Flutter<br><b>Implementado en el backend</b> | Controlar la relación y autorización entre veterinario y ganadero. | id: int<br>veterinarianId: int<br>rancherId: int<br>status:<br>CollaborationStatus<br>requestedAt: DateTime<br>acceptedAt: DateTime?<br>revokedAt: DateTime?<br>scope:<br>AuthorizationScope | Accept(at: DateTime): void<br>Reject(): void<br>Revoke(at: DateTime): void<br>IsActive(): bool                                                                                                                   | VeterinarianClient se relaciona con CollaborationStatus; VeterinarianClient compone AuthorizationScope; CollaborationPolicy depende de VeterinarianClient; IVeterinarianClientRepository depende de VeterinarianClient |
| CollaborationStatus           | Enumeration          | Backend, Android y Flutter (modelo canónico)<br><b>Diseño objetivo</b>                | Definir los valores válidos de CollaborationStatus.                | Pending<br>Accepted<br>Rejected<br>Revoked                                                                                                                                                   | —                                                                                                                                                                                                                | VeterinarianClient se relaciona con CollaborationStatus                                                                                                                                                                |
| AuthorizationScope            | Value Object         | Backend, Android y Flutter (modelo canónico)<br><b>Diseño objetivo</b>                | Delimitar fincas, animales y operaciones autorizadas.              | farmIds: Set~int~<br>animalIds: Set~int~<br>canWriteHealthRecords: bool                                                                                                                      | AllowsAnimal(animalId: int): bool                                                                                                                                                                                | VeterinarianClient compone AuthorizationScope                                                                                                                                                                          |
| CollaborationPolicy           | Domain Service       | Backend y modelos equivalentes Android/Flutter<br><b>Diseño objetivo</b>              | Evaluar aceptación y acceso dentro de una colaboración.            | —                                                                                                                                                                                            | CanAccept(rancherId: int, relation: VeterinarianClient): bool<br>CanAccess(veterinarianId: int, animalId: int): bool                                                                                             | CollaborationPolicy depende de VeterinarianClient                                                                                                                                                                      |
| IVeterinarianClientRepository | Repository Interface | Backend; modelo equivalente en Android y Flutter<br><b>Implementado en el backend</b> | Abstraer la persistencia de VeterinarianClient.                    | —                                                                                                                                                                                            | Find(veterinarianId: int, rancherId: int): VeterinarianClient?<br>FindByVeterinarian(id: int): List~VeterinarianClient~<br>Add(relation: VeterinarianClient): void<br>Update(relation: VeterinarianClient): void | IVeterinarianClientRepository depende de VeterinarianClient                                                                                                                                                            |
| CollaborationRequest          | Entity               | Backend; modelo equivalente en Android y Flutter<br><b>Diseño objetivo</b>            | Representar la solicitud inicial entre veterinario y ganadero.     | veterinarianId: int<br>rancherId: int<br>requestedAt: DateTime                                                                                                                               | Accept(at)<br>Reject()                                                                                                                                                                                           | Da origen o cambia el estado de VeterinarianClient.                                                                                                                                                                    |
| AccessGrant                   | Entity               | Backend; modelo equivalente en Android y Flutter<br><b>Diseño objetivo</b>            | Materializar el permiso concedido al veterinario.                  | relationId: int<br>scope:<br>AuthorizationScope<br>grantedAt: DateTime<br>revokedAt: DateTime?                                                                                               | Allows(animalId): bool<br>Revoke(at)                                                                                                                                                                             | Depende de VeterinarianClient y contiene AuthorizationScope.                                                                                                                                                           |

## 2.6.5.2. Interface Layer

Esta capa recibe las acciones relacionadas con **solicitud, aceptación, revocación y consulta de colaboraciones veterinarias** y las traduce a casos de uso. Los controllers y resources corresponden a la API; las pantallas y controladores de estado representan la presentación objetivo en Android y Flutter. Ninguna de estas clases implementa reglas de negocio.

| Clase                         | Categoría       | Producto y estado                                         | Propósito                                                      | Atributos                                                                                                                                                                                                                       | Métodos u operaciones                                                                                                                                                                                                                     | Relaciones                                                                  |
|-------------------------------|-----------------|-----------------------------------------------------------|----------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| VeterinarianClientsController | REST Controller | Backend ASP.NET Core<br><b>Implementado en el backend</b> | Publicar por HTTP las capacidades de Veterinary Collaboration. | IVeterinarianClientCommandService<br>commandService<br>IVeterinarianClientQueryService<br>queryService<br>IUserQueryService userQueryService<br>IHerdQueryService herdQueryService<br>IAnimalQueryService<br>animalQueryService | GetClients(int veterinarianId, Cancellation token)<br>GetAvailableRanchers(int veterinarianId, Cancellation token)<br>AddClient(int veterinarianId, int rancherId, Cancellation token)<br>RemoveClient(int veterinarianId, int rancherId, | Recibe resources, invoca servicios de aplicación y devuelve resources HTTP. |

| Clase                        | Categoría          | Producto y estado                                            | Propósito                                                                                         | Atributos                                                                                                                                                        | Métodos u operaciones                   | Relaciones                                                                                     |
|------------------------------|--------------------|--------------------------------------------------------------|---------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------|------------------------------------------------------------------------------------------------|
|                              |                    |                                                              |                                                                                                   |                                                                                                                                                                  | CancellationToken<br>cancellationToken) |                                                                                                |
| AvailableRancherResource     | Resource/Assembler | Backend<br>ASP.NET Core<br><b>Implementado en el backend</b> | Definir un contrato estable de entrada o salida para la API REST.                                 | int Id<br>string Username<br>string FullName<br>int Herds<br>int Animals                                                                                         | Create(...)<br>Deconstruct(...)         | Es construido o traducido por assemblers y consumido por el controller y los clientes móviles. |
| VeterinarianClientResource   | Resource/Assembler | Backend<br>ASP.NET Core<br><b>Implementado en el backend</b> | Definir un contrato estable de entrada o salida para la API REST.                                 | int Id<br>int VeterinarianId<br>int RancherId<br>string RancherName<br>string Status<br>int Herds<br>int Animals<br>DateTime RequestedAt<br>DateTime? AcceptedAt | Create(...)<br>Deconstruct(...)         | Es construido o traducido por assemblers y consumido por el controller y los clientes móviles. |
| VeterinarianClientScreen     | Composable         | Android / Jetpack<br>Compose<br><b>Diseño objetivo</b>       | Presentar solicitud, aceptación, revocación y consulta de colaboraciones veterinarias en Android. | uiState<br>onAction<br>navigation                                                                                                                                | Render()<br>Submit()<br>Retry()         | Observa VeterinarianClientViewModel y emite acciones de interfaz.                              |
| VeterinarianClientViewModel  | Presentation Model | Android / Kotlin<br><b>Diseño objetivo</b>                   | Mantener el estado observable y traducir acciones de Android a casos de uso.                      | state<br>observeUseCase<br>syncUseCase                                                                                                                           | Load()<br>Submit(action)<br>RetrySync() | Invoca casos de uso de Application Layer y publica un UI State inmutable.                      |
| VeterinarianClientPage       | Widget             | Flutter / Dart<br><b>Diseño objetivo</b>                     | Presentar solicitud, aceptación, revocación y consulta de colaboraciones veterinarias en Flutter. | state<br>onAction<br>router                                                                                                                                      | build(context)<br>submit()<br>retry()   | Observa VeterinarianClientController y emite intenciones del usuario.                          |
| VeterinarianClientController | State Controller   | Flutter / Dart<br><b>Diseño objetivo</b>                     | Mantener el estado de presentación de Flutter y coordinar casos de uso.                           | state<br>observeUseCase<br>syncUseCase                                                                                                                           | load()<br>submit(action)<br>retrySync() | Invoca Application Layer y publica estados de carga, éxito y error.                            |

## 2.6.5.3. Application Layer

Esta capa coordina las capacidades de **solicitud, aceptación, revocación y consulta de colaboraciones veterinarias**. Los commands y queries expresan intenciones; los handlers cargan aggregates, aplican reglas, persisten cambios y reaccionan a eventos. Los casos de uso móviles coordinan lectura local, actualización remota y sincronización idempotente.

| Clase                            | Categoría           | Producto y estado                                            | Propósito                                                                                                              | Atributos                                                             | Métodos u operaciones                                                                                                                                                        | Relaciones                                                                 |
|----------------------------------|---------------------|--------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| VeterinarianClientCommandService | Application Service | Backend<br>ASP.NET Core<br><b>Implementado en el backend</b> | Orquestar solicitud, aceptación, revocación y consulta de colaboraciones veterinarias sin contener reglas del dominio. | IVeterinarianClientRepository<br>repository<br>IUnitOfWork unitOfWork | Handle(CreateVeterinarianClientCommand command, CancellationToken cancellationToken)<br>Handle(DeleteVeterinarianClientCommand command, CancellationToken cancellationToken) | Invoca agregados repositories; confi la transacción mediante Unit of Work. |
| VeterinarianClientQueryService   | Application Service | Backend<br>ASP.NET Core<br><b>Implementado en el backend</b> | Orquestar solicitud, aceptación, revocación y consulta de                                                              | IVeterinarianClientRepository<br>repository                           | Handle(GetVeterinarianClientsByVeterinarianIdQuery query, CancellationToken cancellationToken)                                                                               | Invoca agregados repositories; confi la transacción mediante Unit of Work. |

| Clase                                       | Categoría       | Producto y estado                                      | Propósito                                                                                | Atributos                                                         | Métodos u operaciones             | Relaciones                                                                                              |
|---------------------------------------------|-----------------|--------------------------------------------------------|------------------------------------------------------------------------------------------|-------------------------------------------------------------------|-----------------------------------|---------------------------------------------------------------------------------------------------------|
|                                             |                 |                                                        | colaboraciones veterinarias sin contener reglas del dominio.                             |                                                                   |                                   |                                                                                                         |
| CreateVeterinarianClientCommand             | Command/Query   | Backend ASP.NET Core <b>Implementado en el backend</b> | Transportar una intención o consulta tipada hacia su handler.                            | int VeterinarianId<br>int RancherId<br>string Status = "Accepted" | —                                 | Es recibida por un handler o servicio aplicación y no contiene lógica de negocio.                       |
| DeleteVeterinarianClientCommand             | Command/Query   | Backend ASP.NET Core <b>Implementado en el backend</b> | Transportar una intención o consulta tipada hacia su handler.                            | int VeterinarianId<br>int RancherId                               | —                                 | Es recibida por un handler o servicio aplicación y no contiene lógica de negocio.                       |
| GetVeterinarianClientsByVeterinarianIdQuery | Command/Query   | Backend ASP.NET Core <b>Implementado en el backend</b> | Transportar una intención o consulta tipada hacia su handler.                            | int VeterinarianId                                                | —                                 | Es recibida por un handler o servicio aplicación y no contiene lógica de negocio.                       |
| ObserveVeterinarianClientUseCase            | Use Case        | Android y Flutter <b>Diseño objetivo</b>               | Entregar primero datos locales y actualizar la consulta cuando exista conectividad.      | localRepository<br>remoteRepository<br>connectivityMonitor        | Execute(criteria): Stream<Result> | Es invocado por ViewModel/Controlador y coordina repositorios móvil                                     |
| SyncVeterinarianClientUseCase               | Use Case        | Android y Flutter <b>Diseño objetivo</b>               | Procesar operaciones móviles pendientes de manera idempotente.                           | outboxRepository<br>remoteRepository<br>conflictResolver          | Execute(): SyncResult             | Lee el outbox local consume la API y actualiza el estado de sincronización.                             |
| RequestCollaborationCommandHandler          | Command Handler | Backend ASP.NET Core <b>Diseño objetivo</b>            | Ejecutar una intención concreta, aplicar reglas del agregado y confirmar la transacción. | repository<br>unitOfWork<br>domainPolicy                          | Handle(command): Result           | Consume un Command, carga el agregado, media su repository y publica un Domain Event.                   |
| AcceptCollaborationCommandHandler           | Command Handler | Backend ASP.NET Core <b>Diseño objetivo</b>            | Ejecutar una intención concreta, aplicar reglas del agregado y confirmar la transacción. | repository<br>unitOfWork<br>domainPolicy                          | Handle(command): Result           | Consume un Command, carga el agregado, media su repository y publica un Domain Event.                   |
| CollaborationAcceptedEventHandler           | Event Handler   | Backend ASP.NET Core <b>Diseño objetivo</b>            | Reaccionar al evento confirmado y actualizar proyecciones o integraciones.               | projectionRepository<br>notificationPort<br>unitOfWork            | Handle(domainEvent): Task         | Consume un Domain Event y utiliza proveedor de infraestructura para modificar directamente el agregado. |

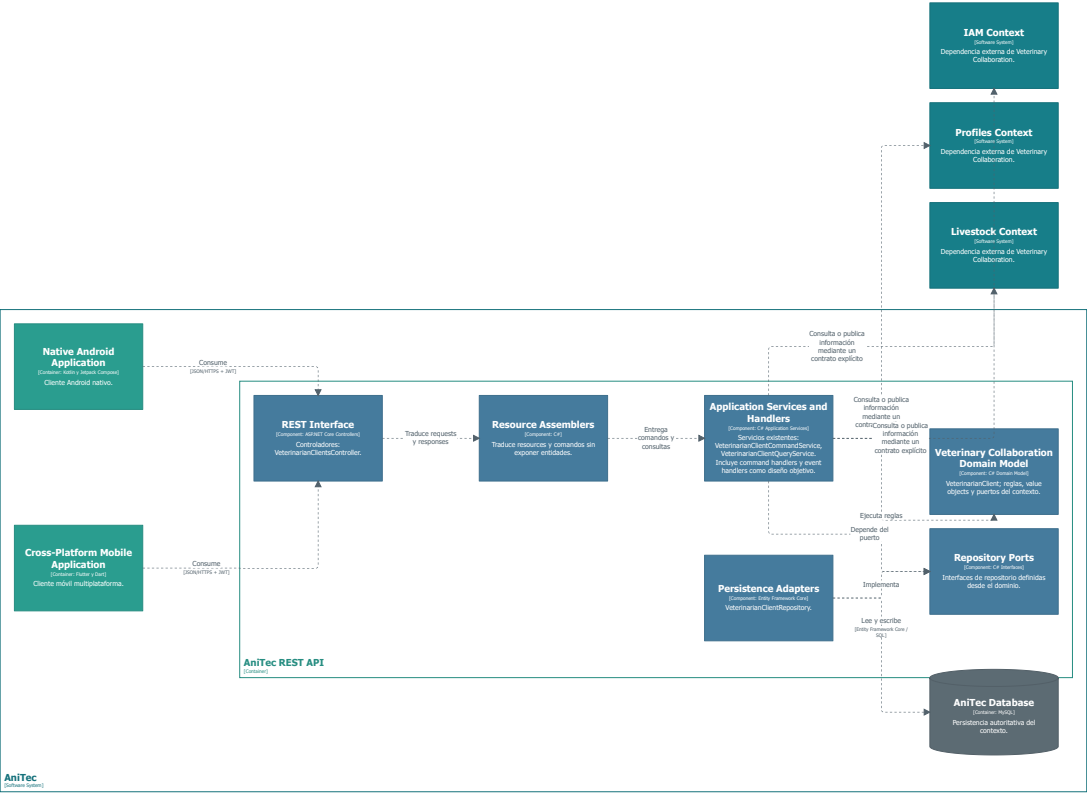
## 2.6.5.4. Infrastructure Layer

Esta capa implementa los puertos definidos hacia el interior de **Veterinary Collaboration** y concentra acceso a base de datos, red, almacenamiento local e integraciones externas. Las clases de infraestructura traducen errores y contratos técnicos antes de devolver resultados a Application Layer.

| Clase                              | Categoría                 | Producto y estado                                                    | Propósito                                                           | Atributos                           | Métodos u operaciones                                                                                                                                                                                                                                                                                                     | Relaciones                                                                                       |
|------------------------------------|---------------------------|----------------------------------------------------------------------|---------------------------------------------------------------------|-------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| VeterinarianClientRepository       | Repository Adapter        | Backend / Entity Framework Core<br><b>Implementado en el backend</b> | Implementar el puerto de persistencia definido por Domain Layer.    | AppDbContext context                | FindByVeterinarianIdAsync(int veterinarianId, Cancellation.Token cancellationToken)<br>FindByVeterinarianIdAndRancherIdAsync(int veterinarianId, int rancherId, Cancellation.Token cancellationToken)<br>ExistsByVeterinarianIdAndRancherIdAsync(int veterinarianId, int rancherId, Cancellation.Token cancellationToken) | Implementa IVeterinarianClientRepository; utiliza AppDbContext/MySQL y reconstruye el aggregate. |
| ModelBuilderExtensions             | Persistence Configuration | Backend / Entity Framework Core<br><b>Implementado en el backend</b> | Mapear entidades y value objects del contexto al modelo relacional. | EntityTypeBuilder configuration     | ApplyConfiguration(modelBuilder)                                                                                                                                                                                                                                                                                          | Configura tablas, claves, relaciones, restricciones y conversiones de Entity Framework Core.     |
| VeterinarianClientApiDataSource    | Remote Adapter            | Android / Kotlin<br><b>Diseño objetivo</b>                           | Implementar el acceso remoto del cliente móvil a la API.            | httpClient tokenProvider serializer | Get(criteria)<br>Create(dto)<br>Update(dto)<br>Delete(id)                                                                                                                                                                                                                                                                 | Consume controllers REST por HTTPS/JSON y traduce errores HTTP al modelo de aplicación.          |
| VeterinarianClientDao              | Room Adapter              | Android / Room<br><b>Diseño objetivo</b>                             | Implementar persistencia local y observación reactiva en Android.   | roomDatabase entityMapper           | Observe(criteria)<br>Upsert(entity)<br>Delete(id)<br>Pending()                                                                                                                                                                                                                                                            | Implementa el puerto local mediante Room y participa en la estrategia de caché/outbox.           |
| VeterinarianClientRemoteDataSource | Remote Adapter            | Flutter / Dart<br><b>Diseño objetivo</b>                             | Implementar el acceso remoto del cliente móvil a la API.            | httpClient tokenProvider serializer | Get(criteria)<br>Create(dto)<br>Update(dto)<br>Delete(id)                                                                                                                                                                                                                                                                 | Consume controllers REST por HTTPS/JSON y traduce errores HTTP al modelo de aplicación.          |
| VeterinarianClientLocalDataSource  | SQLite Adapter            | Flutter / Dart<br><b>Diseño objetivo</b>                             | Implementar persistencia local equivalente en Flutter.              | sqliteDatabase entityMapper         | watch(criteria)<br>upsert(entity)<br>delete(id)<br>pending()                                                                                                                                                                                                                                                              | Implementa el puerto local mediante SQLite y participa en la estrategia de caché/outbox.         |
| IamIdentityAdapter                 | Context Adapter           | Backend ASP.NET Core<br><b>Diseño objetivo</b>                       | Aislar una dependencia externa detrás de un puerto explícito.       | iamFacade                           | GetRole(userId)<br>Exists(userId): bool                                                                                                                                                                                                                                                                                   | Implementa el puerto de identidad y consume IAM.                                                 |
| ProfilesDirectoryAdapter           | Context Adapter           | Backend ASP.NET Core<br><b>Diseño objetivo</b>                       | Aislar una dependencia externa detrás de un puerto explícito.       | profilesFacade                      | GetDisplayName(userId)<br>FindRanchers()                                                                                                                                                                                                                                                                                  | Implementa el puerto de directorio y consume Profile Management.                                 |
| LivestockPatientAdapter            | Context Adapter           | Backend ASP.NET Core<br><b>Diseño objetivo</b>                       | Aislar una dependencia externa detrás de un puerto explícito.       | livestockFacade                     | GetAnimals(rancherId)<br>ValidateScope(scope)                                                                                                                                                                                                                                                                             | Implementa el puerto de pacientes y consume Livestock Management.                                |

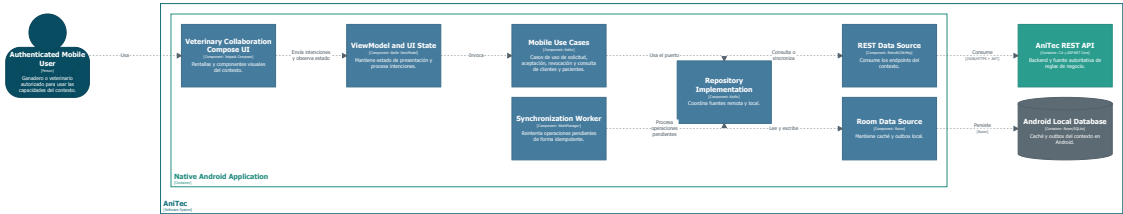
## 2.6.5.5. Bounded Context Software Architecture Component Level Diagrams

El archivo [component-level.dsl](#) contiene las vistas BC5-APIComponents, BC5-AndroidComponents y BC5-FlutterComponents. Las tres parten del mismo modelo C4 y muestran la separación entre presentación, aplicación, dominio y adaptadores.



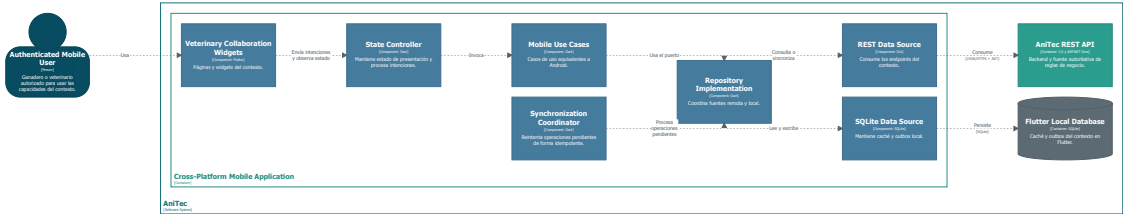
Component View: AniTec - AniTec REST API  
Componentes backend de Veterinary Collaboration.  
Tuesday, September 15, 2026 at 3:26 AM Coordinated Universal Time

Figura 2.6.5.1. Componentes de la API para Veterinary Collaboration. Fuente: elaboración propia con Structurizr DSL.



Component View: AniTec - Native Android Application  
Componentes Android de Veterinary Collaboration.  
Tuesday, September 15, 2026 at 3:26 AM Coordinated Universal Time

Figura 2.6.5.2. Componentes Android para Veterinary Collaboration. Fuente: elaboración propia con Structurizr DSL.



Component View: AniTec - Cross-Platform Mobile Application  
Componentes Flutter de Veterinary Collaboration.  
Tuesday, September 15, 2026 at 3:26 AM Coordinated Universal Time

Figura 2.6.5.3. Componentes Flutter para Veterinary Collaboration. Fuente: elaboración propia con Structurizr DSL.

## 2.6.5.6. Bounded Context Software Architecture Code Level Diagrams

Los diagramas de código detallan el modelo del dominio y los objetos de persistencia. El UML diferencia los elementos existentes de las incorporaciones objetivo, mientras los esquemas SQL señalan mediante comentarios las

columnas propuestas. Los archivos ERD quedan disponibles para completar la importación manual.

### 2.6.5.6.1. Bounded Context Domain Layer Class Diagrams

El Class Diagram incluye agregados, entidades, value objects, enumeraciones, servicios de dominio e interfaces de repositorio con atributos, operaciones, visibilidad y multiplicidades.

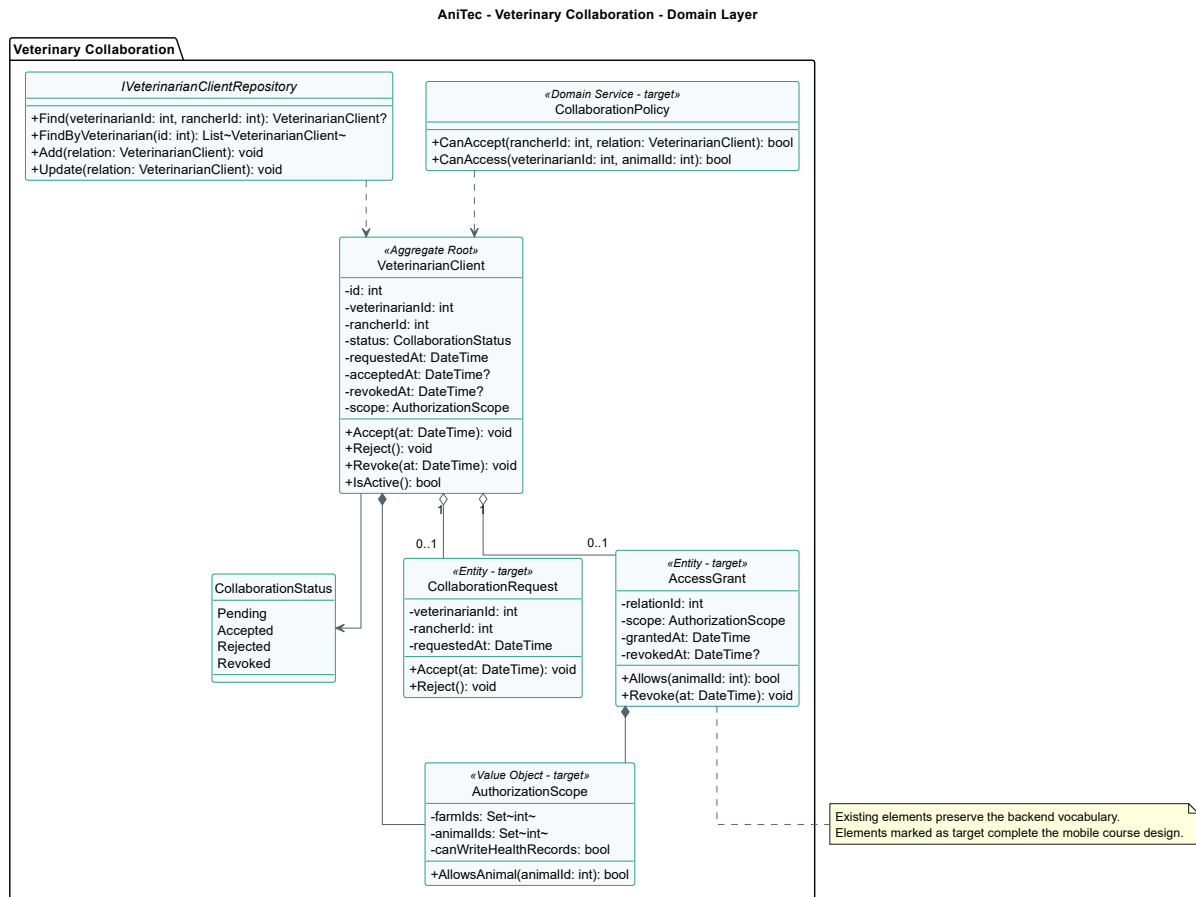


Figura 2.6.5.4. Domain Layer Class Diagram de Veterinary Collaboration. Fuente: elaboración propia con PlantUML.

### 2.6.5.6.2. Bounded Context Database Design Diagram

MySQL mantiene la persistencia autoritativa. Room y SQLite contienen únicamente caché, metadatos de sincronización y operaciones pendientes; no sustituyen las reglas ni la fuente de verdad del backend. En IAM, las credenciales y tokens permanecen fuera de las tablas locales y se almacenan mediante mecanismos seguros del sistema operativo.

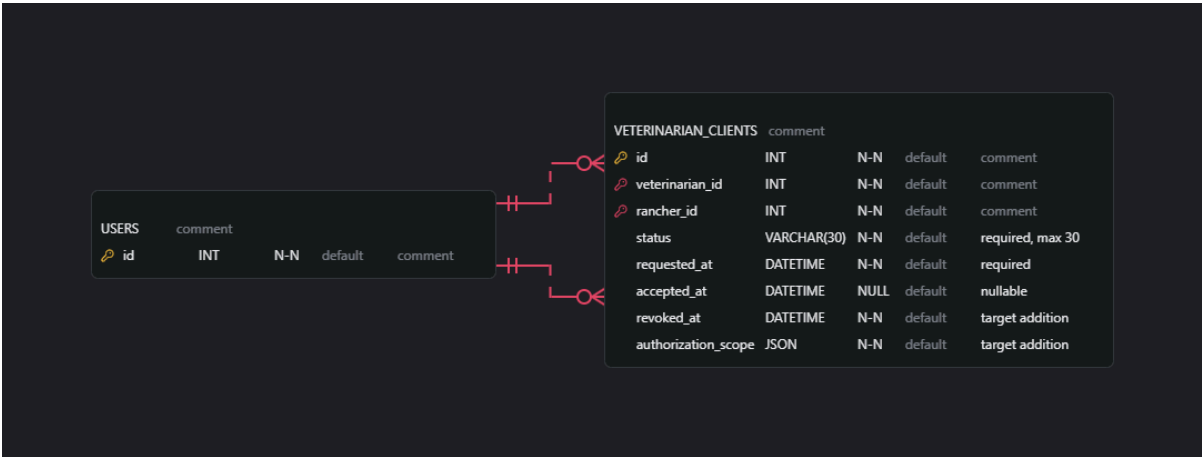


Figura 2.6.5.5. MySQL Database Design de Veterinary Collaboration. Fuente: elaboración propia a partir del esquema SQL.

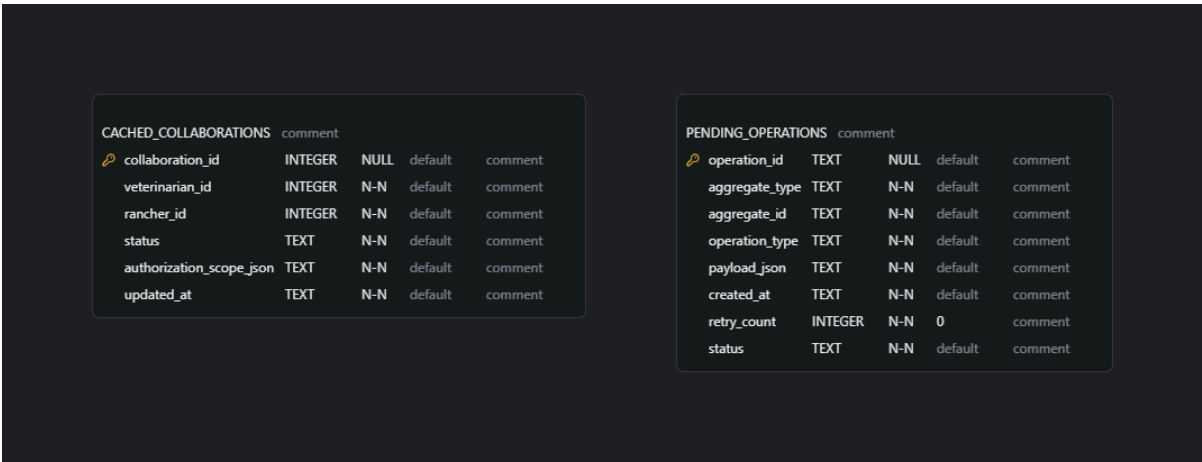


Figura 2.6.5.6. Android Room Database Design de Veterinary Collaboration. Fuente: elaboración propia a partir del esquema SQL.

## 2.6.6. Bounded Context: Activity Management

Planificar actividades ganaderas y sanitarias, controlar su estado y decidir cuándo corresponde generar un recordatorio.

La base implementada se encuentra en el módulo Activities de la API ASP.NET Core. El diseño móvil de Android y Flutter se presenta como **diseño objetivo** porque esos clientes todavía no existen en el workspace. Los tres productos comparten contratos REST y lenguaje ubicuo, mientras la API conserva las reglas autoritativas.

### 2.6.6.1. Domain Layer

Esta capa documenta el modelo que representa el núcleo de **Activity Management**. Las reglas autoritativas se ejecutan en la API; Android y Flutter mantienen modelos equivalentes para presentación, validación inmediata y trabajo offline. La columna de estado distingue el código heredado de la arquitectura objetivo.

| Clase        | Categoría         | Producto y estado                 | Propósito                                      | Atributos                | Métodos u operaciones                           | Relaciones                                                                                                          |
|--------------|-------------------|-----------------------------------|------------------------------------------------|--------------------------|-------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| FarmActivity | Aggregate<br>Root | Backend; modelo<br>equivalente en | Controlar el ciclo de<br>vida de una actividad | id: int<br>ownerId: int? | Reschedule(schedule:<br>ActivitySchedule); void | FarmActivity compone ActivitySchedule; FarmActivity se<br>relaciona con ActivityPriority; FarmActivity se relaciona |

| Clase                   | Categoría            | Producto y estado                                                                     | Propósito                                        | Atributos                                                                                                                                                     | Métodos u operaciones                                                                                                                                          | Relaciones                                                                                                  |
|-------------------------|----------------------|---------------------------------------------------------------------------------------|--------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
|                         |                      | Android y Flutter<br><b>Implementado en el backend</b>                                | programada.                                      | veterinarianId: int?<br>animalId: int?<br>title: string<br>type: string<br>schedule: ActivitySchedule<br>priority: ActivityPriority<br>status: ActivityStatus | Complete(): void<br>Cancel(): void                                                                                                                             | con ActivityStatus; ReminderPolicy depende de FarmActivity; IFarmActivityRepository depende de FarmActivity |
| ActivitySchedule        | Value Object         | Backend, Android y Flutter (modelo canónico)<br><b>Diseño objetivo</b>                | Representar fecha programada y recordatorio.     | scheduledAt: DateTime<br>reminderAt: DateTime?                                                                                                                | IsUpcoming(now: DateTime): bool                                                                                                                                | FarmActivity compone ActivitySchedule                                                                       |
| ActivityPriority        | Enumeration          | Backend, Android y Flutter (modelo canónico)<br><b>Diseño objetivo</b>                | Definir los valores válidos de ActivityPriority. | Low<br>Medium<br>High                                                                                                                                         | —                                                                                                                                                              | FarmActivity se relaciona con ActivityPriority                                                              |
| ActivityStatus          | Enumeration          | Backend, Android y Flutter (modelo canónico)<br><b>Diseño objetivo</b>                | Definir los valores válidos de ActivityStatus.   | Pending<br>InProgress<br>Completed<br>Cancelled                                                                                                               | —                                                                                                                                                              | FarmActivity se relaciona con ActivityStatus                                                                |
| ReminderPolicy          | Domain Service       | Backend, Android y Flutter (modelo canónico)<br><b>Diseño objetivo</b>                | Decidir cuándo debe emitirse una notificación.   | —                                                                                                                                                             | ShouldNotify(activity: FarmActivity, now: DateTime): bool                                                                                                      | ReminderPolicy depende de FarmActivity                                                                      |
| IFarmActivityRepository | Repository Interface | Backend; modelo equivalente en Android y Flutter<br><b>Implementado en el backend</b> | Abstraer la persistencia de FarmActivity.        | —                                                                                                                                                             | FindById(id: int): FarmActivity?<br>FindUpcoming(userId: int): List~FarmActivity~<br>Add(activity: FarmActivity): void<br>Update(activity: FarmActivity): void | IFarmActivityRepository depende de FarmActivity                                                             |

## 2.6.6.2. Interface Layer

Esta capa recibe las acciones relacionadas con **programación, reprogramación, finalización y recordatorio de actividades** y las traduce a casos de uso. Los controllers y resources corresponden a la API; las pantallas y controladores de estado representan la presentación objetivo en Android y Flutter. Ninguna de estas clases implementa reglas de negocio.

| Clase                      | Categoría          | Producto y estado                                         | Propósito                                                         | Atributos                                                                                                               | Métodos u operaciones                                                                                                                                                                                                                                                                                                                                | Relaciones                                                                                     |
|----------------------------|--------------------|-----------------------------------------------------------|-------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| FarmActivitiesController   | REST Controller    | Backend ASP.NET Core<br><b>Implementado en el backend</b> | Publicar por HTTP las capacidades de Activity Management.         | IFarmActivityCommandService<br>commandService<br>IFarmActivityQueryService<br>queryService                              | GetAll(CancellationTokens cancellationTokens)<br>GetById(int id, CancellationTokens cancellationTokens)<br>Create(CreateFarmActivityResource resource, CancellationTokens cancellationTokens)<br>Update(int id, CreateFarmActivityResource resource, CancellationTokens cancellationTokens)<br>Delete(int id, CancellationTokens cancellationTokens) | Recibe resources, invoca servicios de aplicación y devuelve resources HTTP.                    |
| CreateFarmActivityResource | Resource/Assembler | Backend ASP.NET Core<br><b>Implementado en el backend</b> | Definir un contrato estable de entrada o salida para la API REST. | int? OwnerId<br>int? VeterinarianId<br>string Title<br>string Type<br>DateOnly Date<br>string Priority<br>string Status | Create(...)<br>Deconstruct(...)                                                                                                                                                                                                                                                                                                                      | Es construido o traducido por assemblers y consumido por el controller y los clientes móviles. |

| Clase                  | Categoría          | Producto y estado                                            | Propósito                                                                                      | Atributos                                                                                                                         | Métodos u operaciones                   | Relaciones                                                                                     |
|------------------------|--------------------|--------------------------------------------------------------|------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------|------------------------------------------------------------------------------------------------|
| FarmActivityResource   | Resource/Assembler | Backend<br>ASP.NET Core<br><b>Implementado en el backend</b> | Definir un contrato estable de entrada o salida para la API REST.                              | int Id<br>int? OwnerId<br>int? VeterinarianId<br>string Title<br>string Type<br>DateOnly Date<br>string Priority<br>string Status | Create(...)<br>Deconstruct(...)         | Es construido o traducido por assemblers y consumido por el controller y los clientes móviles. |
| FarmActivityScreen     | Composable         | Android / Jetpack<br>Compose<br><b>Diseño objetivo</b>       | Presentar programación, reprogramación, finalización y recordatorio de actividades en Android. | uiState<br>onAction<br>navigation                                                                                                 | Render()<br>Submit()<br>Retry()         | Observa FarmActivityViewModel y emite acciones de interfaz.                                    |
| FarmActivityViewModel  | Presentation Model | Android / Kotlin<br><b>Diseño objetivo</b>                   | Mantener el estado observable y traducir acciones de Android a casos de uso.                   | state<br>observeUseCase<br>syncUseCase                                                                                            | Load()<br>Submit(action)<br>RetrySync() | Invoca casos de uso de Application Layer y publica un UI State inmutable.                      |
| FarmActivityPage       | Widget             | Flutter / Dart<br><b>Diseño objetivo</b>                     | Presentar programación, reprogramación, finalización y recordatorio de actividades en Flutter. | state<br>onAction<br>router                                                                                                       | build(context)<br>submit()<br>retry()   | Observa FarmActivityController y emite intenciones del usuario.                                |
| FarmActivityController | State Controller   | Flutter / Dart<br><b>Diseño objetivo</b>                     | Mantener el estado de presentación de Flutter y coordinar casos de uso.                        | state<br>observeUseCase<br>syncUseCase                                                                                            | load()<br>submit(action)<br>retrySync() | Invoca Application Layer y publica estados de carga, éxito y error.                            |

### 2.6.6.3. Application Layer

Esta capa coordina las capacidades de **programación, reprogramación, finalización y recordatorio de actividades**. Los commands y queries expresan intenciones; los handlers cargan aggregates, aplican reglas, persisten cambios y reaccionan a eventos. Los casos de uso móviles coordinan lectura local, actualización remota y sincronización idempotente.

| Clase                      | Categoría           | Producto y estado                                            | Propósito                                                                                                           | Atributos                                                                                                               | Métodos u operaciones                                                                                                                                                                                                 | Relaciones                                                                           |
|----------------------------|---------------------|--------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| FarmActivityCommandService | Application Service | Backend<br>ASP.NET Core<br><b>Implementado en el backend</b> | Orquestar programación, reprogramación, finalización y recordatorio de actividades sin contener reglas del dominio. | IFarmActivityRepository repository<br>IUnitOfWork<br>unitOfWork                                                         | Handle(CreateFarmActivityCommand command, Cancellation token)<br>Handle(UpdateFarmActivityCommand command, Cancellation token)<br>Handle(DeleteFarmActivityCommand command, Cancellation token)<br>cancellationToken) | Invoca agregados y repositories; confirma la transacción mediante Unit of Work.      |
| FarmActivityQueryService   | Application Service | Backend<br>ASP.NET Core<br><b>Implementado en el backend</b> | Orquestar programación, reprogramación, finalización y recordatorio de actividades sin contener reglas del dominio. | IFarmActivityRepository repository                                                                                      | Handle(GetFarmActivityByIdQuery query, Cancellation token)<br>Handle(GetAllFarmActivitiesQuery query, Cancellation token)<br>cancellationToken)                                                                       | Invoca agregados y repositories; confirma la transacción mediante Unit of Work.      |
| CreateFarmActivityCommand  | Command/Query       | Backend<br>ASP.NET Core<br><b>Implementado en el backend</b> | Transportar una intención o consulta tipada hacia su handler.                                                       | int? OwnerId<br>int? VeterinarianId<br>string Title<br>string Type<br>DateOnly Date<br>string Priority<br>string Status | —                                                                                                                                                                                                                     | Es recibida por un handler o servicio de aplicación y no contiene lógica de negocio. |

| Clase                              | Categoría       | Producto y estado                                            | Propósito                                                                                | Atributos                                                                                                                         | Métodos u operaciones             | Relaciones                                                                                           |
|------------------------------------|-----------------|--------------------------------------------------------------|------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|-----------------------------------|------------------------------------------------------------------------------------------------------|
| UpdateFarmActivityCommand          | Command/Query   | Backend<br>ASP.NET Core<br><b>Implementado en el backend</b> | Transportar una intención o consulta tipada hacia su handler.                            | int Id<br>int? OwnerId<br>int? VeterinarianId<br>string Title<br>string Type<br>DateOnly Date<br>string Priority<br>string Status | —                                 | Es recibida por un handler o servicio de aplicación y no contiene lógica de negocio.                 |
| DeleteFarmActivityCommand          | Command/Query   | Backend<br>ASP.NET Core<br><b>Implementado en el backend</b> | Transportar una intención o consulta tipada hacia su handler.                            | int Id                                                                                                                            | —                                 | Es recibida por un handler o servicio de aplicación y no contiene lógica de negocio.                 |
| GetFarmActivityByIdQuery           | Command/Query   | Backend<br>ASP.NET Core<br><b>Implementado en el backend</b> | Transportar una intención o consulta tipada hacia su handler.                            | int Id                                                                                                                            | —                                 | Es recibida por un handler o servicio de aplicación y no contiene lógica de negocio.                 |
| ObserveFarmActivityUseCase         | Use Case        | Android y Flutter<br><b>Diseño objetivo</b>                  | Entregar primero datos locales y actualizar la consulta cuando exista conectividad.      | localRepository<br>remoteRepository<br>connectivityMonitor                                                                        | Execute(criteria): Stream<Result> | Es invocado por ViewModel/Controller y coordina repositorios móviles.                                |
| SyncFarmActivityUseCase            | Use Case        | Android y Flutter<br><b>Diseño objetivo</b>                  | Procesar operaciones móviles pendientes de manera idempotente.                           | outboxRepository<br>remoteRepository<br>conflictResolver                                                                          | Execute(): SyncResult             | Lee el outbox local, consume la API y actualiza el estado de sincronización.                         |
| CreateFarmActivityCommandHandler   | Command Handler | Backend<br>ASP.NET Core<br><b>Diseño objetivo</b>            | Ejecutar una intención concreta, aplicar reglas del agregado y confirmar la transacción. | repository<br>unitOfWork<br>domainPolicy                                                                                          | Handle(command): Result           | Consume un Command, carga el aggregate mediante su repository y puede publicar un Domain Event.      |
| CompleteFarmActivityCommandHandler | Command Handler | Backend<br>ASP.NET Core<br><b>Diseño objetivo</b>            | Ejecutar una intención concreta, aplicar reglas del agregado y confirmar la transacción. | repository<br>unitOfWork<br>domainPolicy                                                                                          | Handle(command): Result           | Consume un Command, carga el aggregate mediante su repository y puede publicar un Domain Event.      |
| ActivityScheduledEventHandler      | Event Handler   | Backend<br>ASP.NET Core<br><b>Diseño objetivo</b>            | Reaccionar al evento confirmado y actualizar proyecciones o integraciones.               | projectionRepository<br>notificationPort<br>unitOfWork                                                                            | Handle(domainEvent): Task         | Consume un Domain Event y utiliza puertos de infraestructura sin modificar directamente el agregado. |

## 2.6.6.4. Infrastructure Layer

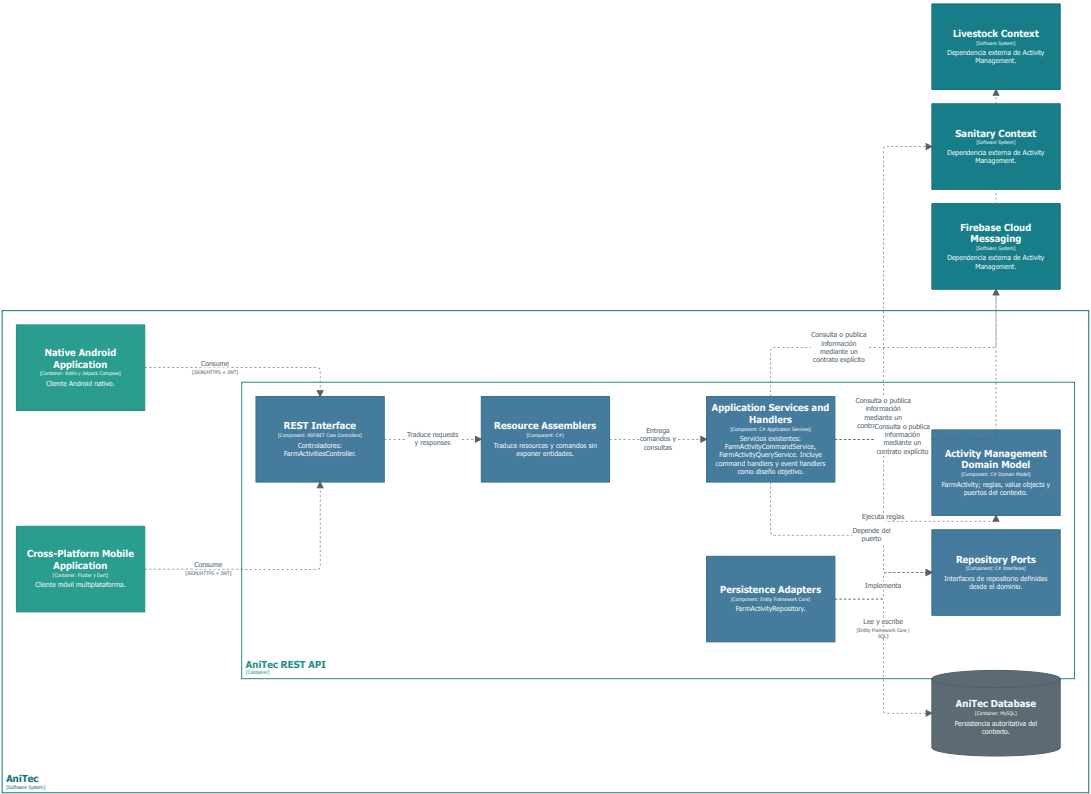
Esta capa implementa los puertos definidos hacia el interior de **Activity Management** y concentra acceso a base de datos, red, almacenamiento local e integraciones externas. Las clases de infraestructura traducen errores y contratos técnicos antes de devolver resultados a Application Layer.

| Clase                     | Categoría                 | Producto y estado                                                    | Propósito                                                           | Atributos                                 | Métodos u operaciones                                           | Relaciones                                                                                   |
|---------------------------|---------------------------|----------------------------------------------------------------------|---------------------------------------------------------------------|-------------------------------------------|-----------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| FarmActivityRepository    | Repository Adapter        | Backend / Entity Framework Core<br><b>Implementado en el backend</b> | Implementar el puerto de persistencia definido por Domain Layer.    | AppDbContext<br>context                   | FindById(id)<br>Add(entity)<br>Update(entity)<br>Delete(entity) | Implementa IFarmActivityRepository; utiliza AppDbContext/MySQL y reconstruye el aggregate.   |
| ModelBuilderExtensions    | Persistence Configuration | Backend / Entity Framework Core<br><b>Implementado en el backend</b> | Mapear entidades y value objects del contexto al modelo relacional. | EntityTypeBuilder<br>configuration        | ApplyConfiguration(modelBuilder)                                | Configura tablas, claves, relaciones, restricciones y conversiones de Entity Framework Core. |
| FarmActivityApiDataSource | Remote Adapter            | Android / Kotlin<br><b>Diseño objetivo</b>                           | Implementar el acceso remoto del cliente móvil a la API.            | httpClient<br>tokenProvider<br>serializer | Get(criteria)<br>Create(dto)<br>Update(dto)<br>Delete(id)       | Consume controllers REST por HTTPS/JSON y traduce errores HTTP al modelo de aplicación.      |

| Clase                        | Categoría         | Producto y estado                              | Propósito                                                         | Atributos                                 | Métodos u operaciones                                          | Relaciones                                                                               |
|------------------------------|-------------------|------------------------------------------------|-------------------------------------------------------------------|-------------------------------------------|----------------------------------------------------------------|------------------------------------------------------------------------------------------|
| FarmActivityDao              | Room Adapter      | Android / Room<br><b>Diseño objetivo</b>       | Implementar persistencia local y observación reactiva en Android. | roomDatabase<br>entityMapper              | Observe(criteria)<br>Upsert(entity)<br>Delete(id)<br>Pending() | Implementa el puerto local mediante Room y participa en la estrategia de caché/outbox.   |
| FarmActivityRemoteDataSource | Remote Adapter    | Flutter / Dart<br><b>Diseño objetivo</b>       | Implementar el acceso remoto del cliente móvil a la API.          | httpClient<br>tokenProvider<br>serializer | Get(criteria)<br>Create(dto)<br>Update(dto)<br>Delete(id)      | Consume controllers REST por HTTPS/JSON y traduce errores HTTP al modelo de aplicación.  |
| FarmActivityLocalDataSource  | SQLite Adapter    | Flutter / Dart<br><b>Diseño objetivo</b>       | Implementar persistencia local equivalente en Flutter.            | sqliteDatabase<br>entityMapper            | watch(criteria)<br>upsert(entity)<br>delete(id)<br>pending()   | Implementa el puerto local mediante SQLite y participa en la estrategia de caché/outbox. |
| LivestockReferenceAdapter    | Context Adapter   | Backend ASP.NET Core<br><b>Diseño objetivo</b> | Aislar una dependencia externa detrás de un puerto explícito.     | livestockFacade                           | ExistsAnimal(animalId): bool                                   | Implementa el puerto de referencia de animales.                                          |
| SanitaryReferenceAdapter     | Context Adapter   | Backend ASP.NET Core<br><b>Diseño objetivo</b> | Aislar una dependencia externa detrás de un puerto explícito.     | sanitaryFacade                            | GetFollowUpDate(eventId): Date?                                | Implementa el puerto de seguimiento sanitario.                                           |
| FirebaseReminderPublisher    | Messaging Adapter | Backend y Android<br><b>Diseño objetivo</b>    | Aislar una dependencia externa detrás de un puerto explícito.     | fcmClient,<br>localNotifier               | Publish(reminder)<br>ScheduleLocal(reminder)                   | Implementa el puerto de notificaciones mediante FCM y notificaciones locales.            |

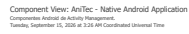
## 2.6.6.5. Bounded Context Software Architecture Component Level Diagrams

El archivo [component-level.dsl](#) contiene las vistas BC6-APIComponents, BC6-AndroidComponents y BC6-FlutterComponents. Las tres parten del mismo modelo C4 y muestran la separación entre presentación, aplicación, dominio y adaptadores.



Component View: AniTec - AniTec REST API  
Componentes backend de Activity Management.  
Tuesday, September 15, 2026 at 3:26 AM Coordinated Universal Time

Figura 2.6.6.1. Componentes de la API para Activity Management. Fuente: elaboración propia con Structurizr DSL.



Component View: AniTec - Cross-Platform Mobile Application  
Componentes Flutter de Activity Management.  
Tuesday, September 15, 2026 at 3:26 AM Coordinated Universal Time

154 / 190

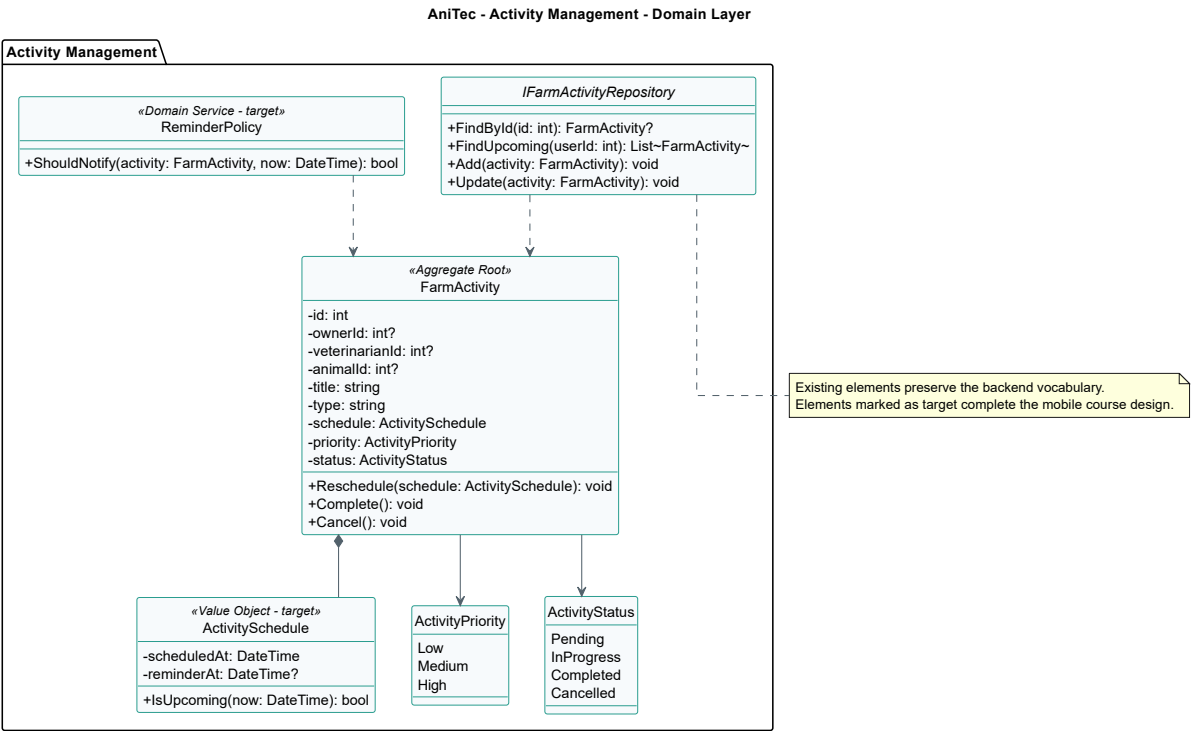


Figura 2.6.6.4. Domain Layer Class Diagram de Activity Management. Fuente: elaboración propia con PlantUML.

2.6.6.6.2. Bounded Context Database Design Diagram

MySQL mantiene la persistencia autoritativa. Room y SQLite contienen únicamente caché, metadatos de sincronización y operaciones pendientes; no sustituyen las reglas ni la fuente de verdad del backend. En IAM, las credenciales y tokens permanecen fuera de las tablas locales y se almacenan mediante mecanismos seguros del sistema operativo.



Figura 2.6.6.5. MySQL Database Design de Activity Management. Fuente: elaboración propia a partir del esquema SQL.

|                            |         |      |         |         |                           |         |      |         |         |
|----------------------------|---------|------|---------|---------|---------------------------|---------|------|---------|---------|
| PENDING_OPERATIONS comment |         |      |         |         | CACHED_ACTIVITIES comment |         |      |         |         |
| operation_id               | TEXT    | NULL | default | comment | activity_id               | INTEGER | NULL | default | comment |
| aggregate_type             | TEXT    | N-N  | default | comment | owner_id                  | INTEGER | N-N  | default | comment |
| aggregate_id               | TEXT    | N-N  | default | comment | veterinarian_id           | INTEGER | N-N  | default | comment |
| operation_type             | TEXT    | N-N  | default | comment | animal_id                 | INTEGER | N-N  | default | comment |
| payload_json               | TEXT    | N-N  | default | comment | title                     | TEXT    | N-N  | default | comment |
| created_at                 | TEXT    | N-N  | default | comment | scheduled_at              | TEXT    | N-N  | default | comment |
| retry_count                | INTEGER | N-N  | 0       | comment | reminder_at               | TEXT    | N-N  | default | comment |
| status                     | TEXT    | N-N  | default | comment | priority                  | TEXT    | N-N  | default | comment |
|                            |         |      |         |         | status                    | TEXT    | N-N  | default | comment |
|                            |         |      |         |         | updated_at                | TEXT    | N-N  | default | comment |

Figura 2.6.6.6. Android Room Database Design de Activity Management. Fuente: elaboración propia a partir del esquema SQL.

## 2.6.7. Bounded Context: Financial Management

Registrar ingresos y egresos operativos y producir resúmenes económicos básicos para apoyar decisiones del ganadero.

La base implementada se encuentra en el módulo Financiamiento de la API ASP.NET Core. El diseño móvil de Android y Flutter se presenta como **diseño objetivo** porque esos clientes todavía no existen en el workspace. Los tres productos comparten contratos REST y lenguaje ubicuo, mientras la API conserva las reglas autoritativas.

### 2.6.7.1. Domain Layer

Esta capa documenta el modelo que representa el núcleo de **Financiamiento**. Las reglas autoritativas se ejecutan en la API; Android y Flutter mantienen modelos equivalentes para presentación, validación inmediata y trabajo offline. La columna de estado distingue el código heredado de la arquitectura objetivo.

| Clase               | Categoría      | Producto y estado                                                                  | Propósito                                           | Atributos                                                                                                                                 | Métodos u operaciones                                                                  | Relaciones                                                                                                                                                                                                                         |
|---------------------|----------------|------------------------------------------------------------------------------------|-----------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| FinancialRecord     | Aggregate Root | Backend; modelo equivalente en Android y Flutter <b>Implementado en el backend</b> | Mantener un ingreso o egreso del ganadero.          | id: int<br>ownerId: int<br>type: FinancialRecordType<br>category: FinancialCategory<br>amount: Money<br>date: Date<br>description: string | ChangeAmount(amount: Money): void<br>ChangeCategory(category: FinancialCategory): void | FinancialRecord compone Money; FinancialRecord se relaciona con FinancialRecordType; FinancialRecord compone FinancialCategory; FinancialSummary depende de FinancialRecord; IFinancialRecordRepository depende de FinancialRecord |
| Money               | Value Object   | Backend, Android y Flutter (modelo canónico) <b>Diseño objetivo</b>                | Representar un importe junto con su moneda.         | amount: decimal<br>currency: string                                                                                                       | Add(other: Money): Money<br>IsPositive(): bool                                         | FinancialRecord compone Money                                                                                                                                                                                                      |
| FinancialRecordType | Enumeration    | Backend, Android y Flutter (modelo canónico) <b>Diseño objetivo</b>                | Definir los valores válidos de FinancialRecordType. | Income<br>Expense                                                                                                                         | —                                                                                      | FinancialRecord se relaciona con FinancialRecordType                                                                                                                                                                               |
| FinancialCategory   | Value Object   | Backend, Android y Flutter (modelo canónico) <b>Diseño objetivo</b>                | Clasificar un movimiento financiero.                | name: string                                                                                                                              | IsValid(): bool                                                                        | FinancialRecord compone FinancialCategory                                                                                                                                                                                          |

| Clase                      | Categoría            | Producto y estado                                                                     | Propósito                                                   | Atributos | Métodos u operaciones                                                                                                                                                  | Relaciones                                            |
|----------------------------|----------------------|---------------------------------------------------------------------------------------|-------------------------------------------------------------|-----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------|
| FinancialSummary           | Domain Service       | Backend, Android y Flutter (modelo canónico)<br><b>Diseño objetivo</b>                | Calcular totales y balance para un conjunto de movimientos. | —         | Calculate(records: List~FinancialRecord~): Money                                                                                                                       | FinancialSummary depende de FinancialRecord           |
| IFinancialRecordRepository | Repository Interface | Backend; modelo equivalente en Android y Flutter<br><b>Implementado en el backend</b> | Abstraer la persistencia de FinancialRecord.                | —         | FindById(id: int): FinancialRecord?<br>FindByOwner(ownerId: int): List~FinancialRecord~<br>Add(record: FinancialRecord): void<br>Update(record: FinancialRecord): void | IFinancialRecordRepository depende de FinancialRecord |

## 2.6.7.2. Interface Layer

Esta capa recibe las acciones relacionadas con **registro de ingresos y egresos, actualización y cálculo de resúmenes** y las traduce a casos de uso. Los controllers y resources corresponden a la API; las pantallas y controladores de estado representan la presentación objetivo en Android y Flutter. Ninguna de estas clases implementa reglas de negocio.

| Clase                         | Categoría          | Producto y estado                                         | Propósito                                                                                  | Atributos                                                                                                        | Métodos u operaciones                                                                                                                                                                                                                                       | Relaciones                                                                                     |
|-------------------------------|--------------------|-----------------------------------------------------------|--------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| FinancialRecordsController    | REST Controller    | Backend ASP.NET Core<br><b>Implementado en el backend</b> | Publicar por HTTP las capacidades de Financial Management.                                 | IFinancialRecordCommandService<br>IFinancialRecordQueryService                                                   | GetAll(Cancellation token)<br>GetById(int id, Cancellation token)<br>Create(CreateFinancialRecordResource resource, Cancellation token)<br>Update(int id, CreateFinancialRecordResource resource, Cancellation token)<br>Delete(int id, Cancellation token) | Recibe resources, invoca servicios de aplicación y devuelve resources HTTP.                    |
| CreateFinancialRecordResource | Resource/Assembler | Backend ASP.NET Core<br><b>Implementado en el backend</b> | Definir un contrato estable de entrada o salida para la API REST.                          | int OwnerId<br>string Type<br>string Category<br>decimal Amount<br>DateOnly Date<br>string Description           | Create(...)<br>Deconstruct(...)                                                                                                                                                                                                                             | Es construido o traducido por assemblers y consumido por el controller y los clientes móviles. |
| FinancialRecordResource       | Resource/Assembler | Backend ASP.NET Core<br><b>Implementado en el backend</b> | Definir un contrato estable de entrada o salida para la API REST.                          | int Id<br>int OwnerId<br>string Type<br>string Category<br>decimal Amount<br>DateOnly Date<br>string Description | Create(...)<br>Deconstruct(...)                                                                                                                                                                                                                             | Es construido o traducido por assemblers y consumido por el controller y los clientes móviles. |
| FinancialRecordScreen         | Composable         | Android / Jetpack Compose<br><b>Diseño objetivo</b>       | Presentar registro de ingresos y egresos, actualización y cálculo de resúmenes en Android. | uiState<br>onAction<br>navigation                                                                                | Render()<br>Submit()<br>Retry()                                                                                                                                                                                                                             | Observa FinancialRecordViewModel y emite acciones de interfaz.                                 |
| FinancialRecordViewModel      | Presentation Model | Android / Kotlin<br><b>Diseño objetivo</b>                | Mantener el estado observable y traducir acciones de Android a casos de uso.               | state<br>observeUseCase<br>syncUseCase                                                                           | Load()<br>Submit(action)<br>RetrySync()                                                                                                                                                                                                                     | Invoca casos de uso de Application Layer y publica un UI State inmutable.                      |

| Clase                     | Categoría        | Producto y estado                                  | Propósito                                                                                  | Atributos                              | Métodos u operaciones                   | Relaciones                                                          |
|---------------------------|------------------|----------------------------------------------------|--------------------------------------------------------------------------------------------|----------------------------------------|-----------------------------------------|---------------------------------------------------------------------|
| FinancialRecordPage       | Widget           | Flutter / Dart<br><b>Diseño</b><br><b>objetivo</b> | Presentar registro de ingresos y egresos, actualización y cálculo de resúmenes en Flutter. | state<br>onAction<br>router            | build(context)<br>submit()<br>retry()   | Observa FinancialRecordController y emite intenciones del usuario.  |
| FinancialRecordController | State Controller | Flutter / Dart<br><b>Diseño</b><br><b>objetivo</b> | Mantener el estado de presentación de Flutter y coordinar casos de uso.                    | state<br>observeUseCase<br>syncUseCase | load()<br>submit(action)<br>retrySync() | Invoca Application Layer y publica estados de carga, éxito y error. |

### 2.6.7.3. Application Layer

Esta capa coordina las capacidades de **registro de ingresos y egresos, actualización y cálculo de resúmenes**. Los commands y queries expresan intenciones; los handlers cargan aggregates, aplican reglas, persisten cambios y reaccionan a eventos. Los casos de uso móviles coordinan lectura local, actualización remota y sincronización idempotente.

| Clase                         | Categoría           | Producto y estado                                            | Propósito                                                                                                       | Atributos                                                                                                        | Métodos u operaciones                                                                                                                                                                                                                                       | Relaciones                                                                           |
|-------------------------------|---------------------|--------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| FinancialRecordCommandService | Application Service | Backend<br>ASP.NET Core<br><b>Implementado en el backend</b> | Orquestar registro de ingresos y egresos, actualización y cálculo de resúmenes sin contener reglas del dominio. | IFinancialRecordRepository repository<br>IUnitOfWork unitOfWork                                                  | Handle(CreateFinancialRecordCommand command, CancellationToken cancellationToken)<br>Handle(UpdateFinancialRecordCommand command, CancellationToken cancellationToken)<br>Handle(DeleteFinancialRecordCommand command, CancellationToken cancellationToken) | Invoca agregados y repositories; confirma la transacción mediante Unit of Work.      |
| FinancialRecordQueryService   | Application Service | Backend<br>ASP.NET Core<br><b>Implementado en el backend</b> | Orquestar registro de ingresos y egresos, actualización y cálculo de resúmenes sin contener reglas del dominio. | IFinancialRecordRepository repository                                                                            | Handle(GetFinancialRecordByIdQuery query, CancellationToken cancellationToken)<br>Handle(GetAllFinancialRecordsQuery query, CancellationToken cancellationToken)                                                                                            | Invoca agregados y repositories; confirma la transacción mediante Unit of Work.      |
| CreateFinancialRecordCommand  | Command/Query       | Backend<br>ASP.NET Core<br><b>Implementado en el backend</b> | Transportar una intención o consulta tipada hacia su handler.                                                   | int OwnerId<br>string Type<br>string Category<br>decimal Amount<br>DateOnly Date<br>string Description           | —                                                                                                                                                                                                                                                           | Es recibida por un handler o servicio de aplicación y no contiene lógica de negocio. |
| UpdateFinancialRecordCommand  | Command/Query       | Backend<br>ASP.NET Core<br><b>Implementado en el backend</b> | Transportar una intención o consulta tipada hacia su handler.                                                   | int Id<br>int OwnerId<br>string Type<br>string Category<br>decimal Amount<br>DateOnly Date<br>string Description | —                                                                                                                                                                                                                                                           | Es recibida por un handler o servicio de aplicación y no contiene lógica de negocio. |
| DeleteFinancialRecordCommand  | Command/Query       | Backend<br>ASP.NET Core<br><b>Implementado en el backend</b> | Transportar una intención o consulta tipada hacia su handler.                                                   | int Id                                                                                                           | —                                                                                                                                                                                                                                                           | Es recibida por un handler o servicio de aplicación y no contiene lógica de negocio. |

| Clase                                 | Categoría       | Producto y estado                                            | Propósito                                                                                | Atributos                                                  | Métodos u operaciones             | Relaciones                                                                                            |
|---------------------------------------|-----------------|--------------------------------------------------------------|------------------------------------------------------------------------------------------|------------------------------------------------------------|-----------------------------------|-------------------------------------------------------------------------------------------------------|
| GetFinancialRecordByIdQuery           | Command/Query   | Backend<br>ASP.NET Core<br><b>Implementado en el backend</b> | Transportar una intención o consulta tipada hacia su handler.                            | int Id                                                     | —                                 | Es recibida por un handler o servicio de aplicación y no contiene lógica de negocio.                  |
| ObserveFinancialRecordUseCase         | Use Case        | Android y Flutter<br><b>Diseño objetivo</b>                  | Entregar primero datos locales y actualizar la consulta cuando exista conectividad.      | localRepository<br>remoteRepository<br>connectivityMonitor | Execute(criteria): Stream<Result> | Es invocado por ViewModel/Controller y coordina repositorios móviles.                                 |
| SyncFinancialRecordUseCase            | Use Case        | Android y Flutter<br><b>Diseño objetivo</b>                  | Procesar operaciones móviles pendientes de manera idempotente.                           | outboxRepository<br>remoteRepository<br>conflictResolver   | Execute(): SyncResult             | Lee el outbox local, consume la API y actualiza el estado de sincronización.                          |
| CreateFinancialRecordCommandHandler   | Command Handler | Backend<br>ASP.NET Core<br><b>Diseño objetivo</b>            | Ejecutar una intención concreta, aplicar reglas del agregado y confirmar la transacción. | repository<br>unitOfWork<br>domainPolicy                   | Handle(command): Result           | Consumes un Command, carga el aggregate mediante su repository y puede publicar un Domain Event.      |
| UpdateFinancialRecordCommandHandler   | Command Handler | Backend<br>ASP.NET Core<br><b>Diseño objetivo</b>            | Ejecutar una intención concreta, aplicar reglas del agregado y confirmar la transacción. | repository<br>unitOfWork<br>domainPolicy                   | Handle(command): Result           | Consumes un Command, carga el aggregate mediante su repository y puede publicar un Domain Event.      |
| FinancialRecordRegisteredEventHandler | Event Handler   | Backend<br>ASP.NET Core<br><b>Diseño objetivo</b>            | Reaccionar al evento confirmado y actualizar proyecciones o integraciones.               | projectionRepository<br>notificationPort<br>unitOfWork     | Handle(domainEvent): Task         | Consumes un Domain Event y utiliza puertos de infraestructura sin modificar directamente el agregado. |

## 2.6.7.4. Infrastructure Layer

Esta capa implementa los puertos definidos hacia el interior de **Financial Management** y concentra acceso a base de datos, red, almacenamiento local e integraciones externas. Las clases de infraestructura traducen errores y contratos técnicos antes de devolver resultados a Application Layer.

| Clase                        | Categoría                 | Producto y estado                                                    | Propósito                                                           | Atributos                                 | Métodos u operaciones                                           | Relaciones                                                                                    |
|------------------------------|---------------------------|----------------------------------------------------------------------|---------------------------------------------------------------------|-------------------------------------------|-----------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| FinancialRecordRepository    | Repository Adapter        | Backend / Entity Framework Core<br><b>Implementado en el backend</b> | Implementar el puerto de persistencia definido por Domain Layer.    | AppDbContext<br>context                   | FindById(id)<br>Add(entity)<br>Update(entity)<br>Delete(entity) | Implementa IFinancialRecordRepository; utiliza AppDbContext/MySQL y reconstruye el aggregate. |
| ModelBuilderExtensions       | Persistence Configuration | Backend / Entity Framework Core<br><b>Implementado en el backend</b> | Mapear entidades y value objects del contexto al modelo relacional. | EntityTypeBuilder<br>configuration        | ApplyConfiguration(modelBuilder)                                | Configura tablas, claves, relaciones, restricciones y conversiones de Entity Framework Core.  |
| FinancialRecordApiDataSource | Remote Adapter            | Android / Kotlin<br><b>Diseño objetivo</b>                           | Implementar el acceso remoto del cliente móvil a la API.            | httpClient<br>tokenProvider<br>serializer | Get(criteria)<br>Create(dto)<br>Update(dto)<br>Delete(id)       | Consumes controllers REST por HTTPS/JSON y traduce errores HTTP al modelo de aplicación.      |
| FinancialRecordDao           | Room Adapter              | Android / Room<br><b>Diseño objetivo</b>                             | Implementar persistencia local y observación reactiva en Android.   | roomDatabase<br>entityMapper              | Observe(criteria)<br>Upsert(entity)<br>Delete(id)<br>Pending()  | Implementa el puerto local mediante Room y participa en la estrategia de caché/outbox.        |

| Clase                           | Categoría       | Producto y estado                              | Propósito                                                     | Atributos                                 | Métodos u operaciones                                        | Relaciones                                                                               |
|---------------------------------|-----------------|------------------------------------------------|---------------------------------------------------------------|-------------------------------------------|--------------------------------------------------------------|------------------------------------------------------------------------------------------|
| FinancialRecordRemoteDataSource | Remote Adapter  | Flutter / Dart<br><b>Diseño objetivo</b>       | Implementar el acceso remoto del cliente móvil a la API.      | httpClient<br>tokenProvider<br>serializer | Get(criteria)<br>Create(dto)<br>Update(dto)<br>Delete(id)    | Consume controllers REST por HTTPS/JSON y traduce errores HTTP al modelo de aplicación.  |
| FinancialRecordLocalDataSource  | SQLite Adapter  | Flutter / Dart<br><b>Diseño objetivo</b>       | Implementar persistencia local equivalente en Flutter.        | sqliteDatabase<br>entityMapper            | watch(criteria)<br>upsert(entity)<br>delete(id)<br>pending() | Implementa el puerto local mediante SQLite y participa en la estrategia de caché/outbox. |
| ProfilesFinancialOwnerAdapter   | Context Adapter | Backend ASP.NET Core<br><b>Diseño objetivo</b> | Aislar una dependencia externa detrás de un puerto explícito. | profilesFacade                            | ValidateOwner(ownerId): bool                                 | Implementa el puerto de propietarios y consume Profile Management.                       |
| FinancialOutboxStore            | Offline Adapter | Android y Flutter<br><b>Diseño objetivo</b>    | Aislar una dependencia externa detrás de un puerto explícito. | database, serializer                      | Enqueue(record)<br>Pending()<br>MarkSynced(id)               | Implementa el puerto de sincronización financiera sobre Room o SQLite.                   |

2.6.7.5. Bounded Context Software Architecture Component Level Diagrams

El archivo [component-level.dsl](#) contiene las vistas BC7-ApiComponents, BC7-AndroidComponents y BC7-FlutterComponents. Las tres parten del mismo modelo C4 y muestran la separación entre presentación, aplicación, dominio y adaptadores.

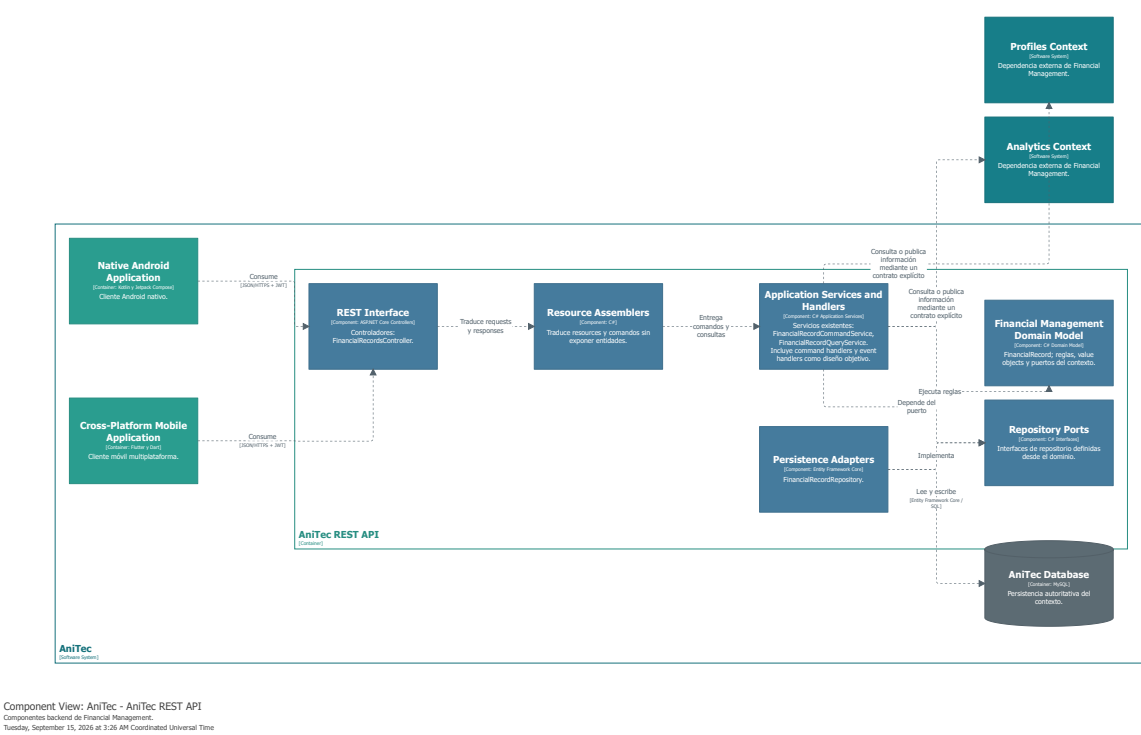


Figura 2.6.7.1. Componentes de la API para Financial Management. Fuente: elaboración propia con Structurizr DSL.

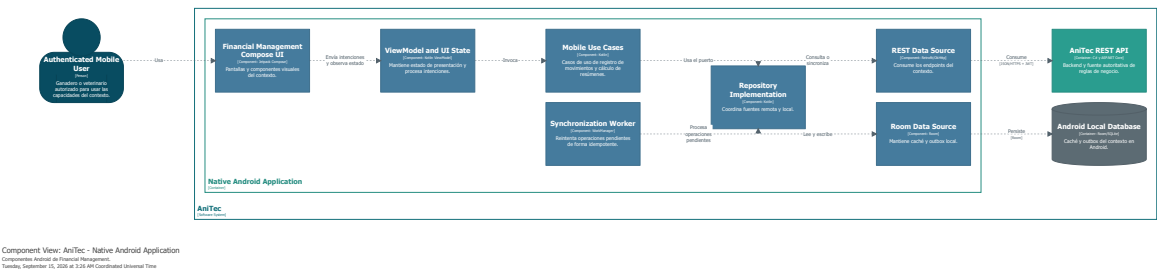


Figura 2.6.7.2. Componentes Android para Financial Management. Fuente: elaboración propia con Structurizr DSL.

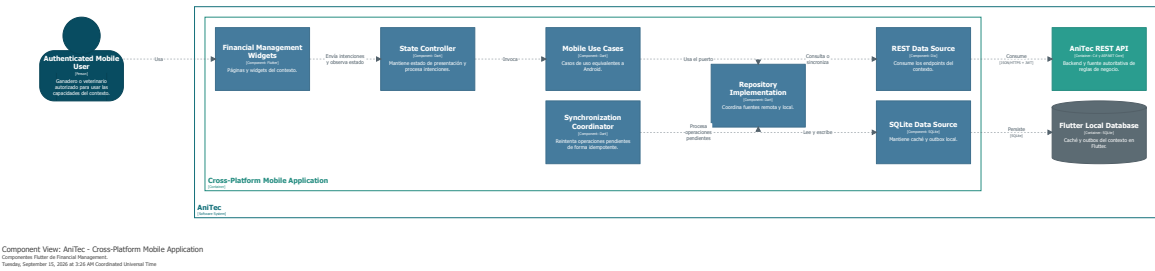


Figura 2.6.7.3. Componentes Flutter para Financial Management. Fuente: elaboración propia con Structurizr DSL.

2.6.7.6. Bounded Context Software Architecture Code Level Diagrams

Los diagramas de código detallan el modelo del dominio y los objetos de persistencia. El UML diferencia los elementos existentes de las incorporaciones objetivo, mientras los esquemas SQL señalan mediante comentarios las columnas propuestas. Los archivos ERD quedan disponibles para completar la importación manual.

2.6.7.6.1. Bounded Context Domain Layer Class Diagrams

El Class Diagram incluye agregados, entidades, value objects, enumeraciones, servicios de dominio e interfaces de repositorio con atributos, operaciones, visibilidad y multiplicidades.

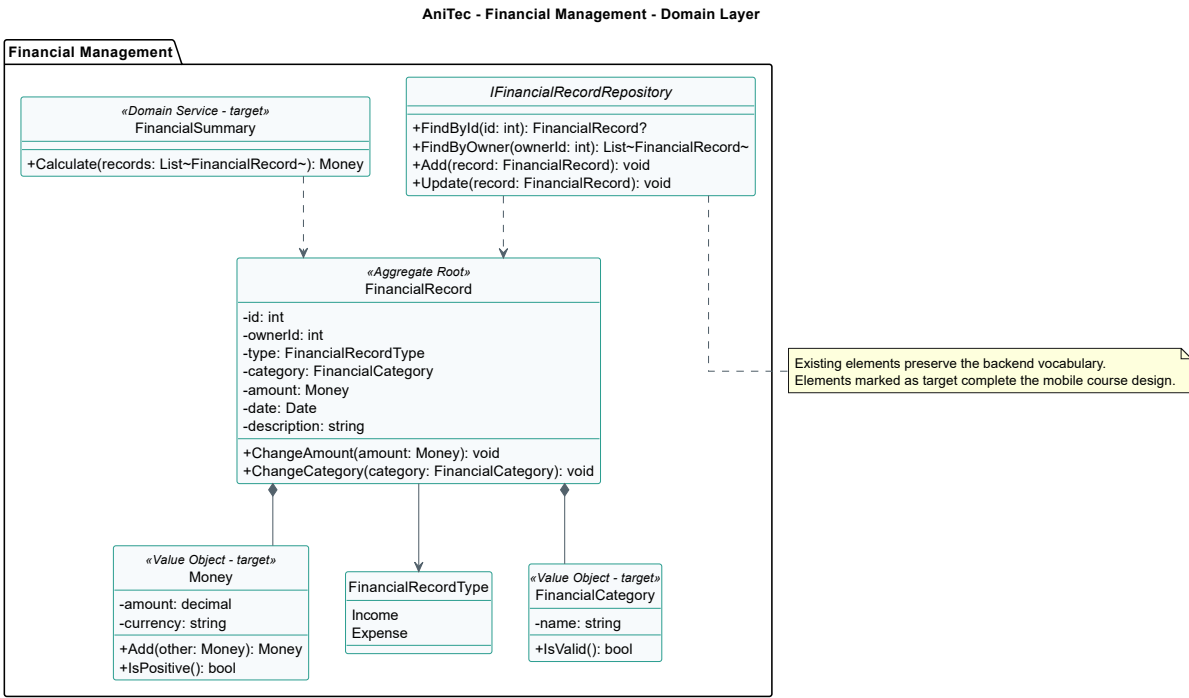


Figura 2.6.7.4. Domain Layer Class Diagram de Financial Management. Fuente: elaboración propia con PlantUML.

2.6.7.6.2. Bounded Context Database Design Diagram

MySQL mantiene la persistencia autoritativa. Room y SQLite contienen únicamente caché, metadatos de sincronización y operaciones pendientes; no sustituyen las reglas ni la fuente de verdad del backend. En IAM, las

credenciales y tokens permanecen fuera de las tablas locales y se almacenan mediante mecanismos seguros del sistema operativo.

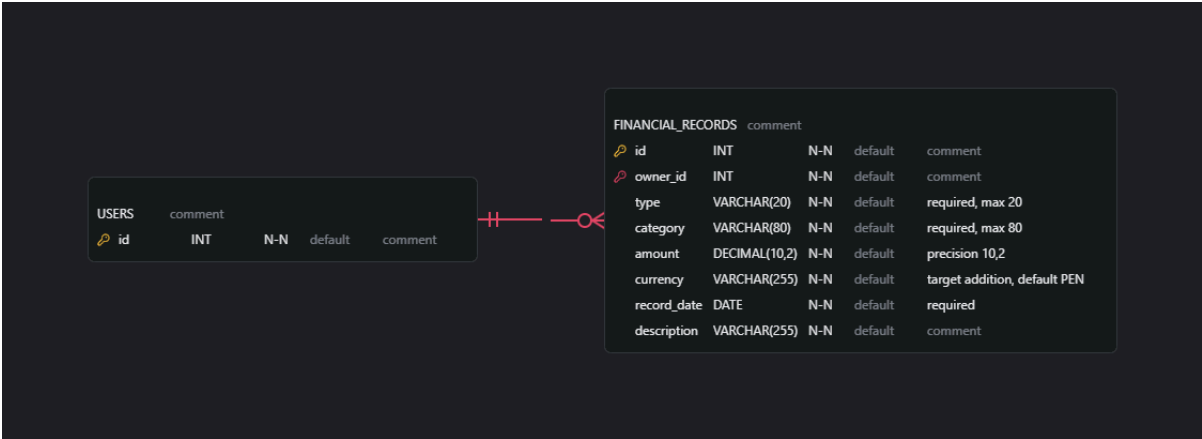


Figura 2.6.7.5. MySQL Database Design de Financial Management. Fuente: elaboración propia a partir del esquema SQL.

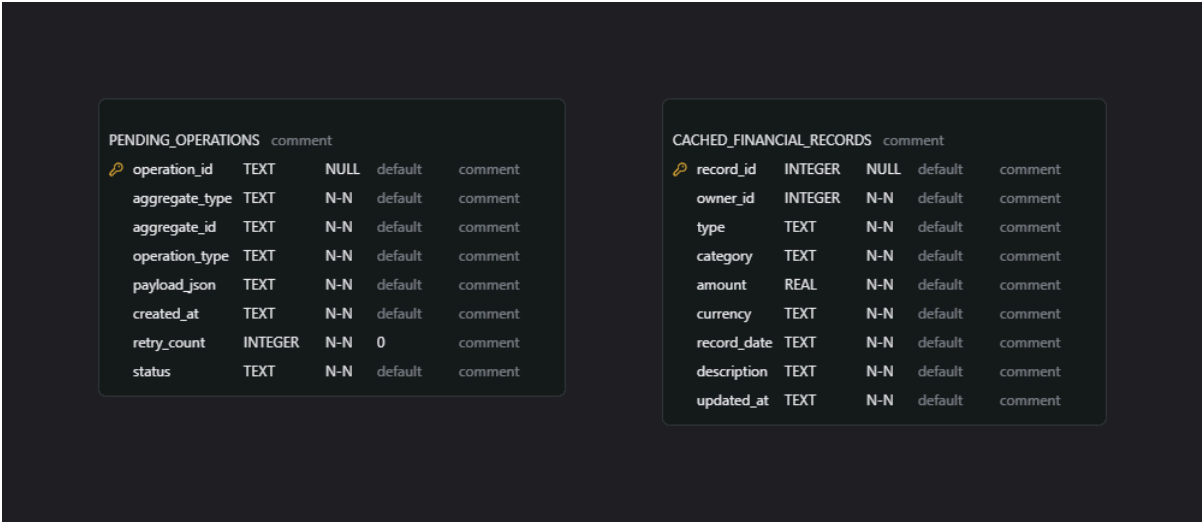


Figura 2.6.7.6. Android Room Database Design de Financial Management. Fuente: elaboración propia a partir del esquema SQL.

## 2.6.8. Bounded Context: Subscription Management

Administrar planes, vigencia de suscripciones y pagos confirmados sin introducir conceptos propios de Stripe en el dominio.

La base implementada se encuentra en el módulo Subscriptions de la API ASP.NET Core. El diseño móvil de Android y Flutter se presenta como **diseño objetivo** porque esos clientes todavía no existen en el workspace. Los tres productos comparten contratos REST y lenguaje ubicuo, mientras la API conserva las reglas autoritativas.

### 2.6.8.1. Domain Layer

Esta capa documenta el modelo que representa el núcleo de **Subscription Management**. Las reglas autoritativas se ejecutan en la API; Android y Flutter mantienen modelos equivalentes para presentación, validación inmediata y trabajo offline. La columna de estado distingue el código heredado de la arquitectura objetivo.

| Clase                       | Categoría            | Producto y estado                                                                     | Propósito                                                    | Atributos                                                                                                                                | Métodos u operaciones                                                                                                                                                 | Relaciones                                                                                                                            |
|-----------------------------|----------------------|---------------------------------------------------------------------------------------|--------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| SubscriptionPlan            | Entity               | Backend; modelo equivalente en Android y Flutter<br><b>Implementado en el backend</b> | Definir condiciones y límites de un plan comercial.          | id: int<br>name: string<br>price: Money<br>providerPriceld: string<br>maxAnimals: int<br>isActive: bool                                  | Deactivate(): void<br>ChangePrice(price: Money): void                                                                                                                 | SubscriptionPlan compone Money                                                                                                        |
| Subscription                | Aggregate Root       | Backend; modelo equivalente en Android y Flutter<br><b>Implementado en el backend</b> | Controlar vigencia y estado de la suscripción de un usuario. | id: int<br>userId: int<br>planId: int<br>status: SubscriptionStatus<br>startedAt: Date<br>endsAt: Date?                                  | Activate(start: Date, end: Date?): void<br>Cancel(end: Date): void<br>IsActive(on: Date): bool                                                                        | Subscription compone Payment;<br>Subscription se relaciona con SubscriptionStatus;<br>ISubscriptionRepository depende de Subscription |
| Payment                     | Entity               | Backend; modelo equivalente en Android y Flutter<br><b>Implementado en el backend</b> | Registrar el resultado de un pago.                           | id: int<br>subscriptionId: int<br>userId: int<br>amount: Money<br>providerPaymentId: string<br>status: PaymentStatus<br>paidAt: DateTime | Confirm(providerId: string, paidAt: DateTime): void<br>Reject(): void                                                                                                 | Subscription compone Payment;<br>Payment se relaciona con PaymentStatus; Payment compone Money                                        |
| Money                       | Value Object         | Backend, Android y Flutter (modelo canónico)<br><b>Diseño objetivo</b>                | Representar un importe junto con su moneda.                  | amount: decimal<br>currency: string                                                                                                      | IsPositive(): bool                                                                                                                                                    | Payment compone Money;<br>SubscriptionPlan compone Money                                                                              |
| SubscriptionStatus          | Enumeration          | Backend, Android y Flutter (modelo canónico)<br><b>Diseño objetivo</b>                | Definir los valores válidos de SubscriptionStatus.           | Pending<br>Active<br>Cancelled<br>Expired                                                                                                | —                                                                                                                                                                     | Subscription se relaciona con SubscriptionStatus                                                                                      |
| PaymentStatus               | Enumeration          | Backend, Android y Flutter (modelo canónico)<br><b>Diseño objetivo</b>                | Definir los valores válidos de PaymentStatus.                | Pending<br>Paid<br>Failed                                                                                                                | —                                                                                                                                                                     | Payment se relaciona con PaymentStatus                                                                                                |
| ISubscriptionRepository     | Repository Interface | Backend; modelo equivalente en Android y Flutter<br><b>Implementado en el backend</b> | Abstraer la persistencia de Subscription.                    | —                                                                                                                                        | FindById(id: int): Subscription?<br>FindActiveByUser(userId: int): Subscription?<br>Add(subscription: Subscription): void<br>Update(subscription: Subscription): void | ISubscriptionRepository depende de Subscription                                                                                       |
| ISubscriptionPlanRepository | Repository Interface | Backend; modelo equivalente en Android y Flutter<br><b>Implementado en el backend</b> | Abstraer la persistencia de planes de suscripción.           | —                                                                                                                                        | FindByAsync(id)<br>ListAsync()<br>AddAsync(plan)<br>Update(plan)                                                                                                      | Persiste SubscriptionPlan y es implementado por SubscriptionPlanRepository.                                                           |
| IPaymentRepository          | Repository Interface | Backend; modelo equivalente en Android y Flutter<br><b>Implementado en el backend</b> | Abstraer la persistencia y consulta de pagos.                | —                                                                                                                                        | FindByUserIdAsync(userId)<br>FindByProviderPaymentIdAsync(providerPaymentId)<br>AddAsync(payment)                                                                     | Persiste Payment y es implementado por PaymentRepository.                                                                             |
| PaymentGateway              | Domain Port          | Backend; modelo equivalente en Android y Flutter<br><b>Diseño objetivo</b>            | Abstraer el proveedor externo de cobros.                     | —                                                                                                                                        | CreateCheckout(subscription, plan)<br>ConfirmPayment(reference)                                                                                                       | Es implementado por StripePaymentGateway en Infrastructure Layer.                                                                     |

## 2.6.8.2. Interface Layer

Esta capa recibe las acciones relacionadas con **consulta de planes, contratación, pago y actualización de suscripciones** y las traduce a casos de uso. Los controllers y resources corresponden a la API; las pantallas y controladores de estado representan la presentación objetivo en Android y Flutter. Ninguna de estas clases implementa reglas de negocio.

| Clase                       | Categoría          | Producto y estado                                            | Propósito                                                                                     | Atributos                                                                                                                                                                                                                                                                         | Métodos u operaciones                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | Relaciones                                                                                     |
|-----------------------------|--------------------|--------------------------------------------------------------|-----------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| SubscriptionPlansController | REST Controller    | Backend<br>ASP.NET Core<br><b>Implementado en el backend</b> | Publicar por HTTP las capacidades de Subscription Management.                                 | ISubscriptionPlanCommandService<br>commandService<br>ISubscriptionPlanQueryService<br>queryService                                                                                                                                                                                | GetAll(CancellationToken cancellationToken)<br>GetById(int id, CancellationToken cancellationToken)<br>Create(CreateSubscriptionPlanResource resource, CancellationToken cancellationToken)<br>Update(int id, CreateSubscriptionPlanResource resource, CancellationToken cancellationToken)<br>Delete(int id, CancellationToken cancellationToken)                                                                                                                                                                                                                                                                                                                                                                                                     | Recibe recursos, invoca servicios de aplicación y devuelve recursos HTTP.                      |
| SubscriptionsController     | REST Controller    | Backend<br>ASP.NET Core<br><b>Implementado en el backend</b> | Publicar por HTTP las capacidades de Subscription Management.                                 | ISubscriptionCommandService<br>commandService<br>ISubscriptionQueryService<br>queryService<br>ISubscriptionPlanQueryService<br>planQueryService<br>IPaymentCommandService<br>paymentCommandService<br>IPaymentQueryService<br>paymentQueryService<br>IConfiguration configuration | GetAll(CancellationToken cancellationToken)<br>GetById(int id, CancellationToken cancellationToken)<br>Create(CreateSubscriptionResource resource, CancellationToken cancellationToken)<br>GetActiveByUser(int userId, CancellationToken cancellationToken)<br>GetPaymentsByUser(int userId, CancellationToken cancellationToken)<br>MockCheckout(MockCheckoutResource resource, CancellationToken cancellationToken)<br>CreateStripeCheckout(StripeCheckoutResource resource, CancellationToken cancellationToken)<br>ConfirmStripeCheckout(string sessionId, CancellationToken cancellationToken)<br>Update(int id, CreateSubscriptionResource resource, CancellationToken cancellationToken)<br>Delete(int id, CancellationToken cancellationToken) | Recibe recursos, invoca servicios de aplicación y devuelve recursos HTTP.                      |
| SubscriptionPlanResource    | Resource/Assembler | Backend<br>ASP.NET Core<br><b>Implementado en el backend</b> | Definir un contrato estable de entrada o salida para la API REST.                             | int Id<br>string Name<br>decimal Price<br>string StripePriceld<br>int MaxAnimals<br>bool IsActive                                                                                                                                                                                 | Create(...)<br>Deconstruct(...)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | Es construido o traducido por assemblers y consumido por el controller y los clientes móviles. |
| SubscriptionResource        | Resource/Assembler | Backend<br>ASP.NET Core<br><b>Implementado en el backend</b> | Definir un contrato estable de entrada o salida para la API REST.                             | int Id<br>int UserId<br>int PlanId<br>string StripeCustomerId<br>string StripeSubscriptionId<br>string Status<br>DateOnly StartedAt<br>DateOnly? EndsAt                                                                                                                           | Create(...)<br>Deconstruct(...)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | Es construido o traducido por assemblers y consumido por el controller y los clientes móviles. |
| PaymentResource             | Resource/Assembler | Backend<br>ASP.NET Core<br><b>Implementado en el backend</b> | Definir un contrato estable de entrada o salida para la API REST.                             | int Id<br>int UserId<br>int SubscriptionId<br>decimal Amount<br>string Currency<br>string Provider<br>string ProviderPaymentId<br>string Status<br>DateTime PaidAt                                                                                                                | Create(...)<br>Deconstruct(...)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | Es construido o traducido por assemblers y consumido por el controller y los clientes móviles. |
| StripeCheckoutResource      | Resource/Assembler | Backend<br>ASP.NET Core<br><b>Implementado en el backend</b> | Definir un contrato estable de entrada o salida para la API REST.                             | int PlanId                                                                                                                                                                                                                                                                        | Create(...)<br>Deconstruct(...)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | Es construido o traducido por assemblers y consumido por el controller y los clientes móviles. |
| SubscriptionScreen          | Composable         | Android /<br>Jetpack<br>Compose<br><b>Diseño objetivo</b>    | Presentar consulta de planes, contratación, pago y actualización de suscripciones en Android. | uiState<br>onAction<br>navigation                                                                                                                                                                                                                                                 | Render()<br>Submit()<br>Retry()                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | Observa SubscriptionViewModel y emite acciones de interfaz.                                    |
| SubscriptionViewModel       | Presentation Model | Android /<br>Kotlin<br><b>Diseño objetivo</b>                | Mantener el estado observable y traducir acciones de                                          | state<br>observeUseCase<br>syncUseCase                                                                                                                                                                                                                                            | Load()<br>Submit(action)<br>RetrySync()                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                | Invoca casos de uso de Application Layer y publica un UI State inmutable.                      |

| Clase                  | Categoría        | Producto y estado                        | Propósito                                                                                     | Atributos                              | Métodos u operaciones                   | Relaciones                                                          |
|------------------------|------------------|------------------------------------------|-----------------------------------------------------------------------------------------------|----------------------------------------|-----------------------------------------|---------------------------------------------------------------------|
|                        |                  |                                          | Android a casos de uso.                                                                       |                                        |                                         |                                                                     |
| SubscriptionPage       | Widget           | Flutter / Dart<br><b>Diseño objetivo</b> | Presentar consulta de planes, contratación, pago y actualización de suscripciones en Flutter. | state<br>onAction<br>router            | build(context)<br>submit()<br>retry()   | Observa SubscriptionController y emite intenciones del usuario.     |
| SubscriptionController | State Controller | Flutter / Dart<br><b>Diseño objetivo</b> | Mantener el estado de presentación de Flutter y coordinar casos de uso.                       | state<br>observeUseCase<br>syncUseCase | load()<br>submit(action)<br>retrySync() | Invoca Application Layer y publica estados de carga, éxito y error. |

### 2.6.8.3. Application Layer

Esta capa coordina las capacidades de **consulta de planes, contratación, pago y actualización de suscripciones**. Los commands y queries expresan intenciones; los handlers cargan aggregates, aplican reglas, persisten cambios y reaccionan a eventos. Los casos de uso móviles coordinan lectura local, actualización remota y sincronización idempotente.

| Clase                          | Categoría           | Producto y estado                                         | Propósito                                                                                                          | Atributos                                                        | Métodos u operaciones                                                                                                                                                                                                                                                | Relaciones                                                                      |
|--------------------------------|---------------------|-----------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| SubscriptionPlanCommandService | Application Service | Backend ASP.NET Core<br><b>Implementado en el backend</b> | Orquestar consulta de planes, contratación, pago y actualización de suscripciones sin contener reglas del dominio. | ISubscriptionPlanRepository repository<br>IUnitOfWork unitOfWork | Handle(CreateSubscriptionPlanCommand command, CancellationTokens cancellationTokens)<br>Handle(UpdateSubscriptionPlanCommand command, CancellationTokens cancellationTokens)<br>Handle(DeleteSubscriptionPlanCommand command, CancellationTokens cancellationTokens) | Invoca agregados y repositories; confirma la transacción mediante Unit of Work. |
| SubscriptionCommandService     | Application Service | Backend ASP.NET Core<br><b>Implementado en el backend</b> | Orquestar consulta de planes, contratación, pago y actualización de suscripciones sin contener reglas del dominio. | ISubscriptionRepository repository<br>IUnitOfWork unitOfWork     | Handle(CreateSubscriptionCommand command, CancellationTokens cancellationTokens)<br>Handle(UpdateSubscriptionCommand command, CancellationTokens cancellationTokens)<br>Handle(DeleteSubscriptionCommand command, CancellationTokens cancellationTokens)             | Invoca agregados y repositories; confirma la transacción mediante Unit of Work. |
| PaymentCommandService          | Application Service | Backend ASP.NET Core<br><b>Implementado en el backend</b> | Orquestar consulta de planes, contratación, pago y actualización de suscripciones sin contener reglas del dominio. | IPaymentRepository repository<br>IUnitOfWork unitOfWork          | Handle(CreatePaymentCommand command, CancellationTokens cancellationTokens)                                                                                                                                                                                          | Invoca agregados y repositories; confirma la transacción mediante Unit of Work. |
| SubscriptionQueryService       | Application Service | Backend ASP.NET Core<br><b>Implementado en el backend</b> | Orquestar consulta de planes, contratación, pago y actualización de suscripciones sin contener reglas del dominio. | ISubscriptionRepository repository                               | Handle(GetSubscriptionByIdQuery query, CancellationTokens cancellationTokens)<br>Handle(GetAllSubscriptionsQuery query, CancellationTokens cancellationTokens)                                                                                                       | Invoca agregados y repositories; confirma la transacción mediante Unit of Work. |
| CreateSubscriptionCommand      | Command/Query       | Backend ASP.NET Core                                      | Transportar una intención o                                                                                        | int UserId<br>int PlanId                                         | —                                                                                                                                                                                                                                                                    | Es recibida por un handler o servicio de                                        |

| Clase                            | Categoría       | Producto y estado                                         | Propósito                                                                                | Atributos                                                                                                                                                | Métodos u operaciones             | Relaciones                                                                                           |
|----------------------------------|-----------------|-----------------------------------------------------------|------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------|------------------------------------------------------------------------------------------------------|
|                                  |                 | <b>Implementado en el backend</b>                         | consulta tipada hacia su handler.                                                        | string StripeCustomerId<br>string StripeSubscriptionId<br>string Status<br>DateOnly StartedAt<br>DateOnly? EndsAt                                        |                                   | aplicación y no contiene lógica de negocio.                                                          |
| CreatePaymentCommand             | Command/Query   | Backend ASP.NET Core<br><b>Implementado en el backend</b> | Transportar una intención o consulta tipada hacia su handler.                            | int UserId<br>int SubscriptionId<br>decimal Amount<br>string Currency<br>string Provider<br>string ProviderPaymentId<br>string Status<br>DateTime PaidAt | —                                 | Es recibida por un handler o servicio de aplicación y no contiene lógica de negocio.                 |
| GetSubscriptionByIdQuery         | Command/Query   | Backend ASP.NET Core<br><b>Implementado en el backend</b> | Transportar una intención o consulta tipada hacia su handler.                            | int Id                                                                                                                                                   | —                                 | Es recibida por un handler o servicio de aplicación y no contiene lógica de negocio.                 |
| GetAllSubscriptionPlansQuery     | Command/Query   | Backend ASP.NET Core<br><b>Implementado en el backend</b> | Transportar una intención o consulta tipada hacia su handler.                            | filters: optional                                                                                                                                        | —                                 | Es recibida por un handler o servicio de aplicación y no contiene lógica de negocio.                 |
| ObserveSubscriptionUseCase       | Use Case        | Android y Flutter<br><b>Diseño objetivo</b>               | Entregar primero datos locales y actualizar la consulta cuando exista conectividad.      | localRepository<br>remoteRepository<br>connectivityMonitor                                                                                               | Execute(criteria): Stream<Result> | Es invocado por ViewModel/Controller y coordina repositorios móviles.                                |
| SyncSubscriptionUseCase          | Use Case        | Android y Flutter<br><b>Diseño objetivo</b>               | Procesar operaciones móviles pendientes de manera idempotente.                           | outboxRepository<br>remoteRepository<br>conflictResolver                                                                                                 | Execute(): SyncResult             | Lee el outbox local, consume la API y actualiza el estado de sincronización.                         |
| CreateSubscriptionCommandHandler | Command Handler | Backend ASP.NET Core<br><b>Diseño objetivo</b>            | Ejecutar una intención concreta, aplicar reglas del agregado y confirmar la transacción. | repository<br>unitOfWork<br>domainPolicy                                                                                                                 | Handle(command): Result           | Consuma un Command, carga el aggregate mediante su repository y puede publicar un Domain Event.      |
| CreatePaymentCommandHandler      | Command Handler | Backend ASP.NET Core<br><b>Diseño objetivo</b>            | Ejecutar una intención concreta, aplicar reglas del agregado y confirmar la transacción. | repository<br>unitOfWork<br>domainPolicy                                                                                                                 | Handle(command): Result           | Consuma un Command, carga el aggregate mediante su repository y puede publicar un Domain Event.      |
| PaymentConfirmedEventHandler     | Event Handler   | Backend ASP.NET Core<br><b>Diseño objetivo</b>            | Reaccionar al evento confirmado y actualizar proyecciones o integraciones.               | projectionRepository<br>notificationPort<br>unitOfWork                                                                                                   | Handle(domainEvent): Task         | Consuma un Domain Event y utiliza puertos de infraestructura sin modificar directamente el agregado. |

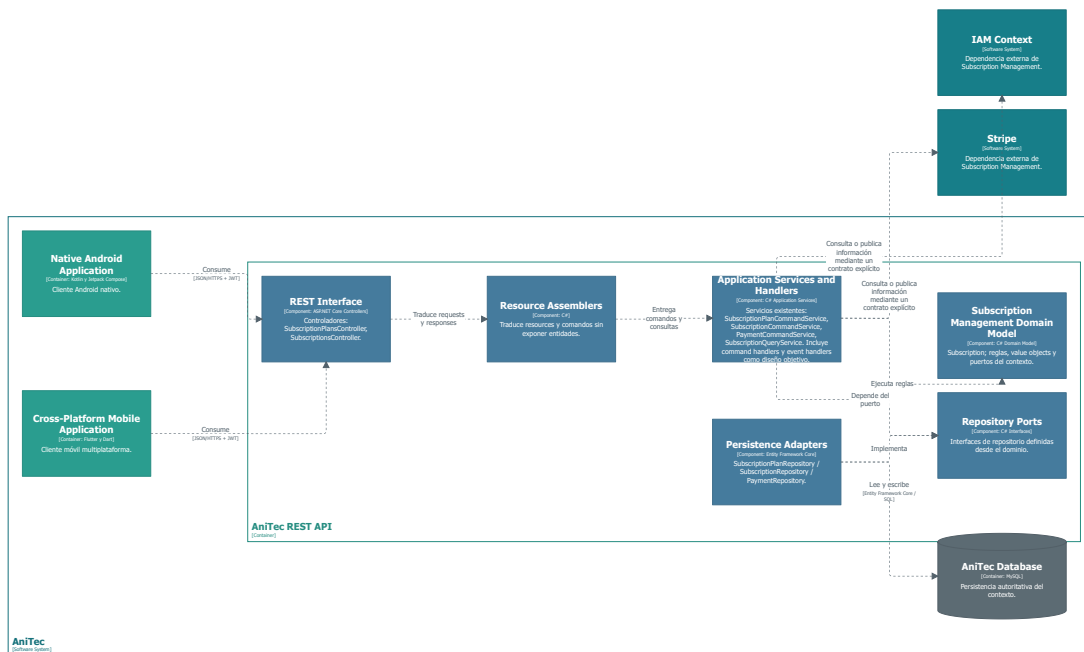
## 2.6.8.4. Infrastructure Layer

Esta capa implementa los puertos definidos hacia el interior de **Subscription Management** y concentra acceso a base de datos, red, almacenamiento local e integraciones externas. Las clases de infraestructura traducen errores y contratos técnicos antes de devolver resultados a Application Layer.

| Clase                                                                   | Categoría                 | Producto y estado                                                 | Propósito                                                           | Atributos                                                                                                                                   | Métodos u operaciones                                                                                                                                                                                 | Relaciones                                                                                                                                    |
|-------------------------------------------------------------------------|---------------------------|-------------------------------------------------------------------|---------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| SubscriptionPlanRepository / SubscriptionRepository / PaymentRepository | Repository Adapter        | Backend / Entity Framework Core <b>Implementado en el backend</b> | Implementar el puerto de persistencia definido por Domain Layer.    | SubscriptionPlanRepository: AppDbContext context<br>SubscriptionRepository: AppDbContext context<br>PaymentRepository: AppDbContext context | PaymentRepository.FindByUserIdAsync(int userId, CancellationToken cancellationToken)<br>PaymentRepository.FindByProviderPaymentIdAsync(string providerPaymentId, CancellationToken cancellationToken) | Implementa ISubscriptionPlanRepository / ISubscriptionRepository / IPaymentRepository; utiliza AppDbContext/MySQL y reconstruye el aggregate. |
| ModelBuilderExtensions                                                  | Persistence Configuration | Backend / Entity Framework Core <b>Implementado en el backend</b> | Mapear entidades y value objects del contexto al modelo relacional. | EntityTypeBuilder configuration                                                                                                             | ApplyConfiguration(modelBuilder)                                                                                                                                                                      | Configura tablas, claves, relaciones, restricciones y conversiones de Entity Framework Core.                                                  |
| SubscriptionApiDataSource                                               | Remote Adapter            | Android / Kotlin <b>Diseño objetivo</b>                           | Implementar el acceso remoto del cliente móvil a la API.            | httpClient<br>tokenProvider<br>serializer                                                                                                   | Get(criteria)<br>Create(dto)<br>Update(dto)<br>Delete(id)                                                                                                                                             | Consume controllers REST por HTTPS/JSON y traduce errores HTTP al modelo de aplicación.                                                       |
| SubscriptionDao                                                         | Room Adapter              | Android / Room <b>Diseño objetivo</b>                             | Implementar persistencia local y observación reactiva en Android.   | roomDatabase<br>entityMapper                                                                                                                | Observe(criteria)<br>Upsert(entity)<br>Delete(id)<br>Pending()                                                                                                                                        | Implementa el puerto local mediante Room y participa en la estrategia de caché/outbox.                                                        |
| SubscriptionRemoteDataSource                                            | Remote Adapter            | Flutter / Dart <b>Diseño objetivo</b>                             | Implementar el acceso remoto del cliente móvil a la API.            | httpClient<br>tokenProvider<br>serializer                                                                                                   | Get(criteria)<br>Create(dto)<br>Update(dto)<br>Delete(id)                                                                                                                                             | Consume controllers REST por HTTPS/JSON y traduce errores HTTP al modelo de aplicación.                                                       |
| SubscriptionLocalDataSource                                             | SQLite Adapter            | Flutter / Dart <b>Diseño objetivo</b>                             | Implementar persistencia local equivalente en Flutter.              | sqliteDatabase<br>entityMapper                                                                                                              | watch(criteria)<br>upsert(entity)<br>delete(id)<br>pending()                                                                                                                                          | Implementa el puerto local mediante SQLite y participa en la estrategia de caché/outbox.                                                      |
| IamSubscriberAdapter                                                    | Context Adapter           | Backend ASP.NET Core <b>Diseño objetivo</b>                       | Aislar una dependencia externa detrás de un puerto explícito.       | iamFacade                                                                                                                                   | Exists(userId): bool<br>GetEmail(userId): string                                                                                                                                                      | Implementa el puerto de suscriptor y consume IAM.                                                                                             |
| StripePaymentGateway                                                    | Anti-Corruption Layer     | Backend ASP.NET Core <b>Diseño objetivo</b>                       | Aislar una dependencia externa detrás de un puerto explícito.       | stripeClient,<br>webhookSecret                                                                                                              | CreateCheckout(command)<br>ConfirmWebhook(payload)<br>Refund(paymentId)                                                                                                                               | Implementa PaymentGateway y traduce Stripe al lenguaje del dominio.                                                                           |
| SubscriptionCacheStore                                                  | Local Cache Adapter       | Android y Flutter <b>Diseño objetivo</b>                          | Aislar una dependencia externa detrás de un puerto explícito.       | subscriptionDao                                                                                                                             | ReadActive(userId)<br>Save(subscription)<br>Invalidate()                                                                                                                                              | Implementa el puerto local de consulta mediante Room o SQLite.                                                                                |

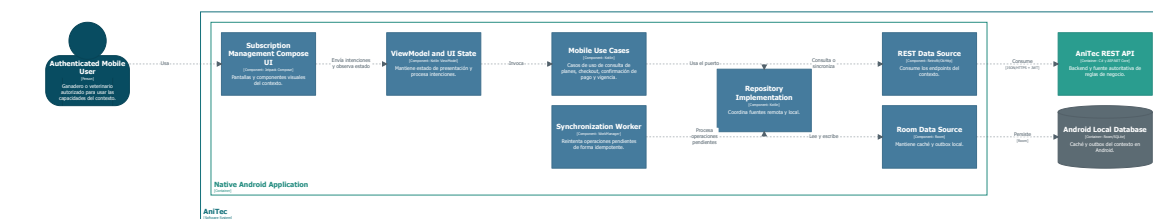
## 2.6.8.5. Bounded Context Software Architecture Component Level Diagrams

El archivo [component-level.dsl](#) contiene las vistas BC8-ApiComponents, BC8-AndroidComponents y BC8-FlutterComponents. Las tres parten del mismo modelo C4 y muestran la separación entre presentación, aplicación, dominio y adaptadores.



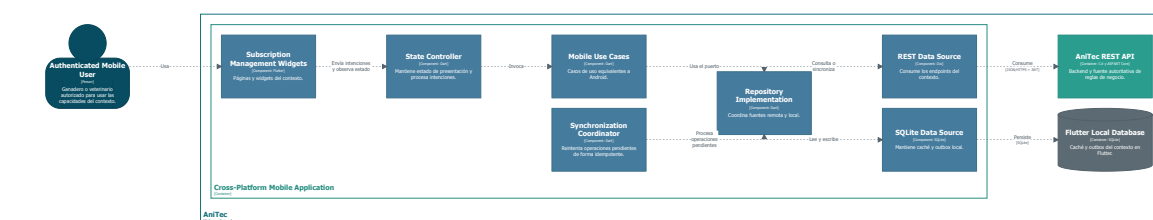
Component View: AniTec - AniTec REST API  
Componentes backend de Subscription Management.  
Tuesday, September 15, 2026 at 3:26 AM Coordinated Universal Time

Figura 2.6.8.1. Componentes de la API para Subscription Management. Fuente: elaboración propia con Structurizr DSL.



Component View: AniTec - Native Android Application  
Componentes backend de Subscription Management.  
Tuesday, September 15, 2026 at 3:26 AM Coordinated Universal Time

Figura 2.6.8.2. Componentes Android para Subscription Management. Fuente: elaboración propia con Structurizr DSL.



Component View: AniTec - Cross-Platform Mobile Application  
Componentes Flutter de Subscription Management.  
Tuesday, September 15, 2026 at 3:26 AM Coordinated Universal Time

Figura 2.6.8.3. Componentes Flutter para Subscription Management. Fuente: elaboración propia con Structurizr DSL.

## 2.6.8.6. Bounded Context Software Architecture Code Level Diagrams

Los diagramas de código detallan el modelo del dominio y los objetos de persistencia. El UML diferencia los elementos existentes de las incorporaciones objetivo, mientras los esquemas SQL señalan mediante comentarios las columnas propuestas. Los archivos ERD quedan disponibles para completar la importación manual.

### 2.6.8.6.1. Bounded Context Domain Layer Class Diagrams

El Class Diagram incluye agregados, entidades, value objects, enumeraciones, servicios de dominio e interfaces de repositorio con atributos, operaciones, visibilidad y multiplicidades.

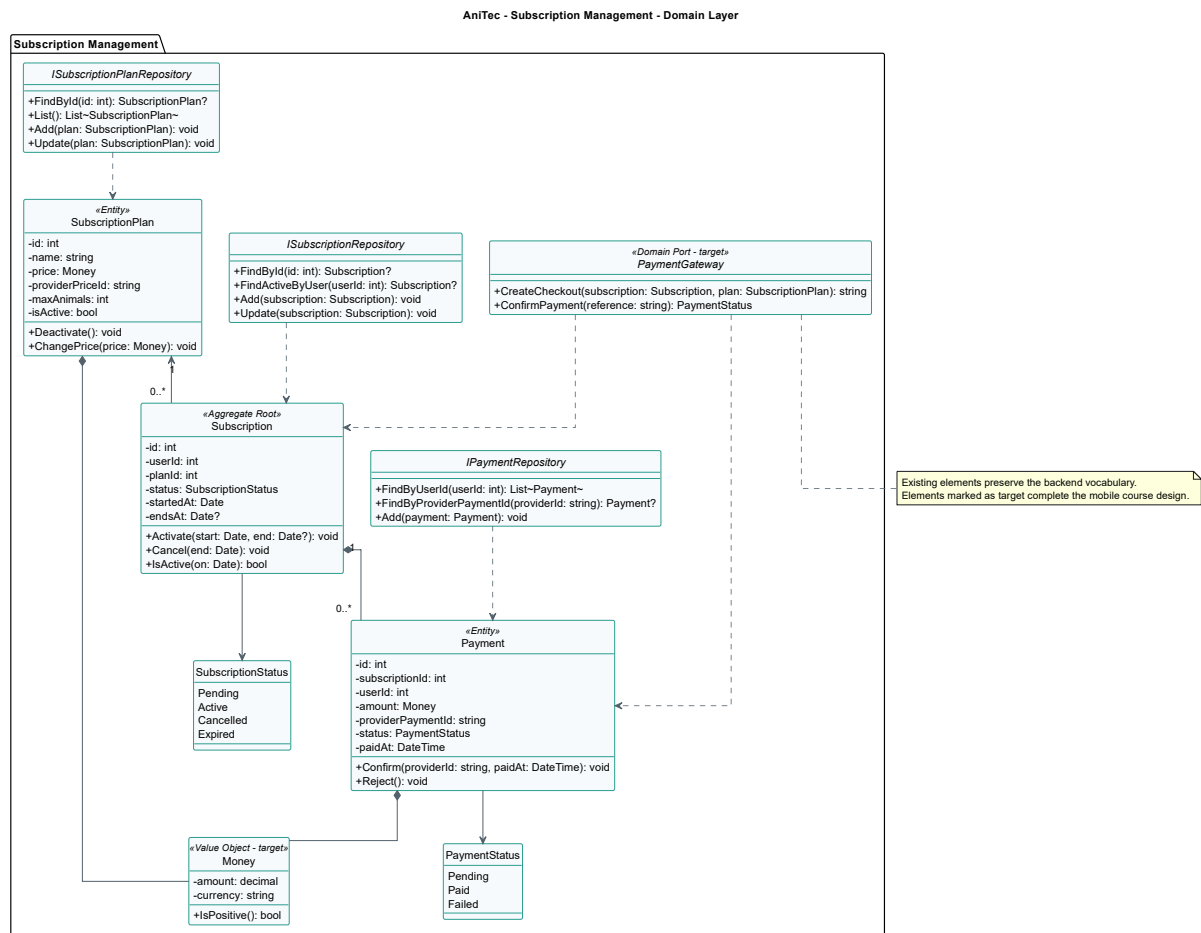


Figura 2.6.8.4. Domain Layer Class Diagram de Subscription Management. Fuente: elaboración propia con PlantUML.

## 2.6.8.6.2. Bounded Context Database Design Diagram

MySQL mantiene la persistencia autoritativa. Room y SQLite contienen únicamente caché, metadatos de sincronización y operaciones pendientes; no sustituyen las reglas ni la fuente de verdad del backend. En IAM, las credenciales y tokens permanecen fuera de las tablas locales y se almacenan mediante mecanismos seguros del sistema operativo.

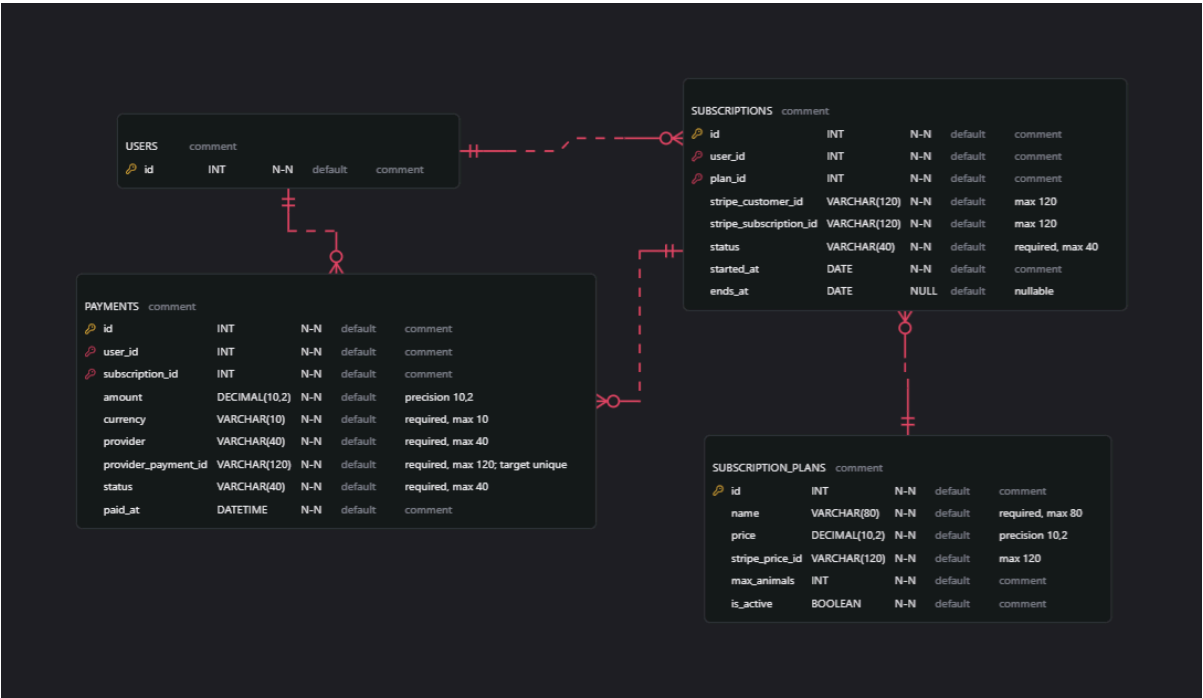


Figura 2.6.8.5. MySQL Database Design de Subscription Management. Fuente: elaboración propia a partir del esquema SQL.

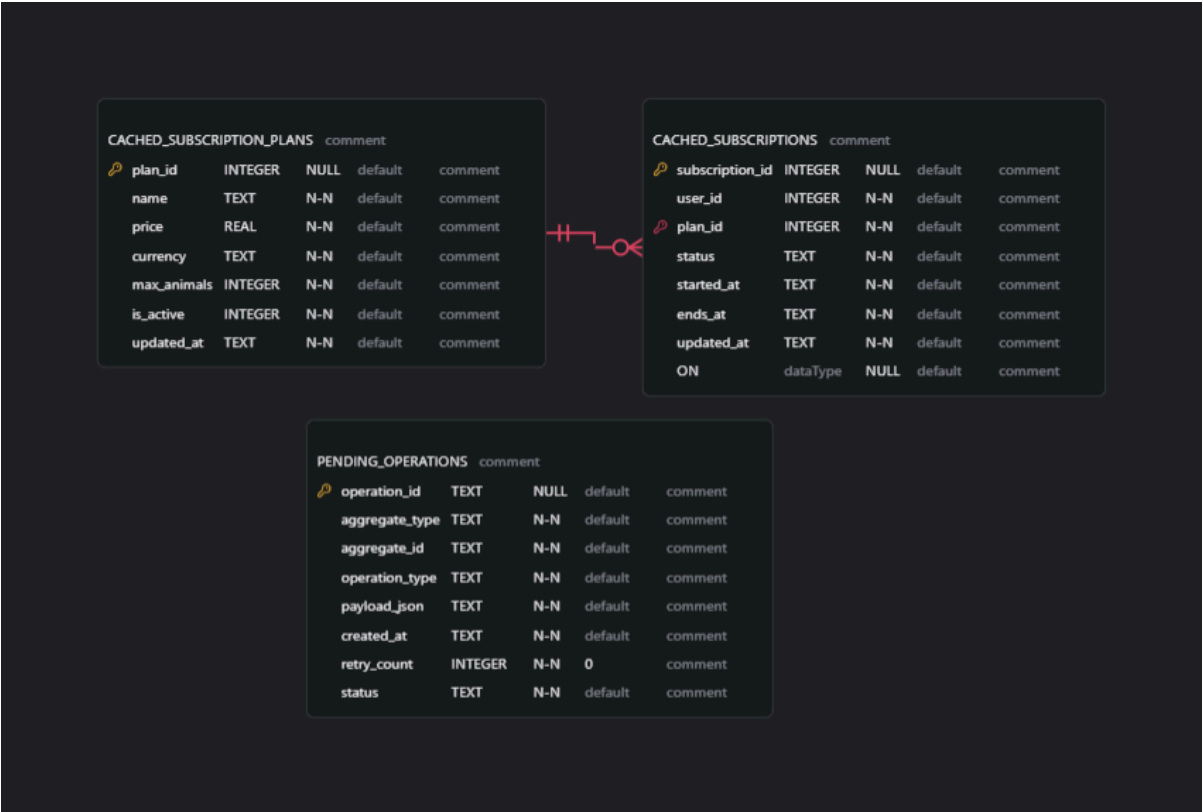


Figura 2.6.8.6. Android Room Database Design de Subscription Management. Fuente: elaboración propia a partir del esquema SQL.

## 2.6.9. Bounded Context: Analytics and Reporting

Construir proyecciones y métricas de consulta para los dashboards sin adquirir propiedad sobre los registros de los contextos fuente.

La base implementada se encuentra en el módulo Analytics de la API ASP.NET Core. El diseño móvil de Android y Flutter se presenta como **diseño objetivo** porque esos clientes todavía no existen en el workspace. Los tres productos comparten contratos REST y lenguaje ubicuo, mientras la API conserva las reglas autoritativas.

### 2.6.9.1. Domain Layer

Esta capa documenta el modelo que representa el núcleo de **Analytics and Reporting**. Las reglas autoritativas se ejecutan en la API; Android y Flutter mantienen modelos equivalentes para presentación, validación inmediata y trabajo offline. La columna de estado distingue el código heredado de la arquitectura objetivo.

| Clase                   | Categoría            | Producto y estado                                                                     | Propósito                                                 | Atributos                                                                                                                   | Métodos u operaciones                                                                                                                          | Relaciones                                                                                                                                                                                            |
|-------------------------|----------------------|---------------------------------------------------------------------------------------|-----------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| DashboardProjection     | Aggregate Root       | Backend, Android y Flutter (modelo canónico)<br><b>Diseño objetivo</b>                | Agrupar métricas calculadas para una audiencia y periodo. | id: int<br>ownerId: int<br>audience: string<br>period: MetricPeriod<br>generatedAt: DateTime<br>metrics: List~ReportMetric~ | ReplaceMetrics(metrics: List~ReportMetric~): void<br>IsStale(now: DateTime): bool                                                              | DashboardProjection compone ReportMetric;<br>DashboardProjection compone MetricPeriod;<br>ProjectionBuilder depende de DashboardProjection;<br>IReportMetricRepository depende de DashboardProjection |
| ReportMetric            | Entity               | Backend; modelo equivalente en Android y Flutter<br><b>Implementado en el backend</b> | Representar una métrica individual del dashboard.         | id: int<br>label: string<br>slice: MetricSlice<br>trend: string<br>sourceContext: string                                    | Update(slice: MetricSlice, trend: string): void                                                                                                | DashboardProjection compone ReportMetric;<br>ReportMetric compone MetricSlice                                                                                                                         |
| MetricSlice             | Value Object         | Backend, Android y Flutter (modelo canónico)<br><b>Diseño objetivo</b>                | Representar un valor, unidad y etiqueta de una métrica.   | label: string<br>value: decimal<br>unit: string                                                                             | IsComparableTo(other: MetricSlice): bool                                                                                                       | ReportMetric compone MetricSlice                                                                                                                                                                      |
| MetricPeriod            | Value Object         | Backend, Android y Flutter (modelo canónico)<br><b>Diseño objetivo</b>                | Representar el intervalo consultado.                      | from: Date<br>to: Date                                                                                                      | Contains(date: Date): bool                                                                                                                     | DashboardProjection compone MetricPeriod                                                                                                                                                              |
| ProjectionBuilder       | Domain Service       | Backend, Android y Flutter (modelo canónico)<br><b>Diseño objetivo</b>                | Construir proyecciones a partir de los contextos fuente.  | —                                                                                                                           | BuildRancher(ownerId: int, period: MetricPeriod): DashboardProjection<br>BuildVeterinarian(id: int, period: MetricPeriod): DashboardProjection | ProjectionBuilder depende de DashboardProjection                                                                                                                                                      |
| IReportMetricRepository | Repository Interface | Backend; modelo equivalente en Android y Flutter<br><b>Implementado en el backend</b> | Abstraer la persistencia de ReportMetric.                 | —                                                                                                                           | FindProjection(ownerId: int, period: MetricPeriod): DashboardProjection?<br>Save(projection: DashboardProjection): void                        | IReportMetricRepository depende de DashboardProjection                                                                                                                                                |

### 2.6.9.2. Interface Layer

Esta capa recibe las acciones relacionadas con **construcción y consulta de dashboards, métricas y proyecciones** y las traduce a casos de uso. Los controllers y resources corresponden a la API; las pantallas y controladores de estado representan la presentación objetivo en Android y Flutter. Ninguna de estas clases implementa reglas de negocio.

| Clase                        | Categoría       | Producto y estado                                         | Propósito                                                     | Atributos                                                                                                                                                                                                | Métodos u operaciones                                                                                                                                                                    | Relaciones                                                                  |
|------------------------------|-----------------|-----------------------------------------------------------|---------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| DashboardAnalyticsController | REST Controller | Backend ASP.NET Core<br><b>Implementado en el backend</b> | Publicar por HTTP las capacidades de Analytics and Reporting. | IHerdQueryService<br>herdQueryService<br>IAnimalQueryService<br>animalQueryService<br>IHealthEventQueryService<br>healthEventQueryService<br>IFinancialRecordQueryService<br>financialRecordQueryService | GetRancherDashboard(int rancherId, Cancellation token)<br>GetVeterinarianDashboard(int veterinarianId, Cancellation token)<br>GetRancherHealthSummary(int rancherId, Cancellation token) | Recibe resources, invoca servicios de aplicación y devuelve resources HTTP. |

| Clase                         | Categoría          | Producto y estado                                            | Propósito                                                                            | Atributos                                                                                                                                                               | Métodos u operaciones                                                                                                                                                                                                                                                                                                                                              | Relaciones                                                                                     |
|-------------------------------|--------------------|--------------------------------------------------------------|--------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
|                               |                    |                                                              |                                                                                      | IFarmActivityQueryService<br>farmActivityQueryService<br>IDeviceQueryService<br>deviceQueryService<br>IVeterinarianClientQueryService<br>veterinarianClientQueryService | cancellationToken)<br>GetRancherFinancialSummary(int<br>rancherId, CancellationToken<br>cancellationToken)                                                                                                                                                                                                                                                         |                                                                                                |
| ReportMetricsController       | REST Controller    | Backend<br>ASP.NET Core<br><b>Implementado en el backend</b> | Publicar por HTTP las capacidades de Analytics and Reporting.                        | IReportMetricCommandService<br>commandService<br>IReportMetricQueryService<br>queryService                                                                              | GetAll(CancellationToken<br>cancellationToken)<br>GetById(int id, CancellationToken<br>cancellationToken)<br>Create(CreateReportMetricResource<br>resource, CancellationToken<br>cancellationToken)<br>Update(int id,<br>CreateReportMetricResource<br>resource, CancellationToken<br>cancellationToken)<br>Delete(int id, CancellationToken<br>cancellationToken) | Recibe resources, invoca servicios de aplicación y devuelve resources HTTP.                    |
| RancherDashboardResource      | Resource/Assembler | Backend<br>ASP.NET Core<br><b>Implementado en el backend</b> | Definir un contrato estable de entrada o salida para la API REST.                    | int Herds<br>int Animals<br>int HealthyAnimals<br>int HealthEvents<br>int UpcomingActivities<br>int Devices<br>decimal Income<br>decimal Expenses<br>decimal Balance    | Create(...)<br>Deconstruct(...)                                                                                                                                                                                                                                                                                                                                    | Es construido o traducido por assemblers y consumido por el controller y los clientes móviles. |
| VeterinarianDashboardResource | Resource/Assembler | Backend<br>ASP.NET Core<br><b>Implementado en el backend</b> | Definir un contrato estable de entrada o salida para la API REST.                    | int Clients<br>int Herds<br>int Patients<br>int HealthEvents<br>int UpcomingVisits<br>int ActiveTreatments                                                              | Create(...)<br>Deconstruct(...)                                                                                                                                                                                                                                                                                                                                    | Es construido o traducido por assemblers y consumido por el controller y los clientes móviles. |
| FinancialSummaryResource      | Resource/Assembler | Backend<br>ASP.NET Core<br><b>Implementado en el backend</b> | Definir un contrato estable de entrada o salida para la API REST.                    | decimal Income<br>decimal Expenses<br>decimal Balance<br>IEnumerable<MetricSliceResource><br>ByCategory                                                                 | Create(...)<br>Deconstruct(...)                                                                                                                                                                                                                                                                                                                                    | Es construido o traducido por assemblers y consumido por el controller y los clientes móviles. |
| HealthSummaryResource         | Resource/Assembler | Backend<br>ASP.NET Core<br><b>Implementado en el backend</b> | Definir un contrato estable de entrada o salida para la API REST.                    | int TotalEvents<br>IEnumerable<MetricSliceResource><br>ByType                                                                                                           | Create(...)<br>Deconstruct(...)                                                                                                                                                                                                                                                                                                                                    | Es construido o traducido por assemblers y consumido por el controller y los clientes móviles. |
| DashboardProjectionScreen     | Composable         | Android /<br>Jetpack<br>Compose<br><b>Diseño objetivo</b>    | Presentar construcción y consulta de dashboards, métricas y proyecciones en Android. | uiState<br>onAction<br>navigation                                                                                                                                       | Render()<br>Submit()<br>Retry()                                                                                                                                                                                                                                                                                                                                    | Observa<br>DashboardProjectionViewModel<br>y emite acciones de interfaz.                       |
| DashboardProjectionViewModel  | Presentation Model | Android /<br>Kotlin<br><b>Diseño objetivo</b>                | Mantener el estado observable y traducir acciones de Android a casos de uso.         | state<br>observeUseCase<br>syncUseCase                                                                                                                                  | Load()<br>Submit(action)<br>RetrySync()                                                                                                                                                                                                                                                                                                                            | Invoca casos de uso de Application Layer y publica un UI State inmutable.                      |
| DashboardProjectionPage       | Widget             | Flutter / Dart<br><b>Diseño objetivo</b>                     | Presentar construcción y consulta de dashboards, métricas y proyecciones en Flutter. | state<br>onAction<br>router                                                                                                                                             | build(context)<br>submit()<br>retry()                                                                                                                                                                                                                                                                                                                              | Observa<br>DashboardProjectionController<br>y emite intenciones del usuario.                   |
| DashboardProjectionController | State Controller   | Flutter / Dart<br><b>Diseño objetivo</b>                     | Mantener el estado de presentación                                                   | state<br>observeUseCase<br>syncUseCase                                                                                                                                  | load()<br>submit(action)<br>retrySync()                                                                                                                                                                                                                                                                                                                            | Invoca Application Layer y publica estados de carga, éxito y error.                            |

| Clase | Categoría | Producto y estado | Propósito                            | Atributos | Métodos u operaciones | Relaciones |
|-------|-----------|-------------------|--------------------------------------|-----------|-----------------------|------------|
|       |           |                   | de Flutter y coordinar casos de uso. |           |                       |            |

### 2.6.9.3. Application Layer

Esta capa coordina las capacidades de **construcción y consulta de dashboards, métricas y proyecciones**. Los commands y queries expresan intenciones; los handlers cargan aggregates, aplican reglas, persisten cambios y reaccionan a eventos. Los casos de uso móviles coordinan lectura local, actualización remota y sincronización idempotente.

| Clase                             | Categoría           | Producto y estado                                      | Propósito                                                                                                 | Atributos                                                    | Métodos u operaciones                                                                                                                                                                                                                              | Relaciones                                                                                      |
|-----------------------------------|---------------------|--------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|--------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| ReportMetricCommandService        | Application Service | Backend ASP.NET Core <b>Implementado en el backend</b> | Orquestar construcción y consulta de dashboards, métricas y proyecciones sin contener reglas del dominio. | IReportMetricRepository repository<br>IUnitOfWork unitOfWork | Handle(CreateReportMetricCommand command, CancellationToken cancellationToken)<br>Handle(UpdateReportMetricCommand command, CancellationToken cancellationToken)<br>Handle(DeleteReportMetricCommand command, CancellationToken cancellationToken) | Invoca agregados y repositories; confirma la transacción mediante Unit of Work.                 |
| ReportMetricQueryService          | Application Service | Backend ASP.NET Core <b>Implementado en el backend</b> | Orquestar construcción y consulta de dashboards, métricas y proyecciones sin contener reglas del dominio. | IReportMetricRepository repository                           | Handle(GetReportMetricByIdQuery query, CancellationToken cancellationToken)<br>Handle(GetAllReportMetricsQuery query, CancellationToken cancellationToken)                                                                                         | Invoca agregados y repositories; confirma la transacción mediante Unit of Work.                 |
| CreateReportMetricCommand         | Command/Query       | Backend ASP.NET Core <b>Implementado en el backend</b> | Transportar una intención o consulta tipada hacia su handler.                                             | string Label<br>string Value<br>string Trend                 | —                                                                                                                                                                                                                                                  | Es recibida por un handler o servicio de aplicación y no contiene lógica de negocio.            |
| UpdateReportMetricCommand         | Command/Query       | Backend ASP.NET Core <b>Implementado en el backend</b> | Transportar una intención o consulta tipada hacia su handler.                                             | int Id<br>string Label<br>string Value<br>string Trend       | —                                                                                                                                                                                                                                                  | Es recibida por un handler o servicio de aplicación y no contiene lógica de negocio.            |
| GetAllReportMetricsQuery          | Command/Query       | Backend ASP.NET Core <b>Implementado en el backend</b> | Transportar una intención o consulta tipada hacia su handler.                                             | filters: optional                                            | —                                                                                                                                                                                                                                                  | Es recibida por un handler o servicio de aplicación y no contiene lógica de negocio.            |
| GetReportMetricByIdQuery          | Command/Query       | Backend ASP.NET Core <b>Implementado en el backend</b> | Transportar una intención o consulta tipada hacia su handler.                                             | int Id                                                       | —                                                                                                                                                                                                                                                  | Es recibida por un handler o servicio de aplicación y no contiene lógica de negocio.            |
| ObserveDashboardProjectionUseCase | Use Case            | Android y Flutter <b>Diseño objetivo</b>               | Entregar primero datos locales y actualizar la consulta cuando exista conectividad.                       | localRepository<br>remoteRepository<br>connectivityMonitor   | Execute(criteria): Stream<Result>                                                                                                                                                                                                                  | Es invocado por ViewModel/Controller y coordina repositorios móviles.                           |
| SyncDashboardProjectionUseCase    | Use Case            | Android y Flutter <b>Diseño objetivo</b>               | Procesar operaciones móviles pendientes de manera idempotente.                                            | outboxRepository<br>remoteRepository<br>conflictResolver     | Execute(): SyncResult                                                                                                                                                                                                                              | Lee el outbox local, consume la API y actualiza el estado de sincronización.                    |
| CreateReportMetricCommandHandler  | Command Handler     | Backend ASP.NET Core <b>Diseño objetivo</b>            | Ejecutar una intención concreta, aplicar reglas del agregado y confirmar la transacción.                  | repository<br>unitOfWork<br>domainPolicy                     | Handle(command): Result                                                                                                                                                                                                                            | Consume un Command, carga el aggregate mediante su repository y puede publicar un Domain Event. |

| Clase                           | Categoría     | Producto y estado                              | Propósito                                                                       | Atributos                                              | Métodos u operaciones     | Relaciones                                                                                            |
|---------------------------------|---------------|------------------------------------------------|---------------------------------------------------------------------------------|--------------------------------------------------------|---------------------------|-------------------------------------------------------------------------------------------------------|
| BuildDashboardQueryHandler      | Query Handler | Backend ASP.NET Core<br><b>Diseño objetivo</b> | Resolver una consulta de aplicación y construir el modelo de lectura requerido. | queryRepository<br>projectionBuilder                   | Handle(query): Result     | Consumes una Query y consulta repositories o proyecciones.                                            |
| SourceRecordChangedEventHandler | Event Handler | Backend ASP.NET Core<br><b>Diseño objetivo</b> | Reaccionar al evento confirmado y actualizar proyecciones o integraciones.      | projectionRepository<br>notificationPort<br>unitOfWork | Handle(domainEvent): Task | Consumes un Domain Event y utiliza puertos de infraestructura sin modificar directamente el agregado. |

## 2.6.9.4. Infrastructure Layer

Esta capa implementa los puertos definidos hacia el interior de **Analytics and Reporting** y concentra acceso a base de datos, red, almacenamiento local e integraciones externas. Las clases de infraestructura traducen errores y contratos técnicos antes de devolver resultados a Application Layer.

| Clase                               | Categoría                 | Producto y estado                                                    | Propósito                                                           | Atributos                                 | Métodos u operaciones                                           | Relaciones                                                                                   |
|-------------------------------------|---------------------------|----------------------------------------------------------------------|---------------------------------------------------------------------|-------------------------------------------|-----------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| ReportMetricRepository              | Repository Adapter        | Backend / Entity Framework Core<br><b>Implementado en el backend</b> | Implementar el puerto de persistencia definido por Domain Layer.    | AppDbContext<br>context                   | FindById(id)<br>Add(entity)<br>Update(entity)<br>Delete(entity) | Implementa IReportMetricRepository; utiliza AppDbContext/MySQL y reconstruye el aggregate.   |
| ModelBuilderExtensions              | Persistence Configuration | Backend / Entity Framework Core<br><b>Implementado en el backend</b> | Mapear entidades y value objects del contexto al modelo relacional. | EntityTypeBuilder<br>configuration        | ApplyConfiguration(modelBuilder)                                | Configura tablas, claves, relaciones, restricciones y conversiones de Entity Framework Core. |
| DashboardProjectionApiDataSource    | Remote Adapter            | Android / Kotlin<br><b>Diseño objetivo</b>                           | Implementar el acceso remoto del cliente móvil a la API.            | httpClient<br>tokenProvider<br>serializer | Get(criteria)<br>Create(dto)<br>Update(dto)<br>Delete(id)       | Consumes controllers REST por HTTPS/JSON y traduce errores HTTP al modelo de aplicación.     |
| DashboardProjectionDao              | Room Adapter              | Android / Room<br><b>Diseño objetivo</b>                             | Implementar persistencia local y observación reactiva en Android.   | roomDatabase<br>entityMapper              | Observe(criteria)<br>Upsert(entity)<br>Delete(id)<br>Pending()  | Implementa el puerto local mediante Room y participa en la estrategia de caché/outbox.       |
| DashboardProjectionRemoteDataSource | Remote Adapter            | Flutter / Dart<br><b>Diseño objetivo</b>                             | Implementar el acceso remoto del cliente móvil a la API.            | httpClient<br>tokenProvider<br>serializer | Get(criteria)<br>Create(dto)<br>Update(dto)<br>Delete(id)       | Consumes controllers REST por HTTPS/JSON y traduce errores HTTP al modelo de aplicación.     |
| DashboardProjectionLocalDataSource  | SQLite Adapter            | Flutter / Dart<br><b>Diseño objetivo</b>                             | Implementar persistencia local equivalente en Flutter.              | sqliteDatabase<br>entityMapper            | watch(criteria)<br>upsert(entity)<br>delete(id)<br>pending()    | Implementa el puerto local mediante SQLite y participa en la estrategia de caché/outbox.     |
| LivestockMetricsAdapter             | Projection Source Adapter | Backend ASP.NET Core<br><b>Diseño objetivo</b>                       | Aislar una dependencia externa detrás de un puerto explícito.       | livestockQueries                          | ReadLivestockMetrics(ownerId, period)                           | Implementa una fuente de proyección desde Livestock Management.                              |
| SanitaryMetricsAdapter              | Projection Source Adapter | Backend ASP.NET Core<br><b>Diseño objetivo</b>                       | Aislar una dependencia externa detrás de un puerto explícito.       | sanitaryQueries                           | ReadHealthMetrics(ownerId, period)                              | Implementa una fuente de proyección desde Sanitary Management.                               |
| ActivityMetricsAdapter              | Projection Source Adapter | Backend ASP.NET Core<br><b>Diseño objetivo</b>                       | Aislar una dependencia externa detrás de un puerto explícito.       | activityQueries                           | ReadActivityMetrics(ownerId, period)                            | Implementa una fuente de proyección desde Activity Management.                               |
| FinancialMetricsAdapter             | Projection Source Adapter | Backend ASP.NET Core<br><b>Diseño objetivo</b>                       | Aislar una dependencia externa detrás de un puerto explícito.       | financialQueries                          | ReadFinancialMetrics(ownerId, period)                           | Implementa una fuente de proyección desde Financial Management.                              |

2.6.9.5. Bounded Context Software Architecture Component Level Diagrams

El archivo [component-level.dsl](#) contiene las vistas BC9-ApiComponents, BC9-AndroidComponents y BC9-FlutterComponents. Las tres parten del mismo modelo C4 y muestran la separación entre presentación, aplicación, dominio y adaptadores.

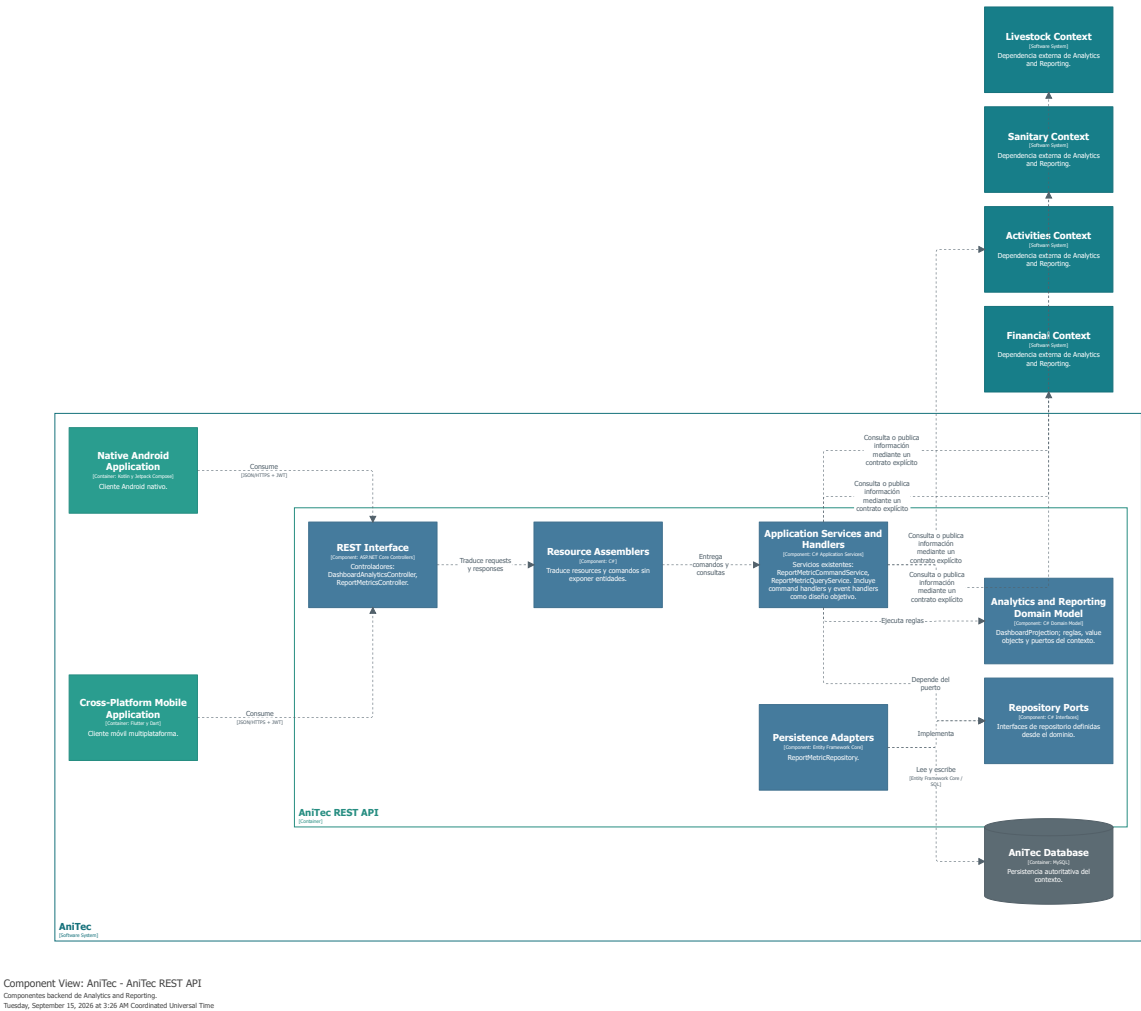


Figura 2.6.9.1. Componentes de la API para Analytics and Reporting. Fuente: elaboración propia con Structurizr DSL.

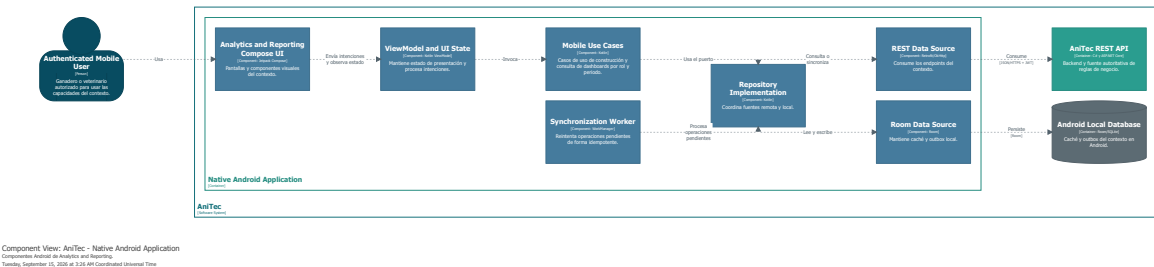


Figura 2.6.9.2. Componentes Android para Analytics and Reporting. Fuente: elaboración propia con Structurizr DSL.

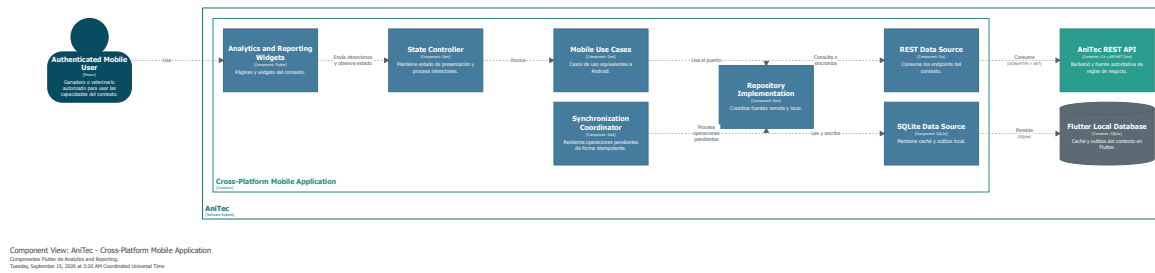


Figura 2.6.9.3. Componentes Flutter para Analytics and Reporting. Fuente: elaboración propia con Structurizr DSL.

## 2.6.9.6. Bounded Context Software Architecture Code Level Diagrams

Los diagramas de código detallan el modelo del dominio y los objetos de persistencia. El UML diferencia los elementos existentes de las incorporaciones objetivo, mientras los esquemas SQL señalan mediante comentarios las columnas propuestas. Los archivos ERD quedan disponibles para completar la importación manual.

### 2.6.9.6.1. Bounded Context Domain Layer Class Diagrams

El Class Diagram incluye agregados, entidades, value objects, enumeraciones, servicios de dominio e interfaces de repositorio con atributos, operaciones, visibilidad y multiplicidades.

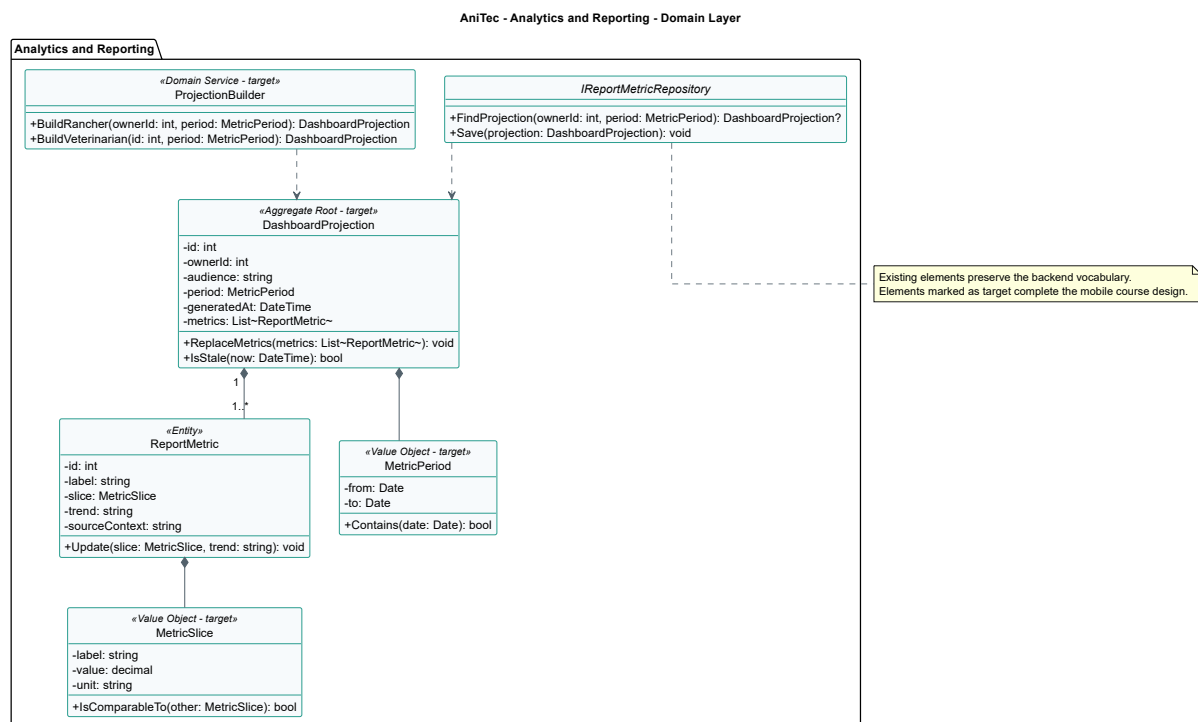


Figura 2.6.9.4. Domain Layer Class Diagram de Analytics and Reporting. Fuente: elaboración propia con PlantUML.

### 2.6.9.6.2. Bounded Context Database Design Diagram

MySQL mantiene la persistencia autoritativa. Room y SQLite contienen únicamente caché, metadatos de sincronización y operaciones pendientes; no sustituyen las reglas ni la fuente de verdad del backend. En IAM, las credenciales y tokens permanecen fuera de las tablas locales y se almacenan mediante mecanismos seguros del sistema operativo.

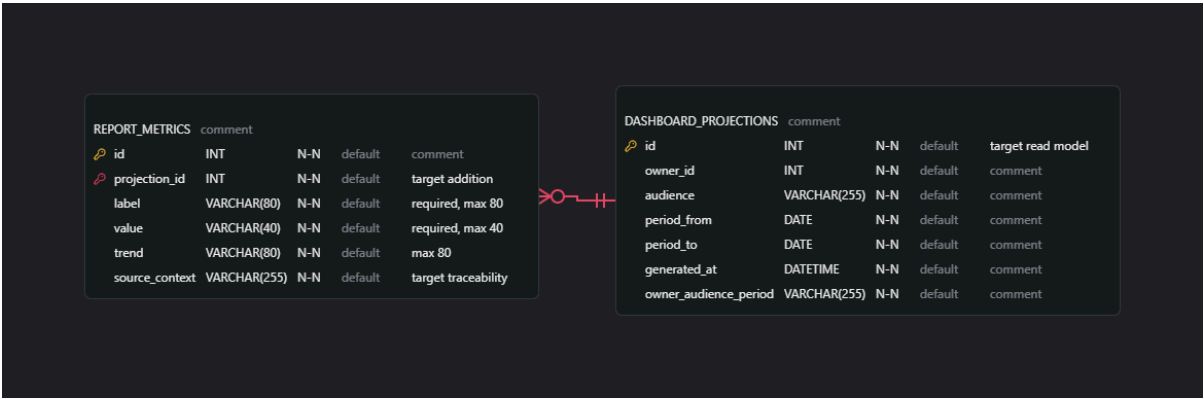


Figura 2.6.9.5. MySQL Database Design de Analytics and Reporting. Fuente: elaboración propia a partir del esquema SQL.

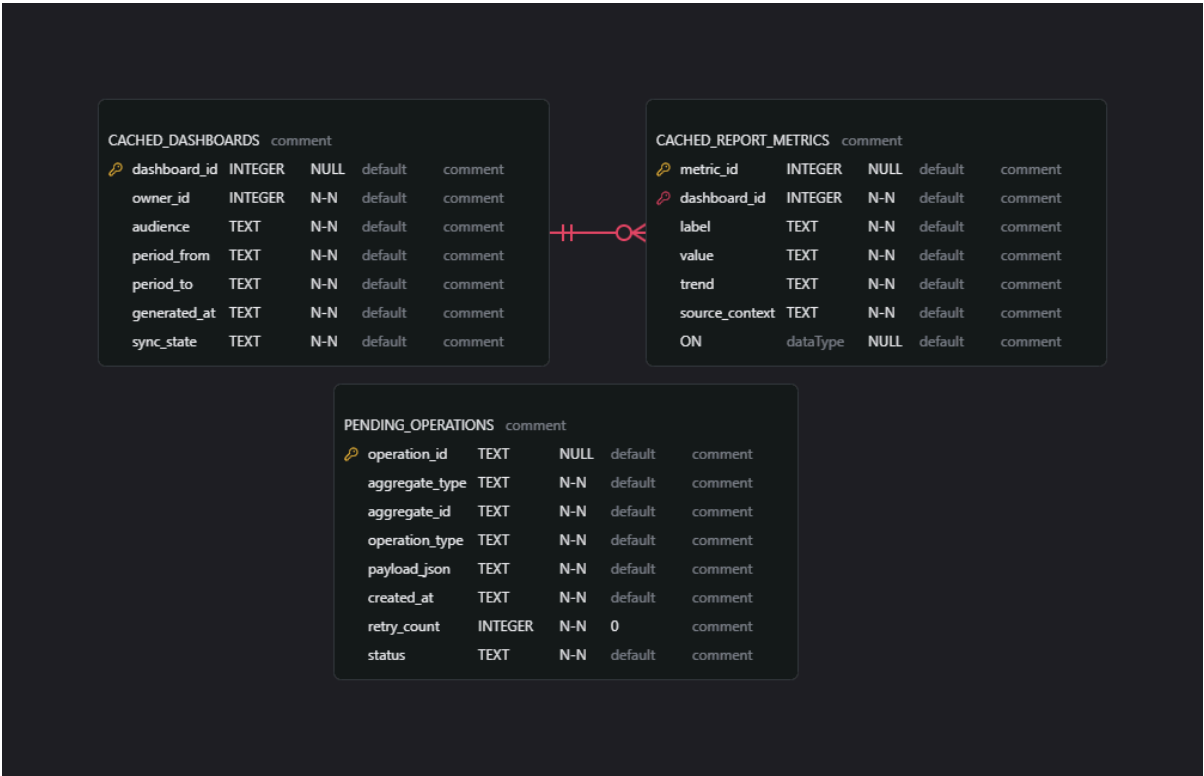


Figura 2.6.9.6. Android Room Database Design de Analytics and Reporting. Fuente: elaboración propia a partir del esquema SQL.

# Capítulo III: Solution UI/UX Design

---

## 3.1. Product design

---

### 3.1.1. Style Guidelines

#### 3.1.1.1. General Style Guidelines

### 3.1.2. Information Architecture

#### 3.1.2.1. Organization Systems

#### 3.1.2.2. Labelling Systems

#### 3.1.2.3. SEO Tags and Meta Tags

#### 3.1.2.4. Searching Systems

#### 3.1.2.5. Navigation Systems

### 3.1.3. Landing Page UI Design

#### 3.1.3.1. Landing Page Wireframe

#### 3.1.3.2. Landing Page Mock-up

### 3.1.4. Mobile Applications UX/UI Design

#### 3.1.4.1. Mobile Applications Wireframes

#### 3.1.4.2. Mobile Applications Wireflow Diagrams

#### 3.1.4.3. Mobile Applications Mock-ups

#### 3.1.4.4. Mobile Applications User Flow Diagrams

#### 3.1.4.5. Mobile Applications Prototyping

# Capítulo IV: Product Implementation & Validation

---

## 4. Product Implementation & Validation

---

### 4.1. Software Configuration Management

---

#### 4.1.1. Software Development Environment Configuration

#### 4.1.2. Source Code Management

#### 4.1.3. Source Code Style Guide & Conventions

#### 4.1.4. Software Deployment Configuration

### 4.2. Landing Page & Mobile Application Implementation

---

#### 4.2.1. Sprint n

##### 4.2.1.1. Sprint Planning n

##### 4.2.1.2. Aspect Leaders and Collaborators

##### 4.2.1.3. Sprint Backlog n

##### 4.2.1.4. Development Evidence for Sprint Review

##### 4.2.1.5. Testing Suite Evidence for Sprint Review

##### 4.2.1.6. Execution Evidence for Sprint Review

##### 4.2.1.7. Services Documentation Evidence for Sprint Review

##### 4.2.1.8. Software Deployment Evidence for Sprint Review

##### 4.2.1.9. Team Collaboration Insights during Sprint

### 4.3. Validation Interviews

---

#### 4.3.1. Diseño de Entrevistas

## 4.3.2. Registro de Entrevistas

## 4.3.3. Evaluaciones según heurísticas

# Conclusiones

---

## Conclusiones y recomendaciones

### Conclusiones

El proyecto AniTec permitio validar la necesidad de una solucion digital orientada a ganaderos y veterinarios, debido a que ambos segmentos requieren mayor orden, trazabilidad y disponibilidad de informacion sobre animales, fincas, actividades, sanidad y gastos. Las entrevistas y validaciones realizadas confirmaron que el problema identificado es real y que una plataforma web puede aportar valor si mantiene una experiencia simple, clara y cercana al contexto ganadero.

El diseño del sistema basado en Domain-Driven Design permitio organizar AniTec mediante bounded contexts como IAM, Profiles, Livestock, Sanitary, Financial, Activities, Analytics, Clients, Devices, Metrics y Subscriptions. Esta separacion favorece la mantenibilidad del sistema, facilita la asignacion de responsabilidades dentro del equipo y permite que cada modulo evolucione sin afectar innecesariamente al resto de la aplicacion.

Durante el Sprint 1 se logro establecer la presencia digital del producto mediante una landing page funcional desplegada en GitHub Pages. Este avance permitio comunicar la propuesta de valor de AniTec, presentar los beneficios principales para ganaderos y veterinarios, y generar una primera base de validacion frente a usuarios potenciales.

Durante el Sprint 2 se implemento la aplicacion web frontend con Vue, organizada por bounded contexts y conectada inicialmente a datos de prueba mediante my-json-server. Este sprint permitio construir las principales vistas funcionales de la plataforma, validar flujos de navegacion y preparar la estructura necesaria para reemplazar progresivamente los datos mock por servicios reales.

Durante el Sprint 3 se desarrollo el backend real de AniTec con ASP.NET Core, Entity Framework Core y MySQL. La API implementa autenticacion con JWT, gestion de usuarios, perfiles, hatos, animales, eventos sanitarios, actividades, registros financieros, clientes veterinarios, analiticas, dispositivos IoT, metricas, planes, suscripciones y pagos mock. Con ello, el proyecto dejo de depender solamente de datos simulados y paso a contar con una base de servicios persistentes.

La documentacion de servicios mediante Swagger/OpenAPI permitio verificar los endpoints implementados y facilitar las pruebas manuales del backend. Asimismo, las migraciones de Entity Framework Core permitieron mantener alineada la estructura de la base de datos con las entidades del dominio, reduciendo errores en la configuracion y mejorando la trazabilidad tecnica del sistema.

El despliegue del backend en Render representa un avance importante para el proyecto, ya que la API queda disponible desde internet y puede ser consumida por el frontend desplegado. Esto permite validar AniTec en un

entorno mas cercano a produccion, comprobar la disponibilidad publica del servicio y preparar futuras pruebas de integracion entre frontend, backend y base de datos.

En relacion con el Student Outcome, el equipo evidencio trabajo colaborativo y liderazgo conjunto durante los tres avances. Cada integrante asumió responsabilidades claras por modulo o artefacto, participo en la planificacion de tareas, colaboro en la integracion del sistema y apporto a la documentacion del producto. Esto permitio cumplir los objetivos de cada sprint de forma progresiva y mantener coherencia entre la propuesta, el diseño, el frontend, el backend y el despliegue.

### **Contraste con Lean UX y validaciones:**

El Problem Statement se mantiene vigente, ya que las validaciones confirmaron que ganaderos y veterinarios siguen enfrentando problemas de desorden, registros manuales y falta de trazabilidad. AniTec responde a una necesidad real al centralizar información sanitaria, productiva, económica y operativa en una sola plataforma.

Las Business Assumptions fueron validadas de forma general, debido a que los usuarios reconocieron valor en contar con una solución web para registrar animales, eventos sanitarios, actividades, finanzas, dispositivos IoT y clientes veterinarios. Sin embargo, todavía se requiere reforzar reportes, alertas y campos específicos para aumentar la utilidad percibida.

Las User Assumptions se confirmaron parcialmente, porque los entrevistados comprendieron la navegación por roles y consideraron útiles los dashboards y módulos principales. Aun así, la experiencia debe seguir simplificándose, especialmente en formularios, mensajes de error y explicación de funciones más técnicas como IoT.

Las Feature Assumptions resultaron pertinentes para una primera versión del producto, ya que los usuarios identificaron valor en el registro de animales, historial sanitario, actividades, finanzas y seguimiento veterinario. Como mejora futura, se deben ampliar los campos clínicos, datos del animal y métricas disponibles para apoyar mejores decisiones.

Las Hypothesis Statements fueron parcialmente validadas. Existe interés inicial y una percepción positiva sobre la utilidad de AniTec, especialmente por la centralización de información; no obstante, la adopción dependerá de fortalecer la confianza, mejorar recordatorios, optimizar reportes y facilitar el uso continuo de la plataforma.

Los criterios de éxito quedan planteados como metas para una siguiente etapa de validación con uso real de la aplicación desplegada. Las entrevistas permiten confirmar intención de uso y valor percibido, pero aún falta medir reducción de errores, mejora en precisión de registros y apoyo efectivo a decisiones basadas en datos.

### **Recomendaciones**

**Corto plazo:** Se recomienda continuar con la integracion completa entre el frontend desplegado en GitHub Pages y el backend desplegado en Render, asegurando que las variables de entorno del frontend apunten a la API real y que los flujos principales funcionen sin depender de my-json-server.

**Corto plazo:** Se recomienda fortalecer la autenticacion y autorizacion del sistema, definiendo permisos por rol para ganaderos, veterinarios y administradores. Esto permitiria proteger los endpoints sensibles y asegurar que cada usuario solo acceda a la informacion correspondiente a su perfil.

**Mediano plazo:** Se recomienda continuar la validacion con ganaderos y veterinarios usando la aplicacion desplegada, no solo prototipos. Esto permitira identificar problemas reales de uso, comprension del lenguaje, dificultad en formularios, utilidad de dashboards y necesidades adicionales en los modulos de sanidad, actividades, finanzas e IoT.

**Mediano plazo:** Se recomienda mejorar la experiencia de usuario del frontend en una siguiente iteracion, priorizando claridad en dashboards, simplificacion de formularios, estados visuales para dispositivos IoT y mejor organizacion del flujo veterinario por clientes asignados.

**Mediano plazo:** Se recomienda completar y fortalecer las pruebas del backend, incluyendo pruebas de endpoints, validacion de reglas de negocio, manejo de errores y pruebas de integracion con la base de datos. Esto ayudara a asegurar mayor estabilidad antes de seguir ampliando funcionalidades.

**Futuro roadmap:** Se recomienda mantener actualizada la documentacion tecnica, incluyendo endpoints, migraciones, evidencias de despliegue, diagramas C4 y decisiones de arquitectura. A medida que AniTec crezca, esta documentacion sera clave para sostener la colaboracion del equipo y facilitar futuras mejoras del producto.

## Video About The Team

El video About The Team presenta la participación de los integrantes del equipo Titan Team 4 durante el desarrollo de AniTec, destacando las actividades realizadas, los logros alcanzados en el curso y el desarrollo de competencias asociadas al trabajo colaborativo, liderazgo conjunto, planificación de tareas y cumplimiento de objetivos.

**Datos del video:**

| Elemento                          | Información                                                                                              |
|-----------------------------------|----------------------------------------------------------------------------------------------------------|
| Título                            | Video About The Team - AniTec                                                                            |
| Duración                          |                                                                                                          |
| Público objetivo                  | Docente del curso, visitantes del landing page e interesados en conocer el proceso de trabajo del equipo |
| URL publicado en Microsoft Stream |                                                                                                          |
| URL publicado en YouTube          |                                                                                                          |
| Uso en landing page               | El video de YouTube se utiliza como evidencia pública del proceso de trabajo del equipo.                 |

**URL del video publicado en Microsoft Stream:**

**URL del video publicado en YouTube:**

## Bibliografía

AgriWebb. (s. f.). *AgriWebb mobile app*. Recuperado el 13 de septiembre de 2026, de <https://www.agriwebb.com/mobile-app/>

- AgriWebb. (s. f.). *Pricing*. Recuperado el 13 de septiembre de 2026, de <https://www.agriwebb.com/us/pricing/>
- Android Developers. (s. f.). *Jetpack Compose*. Recuperado el 13 de septiembre de 2026, de <https://developer.android.com/develop/ui/compose>
- Android Developers. (s. f.). *Save data in a local database using Room*. Recuperado el 13 de septiembre de 2026, de <https://developer.android.com/training/data-storage/room>
- Bourgau, P. (2022, marzo 29). *Step by step guide to run your big picture event storming*. Philippe Bourgau's Blog. <https://bit.ly/bpes-guide>
- Evans, E. (2003). *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley Professional.
- Food and Agriculture Organization of the United Nations. (2022). *Digital agriculture: Technologies transforming agriculture and food systems*. <https://www.fao.org/digital-agriculture/en/>
- Food and Agriculture Organization of the United Nations. (2023). *Livestock and the environment*. <https://www.fao.org/livestock-environment/en/>
- Firebase. (s. f.). *Firebase App Distribution*. Recuperado el 13 de septiembre de 2026, de <https://firebase.google.com/docs/app-distribution>
- Flutter. (s. f.). *Flutter documentation*. Recuperado el 13 de septiembre de 2026, de <https://docs.flutter.dev/>
- Google for Developers. (s. f.). *Scan barcodes with ML Kit on Android*. Recuperado el 13 de septiembre de 2026, de <https://developers.google.com/ml-kit/vision/barcode-scanning/android>
- Chandrasekaran, P. (2023, marzo 20). *Remote EventStorming in practice*. Domain-driven Design: A Practitioner's Guide. <https://ddd-practitioners.com/2023/03/20/remote-eventstorming-workshop/>
- Cohn, M. (2006). *User stories applied: For agile software development*. Addison-Wesley Professional.
- Ddd-By-Examples. (s. f.). *library/docs/big-picture.md at master*. GitHub. <https://github.com/ddd-by-examples/library/blob/master/docs/big-picture.md>
- Ddd-By-Examples. (s. f.-b). *library/docs/design-level.md at master*. GitHub. <https://github.com/ddd-by-examples/library/blob/master/docs/design-level.md>
- Gothelf, J., & Seiden, J. (2021). *Lean UX: Designing great products with agile teams* (3rd ed.). O'Reilly Media.
- Instituto Nacional de Estadística e Informática. (2014). *Características socioeconómicas del productor agropecuario en el Perú: IV Censo Nacional Agropecuario 2012*. [https://www.inei.gob.pe/media/MenuRecursivo/publicaciones\\_digitales/Est/Lib1177/libro.pdf](https://www.inei.gob.pe/media/MenuRecursivo/publicaciones_digitales/Est/Lib1177/libro.pdf)
- Instituto Nacional de Estadística e Informática. (2025a). *Estadísticas de las tecnologías de información y comunicación en los hogares: Enero-febrero-marzo 2025*. [https://www.inei.gob.pe/media/MenuRecursivo/boletines/informe-tecnico\\_tecnologiasdelainformacion\\_ene\\_feb\\_mar2025.pdf](https://www.inei.gob.pe/media/MenuRecursivo/boletines/informe-tecnico_tecnologiasdelainformacion_ene_feb_mar2025.pdf)
- Instituto Nacional de Estadística e Informática. (2026). *Estadísticas de las tecnologías de información y comunicación en los hogares: Octubre-noviembre-diciembre 2025*. <https://www.inei.gob.pe/media/MenuRecursivo/boletines/boletin-tic->

oct\_dic2025.pdf

iLivestock. (s. f.). *Livestock management software for sheep and cattle farmers*. Recuperado el 13 de septiembre de 2026, de <https://www.ilivestock.com/software>

iLivestock. (s. f.). *iLivestock Argentina: Innovación y tecnología agrícola*. Recuperado el 13 de septiembre de 2026, de <https://www.ilivestock.com/es-ar/ar>

Livestock Manager. (s. f.). *Livestock Manager: Made by Pila*. Recuperado el 13 de septiembre de 2026, de <https://app.livestockfarm.co/>

Microsoft. (2026). *ASP.NET Core documentation*. <https://learn.microsoft.com/aspnet/core/>

Microsoft. (2026). *Entity Framework Core documentation*. <https://learn.microsoft.com/ef/core/>

MySQL. (2026). *MySQL 8.0 reference manual*. <https://dev.mysql.com/doc/refman/8.0/en/>

OpenAPI Initiative. (2026). *OpenAPI specification*. <https://spec.openapis.org/oas/latest.html>

Pressman, R. S. (2010). *Ingeniería de software: Un enfoque práctico* (7a ed.). McGraw-Hill.

Scrum.org. (2020). *The Scrum Guide*. <https://scrumguides.org/scrum-guide.html>

Structurizr. (2026). *Structurizr documentation*. <https://docs.structurizr.com/>

Stripe. (s. f.). *Use a prebuilt Stripe-hosted payment page*. Recuperado el 13 de septiembre de 2026, de <https://docs.stripe.com/payments/checkout>

Vue.js. (2026). *Vue.js documentation*. <https://vuejs.org/guide/>

Weerakoon, S. (2020, 1 noviembre). How to Write a User Story for an API Product. Medium. <https://sameera17w.medium.com/how-to-write-a-user-story-for-an-api-product-7af6abd4ad2e>

## Anexos

---

Los siguientes anexos reúnen los enlaces disponibles hasta el AV1. Se incluyen únicamente evidencias utilizadas en los capítulos I y II, repositorios vigentes del curso y despliegues del producto web tomado como base. Los recursos correspondientes a validación de la aplicación móvil y a los siguientes sprints se incorporarán cuando sean elaborados.

## Anexo A. Videos de entrevistas

### Segmento de ganaderos

- [Entrevista 1 - Caterina Rodriguez](#)
- [Entrevista 2 - Rebeca Noemi Quiroz Roldan](#)
- [Entrevista 3 - Ursula Trigozo Salcedo](#)

### Segmento de veterinarios

- [Entrevista 4 - Angela Mendoza](#)
- [Entrevista 5 - Jeferson Castro Pariona](#)

## Anexo B. Materiales de investigación y diseño

- [Lean UX Canvas de AniTec](#)
- [Big Picture EventStorming y artefactos de Strategic-Level DDD](#)

Los enlaces públicos del Impact Mapping y del Product Backlog todavía no han sido incorporados al informe. Se añadirán cuando los tableros utilizados por el equipo estén disponibles públicamente.

## Anexo C. Repositorios del curso

- [Repositorio del informe](#)
- [Repositorio de la landing page](#)
- [Repositorio del frontend web tomado como base](#)
- [Repositorio del backend tomado como base](#)

Los repositorios de Android y Flutter se incorporarán cuando se creen los proyectos móviles correspondientes.

## Anexo D. Despliegues del producto web tomado como base

Estos enlaces corresponden al trabajo anterior que sirve como punto de partida. No constituyen todavía el despliegue de las aplicaciones móviles del curso actual.

- [Landing page de AniTec](#)
- [Aplicación web de AniTec](#)
- [Documentación Swagger del backend](#)

## Anexo E. Herramientas y documentación utilizadas hasta el AV1

### Investigación, requisitos y modelado

- [Miro](#), para Lean UX y EventStorming.
- [UXPressia](#), para User Personas, Empathy Maps y User Journey Maps.
- [Gherkin](#), para expresar criterios de aceptación mediante Given, When y Then.
- [GitHub](#), para control de versiones y colaboración.

### Arquitectura y diagramas

- [Structurizr](#), para los diagramas C4.
- [PlantUML](#), para los diagramas de clases del dominio.
- [Mermaid](#), como referencia para diagramas y modelos visuales.

### Tecnologías consideradas para la solución móvil

- [Jetpack Compose](#), para la interfaz de la aplicación Android nativa.
- [Room](#), para la persistencia local en Android.
- [Flutter](#), para la aplicación multiplataforma.
- [Google ML Kit Barcode Scanning](#), para la investigación de identificación mediante códigos QR.
- [Firebase App Distribution](#), para la distribución de versiones móviles durante la validación.
- [Stripe Checkout](#), para el flujo externo de pagos y suscripciones.

## Anexo F. Videos de exposición y validación

Hasta el AV1 no se han incorporado enlaces públicos para el video de exposición, las entrevistas de validación de la aplicación móvil, el video About the Product ni el video About the Team. Estos recursos se añadirán en los hitos que correspondan según el statement del curso.